VELLORE INSTITUTE OF TECHNOLOGY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

SWARA

Secure Wideband Adaptive Real-time Audio over UDP

A perceptually aware, mutually authenticated voice-chat system over UDP with importance-weighted forward error correction, joint quality control and overlay relay selection

FINAL PROJECT REPORT

SUBMITTED BY

Rohit Arora 24BDS0252
Ansh Gupta 24BDS0119
Amrit Raj 24BDS0104

PROJECT GUIDE

Dr. Anitha E
Assistant Professor Sr. Grade 1
Department of Information Security
School of Computer Science and Engineering
anitha.e@vit.ac.in

Department of Information Security · School of Computer Science and Engineering
Vellore Institute of Technology

Project Details



PROJECT GUIDE

Dr. Anitha E

Assistant Professor Sr. Grade 1

Department of Information Security

School of Computer Science and Engineering

Vellore Institute of Technology

anitha.e@vit.ac.in

TEAM

#	Name	Registration number	Programme
1	Rohit Arora	24BDS0252	B.Tech. · SCOPE, VIT
2	Ansh Gupta	24BDS0119	B.Tech. · SCOPE, VIT
3	Amrit Raj	24BDS0104	B.Tech. · SCOPE, VIT

Project title	SWARA — Secure Wideband Adaptive Real-time Audio over UDP
Course	Computer Networks
Department	Department of Information Security, School of Computer Science and Engineering
Institution	Vellore Institute of Technology
Submission	October 2026
Repository	https://github.com/rohitarora80905-glitch/SWARA_Complete_Project
Deliverables	Working voice-chat application over UDP (server, clients, web console), evaluation harness, research figures, this report

Abstract

Conventional voice-over-IP stacks treat every 20 ms packet of speech as equally valuable, protect all of them with the same redundancy, size the playout buffer against a loss target that ignores the delay it adds, and secure the call with handshakes that were designed for connection-oriented transports. This project presents **SWARA** (Secure Wideband Adaptive Real-time Audio over UDP), a complete, working voice-chat system built from the datagram up, together with the research evaluation that motivated its design.

SWARA's central idea is that packet loss should be judged by what the listener loses, not by how many packets were lost. A causal **Perceptual Importance (PI)** analyser scores each frame from its energy, voicing, zero-crossing rate, spectral flux and onset behaviour, and carries the score as an authenticated-but-readable octet in the packet header. A **PI-weighted adaptive forward error correction** scheme (PI-AFEC) spends a fixed Reed–Solomon parity budget on the frame groups that carry meaning and withholds it from silence, and protects vowel onsets with time-displaced *anchor replicas* that survive loss bursts a block code cannot. The **NAVIK** controller chooses codec bitrate, redundancy and DiffServ marking jointly by maximising the ITU-T G.107 E-model rating, distinguishes congestion from link loss through the one-way-delay trend, and couples the R-factor-optimal jitter buffer to the protection decision. **SPARK**, a stateless password-authenticated key agreement, mutually authenticates users in two round trips without ever sending the password, bounds reflection amplification at 0.31×, and migrates a call between networks in zero round trips; the relay forwards ChaCha20-Poly1305 ciphertext it cannot read. **SETU**, a distance-vector overlay, selects relay paths when the direct Internet route violates the triangle inequality.

The system is implemented in about 5,500 lines of Python over real UDP sockets, IMA-ADPCM speech coding with per-frame state reset, X25519/HKDF/ChaCha20-Poly1305 cryptography, and a FastAPI/uvicorn web console that runs instrumented live calls, A/B benchmarks and an interactive protocol toolkit. In the simulated evaluation, SWARA raises predicted MOS at 12 % loss from 3.38 (unprotected) and 3.37 (uniform FEC with the same budget) to 3.53, reaches the same quality as uniform FEC for 16.7 % less redundancy, keeps the measured optimum playout depth within 1 ms of its own choice, and authenticates in 397 bytes against roughly 1,900 bytes for DTLS-SRTP. In live calls over an impaired loopback path at 15 % loss with mean bursts of four packets, SWARA matches uniform FEC's perceptual damage (7.3 %) with 11 % less bandwidth and reduces it by 16 % relative to unprotected UDP; at 10 % loss it achieves the lowest damage of the three (4.2 %) at 12 % less bandwidth than uniform FEC. The evaluation also uncovered and corrected a scheduler defect in the project's own impairment injector that had silently disabled redundancy mid-call, illustrating why the measurement instrument must be validated before the protocol is judged.

Keywords — VoIP, UDP, forward error correction, unequal loss protection, Reed–Solomon, perceptual quality, E-model, jitter buffer, password-authenticated key exchange, X25519, ChaCha20-Poly1305, DiffServ, overlay routing.

Contents

Project Details i

Abstract ii

Contents iii

List of Figures v

List of Tables vii

Abbreviations viii

1 Introduction 1

1.1 Background 1

1.2 Motivation 1

1.3 Problem statement 2

1.4 Scope and contributions 2

1.5 Organisation of the report 3

2 Literature Survey 4

2.1 Packet loss recovery for streaming audio 4

2.2 Unequal protection and perceptual weighting 4

2.3 Playout scheduling and concealment 4

2.4 Objective quality assessment 5

2.5 Channel models 5

2.6 Secure real-time transport and key establishment 5

2.7 Network QoS and overlay routing 5

3 Research Gap 7

4 Objectives 8

5 Proposed Work 9

5.1 System architecture 9

5.2 Wire protocol 9

5.3 Perceptual importance analysis 11

5.4 PI-AFEC: importance-weighted adaptive FEC 12

5.5 SPARK: stateless password-authenticated key agreement 13

5.6 NAVIK: joint quality control 15

5.7 Receiver: estimation, repair and R-factor-optimal playout 16

5.8 SETU: voice-aware overlay routing 17

5.9 Speech codec with per-frame state reset 17

5.10 Evaluation methodology 18

6 Implementation 19

6.1 Technology stack 19

6.2 Repository layout 20

6.3 Module responsibilities 20

6.4 The live call pipeline 21

6.5 The relay server 22

6.6 The client 22
6.7 Demonstration harness and experiments 22
6.8 The web console and REST API 23
6.9 Server control launcher 24
6.10 Verification during development 25
6.11 Running the system 26

7 Results and Discussion 27

7.1 Experimental setup 27
7.2 Perceptual importance separates what matters 27
7.3 Quality under packet loss 28
7.4 Resilience to bursty loss 29
7.5 Quality returned per byte of redundancy 30
7.6 The jitter-buffer optimum 31
7.7 Handshake cost and security properties 32
7.8 Relay-path selection 33
7.9 Realistic network profiles 33
7.10 Live end-to-end calls 34
7.11 Discussion 37
7.12 Limitations 38

8 Conclusion 39

8.1 Future work 39

References 40

Appendix A Source Code Repository 42

A.1 Obtaining and running the code 42
A.2 Repository contents 42
A.3 Reproducibility 43

Appendix B Screenshots 44

B.1 Starting the server 44
B.2 A real call between three separate programs — both sides 44
B.3 Command-line clients on the relay hosted by the web console 46
B.4 Live Call Lab 50
B.5 A/B Benchmark 53
B.6 Network Simulator 55
B.7 Research Results 56
B.8 Protocol toolkit 64
B.9 Operations and API 71

Appendix C Source Code 74

List of Figures

Why not all losses are equal. Left: the perceptual damage weight used throughout this project rises superlinearly with a frame's importance and is penalised for consecutive losses. Centre: the E-model's delay impairment has a knee at 177.3 ms, so an over-deep jitter buffer is a quality defect. Right: for a fixed loss rate, clustered (bursty) loss is far more damaging than random loss. 2
SWARA end-to-end architecture for one direction of a call. Orange stages are the project's novel components; black stages are conventional. The relay in the middle forwards ciphertext it cannot read. 9
The 16-byte SWARA header and the composition of a wideband media datagram. The header and the optional 16-byte telemetry block are AEAD associated data: readable on the path, unforgeable. Everything after them is ciphertext. 10
PI-AFEC on three consecutive groups of k = 4 frames. The silent group receives no parity; its budget moves to the group carrying the vowel onset, which additionally earns an anchor replica displaced past the expected burst length. 13
The SPARK handshake and 0-RTT path migration, with on-the-wire sizes measured from the implementation. 14
The NAVIK control loop: receiver-side estimation feeds a single R-factor objective whose decisions drive the scheduler, the codec and the packet marking. 16
A triangle-inequality violation: the direct A–B path crosses a congested exchange while the two-hop path through C is far cheaper in voice-cost terms. 17
The Gilbert–Elliott loss model used by both the simulator and the live injector. 18
The four threads of a CallSession and the state they share. 21
The web console's Overview page: headline results, the twelve-stage pipeline, and the hosted relay card. 24
The SWARA Server Control window with the server running. 25
PI score distributions by speech class (violins show the full distribution, bars the mean) and the share of frames versus the share of importance held by silence. 28
Call quality under loss (left; shaded bands ±1 s.e.) and the reduction in perceptually weighted loss relative to the baseline (right). 29
Resilience to bursty loss (left) and the mechanism responsible for each recovery (right). 30
Quality returned per byte of redundancy. 31
Too shallow loses packets, too deep loses the conversation: the R-factor curve (left) and the end-to-end validation (right) 31
Call-setup cost in round trips and bytes. 32
SETU: distribution of the cost reduction on improved pairs (left) and the share of pairs by chosen route length (right). 33
Measured call quality across realistic network profiles (dashed line: the 3.6 “acceptable” threshold). 34
Terminal output of demo.py --compare at 15 % loss, mean burst 4, 12 ms jitter (25 s calls, 2 repeats per arm). 36
Terminal output of demo.py --compare at 10 % loss, mean burst 3, 10 ms jitter. 36
SWARA Server Control, opened by double-clicking “SWARA Server.bat”. The server was started with one click; the window shows the console address, the LAN address for other laptops, the hosted relay on udp:9000 and the live server log. 44
Terminal 1 — the relay (`python server.py --port 9000`) during the call: both users authenticated by SPARK and the call being relayed “keys not held”. 45
Terminal 2 — the callee (`python client.py --user bob … --wait`) during the call: authenticated in two round trips, incoming call accepted, media flowing. 45
Terminal 3 — the caller (`python client.py --user alice … --call bob`) during the call. 45
Terminal 1 after the call: the relay's complete log for the session. 45
Terminal 2 after the call — bob's own report: 1,021 frames sent with 287 parity and 17 replica packets; 1,015 played; 14 rebuilt by FEC; 102 concealed (10.0 %); no authentication failures. 46

Terminal 3 after the call — alice's own report: 1,021 frames sent with 273 parity and 32 replicas; 1,011 played; 7 rebuilt by FEC; 139 concealed (13.7 %); no authentication failures. 46
Relay & Users page while alice and bob (command-line clients) are in a call through the hosted relay. 47
Overview page during the same call: the hosted-relay card reports two users online and one call relayed. 48
The caller's terminal for the hosted-relay call (30 s, 10 % loss, burst 3, 10 ms jitter on alice's socket). 49
The callee's terminal for the hosted-relay call. 49
Live Call Lab before a call: audio source, call length, protection scheme, codec mode and the network sliders (presets from Clean to Poor 3G). 50
Live Call Lab during a call: the frame-fate map fills from left to right (bar height = PI; teal delivered, blue rebuilt from parity, violet anchor replica, red concealed), the sent and received waveforms, and the live gauges. 51
Live Call Lab after the call: the complete frame-fate map, the three time-series charts (network as measured, adaptation, quality), the call report with playable sent and received audio, the call log and the session table. 52
A/B Benchmark page before a run: audio source, seconds per call, repeats, and the network conditions shared by all three arms. 53
A/B Benchmark after a run at 15 % loss, burst 4, 12 ms jitter (20 s per arm): per-arm cards, the verdict, four comparison charts, the like-for-like table and the audio of each arm. 54
Network Simulator: the Congested 4G profile simulated for SWARA, uniform FEC and the baseline with MOS gauges, detailed per-system figures and the loss-accounting chart. 55
Research Results — loss sweep tab, with the interactive chart, the finding, the paper figure, the figure gallery and the regeneration control. 56
Research Results — burst sweep tab. 57
Research Results — overhead efficiency tab. 58
Research Results — PI distribution tab. 59
Research Results — jitter buffer tab. 60
Research Results — handshake tab. 61
Research Results — SETU overlay tab. 62
Research Results — network profiles tab. 63
Perceptual Analyzer: synthetic speech coloured by class with the PI trace, the ground-truth / label / FEC-plan strips, the frame inspector, the PI histogram and per-class statistics. 64
FEC Playground: an RS(6, 4) group with two data shards dropped and recovered; the PI-AFEC scheduler for a vowel-onset group. 64
FEC Playground after the “Burst” button: more shards lost than the code can repair — the frame must be concealed. 65
Quality & NAVIK: the E-model calculator with its impairment breakdown and the controller's decisions across loss rates. 66
SPARK Handshake: the six messages with sizes and timings, the derived keys matching on both sides, and the 0-RTT path migration. 67
SPARK Handshake with a wrong password: the envelope fails to open locally and AUTH_CONFIRM is never sent. 68
Packet Anatomy: the 16-byte header packed by Header.pack(), the hex dump, and what the AEAD protects. 69
SETU Overlay: a seven-node measured mesh, the converged route for the selected pair, and all pairs ranked by gain. 70
Relay & Users (idle): hosted relay status, the ready-to-paste client commands, the relay log and the user registry. 71
System & API: runtime, packages, the source-module inventory and every endpoint. 72
The generated OpenAPI documentation (Swagger UI) at /docs — the first five endpoint groups of thirty-seven documented routes. 73

List of Tables

Summary of the most relevant prior work and its relation to SWARA. 6
Research gaps and the SWARA component that addresses each. 7
Objectives and success criteria. 8
Header fields. 10
Datagram types. 10
Technology stack and versions used for the reported results. 19
Source modules and their responsibilities. 20
Web console pages. 23
Experimental configuration. 27
Perceptual importance score by speech class (0–255). 28
Loss sweep: predicted MOS (G.107) and perceptually weighted loss, mean of 5 seeds. 28
Burst sweep at 12 % loss: MOS and SWARA's recoveries per call by mechanism, mean of 4 seeds. 29
Overhead efficiency at 12 % loss, burst 4: MOS and redundancy actually spent for each nominal budget ρ. 30
Jitter-buffer depth: predicted R-factor and measured end-to-end MOS. 31
Measured handshake and media-protection cost. 32
Security properties of SPARK and how each is obtained. 32
Named profiles: predicted MOS, residual and perceptual loss, mouth-to-ear delay and redundancy, mean of 4 seeds. 34
Diagnosis of the injector defect: one call at 15 % loss, burst 4, 12 ms jitter, sampled once per second. 35
Live A/B comparison (demo.py, 25 s × 2 repeats, injector at both sockets). 35
Repository contents. 42

Abbreviations

ADPCM Adaptive Differential Pulse-Code Modulation (IMA variant, 4 bits per sample)

AEAD Authenticated Encryption with Associated Data

AF41 / EF / BE Assured Forwarding class 4 (DSCP 34), Expedited Forwarding (DSCP 46), Best Effort (DSCP 0)

AIMD Additive-Increase / Multiplicative-Decrease congestion control

API / REST Application Programming Interface / Representational State Transfer

CID Connection Identifier (32-bit, carried in every SWARA header)

CRC Cyclic Redundancy Check

DSCP Differentiated Services Code Point (6 bits of the IP header)

DTLS-SRTP Datagram TLS keying for the Secure Real-time Transport Protocol (the WebRTC baseline)

E-model ITU-T G.107 computational model mapping impairments to a transmission rating R

FEC Forward Error Correction

GF(2^8) Galois field with 256 elements, the arithmetic of the Reed–Solomon code

GIL Global Interpreter Lock (CPython)

HKDF / HMAC HMAC-based Key Derivation Function / Hash-based Message Authentication Code

MDS Maximum Distance Separable (any k of k+r shards reconstruct the group)

MOS Mean Opinion Score (1–5, predicted here from R by G.107)

NAT Network Address Translation

NAVIK Network-Adaptive Voice-quality Integrated Kontroller (the joint optimiser)

OPAQUE Asymmetric password-authenticated key exchange (envelope construction borrowed by SPARK)

OWD One-Way Delay

PBKDF2 Password-Based Key Derivation Function 2

PI Perceptual Importance (0–255, one octet per frame)

PI-AFEC Perceptual-Importance-weighted Adaptive FEC

PLC Packet Loss Concealment

Ppl Packet-loss probability as consumed by the E-model (percent)

QoS Quality of Service

ROC Rollover Counter (SRTP-style reconstruction of a 16-bit sequence field)

RS(n, k) Systematic Reed–Solomon erasure code with k data and n−k parity shards

RTP / RTCP Real-time Transport Protocol / RTP Control Protocol

RTT Round-Trip Time

SETU Self-Evaluating Topology Underlay (the relay-selection overlay; Hindi “bridge”)

SIP Session Initiation Protocol

SPARK Stateless Password-Authenticated Retry & Key-agreement

SSE Server-Sent Events (console streaming)

SWARA Secure Wideband Adaptive Real-time Audio over UDP (Hindi/Sanskrit “voice, note”)

UDP User Datagram Protocol

VAD Voice Activity Detection

VoIP Voice over Internet Protocol

WRED Weighted Random Early Detection

X25519 Elliptic-curve Diffie–Hellman over Curve25519

ZCR Zero-Crossing Rate

1 Introduction

1.1 Background

Interactive voice is the application that first made the Internet's best-effort service model visible to ordinary users. Speech is captured in short frames — 20 ms is the common choice — and each frame is placed in its own datagram, because a conversation cannot wait for a retransmission: by the time a lost frame could be resent and delivered it is already too late to play. Voice therefore runs over UDP [46] with RTP framing [1], accepts that some datagrams will be lost, delayed or reordered, and relies on the receiver to hide the damage. Three impairments dominate the listener's experience: *packet loss*, which removes audio; *delay*, which, past roughly 150 ms mouth-to-ear, makes a conversation feel like a walkie-talkie exchange [3]; and *jitter*, the variation in delay, which must be absorbed by a playout buffer that itself adds delay.

Loss on real paths is not a coin flip. Queues overflow, radio links fade and handovers stall, so losses arrive in bursts [16], [17]. A single lost 20 ms frame can be concealed almost inaudibly by repeating or interpolating its neighbours [15]; a burst of eight consecutive losses cannot, because 160 ms of speech cannot be invented. Any protection scheme and any evaluation that assumes independent loss therefore overstates how well it will work in a hostel Wi-Fi network or on a congested 4G cell.

The standard defence is forward error correction (FEC): the sender transmits redundant information so that the receiver can rebuild a lost frame without a round trip [5], [6]. Modern stacks adapt the amount of redundancy to the measured loss rate [12], [47]. They all share one simplifying assumption that this project sets out to remove: *every frame is worth the same amount of protection*.

1.2 Motivation

Speech intelligibility is carried by a minority of frames. Voiced vowel nuclei carry the formant structure that identifies a phoneme; the onsets of vowels and plosives carry the transient cues that mark word boundaries; unvoiced fricatives such as /s/ and /f/ are noise-like and tolerate substitution; and roughly a third of a talker's frames in a two-way conversation are silence or background noise, which are perceptually free to lose. A loss-driven FEC scheme spends exactly as much protecting a frame of silence as a frame carrying a vowel onset. The parity packets spent on silence are pure overhead; the parity packets *not* spent on the onset are the ones the listener notices.

Three further observations motivated the design. First, the jitter buffer and the protection scheme are usually tuned independently, yet parity can only rescue a frame that is still waiting in the buffer when the parity arrives, so buffer depth bounds the usefulness of redundancy. Second, loss caused by congestion and loss caused by a fading link call for opposite responses — back off, or protect harder — and a controller that cannot tell them apart makes the wrong choice half the time [43]. Third, the security layer is usually bolted on afterwards: DTLS-SRTP [21] costs about three round trips and some two kilobytes before the first frame flows, offers no graceful handover when a phone moves from Wi-Fi to mobile data, and a naive UDP server that answers unauthenticated packets with larger ones becomes a reflection amplifier.

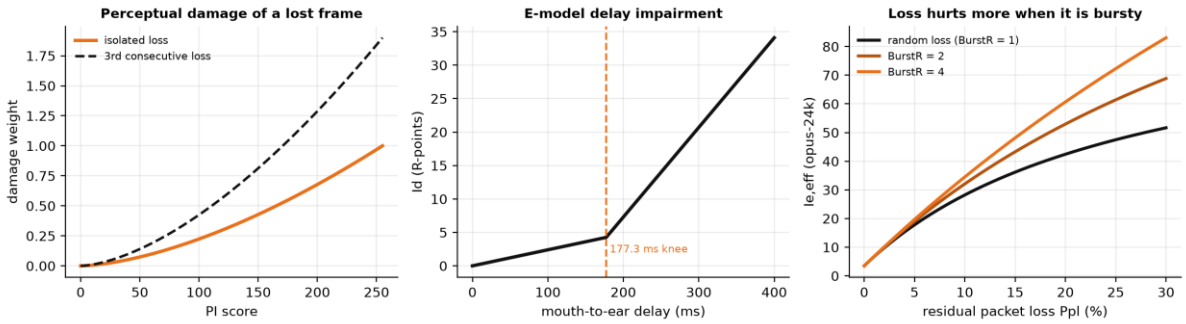


Figure 1. Why not all losses are equal. Left: the perceptual damage weight used throughout this project rises superlinearly with a frame's importance and is penalised for consecutive losses. Centre: the E-model's delay impairment has a knee at 177.3 ms, so an over-deep jitter buffer is a quality defect. Right: for a fixed loss rate, clustered (bursty) loss is far more damaging than random loss.

1.3 Problem statement

Design, implement and evaluate a voice-chat system over UDP that (i) allocates loss protection by the perceptual importance of each frame rather than uniformly, (ii) selects codec bitrate, redundancy, packet marking and playout depth jointly against a single objective quality model, (iii) authenticates users mutually with a password that never crosses the wire, in a minimal number of round trips, without creating amplification or denial-of-service exposure, and (iv) relays and routes media through infrastructure that cannot read it — and demonstrate the resulting quality gains with a like-for-like, reproducible evaluation against the conventional alternatives.

1.4 Scope and contributions

The project delivers a working application, not only a model. Real UDP sockets carry real ADPCM-coded speech, sealed with real X25519 / HKDF / ChaCha20-Poly1305 cryptography, between a relay server and command-line or web-console clients. The contributions are:

1. **PI analyser** — a causal, single-frame-latency perceptual importance scorer whose output travels as authenticated associated data so that every element on the path can act on it without reading the speech (Section 5.3).
2. **PI-AFEC** — a systematic Reed–Solomon erasure code over GF(2^8) whose per-group parity is set by group importance under a fixed budget, plus deadline-bounded anchor replicas for the most important frames and a burst mode that redirects the budget when block coding is futile (Section 5.4).
3. **SPARK** — a stateless, password-authenticated key agreement with an address-validation cookie, an OPAQUE-style credential envelope, triple Diffie–Hellman with forward secrecy, encrypted resumption tickets and zero-round-trip path migration (Section 5.5).
4. **NAVIK** — a joint controller that maximises the predicted E-model rating over a codec-by-redundancy grid every 100 ms, with a burst-aware residual-loss model and a delay-trend discriminator between congestion and link loss (Section 5.6).
5. **R-factor-optimal playout** — a jitter buffer whose depth maximises R rather than meeting a loss target, with perceptually weighted late-packet cost and a floor that guarantees the protection being paid for can actually be used (Section 5.7).

6. **SETU** — a distance-vector overlay with a composite voice-aware link cost in which loss is expressed in decibels so that it adds along a path exactly as delay does (Section 5.8).
7. **A reproducible evaluation harness** — a time-based Gilbert–Elliott impairment injector, an offline simulator that runs identical speech and identical channel realisations through SWARA and two comparators, and a web console with instrumented live calls, A/B benchmarks and an interactive toolkit (Sections 5.10 and 6).

Out of scope are a formal security proof of SPARK, subjective listening tests (quality is predicted with the G.107 model), recorded human speech (the corpus is a source-filter synthesiser, chosen to keep the evaluation self-contained and licence-free), and group calls with more than two participants, although SETU is designed with them in mind.

1.5 Organisation of the report

Chapter 2 surveys the literature on loss recovery, unequal protection, playout scheduling, quality assessment, secure real-time transport and overlay routing. Chapter 3 distils the research gaps and Chapter 4 states the objectives. Chapter 5 presents the proposed design component by component, with the wire protocol, algorithms and equations. Chapter 6 describes the implementation: modules, threads, the relay server and clients, the web console and the launcher. Chapter 7 reports the results of the simulated evaluation, the live end-to-end benchmarks and the diagnostic investigation that accompanied them, and discusses their limits. Chapter 8 concludes. Appendix A gives the repository, Appendix B the screenshots of every part of the running system, and Appendix C the source code.

2 Literature Survey

This chapter reviews the work on which SWARA builds, grouped by the problem each line of research addresses. Table 1 summarises the works most directly relevant to the design and notes what each leaves open.

2.1 Packet loss recovery for streaming audio

Perkins, Hodson and Hardman's survey [5] remains the standard taxonomy of loss recovery for audio: sender-driven techniques (retransmission, forward error correction, interleaving) and receiver-driven concealment (insertion, interpolation, regeneration). Its central observation is that retransmission is unusable for interactive voice, interleaving adds a delay proportional to its depth, and FEC is the only sender-side mechanism that repairs loss within a frame or two of delay. Bolot, Fosse-Parisis and Towsley [6] made FEC *adaptive*, choosing the redundancy level from the measured loss process with a Gilbert model and showing that rate-based adaptation is necessary because fixed redundancy either wastes bandwidth or fails when loss rises. The same loss-driven philosophy underlies the IETF payload formats for generic FEC [47] and the flexible FEC used by WebRTC [12], and the in-band low-bit-rate redundancy (LBRR) of the Opus codec [11].

The erasure codes themselves are well understood. Reed and Solomon's polynomial codes [7] are maximum-distance-separable: any k of k+r code words reconstruct the k data words. Rizzo [8] showed that a systematic Vandermonde-based construction over GF(2^8) can be implemented with table lookups fast enough for software packet-level FEC, and that systematic codes have the decisive practical property that data packets are sent unmodified, so a group that arrives intact costs no decoding at all. SWARA's `fec.py` follows this construction.

2.2 Unequal protection and perceptual weighting

Mohr, Riskin and Ladner [9] introduced *unequal loss protection* for progressively coded images: because early bits of a progressive bit-stream matter more than later ones, assigning more parity to the important prefix degrades quality gracefully instead of catastrophically. The idea was adopted widely for layered video, but far less for speech, where the frames are not naturally ordered by importance — importance must be *estimated from content*. Jiang and Schulzrinne [10] compared loss-repair methods for VoIP under bursty loss and showed that the perceived benefit of FEC depends strongly on burst length and on where the redundancy is placed in time, concluding that loss repair must be optimised against perceived quality rather than raw loss. Clark [18] modelled how burst loss and recency affect subjective quality and argued that an equal-weight loss count is a poor predictor of what a listener reports.

2.3 Playout scheduling and concealment

Ramjee et al. [13] formulated adaptive playout as the problem of trading late-packet loss against added delay and proposed per-talkspurt adjustment driven by delay statistics. Liang, Färber and Girod [14] moved the adjustment inside talkspurts using time-scale modification and showed that playout scheduling and concealment should be designed together. ITU-T G.711 Appendix I [15] standardised a low-complexity concealment algorithm based on pitch-synchronous repetition with attenuation,

establishing the practice that concealment should fade rather than repeat indefinitely. In all of this work the buffer objective is a loss target; the delay the buffer adds enters only as a secondary constraint.

2.4 Objective quality assessment

ITU-T P.800 [4] defines the Mean Opinion Score obtained from listening panels; because panels are expensive, transmission planning uses the ITU-T G.107 E-model [2], which combines a base rating with additive impairments for delay (Id), equipment and packet loss (Ie,eff) and an advantage factor (A) into a rating R on a 0–100 scale that maps to MOS. The model's Annex on bursty loss introduces the burst ratio, by which the same average loss is charged more when it is clustered. Sun and Ifeachor [19] validated and extended E-model-style prediction for VoIP codecs and loss conditions, and recommended it as the objective to optimise inside adaptive systems. SWARA takes this literally: the E-model is the objective of both the controller and the jitter buffer.

2.5 Channel models

Gilbert [16] and Elliott [17] introduced the two-state Markov model of a burst-noise channel; its packet-level form, with a good state that rarely drops and a bad state that almost always drops, is the standard model for Internet packet loss and is parameterised naturally by (loss rate, mean burst length). Its use for evaluating voice protection goes back at least to Bolot et al. [6] and Jiang and Schulzrinne [10].

2.6 Secure real-time transport and key establishment

SRTP [20] protects RTP media with an AEAD-style construction keyed out of band, reconstructing a 48-bit packet index from a 16-bit sequence number and a rollover counter and rejecting replays with a sliding window. DTLS-SRTP [21] derives SRTP keys from a DTLS handshake [22]; it is the WebRTC standard and the baseline against which SWARA's handshake is compared. DTLS introduced the stateless *cookie* exchange (HelloVerifyRequest) so that a server does not allocate state for a spoofed source, and QUIC [23] generalised it into the Retry mechanism, bounded amplification to a 3:1 ratio before address validation, and added connection identifiers that let a connection *migrate* to a new address without a new handshake. OPAQUE [24] is the asymmetric password-authenticated key exchange adopted by the IETF; its key idea is that the server stores an *envelope* — the client's long-term key sealed under a key stretched from the password — and never a password-equivalent, so a stolen database yields only an offline-attack target. SWARA composes these ideas with X25519 [25], HKDF [27], PBKDF2 [28] and ChaCha20-Poly1305 [26].

2.7 Network QoS and overlay routing

Differentiated Services [29] lets a sender mark each packet with a code point that routers map to a per-hop behaviour: Expedited Forwarding [30] for low-delay traffic and the Assured Forwarding classes [31] with their drop precedences. Combined with Random Early Detection [32] and its weighted variant, a congested router drops low-precedence packets first — a mechanism that can be made perceptually informed only if the marking reflects content. Andersen et al. [33] demonstrated with Resilient Overlay Networks that the Internet's routing frequently violates the triangle inequality at the application layer and that a small overlay measuring its own links can route around failures and

congestion within seconds. Distance-vector routing with split horizon and poisoned reverse [34] gives such an overlay a convergent algorithm. Signalling reliability over UDP is handled in SIP [35] by retransmitting INVITE with an identifier that the callee answers idempotently.

Table 1. Summary of the most relevant prior work and its relation to SWARA.

Work	Contribution	Limitation addressed by SWARA
Perkins et al. 1998 [5]	Taxonomy of audio loss recovery; FEC as the only interactive sender-side repair	Treats all frames as equal; no content awareness
Bolot et al. 1999 [6]	Loss-driven adaptive FEC with a Gilbert channel model	Redundancy chosen from network state alone
Rizzo 1997 [8]; Reed–Solomon 1960 [7]	Fast systematic MDS erasure codes over GF(2^8)	Block codes fail when bursts span the group; no time displacement
Mohr et al. 2000 [9]	Unequal loss protection by data importance (images)	Importance is structural, not estimated from speech content
Jiang & Schulzrinne 2002 [10]	Repair methods must be judged by perceived quality under bursty loss	No per-frame importance signal to act on
Ramjee et al. 1994 [13]; Liang et al. 2003 [14]	Adaptive playout against late-loss targets	Delay added by the buffer is not part of the objective
ITU-T G.107 [2]; Sun & Ifeachor 2006 [19]	E-model as an objective quality currency	Used for assessment, rarely as the controller's objective
SRTP / DTLS-SRTP [20], [21]	Standard media protection and keying	3 RTT, ~1.9 kB setup; no password authentication; no 0-RTT migration
QUIC [23]; DTLS cookies [22]	Stateless address validation, amplification limit, connection migration	Not applied to a password-authenticated media relay
OPAQUE [24]	Envelope-based asymmetric PAKE	Composition with a datagram handshake and media keys left open
DiffServ [29]–[31]; RED [32]	Per-packet marking and precedence-aware dropping	Marking is static per flow, not per-frame importance
RON 2001 [33]; RIP [34]	Application-layer overlays beat direct routes; distance-vector convergence	Link cost not expressed in voice-quality terms

3 Research Gap

Reading the literature of Chapter 2 as a whole exposes six gaps between what is known and what deployed voice stacks do. SWARA is organised around closing each of them.

Table 2. Research gaps and the SWARA component that addresses each.

#	Gap	Evidence from the survey	SWARA response
G1	Loss protection is content-blind: every frame receives the same redundancy	Adaptive FEC [6], [12], [47] adapts to loss rate only; unequal protection [9] was developed for images, not speech	PI analyser + PI-AFEC: parity by group importance, anchors for onsets
G2	Block FEC cannot survive bursts that span the group, and interleaving is too slow for conversation	Interleaving delay [5]; burst sensitivity of repair [10]	Burst mode: budget redirected to time-displaced replicas of the most important frames
G3	Playout depth and protection are decided separately although depth bounds what parity and replicas can rescue	Playout work [13], [14] targets late loss; FEC work assumes the buffer is deep enough	R-factor-optimal buffer with a protection floor; replica displacement bounded by depth
G4	Quality proxies (loss, delay, throughput) conflict; no single objective ties codec, redundancy and marking together	E-model recommended as objective [19] but used for assessment	NAVIK maximises predicted R over a codec × redundancy grid with a burst-aware residual-loss model
G5	Congestion loss and link loss are treated alike, so controllers back off when they should protect and protect when they should back off	AIMD reacts to any loss [43]; delay-gradient controllers exist for throughput, not for voice protection	Capacity estimate reacts to loss only when the one-way-delay trend rises
G6	Secure call setup is expensive, stateful and brittle: multi-RTT, large, password-less, no handover; naive UDP servers amplify	DTLS-SRTP cost [21]; cookie/Retry and migration exist in QUIC [23] but not in a password-authenticated media service	SPARK: 2-RTT PAKE with stateless cookie, amplification < 1, ticket-based 0-RTT migration; blind relay
G7	Direct Internet paths are often not the best paths for voice, and overlay costs ignore jitter and loss	RON [33] uses latency/loss heuristics; RIP [34] uses hop count	SETU: cost = owd + 2·jitter + 45·loss_dB, additive along paths; split horizon with poisoned reverse

A methodological gap also became apparent during the project and is recorded here because it shaped the evaluation: comparative measurements of protection schemes are easily contaminated by the measuring instrument. Stepping a loss model once per packet gives a scheme that sends more packets a different channel from the one it is being compared with; a scheduler thread that busy-waits distorts every timing the controller observes. Section 5.10 and Section 7.9 describe how both artefacts were found and removed.

4 Objectives

The project's objectives follow directly from the gaps of Chapter 3. Each is stated with the measurable criterion used to judge it in Chapter 7.

Table 3. Objectives and success criteria.

#	Objective	Success criterion
O1	Score every 20 ms frame by its perceptual importance, causally, and carry the score in the packet header where the network can read but not forge it	Silence, unvoiced, voiced and onset frames occupy distinct score bands; silence holds < 2 % of total importance; score is AEAD associated data
O2	Spend a fixed redundancy budget by importance (PI-AFEC) and protect onsets with deadline-bounded anchor replicas	At equal budget, lower perceptually weighted loss and higher predicted MOS than uniform adaptive FEC across a loss sweep; equal quality for less redundancy
O3	Survive bursty loss where block coding fails	Advantage over uniform FEC persists as mean burst length rises; anchors account for recoveries when FEC recoveries collapse
O4	Choose codec bitrate, redundancy and DSCP marking jointly by maximising the E-model rating, discriminating congestion from link loss	Controller never offers more than the capacity estimate; redundancy is held on lossy-but-uncongested paths; decision cost well below the 100 ms epoch
O5	Size the playout buffer to maximise R and couple it to the protection decision	The depth chosen by the optimiser coincides with the depth that maximises measured end-to-end MOS
O6	Authenticate users mutually with a password that never crosses the wire, in two round trips, with amplification below one and zero-round-trip migration	397-byte, 2-RTT handshake; 20-byte RETRY to a 64-byte HELLO; migration in 0 RTT; the relay holds no media keys
O7	Select relay paths that beat the direct route when the triangle inequality fails	Overlay converges in a handful of rounds; a measurable share of peer pairs gains from relaying
O8	Deliver a usable application and a reproducible evaluation	Real calls between command-line clients through the relay; a web console that runs instrumented calls, benchmarks and tools; every figure regenerable from one script

5 Proposed Work

SWARA is a layered design in which one idea — *judge loss by what the listener loses* — propagates through every layer. This chapter presents the system top-down: the end-to-end architecture, the wire protocol, and then each component with its algorithm and the equations it implements. The names of the components are Hindi/Sanskrit words chosen to be memorable: *swara* (voice, musical note), *navik* (navigator), *setu* (bridge), *spark* (an acronym).

5.1 System architecture

Figure 2 shows one direction of one call. On the sender, 20 ms frames are captured, coded with a per-frame-independent ADPCM codec, scored by the perceptual importance analyser, grouped for Reed–Solomon parity and anchor replicas by the PI-AFEC scheduler, sealed with an AEAD cipher whose associated data is the header, marked with a DiffServ code point derived from the frame's importance, and sent on a plain UDP socket. The relay authenticates users, sets up calls and forwards ciphertext by connection identifier; it never holds a media key. On the receiver, datagrams are authenticated, de-duplicated with a replay window, reassembled — rebuilding missing frames from parity or replicas — scheduled by an R-factor-optimal playout buffer, decoded, concealed where nothing could be rebuilt, and played. A 16-byte telemetry report rides in the header of the reverse-direction media stream, closing the control loop without a single extra packet.

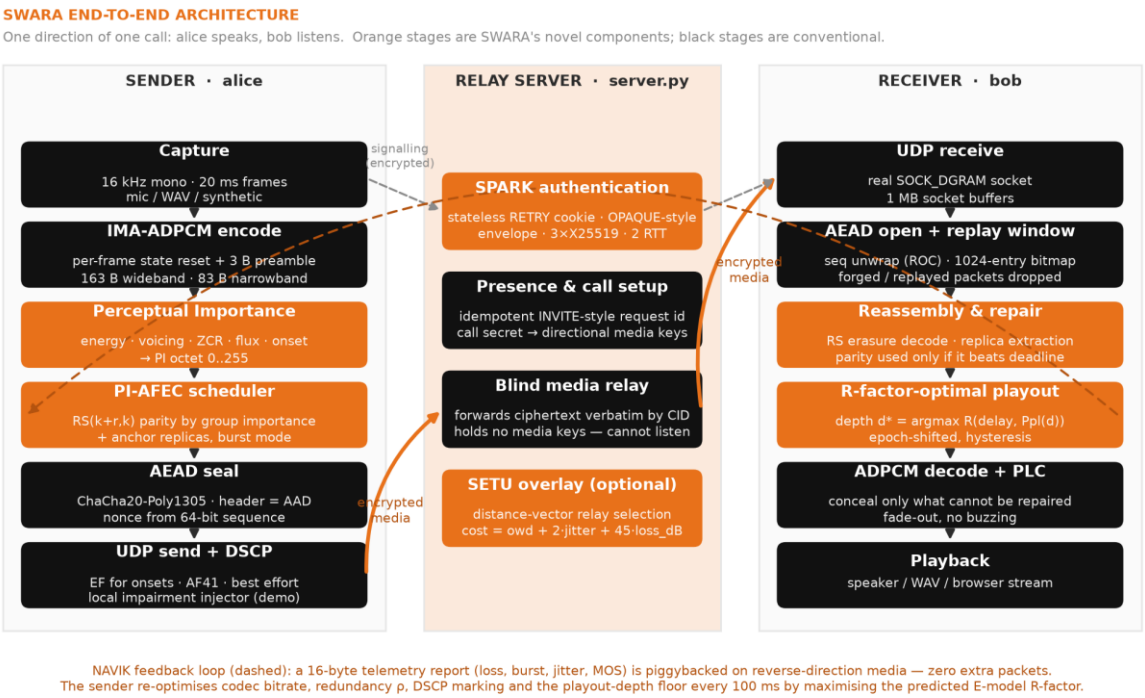


Figure 2. SWARA end-to-end architecture for one direction of a call. Orange stages are the project's novel components; black stages are conventional. The relay in the middle forwards ciphertext it cannot read.

5.2 Wire protocol

Every SWARA datagram is a single UDP payload of at most 1,200 bytes — below the smallest plausible path MTU, so no IP fragmentation ever occurs. There is no stream, no retransmission by default and

no head-of-line blocking: a lost packet is a lost 20 ms of audio and nothing more. All datagrams begin with the 16-byte fixed header of Figure 3, packed in network byte order as struct "!BBHIIBBBB".

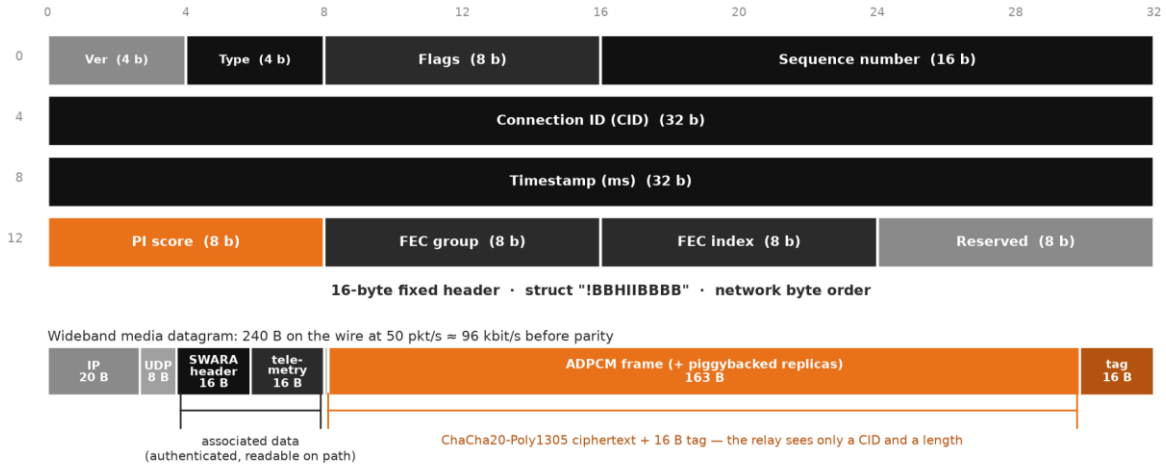


Figure 3. The 16-byte SWARA header and the composition of a wideband media datagram. The header and the optional 16-byte telemetry block are AEAD associated data: readable on the path, unforgeable. Everything after them is ciphertext.

Table 4. Header fields.

Field	Bits	Meaning
Version / Type	4 + 4	Protocol version (1) and datagram type (Table 5)
Flags	8	TELEMETRY_PIGGYBACK 0x01, MARKER 0x02 (first frame of a talkspurt), DSCP_EF 0x04, ENCRYPTED 0x08, LAST_IN_GROUP 0x10, RESUMED 0x20
Sequence number	16	Wire sequence; the receiver unwraps it to a 64-bit counter with an SRTP-style rollover counter, and that counter is the AEAD nonce input
Connection ID	32	Derived from the handshake (sessions) or minted by the relay (calls); survives NAT rebinding and path migration; the relay routes on it
Timestamp	32	Sender clock in milliseconds; feeds the RFC 3550 jitter estimate and the one-way-delay trend
PI score	8	Perceptual importance 0–255 — authenticated but not encrypted, so relays and DiffServ routers can act on it
FEC group / FEC index	8 + 8	Group identifier and position within the group; bit 7 of the index marks a parity shard
Reserved	8	Zero

Table 5. Datagram types.

Type	Code	Direction	Purpose
HELLO	0	client → server	Stateless probe, padded to 64 bytes
RETRY	1	server → client	20-byte address-validation cookie
AUTH_INIT	2	client → server	Cookie + client ephemeral public key + username
AUTH_RESP	3	server → client	Salt + credential envelope + server ephemeral and static keys

Type	Code	Direction	Purpose
AUTH_CONFIRM	4	client → server	Key-confirmation tag
AUTH_TICKET	5	server → client	Connection ID + encrypted resumption ticket
MEDIA	6	peer ↔ peer	One coded speech frame (plus piggybacked replicas)
FEC	7	peer ↔ peer	One Reed–Solomon parity shard
ANCHOR / TELEMETRY / NACK	8–10	peer ↔ peer	Reserved for standalone replicas, standalone telemetry and selective NACK (replicas and telemetry are normally piggybacked)
PATH_MIGRATE / PATH_ACK	11–12	client ↔ server	0-RTT authenticated path migration and its validation
OVERLAY_LSA	13	peer ↔ peer	SETU distance-vector advertisement
BYE	14	any	Teardown
CONTROL	15	client ↔ server	Encrypted JSON signalling: call, incoming, call_ready, call_failed, hangup, ping/pong, who, bye

A **media** datagram's plaintext begins with a one-byte replica count followed by that many `{displacement (1), length (2), payload}` records and then the primary frame. Displacement rather than an absolute sequence number is used because a displacement survives 16-bit wrap. A **parity** datagram carries `{base sequence (2), k (1), r (1), shard length (2)}` followed by the shard. The **telemetry** block packs loss (×10^4), mean burst (×100), jitter (×100 ms) and MOS (×1000) into four 16-bit fields with 8 reserved bytes. For the syllabus's error-control topic, `packet.py` also implements CRC-32 (the Ethernet FCS polynomial), the RFC 1071 Internet checksum that UDP itself carries [38], two-dimensional parity and Hamming(7,4) [39]; the design discussion in Section 5.4 explains why a real-time voice transport uses a detection code plus erasure coding rather than a bit-level correcting code.

5.3 Perceptual importance analysis

The PI analyser converts each 320-sample frame (20 ms at 16 kHz) into a single octet estimating how much intelligibility the listener would lose if that frame were missing. It is strictly causal — every feature uses only the current frame and state accumulated from past frames — because a real encoder cannot look ahead without adding delay. Five features are computed in O(N) or O(N log N):

- **Energy** E in dB, normalised to $E' \in [0, 1]$ against an adaptive noise floor (fast to fall, slow to rise) and an adaptive peak (fast attack, slow release), with a minimum span of 12 dB.
- **Voicing** V: the normalised autocorrelation peak in the 60–400 Hz pitch-lag range, computed by FFT, then rescaled against its measured operating range, V = clip((V_raw − 0.12)/0.45).
- **Zero-crossing rate** ZCR: the classic voiced/unvoiced discriminator; a high ZCR with low voicing identifies fricative noise.
- **Spectral flux** F: the half-wave-rectified difference between successive Hann-windowed magnitude spectra — energy that has just *appeared*, the signature of an onset.
- **Voice activity**: a frame is speech if it is more than 6 dB above the noise floor, or more than 4 dB above it with F > 0.62 (which admits 8 ms plosive bursts whose average energy is low).

$$PI_raw = 0.30 \cdot E' + 0.34 \cdot V_eff + 0.20 \cdot F + 0.16 \cdot onset, \quad V_eff = 0.6 \cdot V \text{ if } (V < 0.4 \text{ and } ZCR > 0.20) \text{ else } V \tag{1}$$

$$PI = 255 \cdot \min(1, boost \cdot (0.12 + 0.88 \cdot clip(PI_raw, 0, 1))) \text{ for speech; } PI = 255 \cdot 0.06 \cdot E' \text{ otherwise} \tag{2}$$

The onset term is a rising-edge detector, onset = clip(3.2 · (E′ − E′_prev)); a *hard* onset — the first speech frame after silence — forces onset = 1 and applies a multiplicative boost of 1.45, because losing the frame that begins a talkspurt destroys the word-boundary cue. The design targets, verified on the synthetic corpus (Section 7.2), are silence 0–25, steady fricatives 50–110, steady vowels 150–210 and onsets 210–255. The score's companion is the **damage weight** that turns a loss into a perceptual cost:

$$d(PI, run) = (PI / 255)^{1.6} \cdot (1 + 0.45 \cdot \min(run, 4)) \tag{3}$$

The curve is superlinear — a frame twice as important is roughly three times as costly to lose — and consecutive losses are penalised because concealment degrades sharply once it has to invent more than about 40 ms of speech. Equation (3) is used in three places with the same constants: to score lost frames in the evaluation, to weight late packets in the jitter buffer, and to convert accumulated damage into the *perceptual* loss percentage fed to the E-model, Ppl_perc = 100 · Σ d / N. Using one function everywhere means the optimiser and the evaluator agree by construction.

5.4 PI-AFEC: importance-weighted adaptive FEC

Erasure code. Groups of k media frames are extended with r parity shards by a systematic Reed–Solomon code over GF(2^8) with primitive polynomial 0x11D (the field of CD-ROM, QR codes and RAID-6). Multiplication is a 256 × 256 table so encoding is a sequence of vectorised XORs; the generator rows are Cauchy-style, m[i, j] = $(a_i \oplus b_j)^{-1}$ with distinct a_i = i + 1 and b_j = 128 + j, normalised so that the r = 1 row reduces to plain XOR. Any square sub-matrix is invertible, which is the MDS property: any k of the k + r shards reconstruct the group. Decoding solves the small system for the missing data shards by Gauss–Jordan elimination over the field. The module verifies the MDS property at import time.

Why erasure coding rather than Hamming. A UDP datagram either arrives intact or not at all: the IP and UDP checksums and the AEAD tag discard corrupted packets, so the channel seen by the application is an *erasure* channel in units of whole frames. A bit-level correcting code such as Hamming(7,4) spends 75 % overhead correcting single-bit errors that the link layer has already removed; a packet-level erasure code spends its overhead on the failure mode that actually occurs.

Scheduler. For each group the scheduler computes the mean normalised importance $\bar{G}$ = mean(PI)/255 and a weight w = $\bar{G}^{\gamma}$ with γ = 0.65 (so moderately important groups are not starved). The weight is normalised by its own running mean $\bar{w}$ so that the *expected* spend equals exactly the budget k·ρ handed down by the controller — the scheme redistributes the budget, it does not quietly discount it — and the group's budget in packets is

$$budget = k \cdot \rho \cdot clip(w / \bar{w}, 0, 4), \quad with \ budget = 0 \ whenever \ \bar{G} < 0.08 \ (a \ silent \ group) \tag{4}$$

The budget is split between parity and anchor replicas according to the estimated mean burst length B. An RS(k + r, k) group survives only if at least k of its k + r packets arrive; when the burst approaches the group's own span that condition fails almost every time the channel goes bad and every parity byte is wasted. Classical coding theory answers with interleaving, but an interleaver of depth D over k

frames costs k·D frame times of delay — 480 ms for D = 3, k = 8 — and destroys interactivity. SWARA instead interleaves *only what matters*: in **burst mode** (B ≥ 0.75 k) the shares are 45 % parity / 55 % replicas, otherwise 85 % / 15 %. Fractional budgets are carried between groups rather than rounded away (with k = 4 a per-group budget of 0.4 packets would otherwise round to zero every time). Up to four replicas go to the highest-PI frames of the group, provided PI ≥ 170, each displaced by

$$D = clip(round(B) + 2, 3, 9) + 2 \cdot slot \quad frames, \quad bounded\ in\ the\ live\ system\ by\ (depth - spread_{95} - 20\ ms) / 20\ ms \tag{5}$$

so that the copy clears the burst that killed the original *and* still beats the original's own playout instant after suffering the same delay spread as everything else. A replica is piggybacked onto an existing media packet — it costs only its payload bytes, not a second set of IP, UDP and SWARA headers. The uniform comparator used throughout the evaluation is the same code with r = round(k·ρ) for every group (probabilistically honouring fractions): a strong, correctly adapting baseline that simply cannot tell a vowel from a silence.

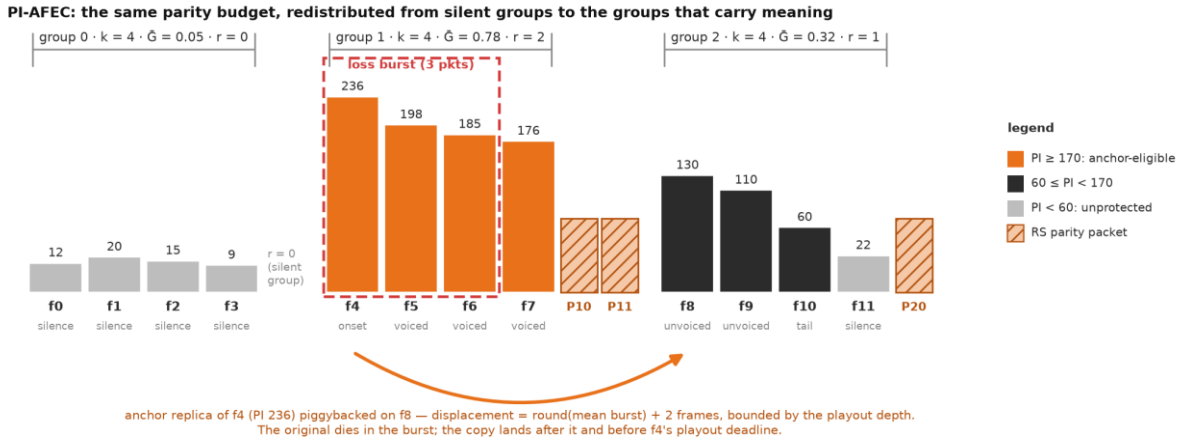


Figure 4. PI-AFEC on three consecutive groups of k = 4 frames. The silent group receives no parity; its budget moves to the group carrying the vowel onset, which additionally earns an anchor replica displaced past the expected burst length.

5.5 SPARK: stateless password-authenticated key agreement

The authentication problem for voice over UDP is not “how do I check a password”. It is: (a) UDP is trivially spoofable, so an unauthenticated first packet must never make the server allocate memory or emit a larger reply, or the service becomes a DDoS amplifier; (b) a password must never cross the wire, and a stolen server database must not yield anything replayable; (c) the handshake must not cost so many round trips that call setup feels slow — and after a Wi-Fi-to-mobile handover it must cost none, because the user is mid-sentence. SPARK addresses all three in the six messages of Figure 5.

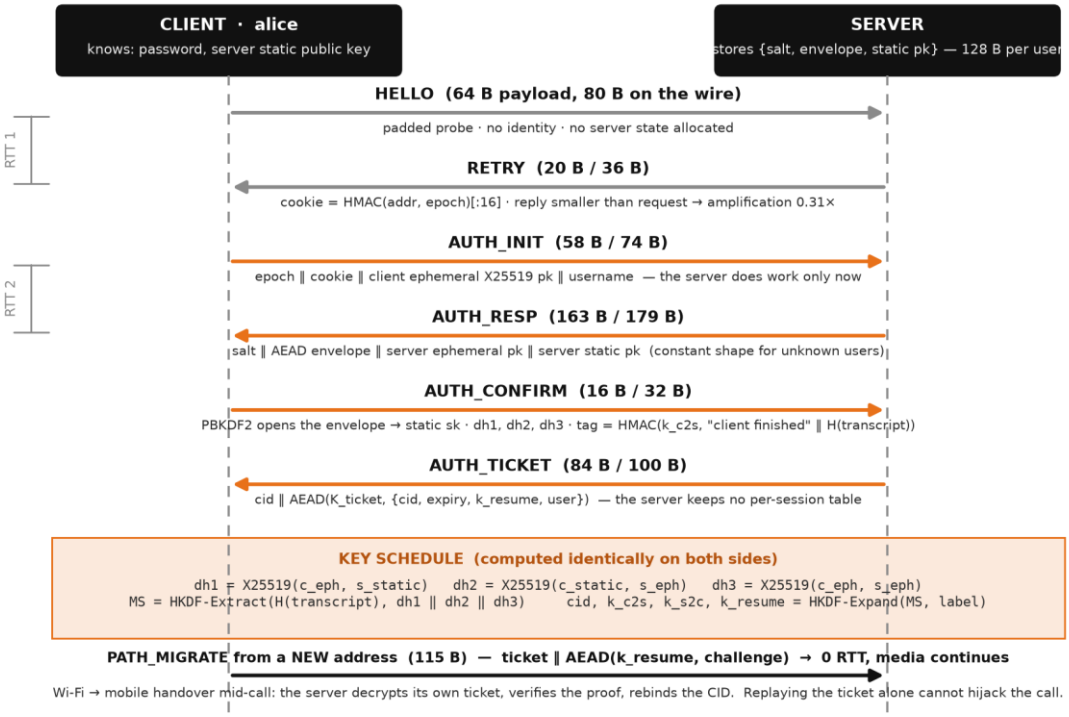


Figure 5. The SPARK handshake and 0-RTT path migration, with on-the-wire sizes measured from the implementation.

1. **Registration (once, out of band).** The password is stretched with PBKDF2-HMAC-SHA256 (200,000 iterations, 16-byte salt) [28] into a key that seals an *envelope*: the client's static X25519 private and public keys under ChaCha20-Poly1305 with the username as associated data. The server stores only {salt, envelope, client static public key} — 128 bytes and nothing resembling a password or a verifier [24].
2. **HELLO → RETRY.** The client sends a 64-byte padded probe revealing no identity. The server replies with a 20-byte cookie, HMAC(cookie_key, ip ‖ port ‖ epoch)[:16] with a 30-second epoch, allocating no state. Because the reply is smaller than the request the amplification factor is 20/64 = 0.31 [22], [23].
3. **AUTH_INIT.** The client echoes the cookie — proving it can receive at the address it claims — with an ephemeral X25519 public key and its username. Only now does the server do work. For an unknown user it answers with a random envelope of identical shape, so the protocol is not a user-enumeration oracle.
4. **AUTH_RESP.** Salt, envelope, a fresh server ephemeral public key and the server's static public key.
5. **AUTH_CONFIRM.** The client re-derives the stretched key, opens the envelope (a wrong password fails *locally* and nothing is sent), and computes three Diffie–Hellman shares: dh1 = X25519(c_eph, s_static) authenticates the server, dh2 = X25519(c_static, s_eph) authenticates the client, dh3 = X25519(c_eph, s_eph) gives forward secrecy. With th = SHA-256 over every length-prefixed handshake byte, MS = HKDF-Extract(th, dh1 ‖ dh2 ‖ dh3) and the connection id and the keys k_c2s, k_s2c, k_resume are HKDF-Expand(MS, label) [25], [27]. The client proves key knowledge with tag = HMAC(k_c2s, “client finished” ‖ th)[:16].

6. **AUTH_TICKET.** The server verifies the tag, binds the CID to the client's address, and issues ticket = nonce ‖ AEAD(K_ticket, cid ‖ expiry ‖ k_resume ‖ user). It keeps no per-session table for resumption.
7. **Path migration (0 RTT).** From a new address the client sends ticket ‖ AEAD(k_resume, challenge). The server decrypts its own ticket, verifies the proof — an off-path attacker replaying the ticket alone cannot forge it — and rebinds the CID. Media continues without a round trip of silence.

Media protection. Each call receives a fresh 32-byte secret minted by the relay, from which two *directional* keys are derived; each party receives both keys inside a CONTROL message sealed under its own session key. Media packets are sealed with ChaCha20-Poly1305 [26]; the nonce is derived from (direction, 64-bit sequence) so none is transmitted — 12 bytes saved per packet, 4.8 kbit/s per stream. The 16-bit wire sequence is unwrapped with an SRTP-style rollover counter (otherwise every packet would fail authentication after 21.8 minutes), and a 1,024-entry sliding bitmap rejects replays [20]. Because the relay forwards datagrams verbatim and holds neither key, it cannot listen to the call it carries.

5.6 NAVIK: joint quality control

The sender has three dials and one budget: codec bitrate b (higher b lowers the codec impairment Ie), redundancy ρ (higher ρ lowers residual loss), DSCP marking, and the available capacity C with b(1 + ρ) + H ≤ C. Conventional stacks resolve this with a cascade of independent heuristics that can fight each other — the classic pathology is FEC added in response to congestion loss, which increases the offered load and deepens the congestion. NAVIK replaces the cascade with one objective, evaluated every 100 ms over a grid of seven nominal codec rates and nine redundancy levels (63 evaluations, tens of microseconds):

$$\text{maximise } R(\text{codec}(b),\ \text{owd} + \text{depth} + 25\ \text{ms},\ \text{Ppl_res}(\rho, p, B),\ B) \quad \text{s.t. } b(1+\rho) + H \le 0.85 \cdot C_est,\ \rho \le \rho_max(p) \tag{6}$$

Because R already contains the delay term, the congestion pathology cannot occur: redundancy that would push the offered rate past the estimate raises queueing delay, lowers R, and is refused. The safety rail ρ_max(p) = 0.65 for p < 3 % and max(0.15, 0.65 − 1.8 (p − 0.03)) above tapers redundancy as loss rises, since very high loss usually *is* congestion. H = 24 kbit/s is the header overhead of 50 packets per second of IP + UDP + SWARA + AEAD tag.

Residual-loss model. The probability that an RS(k + r, k) group cannot be decoded is the probability of more than r erasures among k + r packets. Under independent loss this is a binomial tail; under clustered loss the erasure count is over-dispersed, so NAVIK uses a beta-binomial with intra-class correlation ρ_c = (B − 1)/(B + 1). The difference matters: at 10 % loss with mean burst 4 the binomial predicts a group-failure rate about six times lower than reality, and a controller that believed it would under-provision exactly when protection is most needed. Anchor replicas are modelled independently of the block code — they survive unless an independent loss event hits them — because treating them as a modifier on RS made the controller believe protection was futile in exactly the burst-dominated conditions where the anchor is the only thing that still works.

Congestion versus loss. Naive AIMD backs off on any loss [43] — right on a wired path, wrong on a wireless one, where backing off starves the very redundancy the loss made necessary. Congestion has a signature random loss lacks: the queue fills before it overflows, so one-way delay *rises* first. NAVIK

fits a line to the last 100 one-way-delay samples and treats loss as congestion only when the slope exceeds 8 ms/s, shrinking the capacity estimate by 12 % per epoch; loss with a flat trend is read as link loss and the rate is held (a gentle 0.5 % probe down), while a clean path lets the estimate track 1.25 × goodput.

DiffServ marking. The decision also fixes per-packet marking driven by importance: PI ≥ 160 → EF (46), 80 ≤ PI < 160 → AF41 (34), otherwise best effort [29]–[31]. A router applying WRED [32] to a congested queue drops from the lowest precedence first, so the network's own drop decision becomes perceptually informed without the router understanding — or being able to read — the speech.

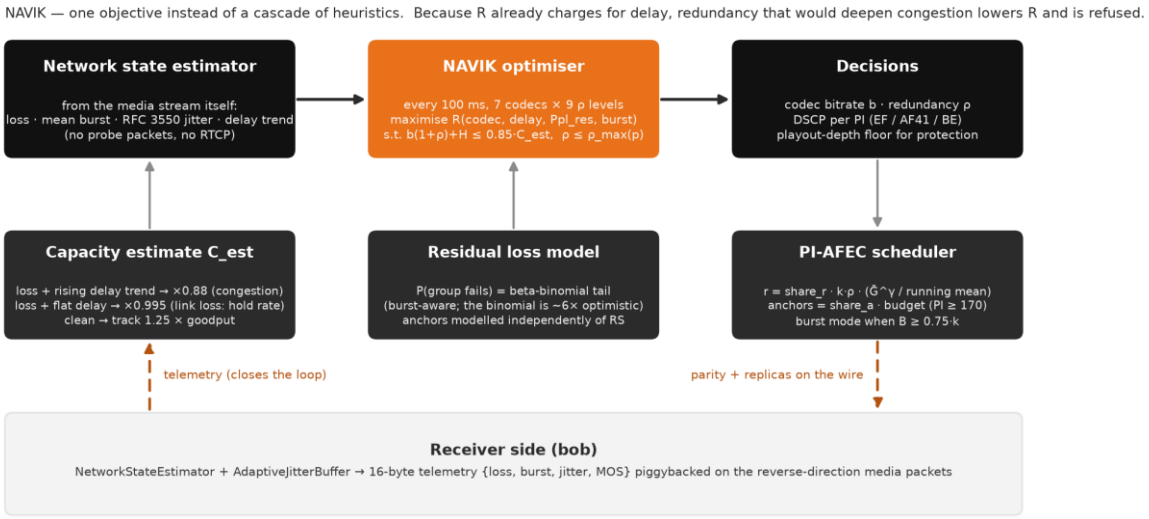


Figure 6. The NAVIK control loop: receiver-side estimation feeds a single R-factor objective whose decisions drive the scheduler, the codec and the packet marking.

5.7 Receiver: estimation, repair and R-factor-optimal playout

All network statistics are estimated from the media stream itself — no probe packets, no separate RTCP reports. The estimator tracks loss as 1 − seen/expected over the highest sequence number, mean burst as the mean length of the last 80 gaps, interarrival jitter with the RFC 3550 recursion J ← J + (|D| − J)/16 [1], the delay spread as the 95th minus 5th percentile of relative one-way delay, reordering and goodput.

A conventional adaptive jitter buffer targets a loss objective — “size the buffer so that at most 1 % of packets arrive late” — which treats the delay it adds as free. It is not: the E-model charges 0.024 R-points per millisecond and 0.134 per millisecond beyond 177.3 ms. SWARA chooses the depth that maximises the predicted rating directly,

$$d^* = \operatorname{argmax}_d \; R(\text{codec}, \text{other_delay} + d,\ 100 \cdot \text{late}(d) \cdot (0.25 + 1.5 \cdot I_{\text{late}}),\ B) \quad \text{over } d \in \{0, 10, \ldots, 180\}\ \text{ms} \tag{7}$$

where late(d) is the fraction of packets whose relative delay exceeds d and I_{late} is the mean damage weight of the packets currently arriving late. The optimum shifts deeper when late packets are important and shallower when they are silence. Depth changes are smoothed (hysteresis 0.35) because they are audible, and in the live system the depth is floored at the shallowest value at which the protection being paid for can work — (k + 1) frames for parity, (B + 2) frames plus the delay spread

for replicas — on the principle that buying redundancy the buffer is too shallow to use is pure waste. Depth is changed on a running call by shifting the playout epoch: deepening is applied at once, shallowing at most 10 ms per epoch. At each 20 ms slot the playout thread takes the frame in sequence; if it is missing it attempts a Reed–Solomon reconstruction of its group (and the previous group, whose parity may have just arrived); if nothing can be rebuilt it conceals by repeating the previous frame with a gain of 1 − 0.35·run and tapered edges, fading to silence rather than buzzing.

5.8 SETU: voice-aware overlay routing

BGP picks paths on policy and AS-hop count, not on latency, jitter or loss, and the triangle inequality is frequently violated at the application layer: A → C → B can be measurably better than A → B [33]. SETU is a small application-layer overlay among call participants and relays that runs a distance-vector computation — Bellman–Ford with split horizon and poisoned reverse, the algorithm family of RIP [34] — over a composite link cost that expresses what voice cares about in one number:

$$cost(u, v) = 1.0 \cdot owd_ms + 2.0 \cdot jitter_ms + 45 \cdot (-10 \cdot \log_{10}(1 - p)), \qquad relay\ hop\ penalty = 12 \tag{8}$$

Expressing loss in decibels of survival probability is what makes the cost valid for shortest-path routing: losses then *add* along a path exactly as delays do (a 1 % link and a 2 % link compose to 2.97 %, and −10 log(0.99) − 10 log(0.98) = −10 log(0.9702) exactly). Jitter is weighted above raw delay because it forces a deeper playout buffer. Count-to-infinity is prevented by poisoned reverse and a cost ceiling; with tens of nodes convergence takes a handful of rounds.

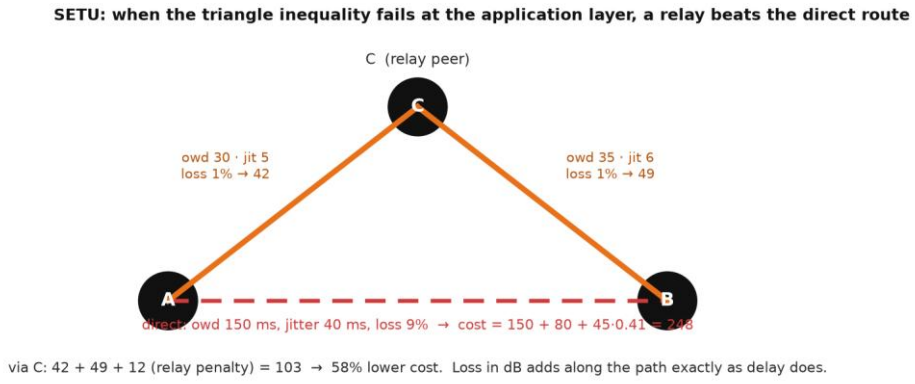


Figure 7. A triangle-inequality violation: the direct A–B path crosses a congested exchange while the two-hop path through C is far cheaper in voice-cost terms.

5.9 Speech codec with per-frame state reset

SWARA's codec is a real IMA ADPCM [36], [37]: 4 bits per sample, 4 : 1 compression of 16-bit PCM, with one deliberate modification. Standard ADPCM carries its predictor and step index continuously from block to block; over a lossy transport that is fatal, because one lost packet makes the decoder's predictor diverge from the encoder's and every subsequent frame decodes to noise until something resynchronises. SWARA resets the state at every frame boundary and transmits the starting predictor and step index in a 3-byte preamble, so every frame is independently decodable: packet loss damages exactly the frames that were lost and nothing else. The cost is a small loss of coding efficiency; the benefit is the property on which every repair mechanism above depends. Two modes give the

controller something real to switch between: wideband (16 kHz, 320 samples, 163 bytes per frame, 65.2 kbit/s) and narrowband (8 kHz via an 8-tap anti-alias FIR, 83 bytes, 33.2 kbit/s). *A codec designed for a file is the wrong codec for a network.*

5.10 Evaluation methodology

Channel model. Both the offline simulator and the live impairment injector use the Gilbert–Elliott chain of Figure 8, parameterised as an operator would — "10 % average loss with a mean burst of 4 packets" — and solved as p_BG = 1/B, p_GB = p_BG · p/(1 − p). The simulator adds a base one-way delay, a heavy-tailed log-normal jitter component (which matches wide-area delay distributions far better than a Gaussian and is what makes buffer sizing genuinely hard), a token-bucket bottleneck queue with tail drop beyond 250 ms of queueing, and optional reordering and duplication. Seven named profiles span an ideal LAN, good and busy Wi-Fi, congested 4G, poor 3G, a hostel Wi-Fi network and a satellite link.

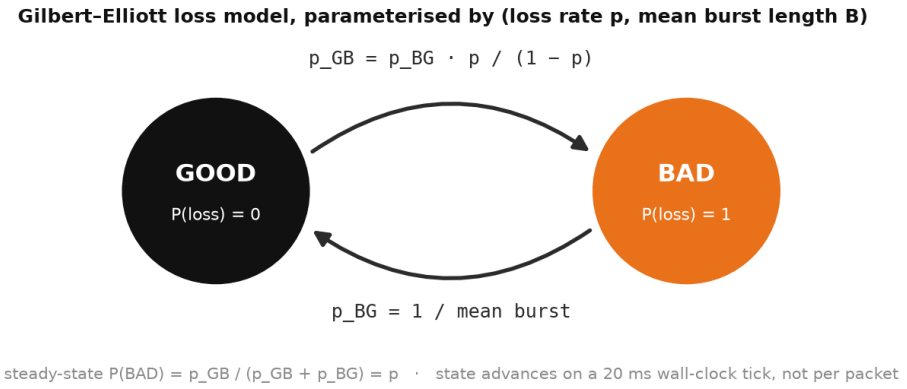


Figure 8. The Gilbert–Elliott loss model used by both the simulator and the live injector.

Time-based impairment. The live injector advances the chain on a 20 ms *wall-clock* tick, not once per packet. Physically a fade lasts a duration, not a number of packets; methodologically, stepping per packet means a configuration that transmits redundancy walks the chain further per unit time and therefore experiences a different channel from the one it is compared with — the project hit exactly that, with three arms configured for 15 % loss measuring 9.5 %, 12.3 % and 12.5 %. The injector also keeps two independent random streams (loss chain and jitter) and is reset to the same seed for every arm, so every arm faces the identical sequence of good and bad periods.

Comparators. Every result compares three systems on identical speech, identical channel realisation and — crucially — identical redundancy budget: the **baseline** (plain UDP, no FEC, a conventional buffer at the 95th percentile of delay plus 10 ms), **uniform FEC** (loss-driven adaptive FEC given exactly the redundancy SWARA spent, spread evenly over all groups — RFC 5109 / FlexFEC behaviour), and **SWARA**. Any gain over the second comparator is attributable to perceptual placement, not to "SWARA has FEC". Quality is scored with the G.107 E-model using the perceptually weighted residual loss and the measured burst ratio; the corpus is a source-filter speech synthesiser (glottal pulse train or noise through formant resonators, talkspurts of 0.9–2.8 s with realistic phone durations), seeded so that every run is reproducible.

6 Implementation

6.1 Technology stack

Table 6. Technology stack and versions used for the reported results.

Layer	Technology	Role
Language / runtime	Python 3.14.3 (CPython), Windows 11, 20 logical CPUs	Protocol stack, relay, clients, simulator, web back end, launcher
Numerics	NumPy 2.5.3	Vectorised GF(2^8) arithmetic, FFT-based voicing and flux, estimators
Cryptography	pyca/cryptography 50.0.1	X25519, HKDF-SHA256, ChaCha20-Poly1305; PBKDF2 and HMAC from the standard library
Transport	socket (AF_INET, SOCK_DGRAM), IP_TOS	Real UDP datagrams with DSCP marking and 1 MB socket buffers
Web API	FastAPI 0.141.1, Starlette 1.6.0, Pydantic 2.13.5, Uvicorn 0.52.4	REST + Server-Sent Events console back end, OpenAPI documentation
Front end	Vanilla JavaScript (ES2020), Chart.js 4.4.1 (vendored), Web Audio API	Single-page console: live charts, frame-fate map, in-browser audio playback and recording
Figures	Matplotlib 3.11.1	Research figures and report diagrams
Audio I/O	sounddevice 0.5.6 (optional)	Microphone and speaker backends; WAV and synthetic backends need nothing
Desktop launcher	Tkinter	Start / stop / restart the server and relay without a terminal

6.2 Repository layout

```
SWARA/
├── server.py              registration, SPARK authentication, presence, call setup, blind media
relay
├── client.py              the voice-chat application: login, call placement, capture, playback
├── demo.py                self-contained end-to-end demonstration and A/B harness
├── api_server.py          FastAPI web console + REST API, hosts the relay inside uvicorn
├── launcher.py            Tkinter "SWARA Server Control" window
├── swara/                 the protocol stack (15 modules, ~4,000 lines)
│   ├── packet.py          wire format, header pack/unpack, CRC-32, checksum, 2-D parity,
Hamming(7,4)
│   ├── perceptual.py      PI analyser and damage weight
│   ├── fec.py             GF(2^8) Reed–Solomon, PI-AFEC scheduler, uniform comparator
│   ├── auth.py            SPARK client/server, key schedule, media AEAD, ROC, replay window
│   ├── controller.py      NAVIK joint optimiser, residual-loss model, DSCP mapping
│   ├── receiver.py        network-state estimator, R-factor-optimal jitter buffer
│   ├── session.py         live call pipeline: four threads per call end
│   ├── codec.py           IMA ADPCM with per-frame reset, concealment
│   ├── net.py             UDP endpoint, DSCP, time-based Gilbert–Elliott impairment injector
│   ├── emodel.py          ITU-T G.107 E-model, perceptual Ppl
│   ├── overlay.py         SETU distance-vector overlay
│   ├── channel.py         Gilbert–Elliott channel for the offline evaluation, named profiles
│   ├── engine.py          offline harness: SWARA vs uniform FEC vs baseline
│   ├── speech.py          source-filter synthetic speech corpus
│   └── audio_io.py        microphone / speaker, WAV and synthetic sources and sinks
├── webapp/                console back end: lab.py (instrumented calls), relay.py, tools.py
├── static/                console front end: index.html, css/app.css, js/core.js, js/pages/*.js,
vendor/chart.umd.min.js
├── experiments/run_all.py the eight evaluation experiments → results.json + figures/
├── figures/               fig1 … fig8 (regenerated by run_all.py)
├── users.json             credential records {salt, envelope, public key} — no passwords
└── SWARA Server.bat / start_console.bat   one-click launchers
```

6.3 Module responsibilities

Table 7. Source modules and their responsibilities.

Module	Lines	Responsibility
swara/packet.py	207	16-byte header, datagram types and flags, media/parity layouts, integrity primitives
swara/perceptual.py	240	Causal PI analyser (energy, voicing, ZCR, flux, onset, VAD) and damage weight
swara/fec.py	398	Table-driven GF(2^8), systematic Reed–Solomon encode/decode, PI-AFEC and uniform schedulers
swara/auth.py	475	Registration, SPARK handshake (server and client), tickets, migration, MediaCipher, SequenceReconstructor, ReplayWindow
swara/controller.py	281	NAVIK: capacity estimate, beta-binomial group-failure model, grid search over codec × ρ, DSCP mapping
swara/receiver.py	233	NetworkStateEstimator, AdaptiveJitterBuffer (R-optimal), GroupBuffer
swara/session.py	607	CallSession: sender, receiver, playout and control threads; replica piggybacking; epoch-based playout; telemetry
swara/codec.py	199	IMA ADPCM encode/decode with per-frame preamble, narrowband resampling, concealment
swara/net.py	240	UdpEndpoint with counters and DSCP; Impairment injector (wall-clock Gilbert–Elliott, delay scheduler)
swara/emodel.py	171	R-factor, Id, Ie,eff with burst ratio, R→MOS, perceptual Ppl, quality labels
swara/overlay.py	164	SETU: link cost, distance-vector node, split horizon with poisoned reverse, convergence, path report
swara/channel.py	193	ChannelConfig, GilbertElliottChannel with queueing/reordering/duplication, named profiles

Module	Lines	Responsibility
swara/engine.py	342	CallSimulator (pilot pass, planning, transmission, scoring) and the three-way compare()
swara/speech.py	156	SpeechGenerator: vowels, fricatives, plosives, silences; per-frame ground truth
swara/audio_io.py	178	AudioSource / AudioSink backends: live, WAV, synthetic, null
server.py	246	Relay: user store, SPARK phases, presence, idempotent call setup, verbatim media relay
client.py	266	Client: retransmitting handshake, INVITE-style call placement, keepalives, run_call
demo.py	229	One-process end-to-end demonstration; A/B harness with identical channel realisations
experiments/run_all.py	511	Eight experiments, results.json and all figures
api_server.py + webapp/	1,284	REST API, SSE streaming, instrumented TracedSession, Benchmark, hosted relay, toolkit wrappers
static/	~3,000	Console front end (14 JavaScript modules, stylesheet, HTML shell)
launcher.py	482	Desktop server-control window

6.4 The live call pipeline

CallSession owns one end of one call: the codec, the protection state and the two directional keys. It starts four daemon threads (Figure 9). The **sender** reads a frame, encodes it, scores it, prepends any replicas queued for this slot, seals the packet with the header and telemetry as associated data, marks the socket's DSCP from the frame's importance and sends; every k frames it closes the group, asks the scheduler for a plan, queues the chosen replicas at their displacements and emits the parity shards. It is paced against a monotonic clock so that scheduling error does not accumulate into drift. The **receiver** authenticates each datagram, unwraps the sequence number, rejects replays, stores media payloads and parity shards, extracts piggybacked replicas and updates the estimator. The **playout** thread fills to the target depth, then runs an absolute schedule anchored at an epoch — this is what allows the depth to be changed on a running call — and for each slot plays, repairs or conceals. The **control** thread, every 100 ms, snapshots the estimator, updates the capacity estimate from the delay trend, asks NAVIK for a decision, hands ρ and the burst estimate to the scheduler, optimises the buffer depth, floors it at the protection requirement and retargets the epoch.

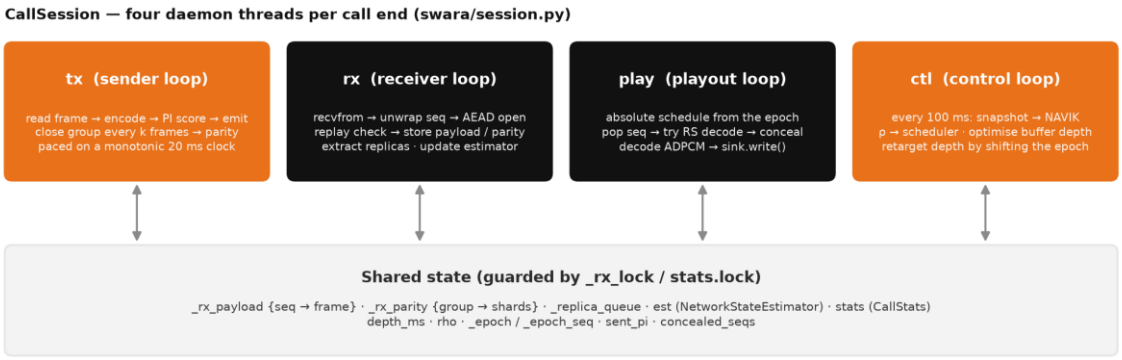


Figure 9. The four threads of a CallSession and the state they share.

Two design details were forced by measurement. First, the playout schedule is absolute rather than a free-running 20 ms tick: with a free-running tick the buffer depth is whatever happened to accumulate at start-up and can never change, so raising the depth later has no effect and parity that needs the extra slack arrives too late to be used. Second, replica displacement subtracts the 95th-percentile delay spread: ignoring it placed the copy about one frame too late and only 7 % of transmitted replicas were ever used.

6.5 The relay server

`server.py` binds one UDP socket and runs a single receive loop. The media relay is checked first because it has the highest volume: if the header's connection id names an active call, the datagram is forwarded verbatim to the other party and nothing else happens — no decryption, no parsing. Otherwise the datagram type selects a SPARK phase: HELLO is answered statelessly; AUTH_INIT is validated against the cookie and silently dropped if it fails (a spoofed source gets nothing); a retransmitted AUTH_INIT re-sends the *first* response so the transcript the client committed to is preserved; AUTH_CONFIRM completes the handshake and records the session by username and by CID. Encrypted CONTROL messages carry JSON signalling. Call placement mints a random 32-byte secret, derives the two directional keys and delivers them to each party under that party's own session key. Because UDP may lose either answer, the caller attaches a request id and retransmits; the server remembers the answer it already gave and repeats the same `call_ready` / `incoming` pair instead of minting a second call — the idempotent server half of SIP-style INVITE retransmission [35]. Keepalive `ping` messages refresh NAT bindings and let the server track the address at which a client is currently reachable.

6.6 The client

`client.py` is the user-facing application: `python client.py --user alice --password secret1 --server host:9000 --call bob` places a call and `--wait` answers one. Every handshake step is retried with a linearly growing timeout because UDP supplies no retransmission and SPARK's messages are all idempotent at the server. Audio comes from the microphone (`--audio live`, requiring sounddevice), a WAV file, or the synthetic generator; `--protect swara|uniform|none` is the A/B switch; `--loss`, `--burst`, `--extra-delay` and `--jitter` enable the local impairment injector for demonstrations; `--narrowband` selects the 33 kbit/s mode. A waiting client sends a keepalive every 15 s so that a NAT pinhole stays open while it is idle — the detail that makes a client that “works perfectly in testing” also work for real users.

6.7 Demonstration harness and experiments

`demo.py` starts a real server and two real clients in one process, each on its own UDP socket, runs an actual call through codec → PI → FEC → AEAD → UDP → relay → UDP → AEAD → repair → jitter buffer → codec, and reports frames sent, parity and replicas, frames rebuilt by FEC or saved by replicas, concealed frames, loss after repair, wire rate, waveform correlation and SNR (by exact FFT cross-correlation alignment of what was sent against what was heard), and authentication failures. With `--compare` the same call is performed three times — SWARA, uniform FEC, unprotected — on the identical channel realisation; `--repeat` averages runs. `experiments/run_all.py` runs the eight

simulated experiments of Chapter 7 in about three minutes and regenerates `results.json` and all figures.

6.8 The web console and REST API

The web console (api_server.py, webapp/, static/) exposes the whole system through a browser. It is a FastAPI application served by uvicorn that also hosts the relay inside the same process, so command-line clients on this or another machine can log in and call each other while the console shows who is online and which calls are being relayed. Its thirteen pages are summarised in Table 8 and shown in Appendix B.

Table 8. Web console pages.

Page	What it does
Overview	Headline results from results.json, the twelve-stage pipeline (each stage links to its tool), an animated frame stream and the hosted-relay card with ready-to-paste client commands
Live Call Lab	Places a real instrumented call (relay + SPARK + AEAD over UDP) and streams it: per-frame fate map (delivered / rebuilt / replica / concealed, bar height = PI), sent vs received waveforms, live charts of network, adaptation and quality; “Listen live” plays the received audio in the browser; sliders change loss, burst, jitter and delay *during* the call; the caller's audio can be the user's own recording
A/B Benchmark	The same call under SWARA, uniform FEC and no protection over the identical channel, with charts, a like-for-like table and the audio of each arm
Network Simulator	The offline engine on demand with the named profiles, and a loss sweep
Research Results	Interactive versions of all eight experiments, the paper figures, and a button that regenerates them
Perceptual Analyzer	PI scores, class labels and the FEC plan for synthetic speech or the user's own recording, frame by frame
FEC Playground	Reed–Solomon encode / erase / decode by clicking packets; the PI-AFEC scheduler for one group
Quality & NAVIK	E-model calculator with the impairment breakdown; NAVIK decisions across loss rates
SPARK Handshake	Runs the real handshake in memory, message by message, including a wrong-password attempt and path migration
Packet Anatomy	Interactive 16-byte header packed by Header.pack() with the datagram composition
SETU Overlay	A random measured mesh, its converged routes and the pairs a relay improves
Relay & Users	Hosted relay status, signed-in users, active calls, log; register and delete users
System & API	Runtime, packages, module inventory and every endpoint; Swagger and ReDoc at /docs and /redoc

The instrumented call is a `TracedSession` subclass of `CallSession` that observes the reassembly buffer immediately before and after the base-class logic runs and records how every received frame was obtained, without changing the protocol. Telemetry, frame fates and the received PCM are streamed to the browser with Server-Sent Events five times a second; the audio is played gaplessly through the Web Audio API. Only one call or benchmark runs at a time because the media threads are paced against a wall clock and two concurrent calls would measure each other rather than the network. The 37 REST endpoints are documented by the generated OpenAPI schema (Figure 57).

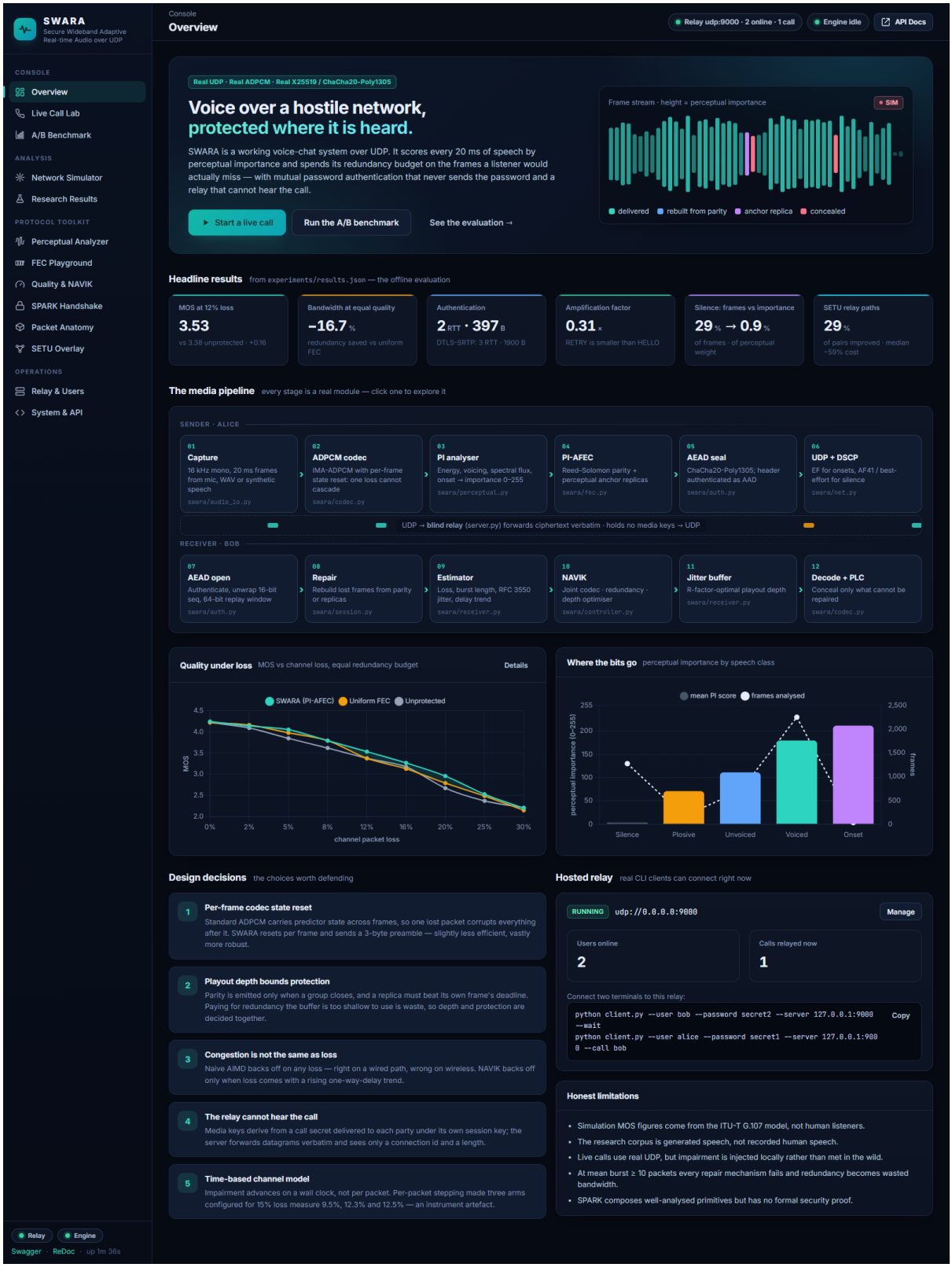


Figure 10. The web console's Overview page: headline results, the twelve-stage pipeline, and the hosted relay card.

6.9 Server control launcher

`launcher.py` is a small Tkinter window that starts, stops and restarts uvicorn and the hosted relay, shows the console URL and the LAN address for other laptops, the relay's signed-in users and calls, and the live server log (with routine polling requests filtered out). It remembers the chosen ports,

moves to the next free port when another program holds the default (which happened on the development machine, where port 8000 was occupied), and offers to stop the server when the window is closed so nothing is left running. `SWARA Server.bat` opens it with one double-click.

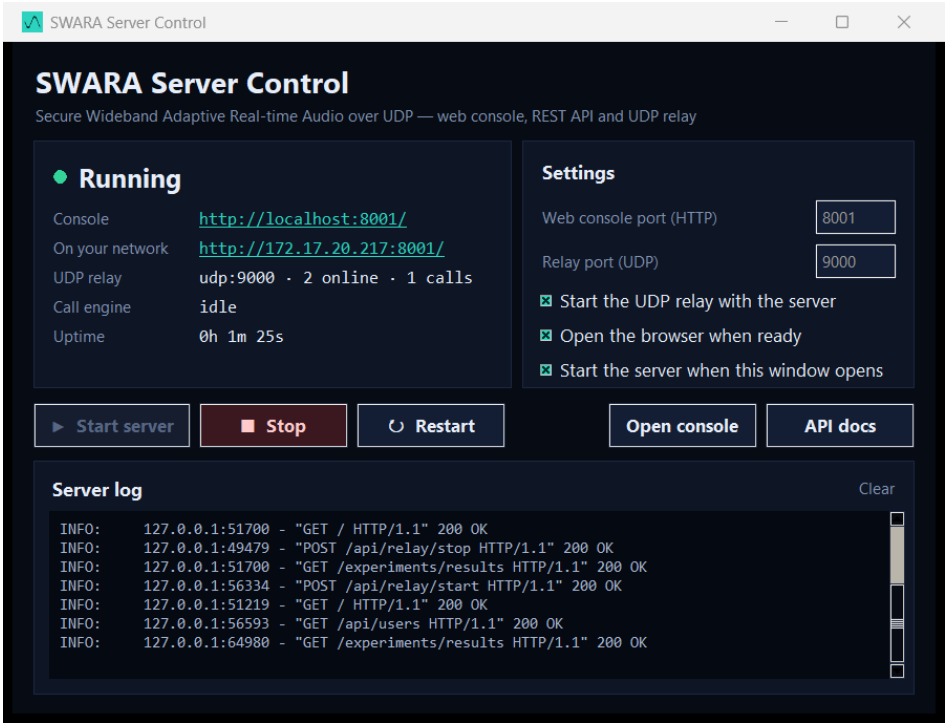


Figure 11. The SWARA Server Control window with the server running.

6.10 Verification during development

Every endpoint was exercised over HTTP with a smoke-test script; every console page was loaded in headless Chrome with the console log captured, and no JavaScript errors remained. The Reed–Solomon implementation is checked for the MDS property at import. The fairness properties of the impairment injector (two random streams, per-arm reset, wall-clock stepping) exist because earlier versions of the harness produced measurable artefacts, each of which is documented in the source. The final verification pass for this report found one more such artefact — a busy-waiting scheduler thread in the injector that distorted every timing the live controller observed — and corrected it; Section 7.9 reports the diagnosis and the before/after measurements, because an evaluation is only as trustworthy as its instrument.

6.11 Running the system

```
pip install -r requirements.txt            # numpy, matplotlib, cryptography, fastapi, uvicorn (+
sounddevice)

# A real call between three terminals
python server.py --add-user alice --password secret1
python server.py --add-user bob   --password secret2
python server.py --port 9000                                        # terminal 1
python client.py --user bob   --password secret2 --wait             # terminal 2
python client.py --user alice --password secret1 --call bob         # terminal 3   (add --audio live for
mic & speakers)

# One-command demonstration and A/B comparison
python demo.py --seconds 25 --loss 0.15 --burst 4 --jitter 12 --compare --repeat 2

# Web console (hosts the relay on udp:9000) and the desktop launcher
python -m uvicorn api_server:app --host 0.0.0.0 --port 8000         # then open http://localhost:8000
"SWARA Server.bat"                                                  # Windows: one-click launcher window

# Regenerate all research figures (~3 minutes)
python experiments/run_all.py
```

7 Results and Discussion

Two kinds of evidence are reported. The **simulated evaluation** (Sections 7.2–7.9) runs identical synthetic speech through identical channel realisations for the three systems and scores each with the E-model; it is the stable, repeatable picture averaged over seeds, and every figure is regenerated by `experiments/run_all.py`. The **live evaluation** (Section 7.10) places real calls — real sockets, real cryptography, real time — through the relay over an impaired loopback path; it is noisier, but it is the system as a user would run it. Section 7.11 discusses what the two together show, and Section 7.12 states the limits of the evidence.

7.1 Experimental setup

Table 9. Experimental configuration.

Experiment	Configuration
Machine	Windows-11-10.0.26200-SP0, 20 logical CPUs, Python 3.14.3; all three systems run on the same machine so that CPU effects are common to every arm
Speech corpus	Source-filter synthesiser, 16 kHz, 20 ms frames; talkspurts 0.9–2.8 s; seeds 5, 17, 29, 41 (53) unless stated
Loss sweep	Loss 0–30 %, mean burst 3, delay 40 ± 14 ms, capacity 800 kbit/s, 22 s calls, 5 seeds; k = 8
Burst sweep	Loss 12 %, mean burst 1–10, otherwise as above, 4 seeds
Overhead efficiency	Loss 12 %, burst 4; forced redundancy $\rho \in \{0 \ldots 0.5\}$ for SWARA and uniform FEC, 4 seeds
PI distribution	Six 14 s utterances (seeds 11, 23, 44, 57, 68, 79) with per-frame ground truth from the synthesiser
Jitter buffer	Loss 6 %, burst 3, delay 45 ± 22 ms with a heavier tail ($\sigma = 0.7$); forced depths 20–180 ms, 3 seeds
Handshake	In-process timing of the real SPARK implementation; 20,000 AEAD seals for the per-packet cost; DTLS-SRTP figures are published typical values
SETU overlay	300 random 6-node overlays, 28 % of links through a congested exchange (70–190 ms, 20–60 ms jitter, 4–18 % loss), 4,500 pairs
Network profiles	Six named profiles (Section 5.10), 22 s calls, 4 seeds
Live calls	demo.py: 25 s calls, 2 repeats per arm, injector at both sockets; two conditions, 15 % / burst 4 / 12 ms jitter and 10 % / burst 3 / 10 ms jitter. Console benchmark: 20 s per arm at 15 % / 4 / 12. Three-terminal call: 20 s at 15 % / 4 / 12 on both clients

7.2 Perceptual importance separates what matters

Table 10 and Figure 12 show the PI score by speech class on 4,419 labelled frames. The analyser places the four classes in the bands it was designed for, with silence at a mean of 3.4 and talkspurt onsets at 211.5. The decisive statistic for the whole project is the last row: silence is **29.0 %** of all frames but carries only **0.86 %** of the total importance mass. Those frames are what PI-AFEC declines to protect, and that budget is what it moves to the frames that carry meaning.

Table 10. Perceptual importance score by speech class (0–255).

Class	Frames	Mean PI	10th percentile	90th percentile
Silence / noise	1273	3.4	0.0	0.0
Plosive	171	70.7	0.0	164.0
Unvoiced fricative	694	111.1	91.0	130.0
Voiced vowel	2251	179.6	160.0	209.0
Talkspurt onset	30	211.5	135.9	255.0
Silence share of frames / of importance		29.0 % / 0.86 %		

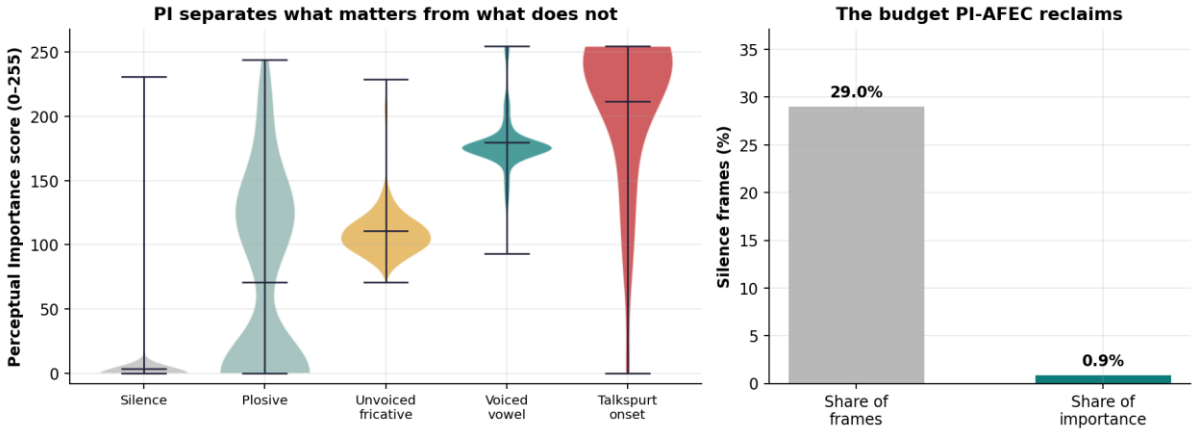


Figure 12. PI score distributions by speech class (violins show the full distribution, bars the mean) and the share of frames versus the share of importance held by silence.

7.3 Quality under packet loss

Table 11 and Figure 13 give the headline comparison: predicted MOS and perceptually weighted loss as channel loss rises from 0 to 30 % with a mean burst of 3, all three systems on the same speech, the same channel realisation and — for uniform FEC — the same redundancy SWARA spent. At 12 % loss SWARA scores **3.53** against 3.37 for uniform FEC and 3.38 unprotected, and its perceptual loss is 5.8 % against 7.3 % and 7.3 %. The advantage is largest in the 5–20 % band that corresponds to busy Wi-Fi and congested cellular links; at 30 % every scheme converges (2.20 / 2.15 / 2.21) because no amount of redundancy can replace a channel that drops almost a third of what is sent. Uniform FEC, note, is *not* uniformly better than no FEC: at 12–16 % loss its evenly spread parity buys nothing the baseline does not already get from concealment, while SWARA's placement still does.

Table 11. Loss sweep: predicted MOS (G.107) and perceptually weighted loss, mean of 5 seeds.

Channel loss	MOS baseline	MOS uniform FEC	MOS SWARA	Perc. loss baseline	Perc. loss uniform	Perc. loss SWARA
0 %	4.22	4.22	4.24	0.5 %	0.5 %	0.0 %
2 %	4.09	4.16	4.14	1.7 %	1.0 %	1.0 %
5 %	3.85	3.98	4.05	3.7 %	2.7 %	1.8 %
8 %	3.62	3.79	3.79	5.5 %	4.1 %	3.8 %

Channel loss	MOS baseline	MOS uniform FEC	MOS SWARA	Perc. loss baseline	Perc. loss uniform	Perc. loss SWARA
12 %	3.38	3.37	3.53	7.3 %	7.3 %	5.8 %
16 %	3.17	3.13	3.27	8.8 %	9.2 %	7.8 %
20 %	2.67	2.79	2.96	12.7 %	11.7 %	10.1 %
25 %	2.37	2.48	2.53	15.3 %	14.4 %	13.6 %
30 %	2.21	2.15	2.20	16.9 %	17.6 %	16.6 %

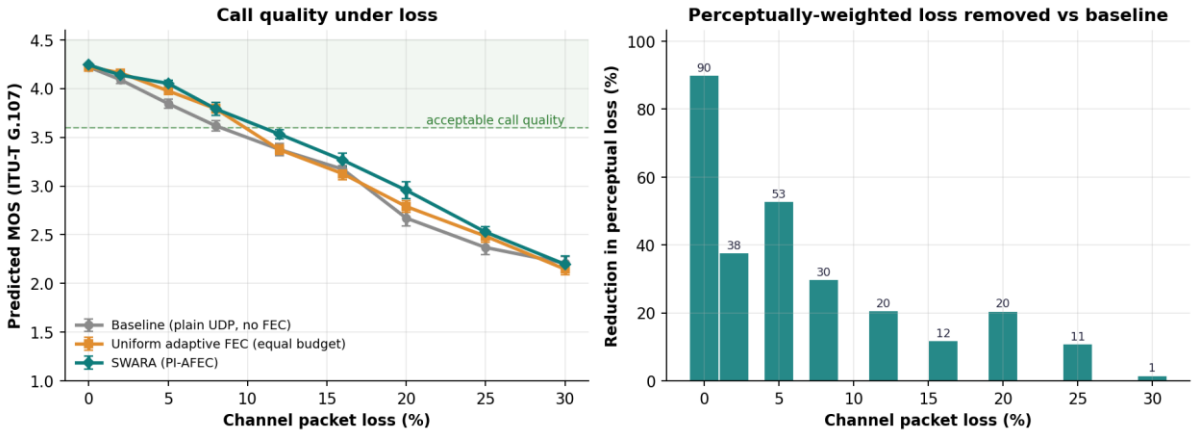


Figure 13. Call quality under loss (left; shaded bands ±1 s.e.) and the reduction in perceptually weighted loss relative to the baseline (right).

7.4 Resilience to bursty loss

Figure 14 fixes the loss rate at 12 % and varies the mean burst length from 1 to 10 packets. Two things are visible. First, SWARA holds its lead over both comparators up to a burst of 8 and only converges with them at 10, where — as the project's own README states — every repair mechanism fails and redundancy becomes wasted bandwidth. Second, the right-hand panel shows *which mechanism does the rescuing*: with isolated losses Reed–Solomon parity recovers 41.3 frames per call and anchors 4.3; by a burst of 6 the parity recoveries have collapsed to 2.3 while anchor recoveries have risen to 6.3. This is the burst-mode design decision of Section 5.4 doing exactly what it was built to do: when a block code cannot survive the burst, the time-displaced replica still can.

Table 12. Burst sweep at 12 % loss: MOS and SWARA's recoveries per call by mechanism, mean of 4 seeds.

Mean burst	MOS baseline	MOS uniform FEC	MOS SWARA	FEC recoveries	Anchor recoveries
1	3.79	3.89	4.02	41.3	4.3
2	3.58	3.64	3.80	21.3	4.3
3	3.37	3.37	3.53	15.5	3.5
4	3.28	3.49	3.55	14.0	1.8
6	2.96	3.13	3.41	2.3	6.3
8	3.03	3.17	3.28	3.0	6.8
10	2.89	2.84	2.80	1.8	5.3

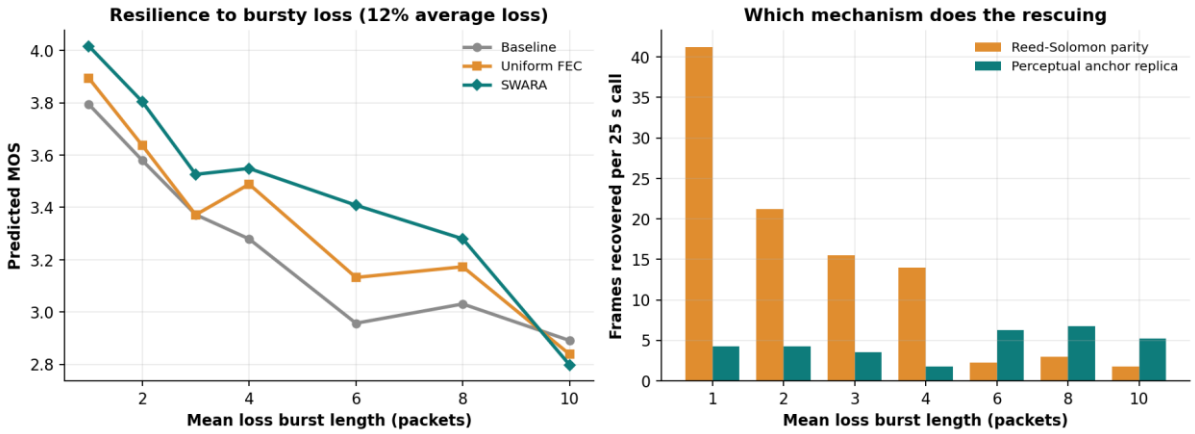


Figure 14. Resilience to bursty loss (left) and the mechanism responsible for each recovery (right).

7.5 Quality returned per byte of redundancy

Figure 15 plots MOS against the redundancy actually transmitted when both schemes are forced through the same budget ladder at 12 % loss and burst 4. Because PI-AFEC never spends on silent groups, it transmits less than the nominal budget (41.6 % at ρ = 0.5 against 50.0 % for uniform FEC) and yet reaches a higher MOS (3.44 versus 3.19). Reading the curves horizontally, SWARA reaches the quality uniform FEC attains at its maximum budget for **16.7 % less** redundancy. This is the project's bandwidth claim: the same quality for fewer bytes, or more quality for the same bytes.

Table 13. Overhead efficiency at 12 % loss, burst 4: MOS and redundancy actually spent for each nominal budget ρ.

Nominal ρ	SWARA redundancy	SWARA MOS	Uniform redundancy	Uniform MOS
0.000	0.0 %	3.18	0.0 %	3.18
0.075	6.2 %	3.07	7.6 %	3.08
0.150	12.4 %	3.16	15.3 %	3.16
0.225	18.7 %	3.17	22.4 %	3.02
0.300	24.9 %	3.04	30.4 %	3.08
0.400	33.3 %	3.30	40.3 %	3.21
0.500	41.6 %	3.44	50.0 %	3.19

Equal-quality saving: 16.7 % of media bandwidth.

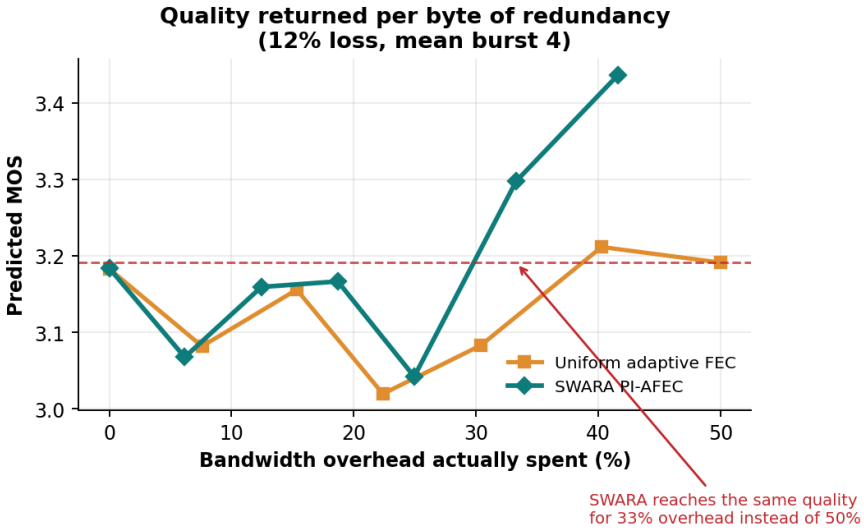


Figure 15. Quality returned per byte of redundancy.

7.6 The jitter-buffer optimum

Section 5.7 argued that “minimise late loss” is the wrong objective for a playout buffer. Figure 16 tests the alternative. The left panel is the optimiser's own view: the predicted R-factor over depth, peaking at 100 ms, from which SWARA chose 99.9 ms. The right panel is the ground truth: the end-to-end simulation re-run with the depth *forced* to each value. Measured MOS peaks at 100 ms (3.81); a shallow 20 ms buffer collapses to 2.11 because almost everything arrives late, and a deep 180 ms buffer falls back to 3.52 because the delay impairment past the 177 ms knee outweighs the packets it rescues. The optimiser's choice and the measured optimum coincide to within 1 ms, which validates both the model and the decision to make it the controller's objective.

Table 14. Jitter-buffer depth: predicted R-factor and measured end-to-end MOS.

Depth (ms)	20	40	60	80	100	120	150	180
Predicted R	23.6	60.8	77.0	82.4	84.2	82.6	79.1	75.3
Measured MOS	2.11	3.46	3.69	3.78	3.81	3.79	3.68	3.52

Optimiser's choice 99.9 ms; measured optimum 100 ms.

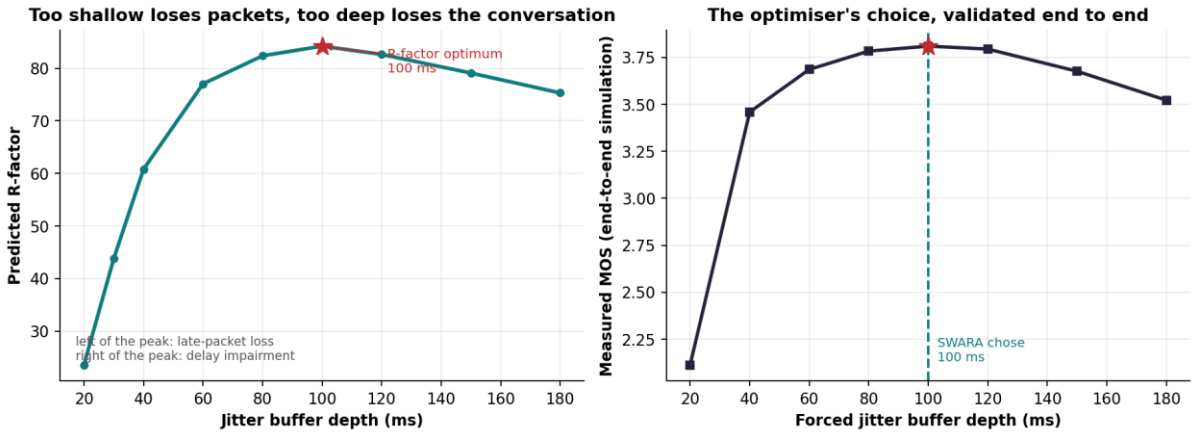


Figure 16. Too shallow loses packets, too deep loses the conversation: the R-factor curve (left) and the end-to-end validation (right).

7.7 Handshake cost and security properties

Table 15 and Figure 17 measure the real SPARK implementation. Mutual password authentication with forward secrecy completes in **2 round trips and 397 bytes**, against three round trips and roughly 1,900 bytes for DTLS-SRTP — about 4.8× less setup traffic. The 73 ms of CPU is dominated by the deliberately expensive PBKDF2 stretching on the client; the server's own work per handshake is three X25519 operations and a few hashes. A network handover costs **zero** round trips and 115 bytes, and the server spends 11 µs on it. The RETRY reply is 20 bytes to a 64-byte HELLO, so the amplification factor is **0.31**: an attacker spoofing HELLOs at a victim makes the victim receive less than it sent. The per-user record is 128 bytes and contains no password-equivalent. Sealing a media packet costs 3.0 µs — 0.015 % of a 20 ms frame budget — so encryption is never on the critical path.

Table 15. Measured handshake and media-protection cost.

Quantity	SPARK	DTLS-SRTP (typical)	Plain UDP
Round trips before media flows	2	3	0
Handshake bytes on the wire	397	1,900	0
Round trips to migrate a live call	0	new handshake	n/a
Migration bytes	115	—	—
Amplification factor (reply ÷ first packet)	0.313	≤ 3 (DTLS 1.2 cookie)	—
Server record per user	128 B, no verifier	certificate / SRTP keys	none
Mutual password authentication	yes	no (certificate fingerprints)	no
AEAD seal per packet	3.02 µs	similar	none

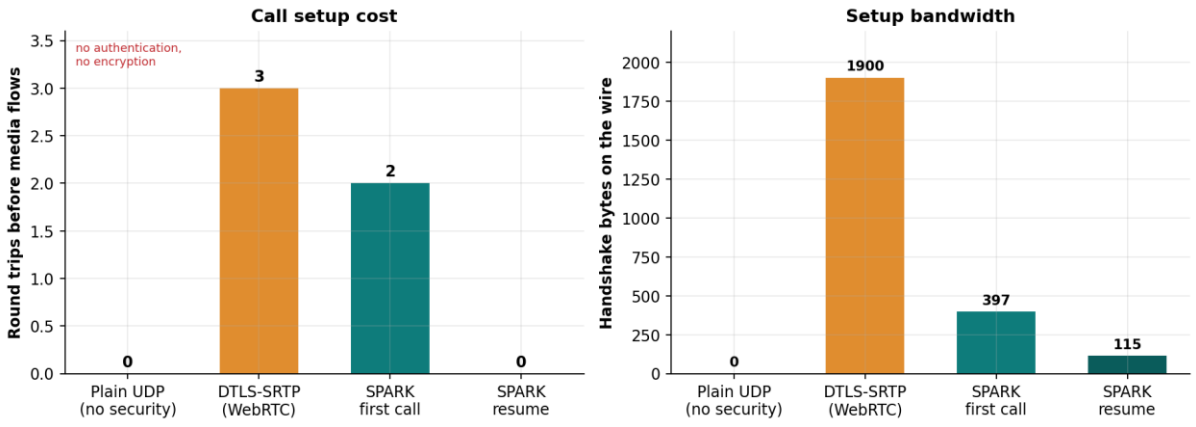


Figure 17. Call-setup cost in round trips and bytes.

Table 16. Security properties of SPARK and how each is obtained.

Property	Mechanism
No state before address validation	Stateless HMAC cookie bound to (address, 30 s epoch); AUTH_INIT without a valid cookie is silently dropped
No amplification	HELLO padded to 64 B; RETRY is 20 B; nothing larger is sent before the cookie returns
Password never on the wire	OPAQUE-style envelope opened locally with PBKDF2; a wrong password fails before anything is sent

Property	Mechanism
No user enumeration	Unknown users receive a constant-shape random envelope
Mutual authentication	dh1 proves the server holds its static key; dh2 proves the client could open the envelope
Forward secrecy	dh3 between the two ephemeral keys; the transcript hash binds every byte of every message
Stolen database resistance	Server stores {salt, envelope, public key}; nothing is replayable at the server
Replay and forgery resistance on media	Deterministic nonces from the 64-bit sequence; 1,024-entry replay bitmap; header as AEAD associated data
Blind relay	Per-call directional keys delivered under each party's session key; the relay forwards ciphertext verbatim
Graceful handover	Encrypted resumption ticket + proof under k_resume; 0 RTT; replaying the ticket alone cannot hijack a call

7.8 Relay-path selection

Across 4,500 peer pairs in 300 random overlays in which 28 % of links cross a congested exchange, SETU found a relayed path cheaper than the direct route for **29.3 %** of pairs, with a mean cost reduction of 55.0 % (median 58.8 %) on the pairs it improved. 3,174 pairs stayed direct, 1,244 took one relay, 78 took two and 4 took three; the distance-vector computation converged in 6 rounds. The result reproduces the RON observation [33] in voice-cost terms: the triangle inequality fails often enough that a small measured overlay is worth having.

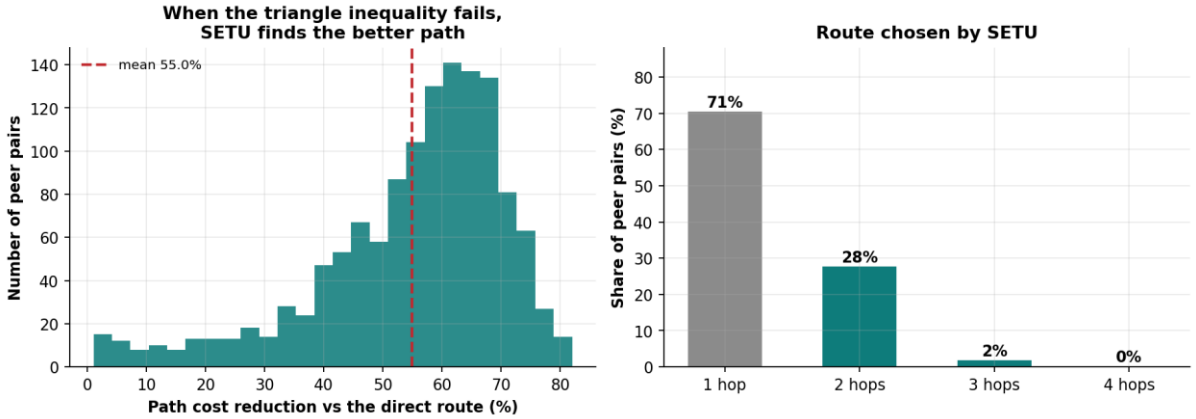


Figure 18. SETU: distribution of the cost reduction on improved pairs (left) and the share of pairs by chosen route length (right).

7.9 Realistic network profiles

Table 17 and Figure 19 summarise the six named profiles. On good Wi-Fi all three systems are at the ceiling (4.23); SWARA spends nothing it does not need. On busy Wi-Fi and congested 4G it leads by 0.17 and 0.14 MOS over uniform FEC and by 0.26 and 0.33 over the baseline, halving the perceptual loss on busy Wi-Fi (1.8 % against 4.4 %). On the hostel profile — 15 % loss in bursts of 8, the condition that motivated the project — it is the only system above 3.0. Two profiles are honest exceptions. On poor 3G (20 % loss, 110 ms base delay) uniform FEC is marginally ahead (1.73 vs 1.72): at that loss rate the controller's safety rail allows little redundancy and the deeper buffer SWARA chooses costs more

than its placement gains. On the satellite profile (280 ms base delay) the same delay arithmetic applies: every millisecond of extra buffer is charged at the steep post-knee rate, so uniform FEC with the conventional shallower buffer edges ahead (2.88 vs 2.82) even though SWARA's residual and perceptual loss are the lowest of the three. SWARA's gains, in other words, come where loss rather than propagation delay is the binding impairment — which is where most real conversations are.

Table 17. Named profiles: predicted MOS, residual and perceptual loss, mouth-to-ear delay and redundancy, mean of 4 seeds.

Profile	MOS base / uniform / SWARA	Residual loss base / uni / SWARA	Perceptual loss base / uni / SWARA	Delay (ms) base / SWARA	Redundancy uni / SWARA
good wifi	4.23 / 4.21 / 4.23	1.9 / 1.7 / 1.0 %	0.6 / 0.8 / 0.4 %	68 / 102	46.5 / 46.3 %
busy wifi	3.77 / 3.85 / 4.02	8.3 / 7.1 / 4.0 %	4.4 / 3.6 / 1.8 %	108 / 163	46.5 / 46.3 %
congested 4g	3.24 / 3.43 / 3.57	13.4 / 11.5 / 9.5 %	7.8 / 6.3 / 4.3 %	168 / 194	37.7 / 37.8 %
hostel wifi	2.80 / 2.79 / 3.05	15.8 / 14.2 / 13.1 %	10.6 / 10.5 / 7.8 %	177 / 195	25.2 / 24.9 %
poor 3g	1.55 / 1.73 / 1.72	22.4 / 21.1 / 21.0 %	15.9 / 13.7 / 12.9 %	276 / 292	10.0 / 10.1 %
satellite	2.75 / 2.88 / 2.82	5.8 / 4.2 / 3.6 %	2.4 / 1.6 / 1.4 %	361 / 375	46.2 / 46.3 %

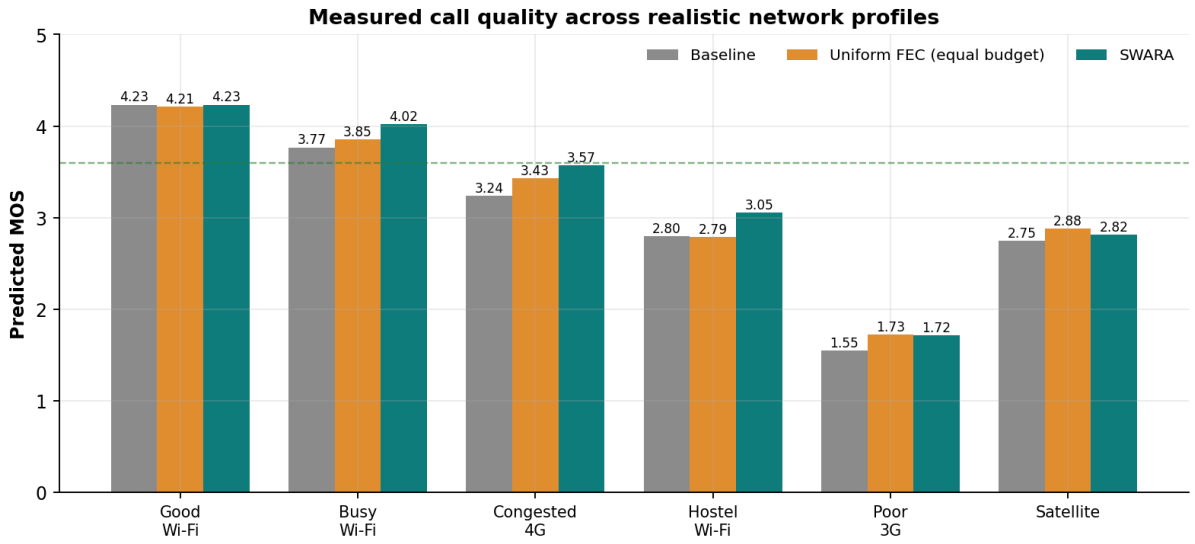


Figure 19. Measured call quality across realistic network profiles (dashed line: the 3.6 “acceptable” threshold).

7.10 Live end-to-end calls

7.10.1 Validating the instrument first

Before the live benchmarks were accepted, the controller's behaviour during a real call was inspected by sampling NAVIK's state once per second (Table 18). The first run showed the capacity estimate collapsing from 400 kbit/s to its 24 kbit/s floor within eight seconds, after which the optimiser could fit no codec and emitted *no redundancy at all* for the rest of the call; the delay trend it was reacting to swung between −101 and +73 ms/s although the configured jitter was 12 ms, and the receiver-side one-way delay had a standard deviation of 35–120 ms with a 95th-percentile spread of up to 410 ms. On a clean link the same quantities were 1.7 ms and 5 ms. The cause was not the controller. The impairment injector's delayed-delivery scheduler consumed its wake-up event only when its queue

ran empty — which, during a call, never happens — so its wait returned immediately and the thread busy-spun, starving the four media threads of the interpreter lock. The two-line correction (consume the flag on every iteration; `swara/net.py`, Listing 9) restored a delay spread consistent with the configured jitter, a trend within ±4 ms/s, a redundancy of ρ = 0.375 held for the whole call, and 169 parity plus 33 replica packets in 15 s where there had been 46 plus 2. Every live result below was measured after the correction. The episode is reported in full because it is the project's methodological thesis in miniature: a protocol can only be judged through an instrument that has itself been validated.

Table 18. Diagnosis of the injector defect: one call at 15 % loss, burst 4, 12 ms jitter, sampled once per second.

Quantity	Clean link (reference)	Impaired, before correction	Impaired, after correction
One-way delay std. dev., last 100 packets	1.7 ms	35 – 120 ms	7.7 – 11 ms
One-way delay spread (p95 − min)	5 ms	120 – 410 ms	29 – 34 ms
Delay-trend samples seen by NAVIK	±0.8 ms/s	−101 … +73 ms/s	−4 … +1.5 ms/s
Capacity estimate after 8 s	—	24 kbit/s (floor)	283 kbit/s
Redundancy ρ after 8 s	0 (no loss)	0 (could not fit)	0.375
Parity + replica packets sent	0 + 0	46 + 2 in 18 s	169 + 33 in 15 s
Frames concealed at the receiver	0.0 %	20.0 %	15.1 %

7.10.2 demo.py: three arms on an identical channel

Table 19 reports the one-command A/B harness: 25-second calls, two repeats per arm, the injector at both sockets, every arm on the identical realisation of loss and jitter. At **15 % loss with bursts of four**, SWARA matches uniform FEC's perceptually weighted damage (7.3 % each) while transmitting 112 kbit/s against 126 — **11 % less bandwidth** — and both cut the damage of unprotected UDP (8.7 %) by 16 %. The raw frame count tells the expected other half of the story: uniform FEC leaves fewer frames unrecoverable (11.1 % against 12.3 %) because it protects everything, silence included, and does so with 600 parity packets to SWARA's 328 plus 56 replicas. At **10 % loss with bursts of three**, SWARA achieves the lowest damage of the three — 4.2 % against 4.3 % and 5.3 % — on **12 % less bandwidth** than uniform FEC. Both runs are captured as real terminal output in Figures 20 and 21.

Table 19. Live A/B comparison (demo.py, 25 s × 2 repeats, injector at both sockets).

Condition	Arm	Parity / replicas	Rebuilt (FEC / replica)	Concealed	Unrecoverable	Wire kbit/s	Corr.	SNR dB	Perceptual damage
15 % / burst 4 / 12 ms	SWARA	328 / 56	8.0 / 5.5	156.5	12.3 %	111.8	0.924	8.5	7.3 %
	Uniform FEC	600 / 0	31.0 / 0	141.5	11.1 %	126.3	0.924	8.5	7.3 %
	Unprotected	0 / 0	0 / 0	169.0	13.3 %	87.7	0.914	8.0	8.7 %
10 % / burst 3 / 10 ms	SWARA	356 / 48	11.0 / 5.0	105.5	8.3 %	113.2	0.942	9.5	4.2 %
	Uniform FEC	628 / 0	25.5 / 0	96.0	7.5 %	128.1	0.942	9.5	4.3 %
	Unprotected	0 / 0	0 / 0	119.0	9.3 %	87.7	0.934	9.0	5.3 %

1,293 frames sent per call; authentication failures 0 in every run. Correlation and SNR compare the waveform heard by bob with the one alice sent, after FFT alignment.

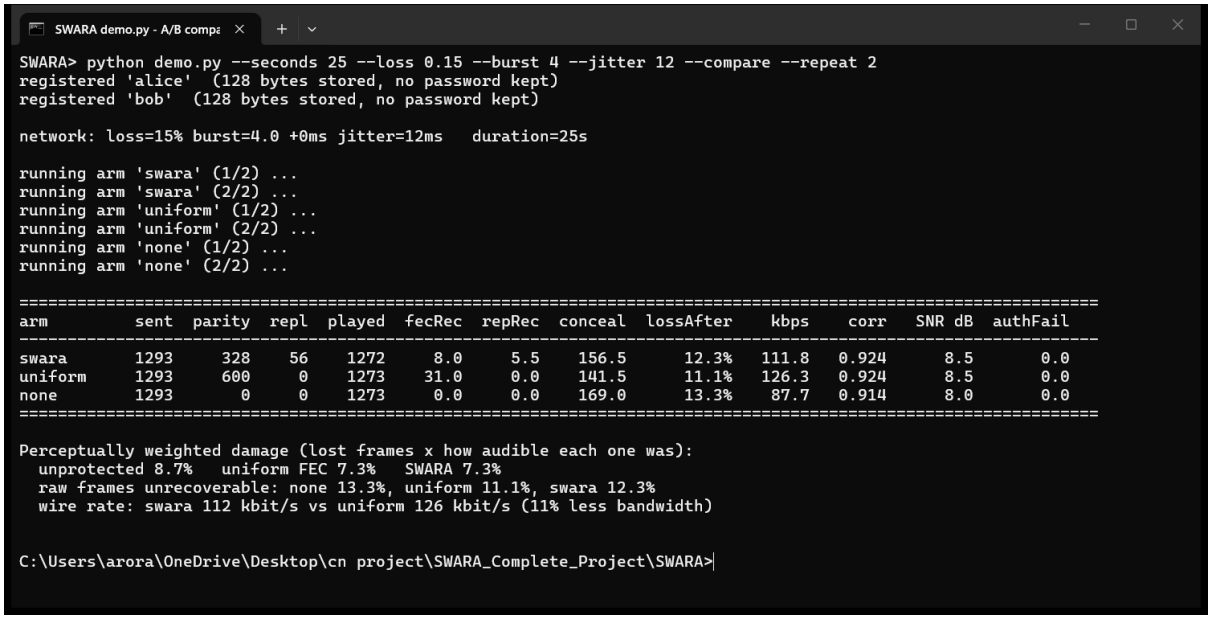


Figure 20. Terminal output of demo.py --compare at 15 % loss, mean burst 4, 12 ms jitter (25 s calls, 2 repeats per arm).

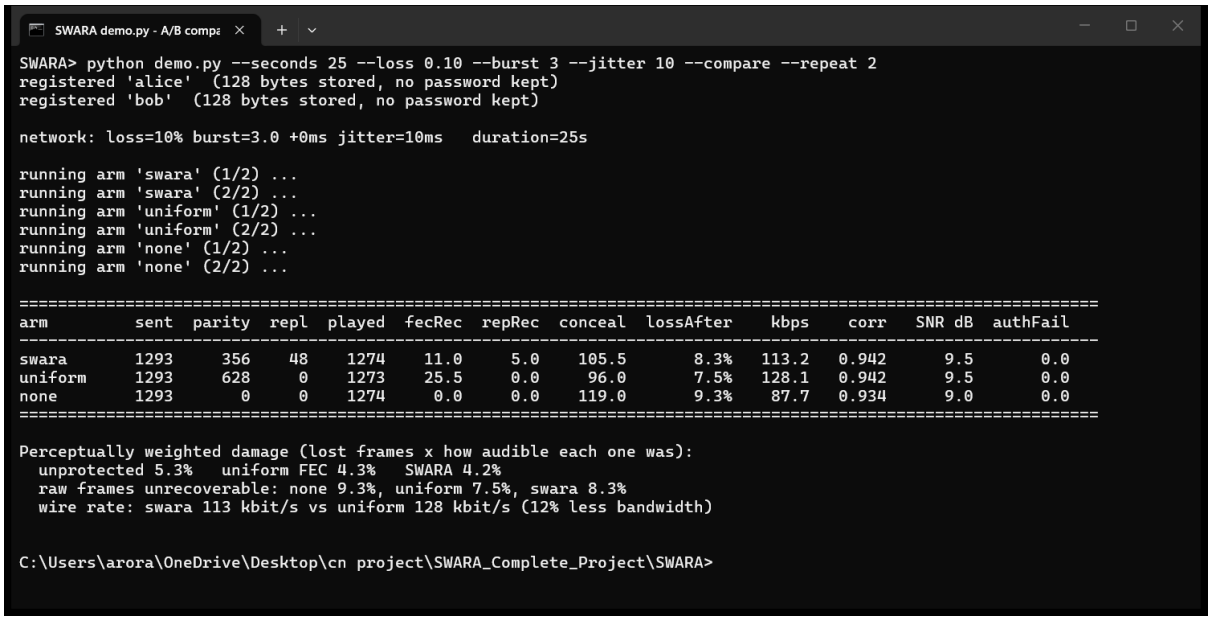


Figure 21. Terminal output of demo.py --compare at 10 % loss, mean burst 3, 10 ms jitter.

7.10.3 The web console benchmark and an instrumented call

The console's A/B Benchmark page (Figure 37) runs the same three arms as full instrumented calls — 20 s per arm at 15 % / burst 4 / 12 ms, one run each — and reports a like-for-like table with the audio of each arm. SWARA cut perceptual damage by 8 % versus unprotected UDP (10.9 % against 11.8 %) and by 3 % versus uniform FEC (11.2 %) while using 12 % less bandwidth (107.8 against 122.6 kbit/s); the estimated MOS ordering was 2.64 / 2.59 / 2.55. The single-call Live Call Lab (Figure 35) shows the mechanism frame by frame: in a 15 s call at 10 % loss with bursts of three, 750 frames were sent with 197 parity and 12 replica packets (26 % overhead, 107 kbit/s), 674 arrived, 6 were rebuilt from parity and 1 from a replica, 69 were concealed, SPARK authenticated the two parties in 190 and 858 ms, call

setup took 826 ms, the jitter buffer settled at 118 ms, and the received waveform correlated with the sent one at 0.941 (SNR 9.4 dB) for a predicted MOS of 3.59. The frame-fate map in the same figure makes the design visible: the tallest bars — the onsets and vowels — are almost all teal, while the red concealments cluster in the low, silent valleys.

7.10.4 A call between separate programs

Finally, the system was run as a user would run it: `server.py` in one terminal, `client.py --wait` as bob in a second and `client.py --call bob` as alice in a third, with the injector enabled on both clients at 15 % / burst 4 / 12 ms (Figures 26–28). Both clients authenticated in two round trips (776 ms and 1,681 ms including PBKDF2), the server logged the call with the note that it holds no keys, and after 20 s each side printed its own report: bob sent 1,021 frames with 287 parity and 17 replica packets, rebuilt 14 frames from parity and concealed 102 (10.0 %); alice sent 1,021 frames with 273 parity and 32 replicas, rebuilt 7 and concealed 139 (13.7 %). Neither side saw a single authentication failure. Section B.2 shows the same three windows during the call.

7.11 Discussion

- **The gain is where the theory says it should be.** Across the loss sweep, burst sweep and profiles, SWARA's advantage over uniform FEC at an identical budget is concentrated at 5–20 % loss with bursts of 2–8 — the regime of busy Wi-Fi, hostel networks and congested cellular links — and it vanishes at 30 % loss and bursts of 10, exactly where no repair mechanism can work. Placement, not quantity, is the source of the gain: Table 13 shows SWARA transmitting *less* redundancy than uniform FEC at every budget and scoring higher.
- **Raw and perceptual loss tell different stories, and both must be reported.** In every live run uniform FEC leaves fewer frames unrecoverable while SWARA leaves fewer *audible* frames unrecoverable. A reader who looks only at the raw count will conclude SWARA is worse; a reader who looks only at the perceptual figure will not understand why. The project reports both and argues that the second is the one a listener experiences.
- **Anchors and parity are complementary, not redundant.** The burst sweep (Figure 14) shows parity doing the work at short bursts and replicas at long ones; a scheme with only one of them is good at only one end of the range.
- **The buffer is part of the protection scheme.** The jitter-buffer experiment validated the R-factor objective to within 1 ms, and the profiles show the price of it: SWARA's buffer is 20–55 ms deeper than the conventional one because it refuses to pay for parity it cannot use. On links where propagation delay is already past the knee (satellite, poor 3G) that price exceeds the benefit, and uniform FEC with a shallow buffer is marginally better.
- **Live gains are smaller than simulated gains, for identifiable reasons.** Live calls are short (the controller needs about half a second of arrivals before it acts), the capacity estimate decays slowly under sustained loss, the simulator's k = 8 groups are replaced by k = 4 to keep the playout depth conversational, and each sender sizes its redundancy from the loss on its *receiving* path — the telemetry report from the far end is carried and parsed but not yet used, so an asymmetric path is under-protected. None of these is intrinsic to the idea; all are listed as future work.
- **Single runs are noisy.** With a mean burst of four a 25-second call contains only about 47 loss bursts, few enough that which bursts land where matters as much as the scheme under test. The identical-realisation design removes the between-arm component of that noise, but not

the between-run component; the two-repeat tables above are stable to roughly ±1 percentage point of perceptual damage, which is why the simulated, multi-seed evaluation remains the primary evidence.

- **Security came at negligible cost.** Two round trips, 397 bytes, 3 µs per packet and a relay that cannot listen — against a naive UDP service that is both a reflection amplifier and an open microphone.

7.12 Limitations

- Quality figures come from the ITU-T G.107 model fed with a perceptually weighted loss; no human listening tests were conducted.
- The research corpus is synthesised speech with the right structure (talkspurts, voiced and unvoiced phones, onsets) but not recorded human speech; the PI analyser's bands were tuned on it.
- Live calls use real UDP, but impairment is injected at the sending socket rather than encountered in the wild, and both ends run on one machine, so the delay statistics are those of the loopback path plus the injector.
- The control loop drives redundancy from the local receive-side estimator; the piggybacked telemetry is not yet consumed by the far end's decision, so paths that are lossy in only one direction are under-protected.
- At mean bursts of ten packets or more every repair mechanism fails and redundancy is wasted; the controller's safety rail limits but does not eliminate that waste.
- CPython's interpreter lock bounds the timing precision of a pure-Python media pipeline; the injector defect of Section 7.10.1 is an example of how sensitive the measurements are to it.
- SPARK composes well-analysed primitives but has no formal security proof; the cookie epoch (30 s) and ticket lifetime (1 h) are reasonable defaults, not tuned values.
- The web console runs one instrumented call or benchmark at a time by design.

8 Conclusion

This project set out to show that packet loss in a voice call should be judged by what the listener loses, and to build a complete system around that principle. SWARA scores every 20 ms frame by its perceptual importance, carries the score where the network can read but not forge it, spends a fixed redundancy budget on the frames that carry meaning, protects the most important of them with replicas that survive the bursts a block code cannot, chooses codec, redundancy, marking and playout depth jointly against a single standard quality model, and secures the whole conversation with a two-round-trip password-authenticated handshake behind a relay that cannot hear the call.

The evaluation supports the principle. At an identical redundancy budget SWARA raises predicted MOS at 12 % loss from 3.37 to 3.53, keeps its lead up to bursts of eight packets, reaches the same quality as uniform FEC for 16.7 % less redundancy, and leads on five of six realistic profiles — the exceptions being links where propagation delay, not loss, is the binding impairment. The R-factor-optimal playout buffer chose the depth that an exhaustive sweep confirmed as optimal. SPARK authenticates in 397 bytes with an amplification factor of 0.31 and migrates in zero round trips. In live calls the system matched uniform FEC's perceptual quality on 11 % less bandwidth at 15 % loss and bettered it at 10 %, and a three-terminal call between separate programs ran exactly as the design says it should.

Two lessons go beyond the numbers. The first is methodological: three of the artefacts the team had to remove — per-packet channel stepping, a shared random stream, and a busy-waiting scheduler — were defects of the measuring instrument that looked like properties of the protocol, and the final one was found only by instrumenting the controller during a real call. The second is architectural: the components that make SWARA work are not independent knobs. The buffer depth bounds the replica displacement; the replica displacement bounds what burst mode can rescue; the controller's objective is the same function the evaluator uses. Designing them together is what made the gains measurable.

8.1 Future work

- Use the far end's piggybacked telemetry, not only local receive-side statistics, to size redundancy on asymmetric paths.
- Replace IMA ADPCM with a modern codec such as Opus so that the controller's bitrate ladder changes real coding quality and not only the nominal E-model parameters.
- Validate the perceptual weighting with listening tests on recorded speech, and tune the PI bands on a human corpus.
- Complete SETU's role as a NAT-traversal fallback and extend the relay to multi-party calls with per-participant importance-aware forwarding.
- Study SPARK formally — in particular the composition of the cookie, the OPAQUE-style envelope and the resumption ticket — and add rate limiting on the server's PBKDF2 path.
- Port the media pipeline to a language without an interpreter lock, or to an asynchronous design, to tighten timing and allow several instrumented calls at once.

References

[1] H. Schulzrinne, S. Casner, R. Frederick and V. Jacobson, “RTP: A Transport Protocol for Real-Time Applications,” IETF RFC 3550, Jul. 2003.

[2] ITU-T Recommendation G.107, “The E-model: a computational model for use in transmission planning,” International Telecommunication Union, Geneva, Jun. 2015.

[3] ITU-T Recommendation G.114, “One-way transmission time,” International Telecommunication Union, Geneva, May 2003.

[4] ITU-T Recommendation P.800, “Methods for subjective determination of transmission quality,” International Telecommunication Union, Geneva, Aug. 1996.

[5] C. Perkins, O. Hodson and V. Hardman, “A survey of packet loss recovery techniques for streaming audio,” IEEE Network, vol. 12, no. 5, pp. 40–48, Sep.–Oct. 1998.

[6] J.-C. Bolot, S. Fosse-Parisis and D. Towsley, “Adaptive FEC-based error control for Internet telephony,” in Proc. IEEE INFOCOM, New York, NY, USA, 1999, pp. 1453–1460.

[7] I. S. Reed and G. Solomon, “Polynomial codes over certain finite fields,” Journal of the Society for Industrial and Applied Mathematics, vol. 8, no. 2, pp. 300–304, 1960.

[8] L. Rizzo, “Effective erasure codes for reliable computer communication protocols,” ACM SIGCOMM Computer Communication Review, vol. 27, no. 2, pp. 24–36, Apr. 1997.

[9] A. E. Mohr, E. A. Riskin and R. E. Ladner, “Unequal loss protection: Graceful degradation of image quality over packet erasure channels through forward error correction,” IEEE Journal on Selected Areas in Communications, vol. 18, no. 6, pp. 819–828, Jun. 2000.

[10] W. Jiang and H. Schulzrinne, “Comparison and optimization of packet loss repair methods on VoIP perceived quality under bursty loss,” in Proc. 12th Int. Workshop on Network and Operating Systems Support for Digital Audio and Video (NOSSDAV), Miami Beach, FL, USA, 2002, pp. 73–81.

[11] J.-M. Valin, K. Vos and T. Terriberry, “Definition of the Opus Audio Codec,” IETF RFC 6716, Sep. 2012.

[12] M. Zanaty, V. Singh, A. Begen and G. Mandyam, “RTP Payload Format for Flexible Forward Error Correction (FEC),” IETF RFC 8627, Jul. 2019.

[13] R. Ramjee, J. Kurose, D. Towsley and H. Schulzrinne, “Adaptive playout mechanisms for packetized audio applications in wide-area networks,” in Proc. IEEE INFOCOM, Toronto, Canada, 1994, pp. 680–688.

[14] Y. J. Liang, N. Färber and B. Girod, “Adaptive playout scheduling and loss concealment for voice communication over IP networks,” IEEE Transactions on Multimedia, vol. 5, no. 4, pp. 532–543, Dec. 2003.

[15] ITU-T Recommendation G.711 Appendix I, “A high quality low-complexity algorithm for packet loss concealment with G.711,” International Telecommunication Union, Geneva, Sep. 1999.

[16] E. N. Gilbert, “Capacity of a burst-noise channel,” Bell System Technical Journal, vol. 39, no. 5, pp. 1253–1265, Sep. 1960.

[17] E. O. Elliott, “Estimates of error rates for codes on burst-noise channels,” Bell System Technical Journal, vol. 42, no. 5, pp. 1977–1997, Sep. 1963.

[18] A. D. Clark, “Modeling the effects of burst packet loss and recency on subjective voice quality,” in Proc. IP Telephony Workshop, Columbia University, New York, NY, USA, Apr. 2001.

[19] L. Sun and E. C. Ifeachor, “Voice quality prediction models and their application in VoIP networks,” IEEE Transactions on Multimedia, vol. 8, no. 4, pp. 809–820, Aug. 2006.

[20] M. Baugher, D. McGrew, M. Naslund, E. Carrara and K. Norrman, “The Secure Real-time Transport Protocol (SRTP),” IETF RFC 3711, Mar. 2004.

[21] D. McGrew and E. Rescorla, “Datagram Transport Layer Security (DTLS) Extension to Establish Keys for the Secure Real-time Transport Protocol (SRTP),” IETF RFC 5764, May 2010.

[22] E. Rescorla and N. Modadugu, “Datagram Transport Layer Security Version 1.2,” IETF RFC 6347, Jan. 2012.

[23] J. Iyengar and M. Thomson (Eds.), “QUIC: A UDP-Based Multiplexed and Secure Transport,” IETF RFC 9000, May 2021.

[24] S. Jarecki, H. Krawczyk and J. Xu, “OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks,” in Advances in Cryptology – EUROCRYPT 2018, Lecture Notes in Computer Science, vol. 10822, Springer, 2018, pp. 456–486.

[25] A. Langley, M. Hamburg and S. Turner, “Elliptic Curves for Security,” IETF RFC 7748, Jan. 2016.

[26] Y. Nir and A. Langley, “ChaCha20 and Poly1305 for IETF Protocols,” IETF RFC 8439, Jun. 2018.

[27] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” IETF RFC 5869, May 2010.

[28] K. Moriarty, B. Kaliski and A. Rusch, “PKCS #5: Password-Based Cryptography Specification Version 2.1,” IETF RFC 8018, Jan. 2017.

[29] K. Nichols, S. Blake, F. Baker and D. Black, “Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers,” IETF RFC 2474, Dec. 1998.

[30] B. Davie et al., “An Expedited Forwarding PHB (Per-Hop Behavior),” IETF RFC 3246, Mar. 2002.

[31] J. Heinanen, F. Baker, W. Weiss and J. Wroclawski, “Assured Forwarding PHB Group,” IETF RFC 2597, Jun. 1999.

[32] S. Floyd and V. Jacobson, “Random early detection gateways for congestion avoidance,” IEEE/ACM Transactions on Networking, vol. 1, no. 4, pp. 397–413, Aug. 1993.

[33] D. Andersen, H. Balakrishnan, F. Kaashoek and R. Morris, “Resilient overlay networks,” in Proc. 18th ACM Symposium on Operating Systems Principles (SOSP), Banff, Canada, 2001, pp. 131–145.

[34] G. Malkin, “RIP Version 2,” IETF RFC 2453, Nov. 1998.

[35] J. Rosenberg et al., “SIP: Session Initiation Protocol,” IETF RFC 3261, Jun. 2002.

[36] Interactive Multimedia Association, “Recommended Practices for Enhancing Digital Audio Compatibility in Multimedia Systems,” Revision 3.00, Oct. 1992 (IMA ADPCM).

[37] H. Schulzrinne and S. Casner, “RTP Profile for Audio and Video Conferences with Minimal Control,” IETF RFC 3551, Jul. 2003.

[38] R. Braden, D. Borman and C. Partridge, “Computing the Internet Checksum,” IETF RFC 1071, Sep. 1988.

[39] R. W. Hamming, “Error detecting and error correcting codes,” Bell System Technical Journal, vol. 29, no. 2, pp. 147–160, Apr. 1950.

[40] S. Ramírez, “FastAPI,” https://fastapi.tiangolo.com, and Encode, “Uvicorn,” https://www.uvicorn.org (accessed Oct. 2026).

[41] Python Cryptographic Authority, “pyca/cryptography,” https://cryptography.io (accessed Oct. 2026).

[42] Chart.js contributors, “Chart.js,” version 4.4.1, https://www.chartjs.org (accessed Oct. 2026).

[43] V. Jacobson, “Congestion avoidance and control,” in Proc. ACM SIGCOMM, Stanford, CA, USA, 1988, pp. 314–329.

[44] ITU-T Recommendation G.113 Appendix I, “Provisional planning values for the equipment impairment factor Ie and packet-loss robustness factor Bpl,” International Telecommunication Union, Geneva, 2009.

[45] Python Software Foundation, “The Python Language Reference, version 3.14,” https://docs.python.org/3.14/ (accessed Oct. 2026).

[46] J. Postel, “User Datagram Protocol,” IETF RFC 768, Aug. 1980.

[47] A. Li (Ed.), “RTP Payload Format for Generic Forward Error Correction,” IETF RFC 5109, Dec. 2007.

Appendix A Source Code Repository

The complete project — protocol stack, relay and clients, demonstration harness, experiments, web console, launcher, figures and this report — is published on GitHub:

https://github.com/rohitarora80905-glitch/SWARA_Complete_Project

> **NOTE**
>
> The local working copy that produced every figure and screenshot in this report is `SWARA_Complete_Project/SWARA`.

A.1 Obtaining and running the code

```
git clone https://github.com/rohitarora80905-glitch/SWARA_Complete_Project
cd SWARA_Complete_Project/SWARA
python -m venv .venv && .venv\Scripts\activate          # Windows (use source .venv/bin/activate on
Linux/macOS)
pip install -r requirements.txt

python demo.py --seconds 20                                  # the most reliable demonstration: 0 %
loss, correlation ≈ 0.99
python demo.py --seconds 25 --loss 0.15 --burst 4 --jitter 12 --compare --repeat 2
python experiments/run_all.py                                # regenerates results.json and figures/
in ~3 minutes
python -m uvicorn api_server:app --host 0.0.0.0 --port 8000 # web console at http://localhost:8000,
API docs at /docs
```

A.2 Repository contents

Table 20. Repository contents.

Path	Contents
swara/	The protocol stack: 15 modules, listed in Appendix C
server.py, client.py, demo.py	Relay server, voice-chat client, one-process demonstration and A/B harness
api_server.py, webapp/, static/	Web console back end (FastAPI), instrumented calls and tools, single-page front end
launcher.py, SWARA Server.bat, start_console.bat	Desktop server-control window and one-click launchers
experiments/run_all.py, experiments/results.json	The eight evaluation experiments and their machine-readable results
figures/fig1 … fig8	Research figures reproduced in Chapter 7
users.json	Credential records (salt, envelope, public key) for the registered users
README.md, RUNME.txt, requirements.txt	Documentation, step-by-step run instructions, dependencies

A.3 Reproducibility

- Every simulated figure is regenerated by one command from fixed seeds; `results.json` is the single source of the numbers in Chapter 7 and the console's Research Results page.
- Every live comparison uses the identical-realisation injector (two random streams, per-arm reset, wall-clock stepping) so the three arms face the same sequence of good and bad periods.
- Every screenshot in Appendix B was captured from the running system on the date of this report; terminal outputs were additionally saved as text and are reproduced verbatim in the tables of Section 7.10.

Appendix B Screenshots

All screenshots are actual captures of the system running on the development machine (Windows 11) on the date of this report. Browser pages are full-length captures of the web console at 1600 px width; terminal windows are captured as displayed. Captions give the exact command or action that produced each view.

B.1 Starting the server

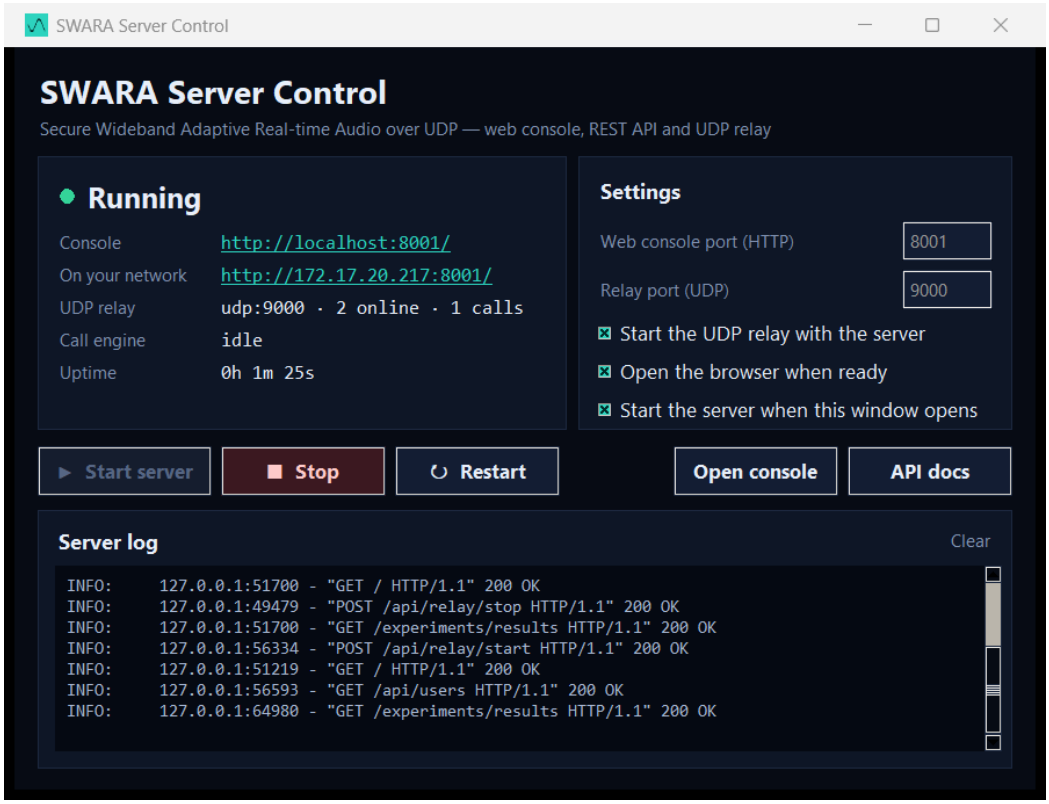


Figure 22. SWARA Server Control, opened by double-clicking “SWARA Server.bat”. The server was started with one click; the window shows the console address, the LAN address for other laptops, the hosted relay on udp:9000 and the live server log.

B.2 A real call between three separate programs — both sides

The classic three-terminal demonstration from the README: the relay in terminal 1, the callee (bob) waiting in terminal 2 and the caller (alice) dialling in terminal 3. The impairment injector was enabled on both clients (15 % loss, mean burst 4, 12 ms jitter) so that each side both suffers and repairs loss. The first three figures were captured during the call; the next three after it ended, when each client printed its own report.

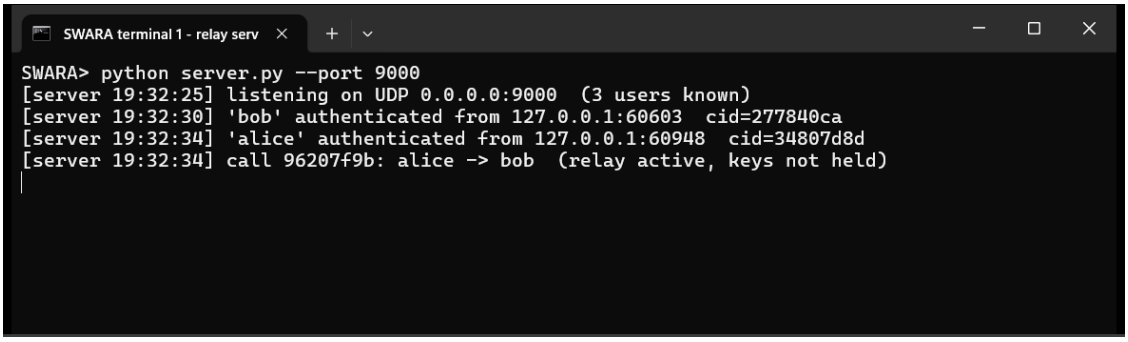


Figure 23. Terminal 1 — the relay (`python server.py --port 9000`) during the call: both users authenticated by SPARK and the call being relayed “keys not held”.

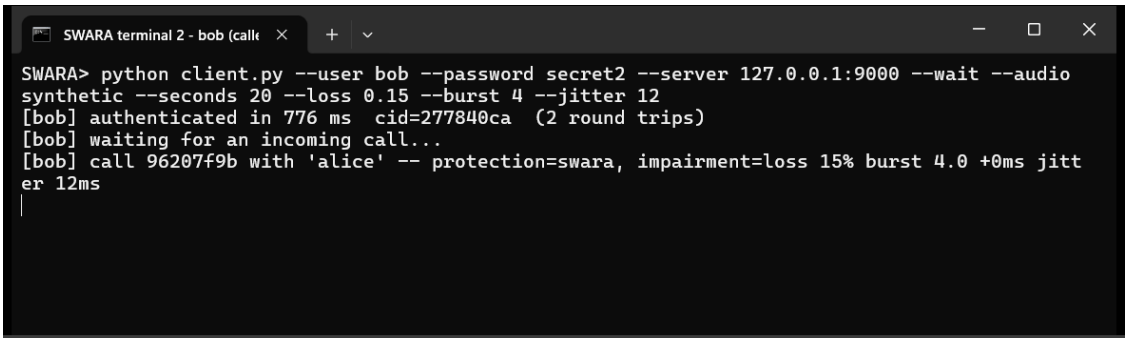


Figure 24. Terminal 2 — the callee (`python client.py --user bob … --wait`) during the call: authenticated in two round trips, incoming call accepted, media flowing.

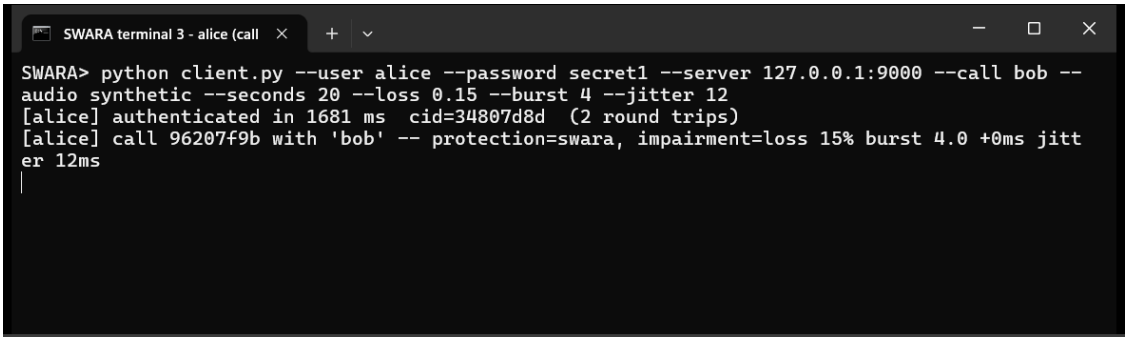


Figure 25. Terminal 3 — the caller (`python client.py --user alice … --call bob`) during the call.

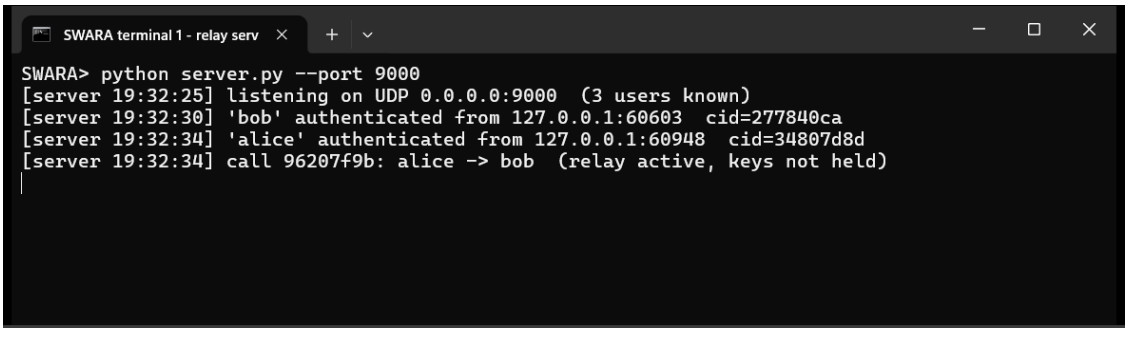


Figure 26. Terminal 1 after the call: the relay's complete log for the session.



Figure 27. Terminal 2 after the call — bob's own report: 1,021 frames sent with 287 parity and 17 replica packets; 1,015 played; 14 rebuilt by FEC; 102 concealed (10.0 %); no authentication failures.

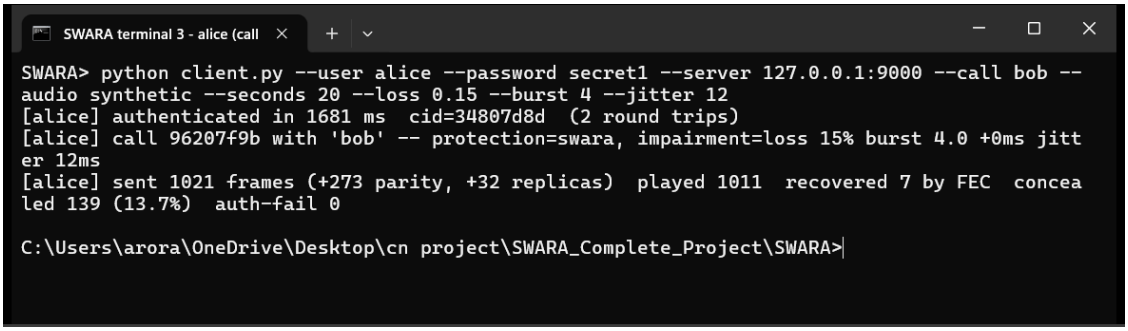


Figure 28. Terminal 3 after the call — alice's own report: 1,021 frames sent with 273 parity and 32 replicas; 1,011 played; 7 rebuilt by FEC; 139 concealed (13.7 %); no authentication failures.

B.3 Command-line clients on the relay hosted by the web console

The same two clients connected to the relay running inside the uvicorn process while the console watched. The Relay & Users page lists both users with their addresses and connection ids, the active call and its two endpoints, and the relay log; the Overview page's relay card shows “2 online · 1 call”.

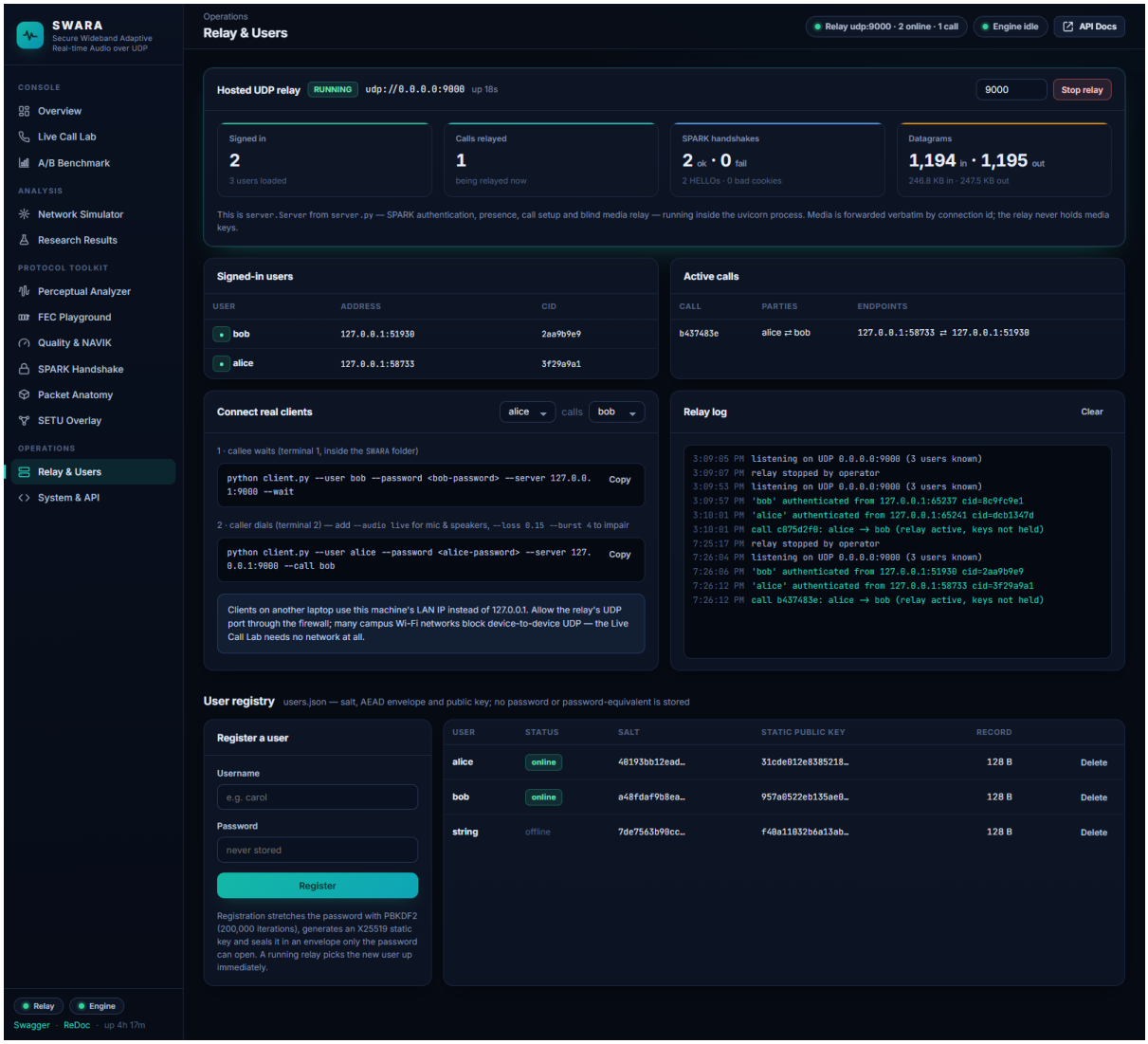


Figure 29. Relay & Users page while alice and bob (command-line clients) are in a call through the hosted relay.

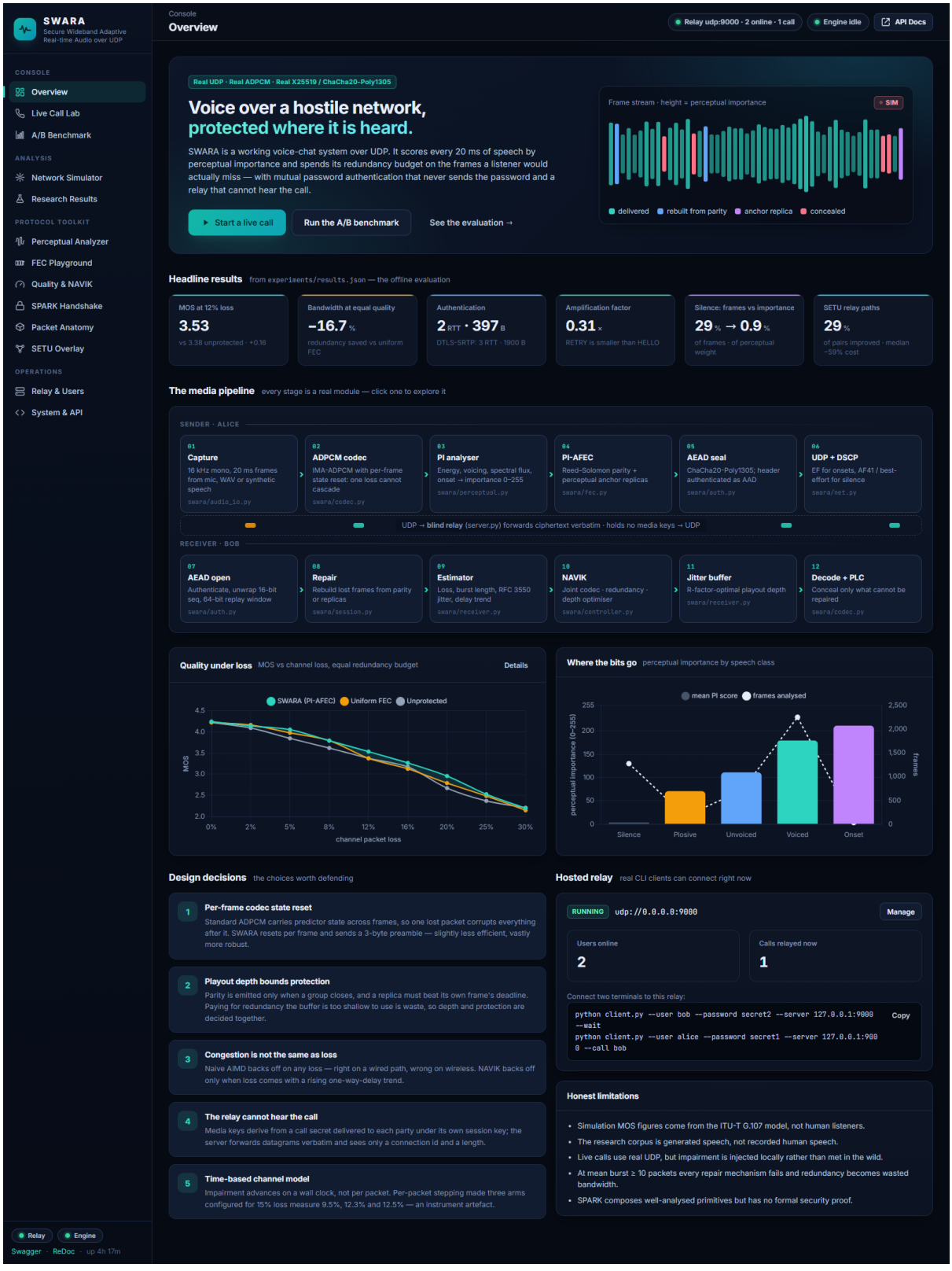


Figure 30. Overview page during the same call: the hosted-relay card reports two users online and one call relayed.

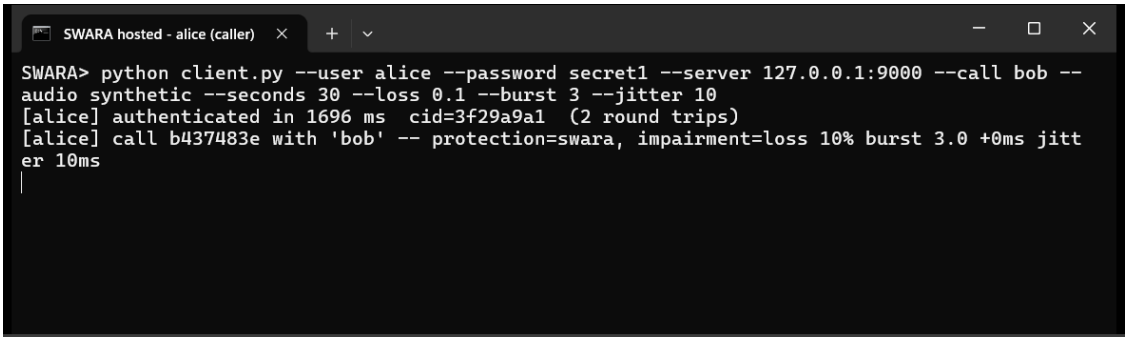


Figure 31. The caller's terminal for the hosted-relay call (30 s, 10 % loss, burst 3, 10 ms jitter on alice's socket).

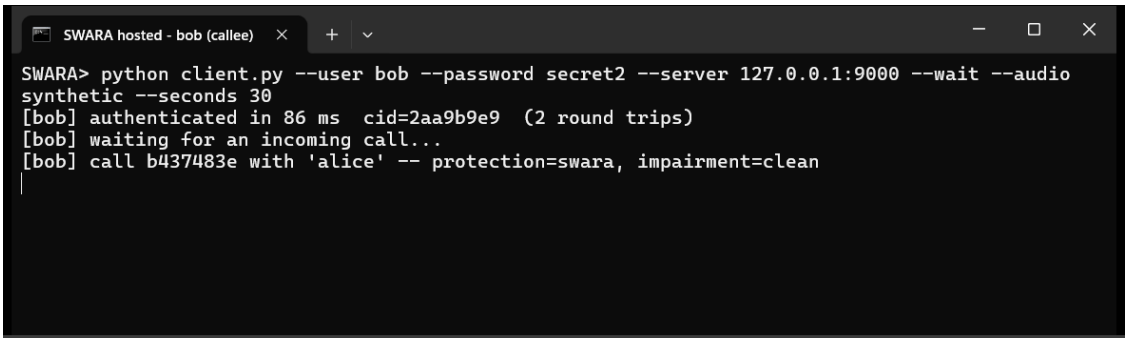


Figure 32. The callee's terminal for the hosted-relay call.

B.4 Live Call Lab

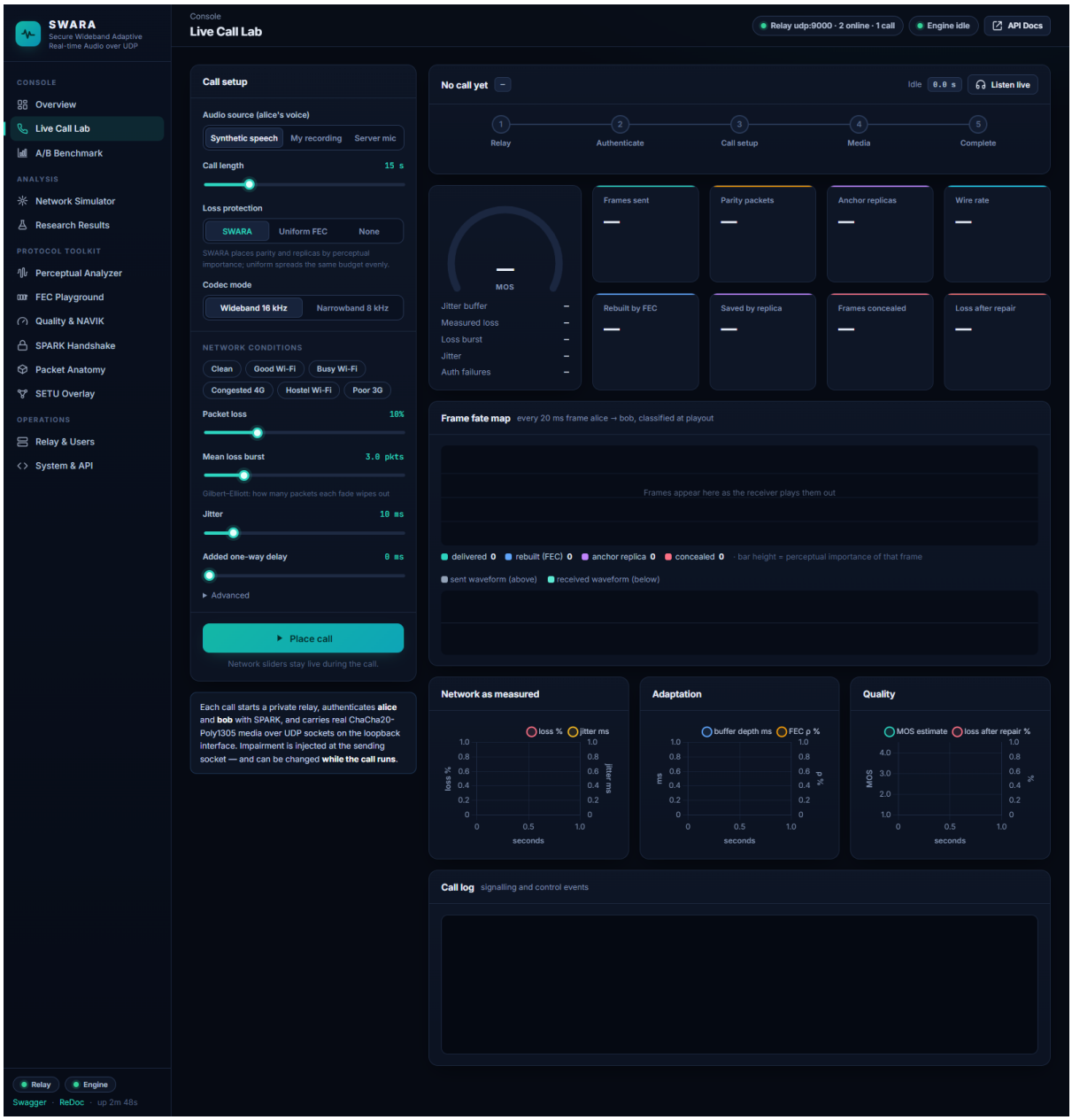


Figure 33. Live Call Lab before a call: audio source, call length, protection scheme, codec mode and the network sliders (presets from Clean to Poor 3G).



Figure 34. Live Call Lab during a call: the frame-fate map fills from left to right (bar height = PI; teal delivered, blue rebuilt from parity, violet anchor replica, red concealed), the sent and received waveforms, and the live gauges.

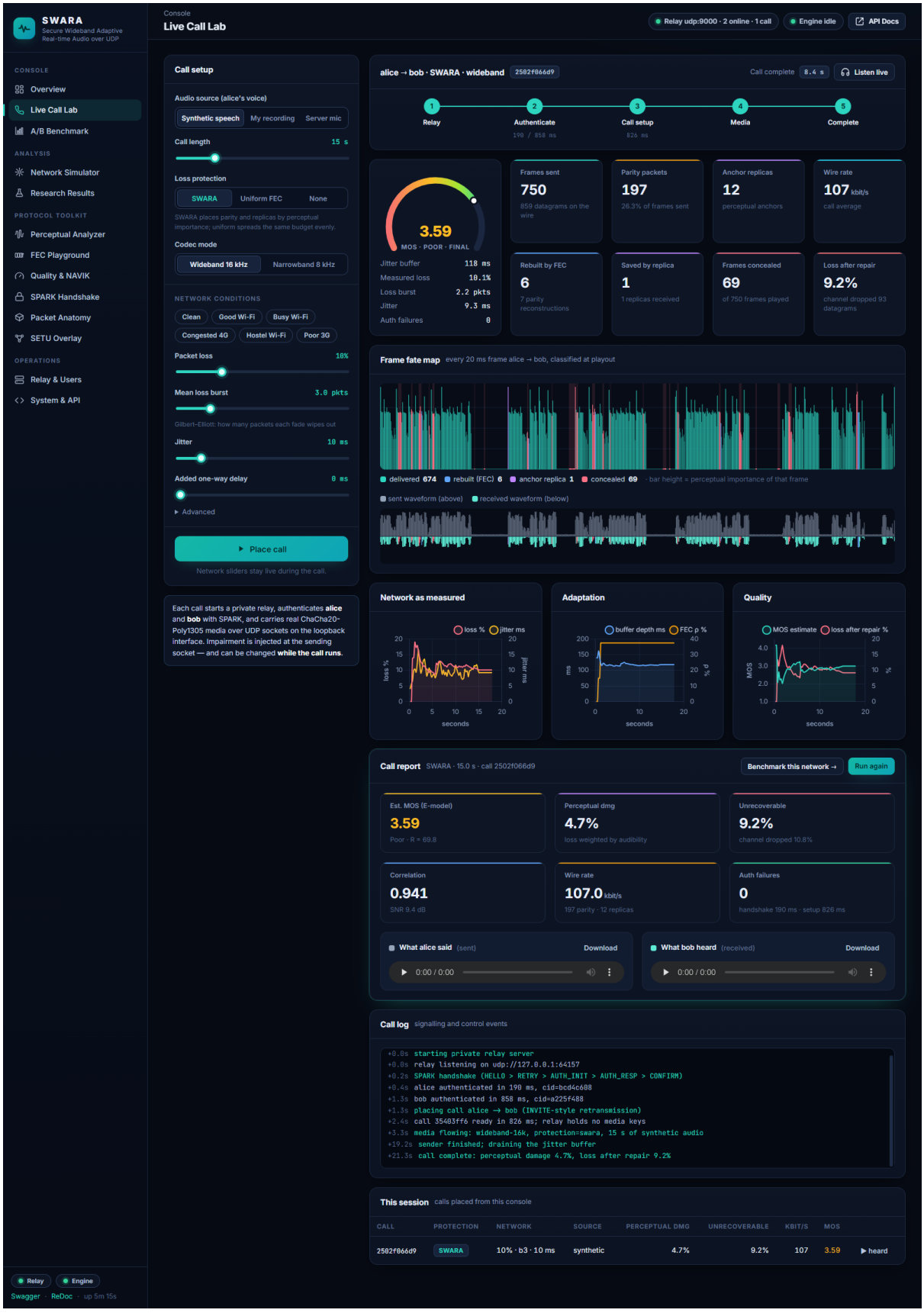


Figure 35. Live Call Lab after the call: the complete frame-fate map, the three time-series charts (network as measured, adaptation, quality), the call report with playable sent and received audio, the call log and the session table.

B.5 A/B Benchmark

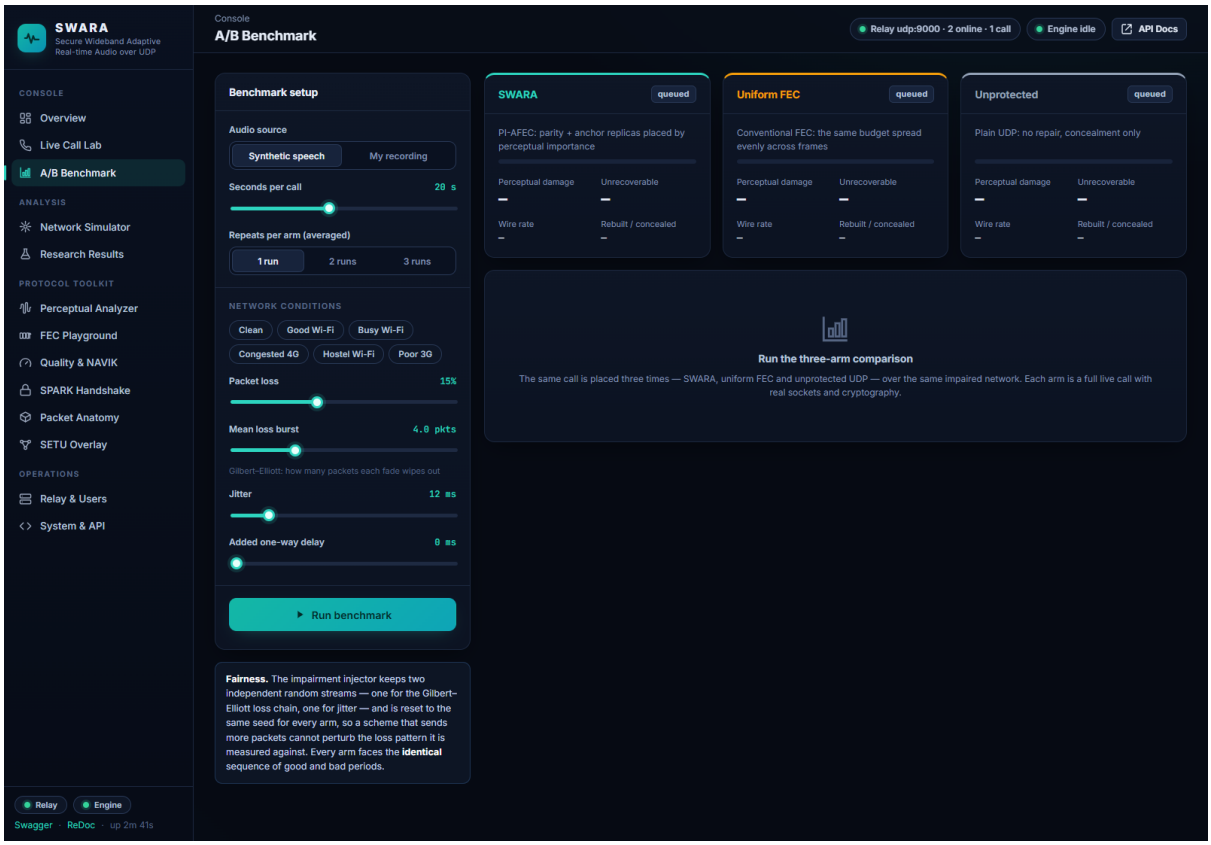


Figure 36. A/B Benchmark page before a run: audio source, seconds per call, repeats, and the network conditions shared by all three arms.

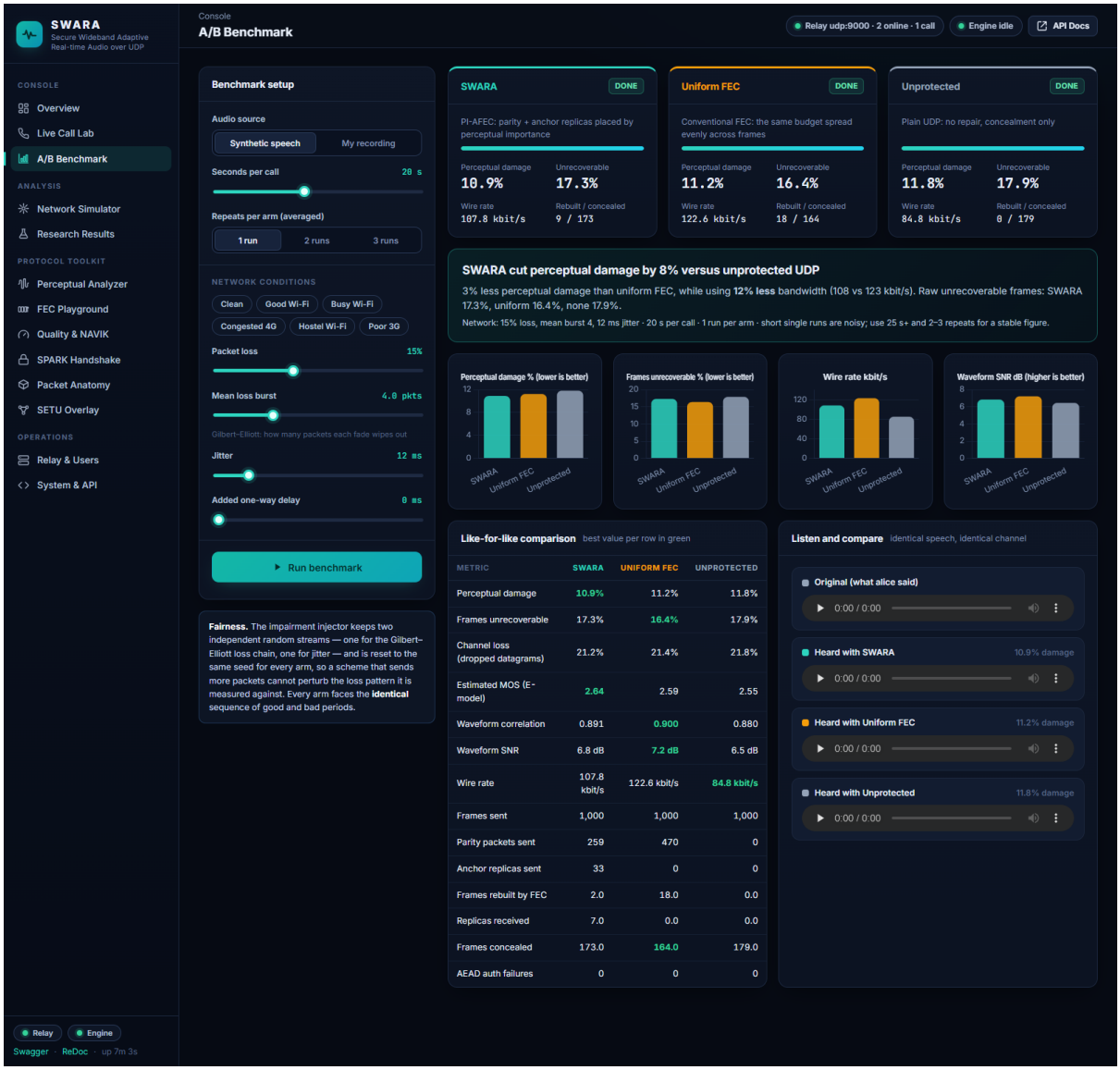


Figure 37. A/B Benchmark after a run at 15 % loss, burst 4, 12 ms jitter (20 s per arm): per-arm cards, the verdict, four comparison charts, the like-for-like table and the audio of each arm.

B.6 Network Simulator

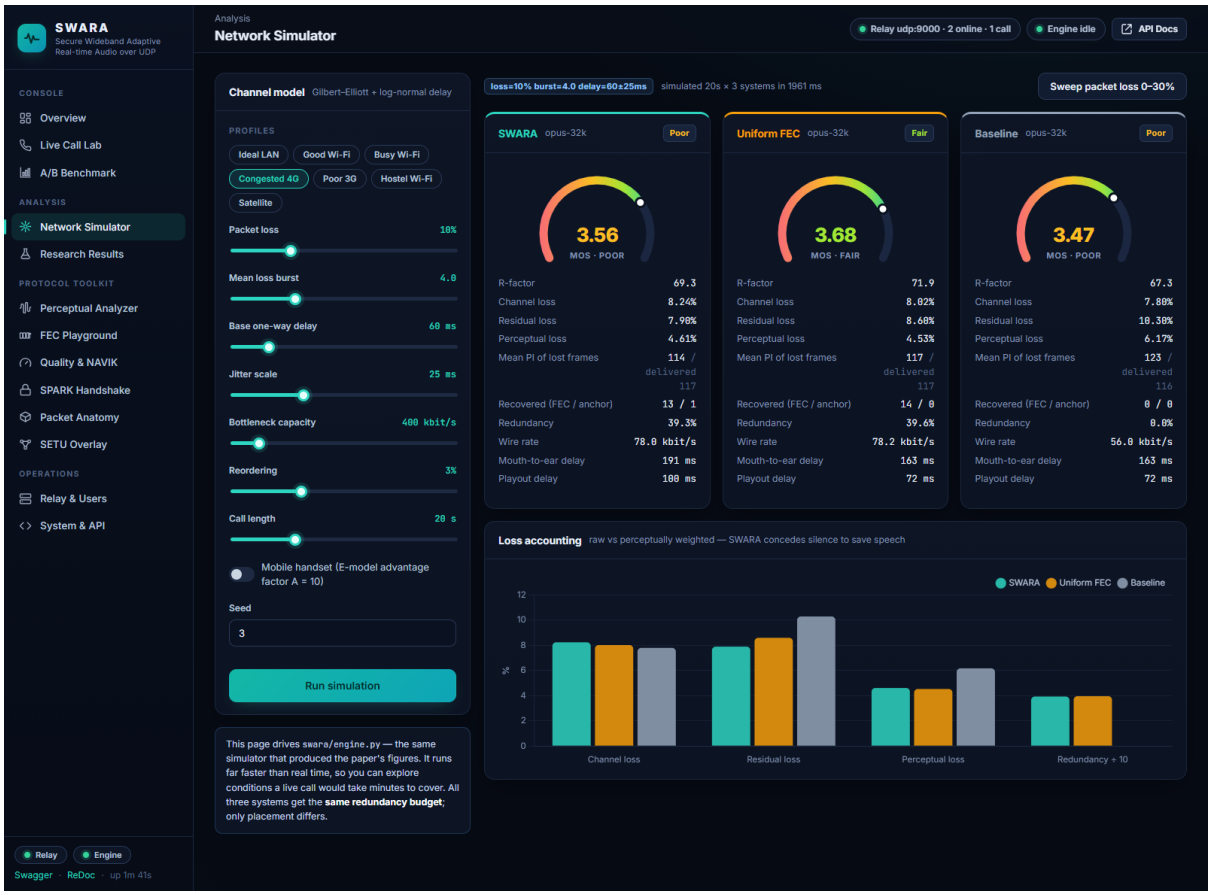


Figure 38. Network Simulator: the Congested 4G profile simulated for SWARA, uniform FEC and the baseline with MOS gauges, detailed per-system figures and the loss-accounting chart.

B.7 Research Results

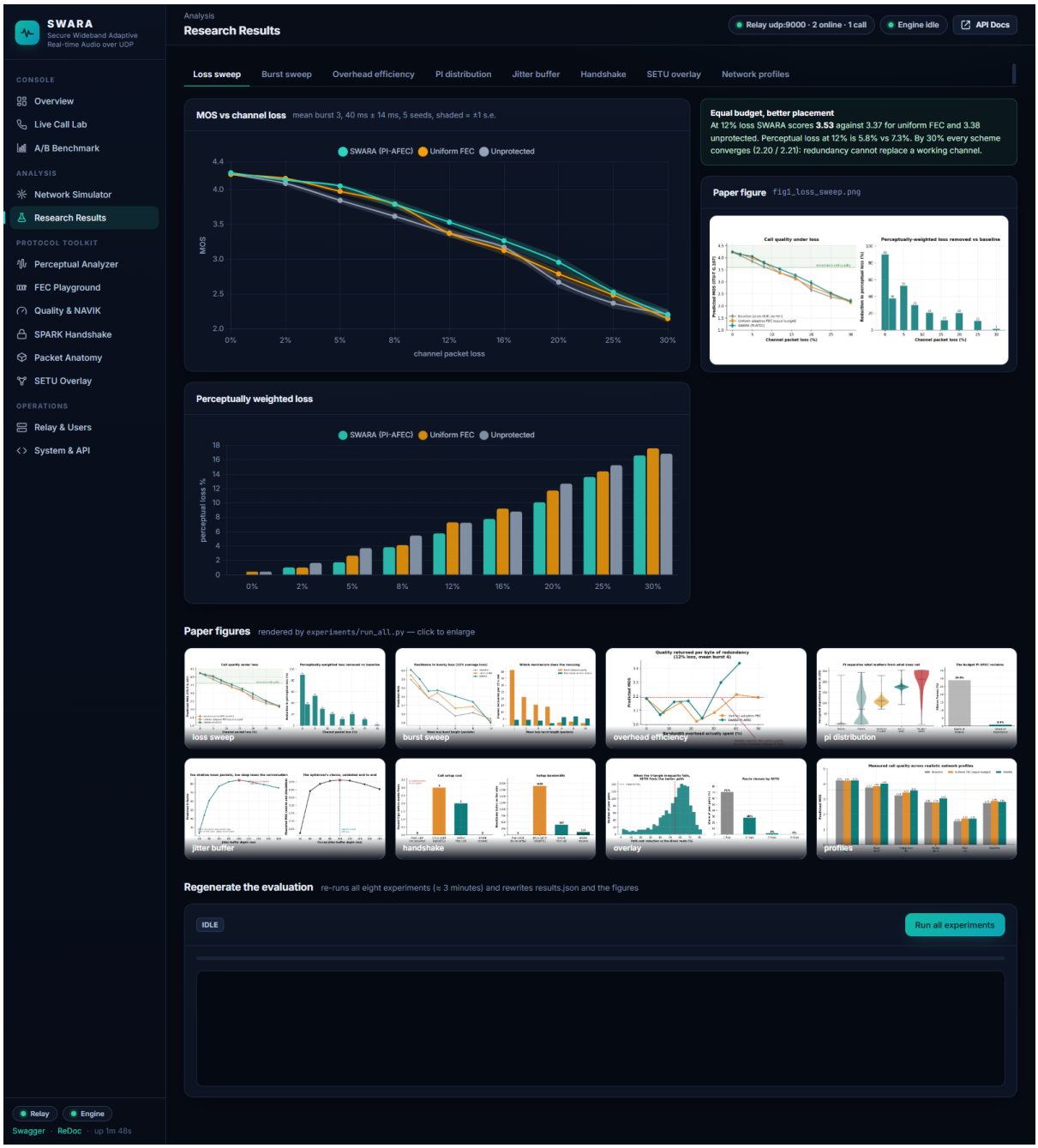


Figure 39. Research Results — loss sweep tab, with the interactive chart, the finding, the paper figure, the figure gallery and the regeneration control.

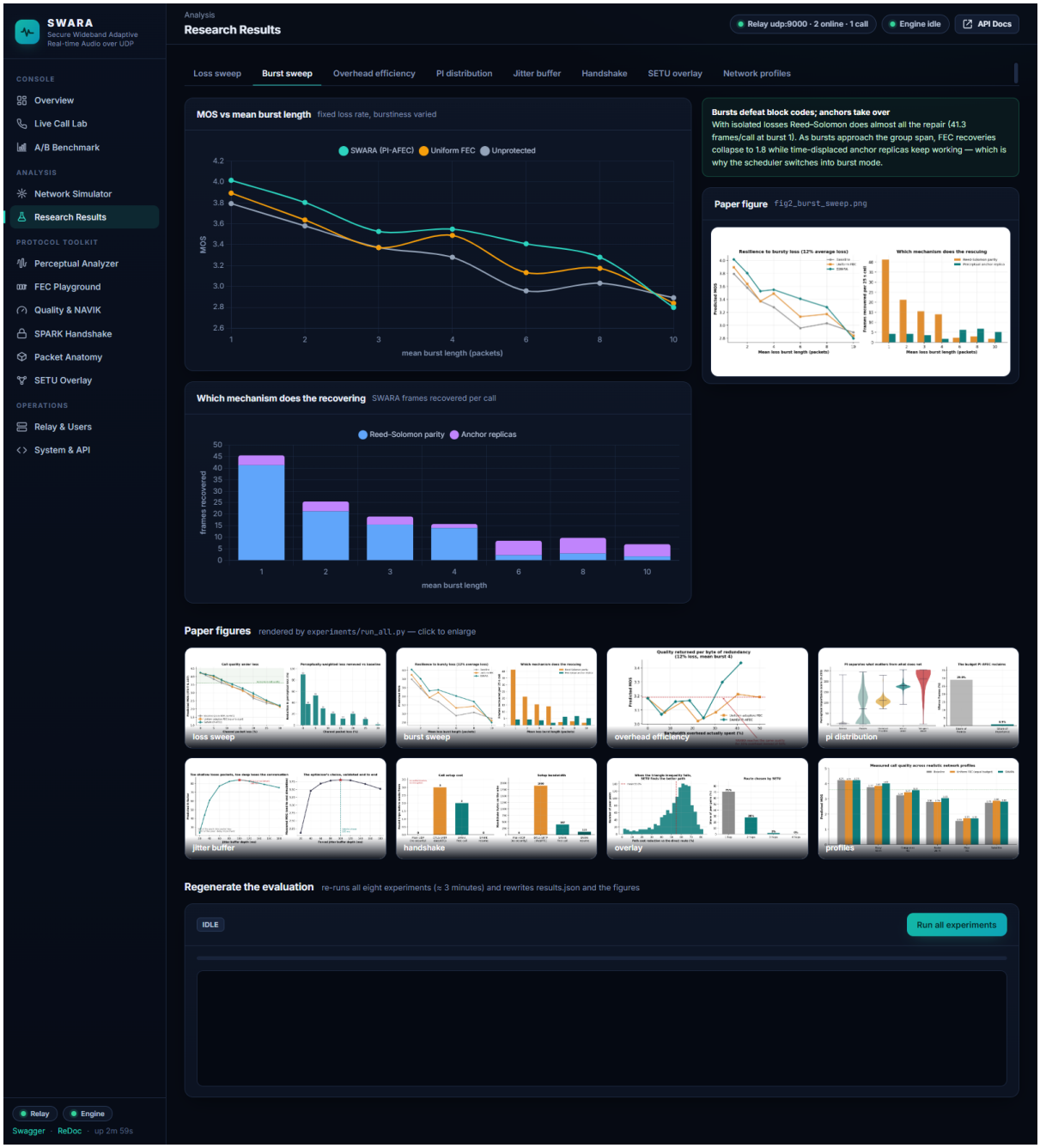


Figure 40. Research Results — burst sweep tab.

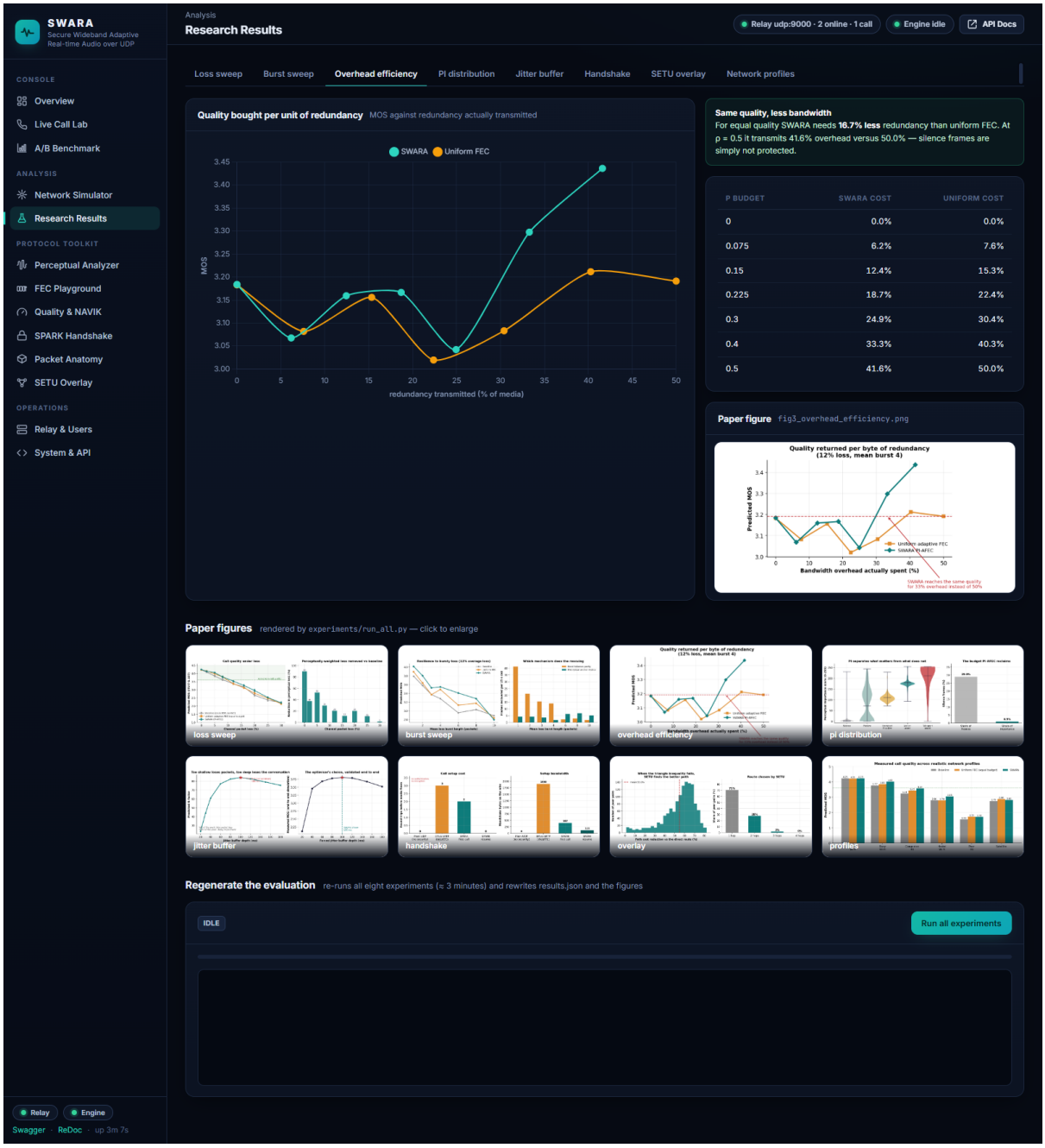


Figure 41. Research Results — overhead efficiency tab.

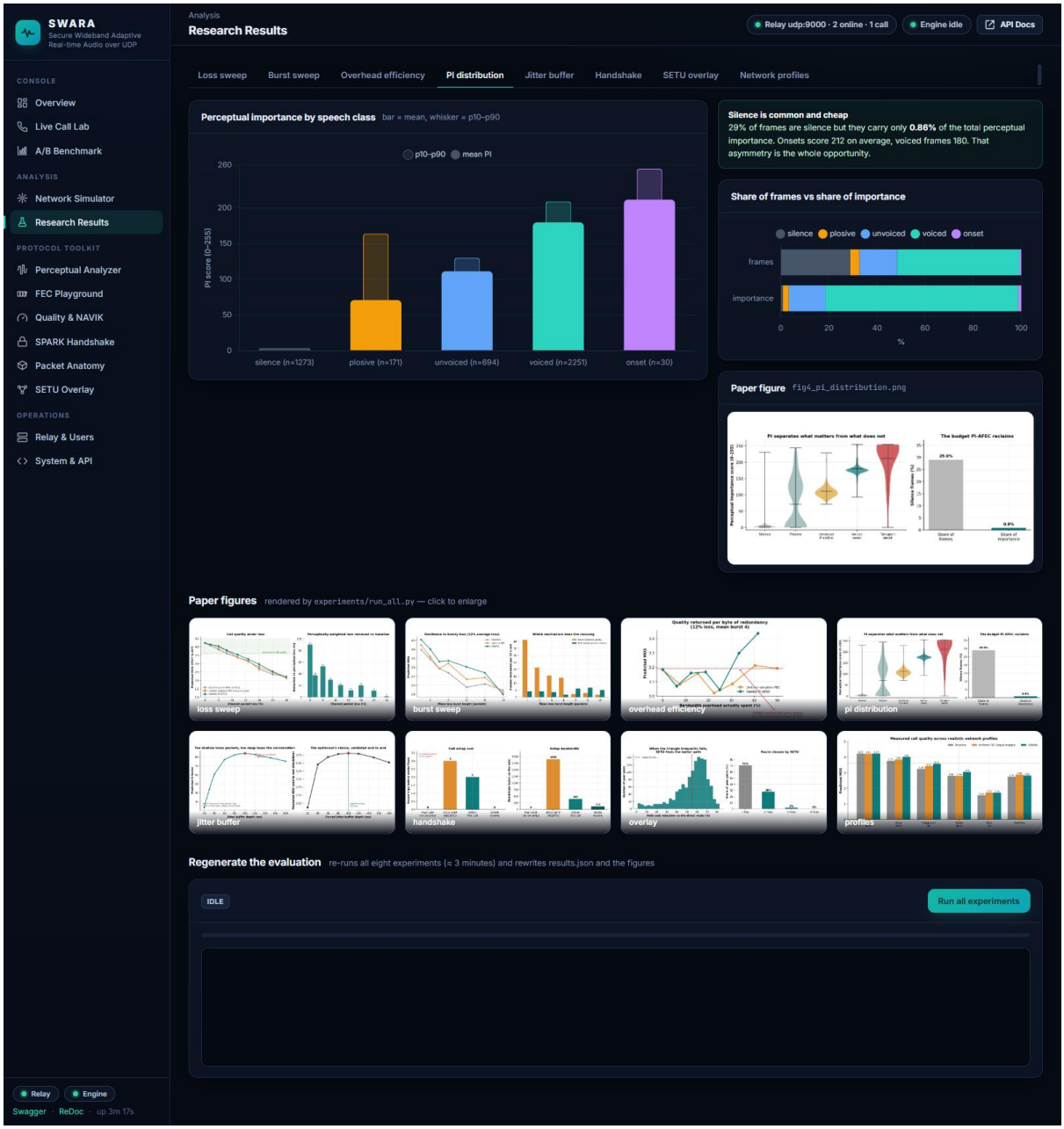


Figure 42. Research Results — PI distribution tab.

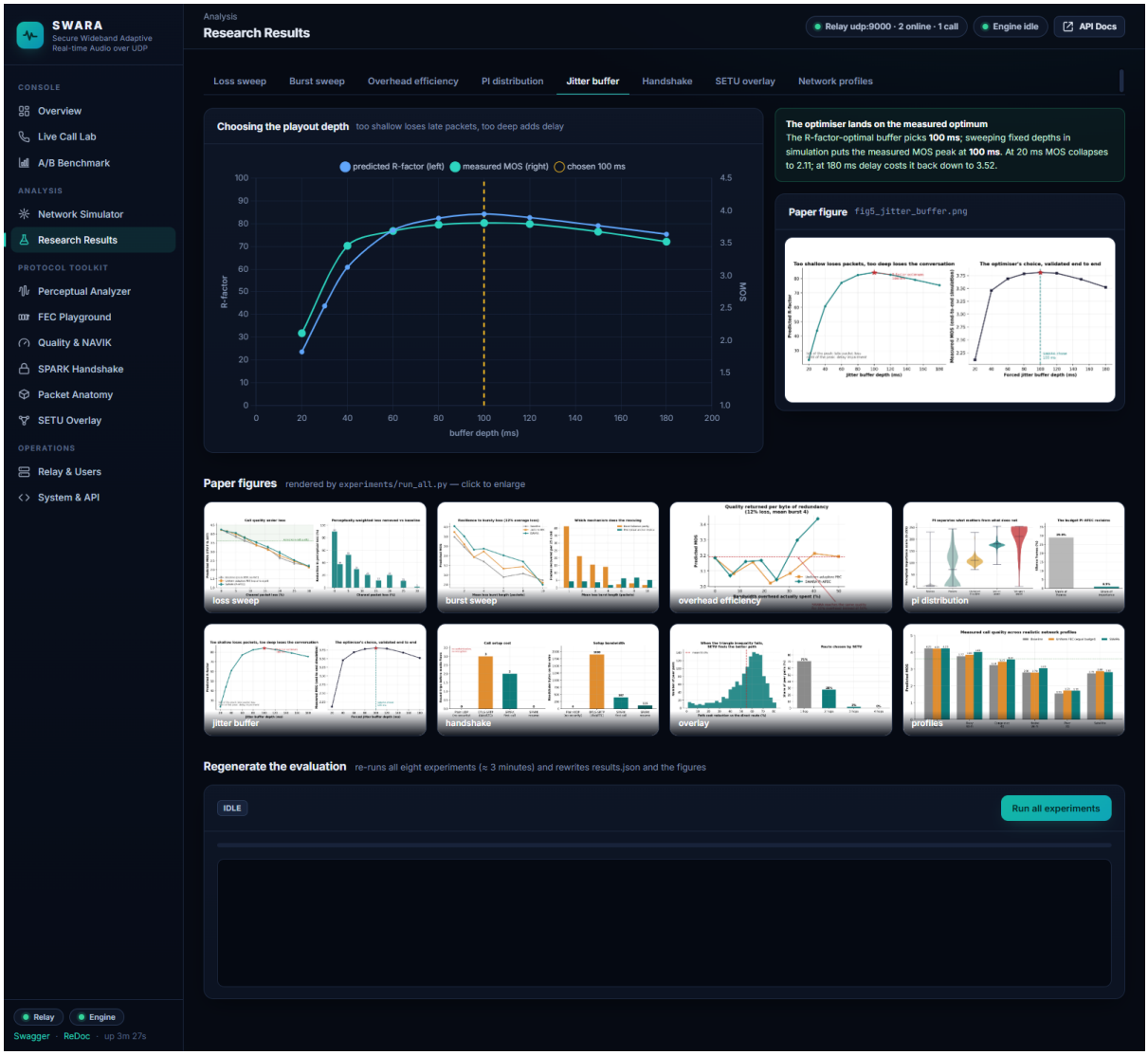


Figure 43. Research Results — jitter buffer tab.

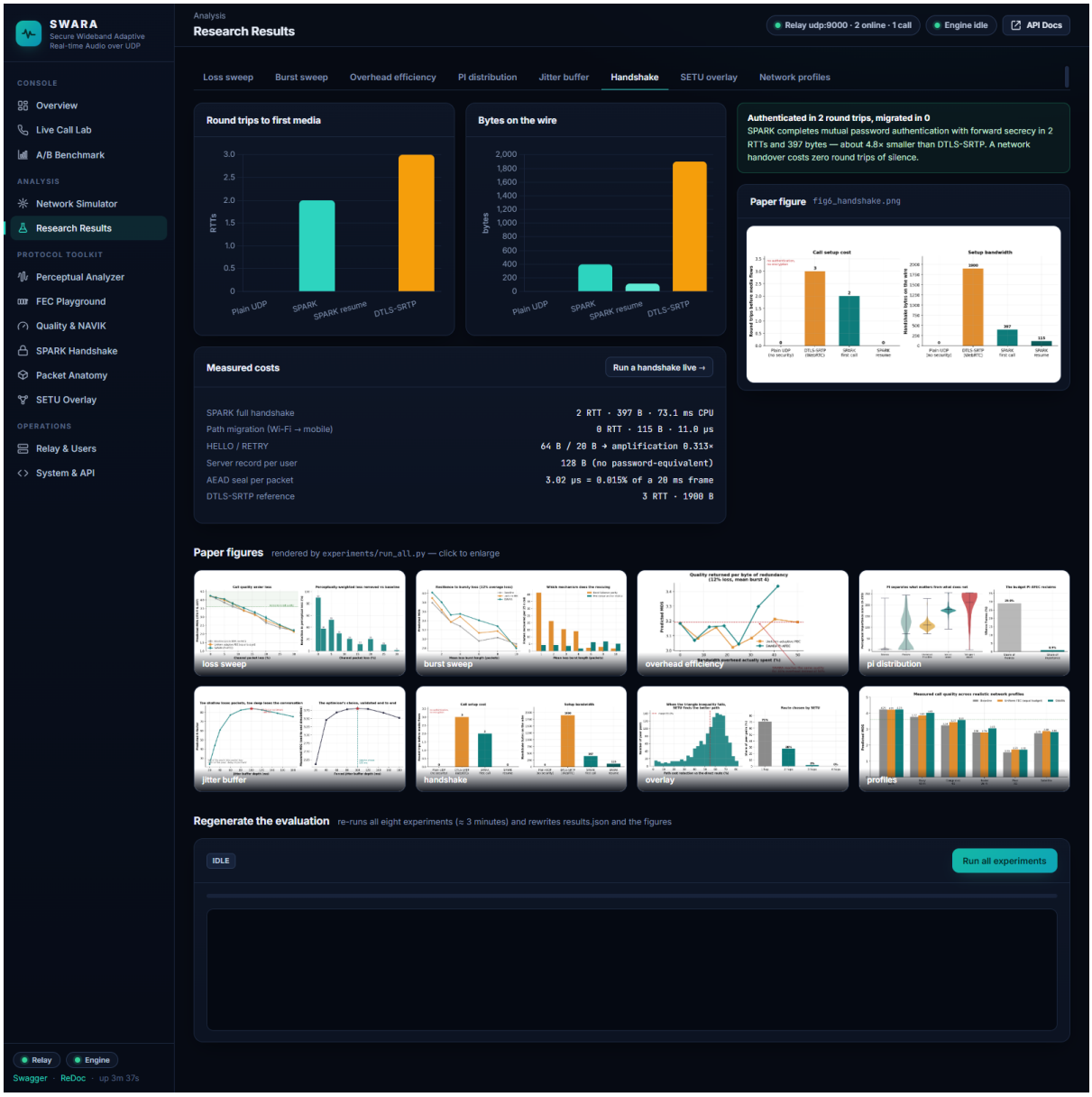


Figure 44. Research Results — handshake tab.

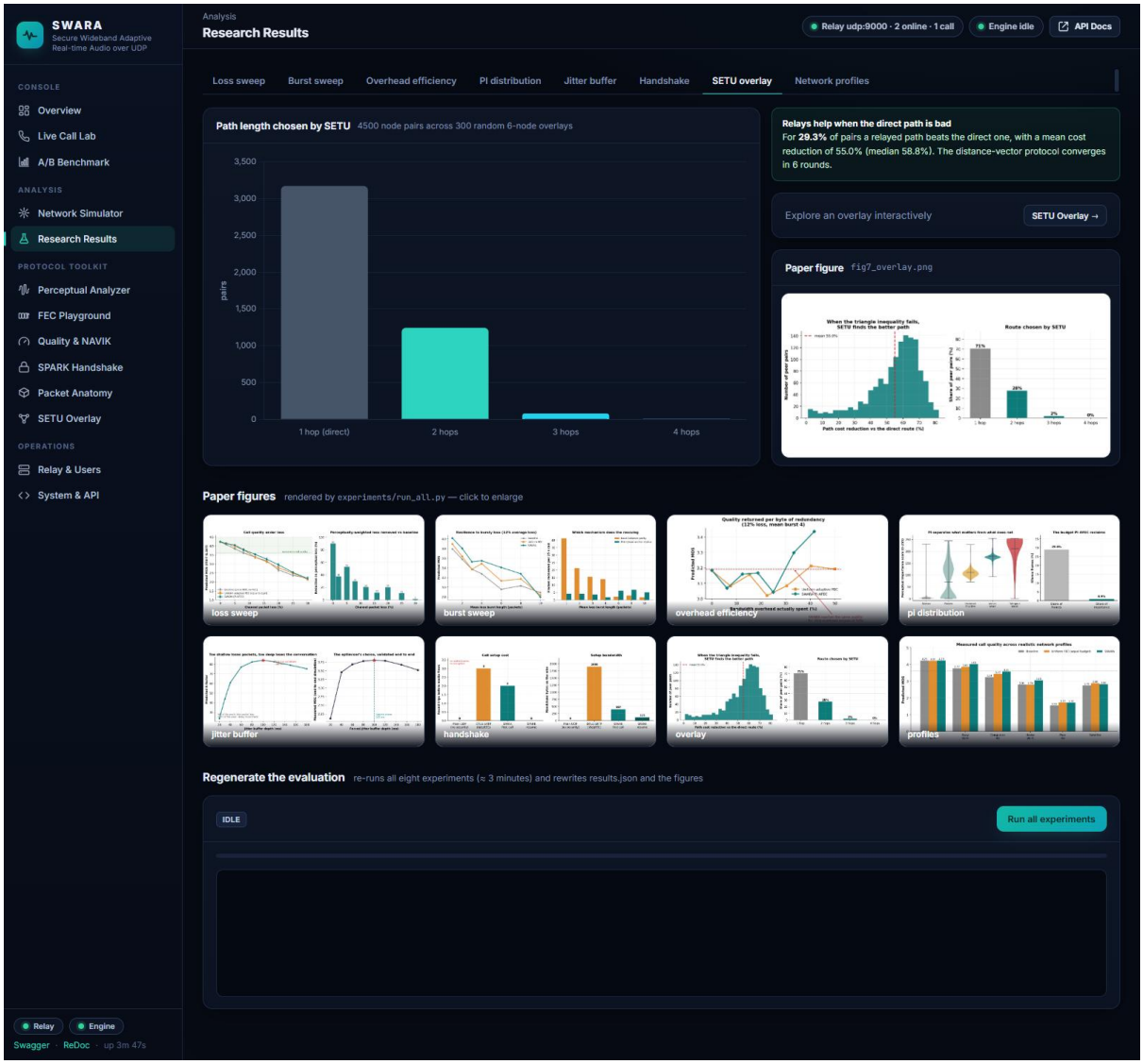


Figure 45. Research Results — SETU overlay tab.

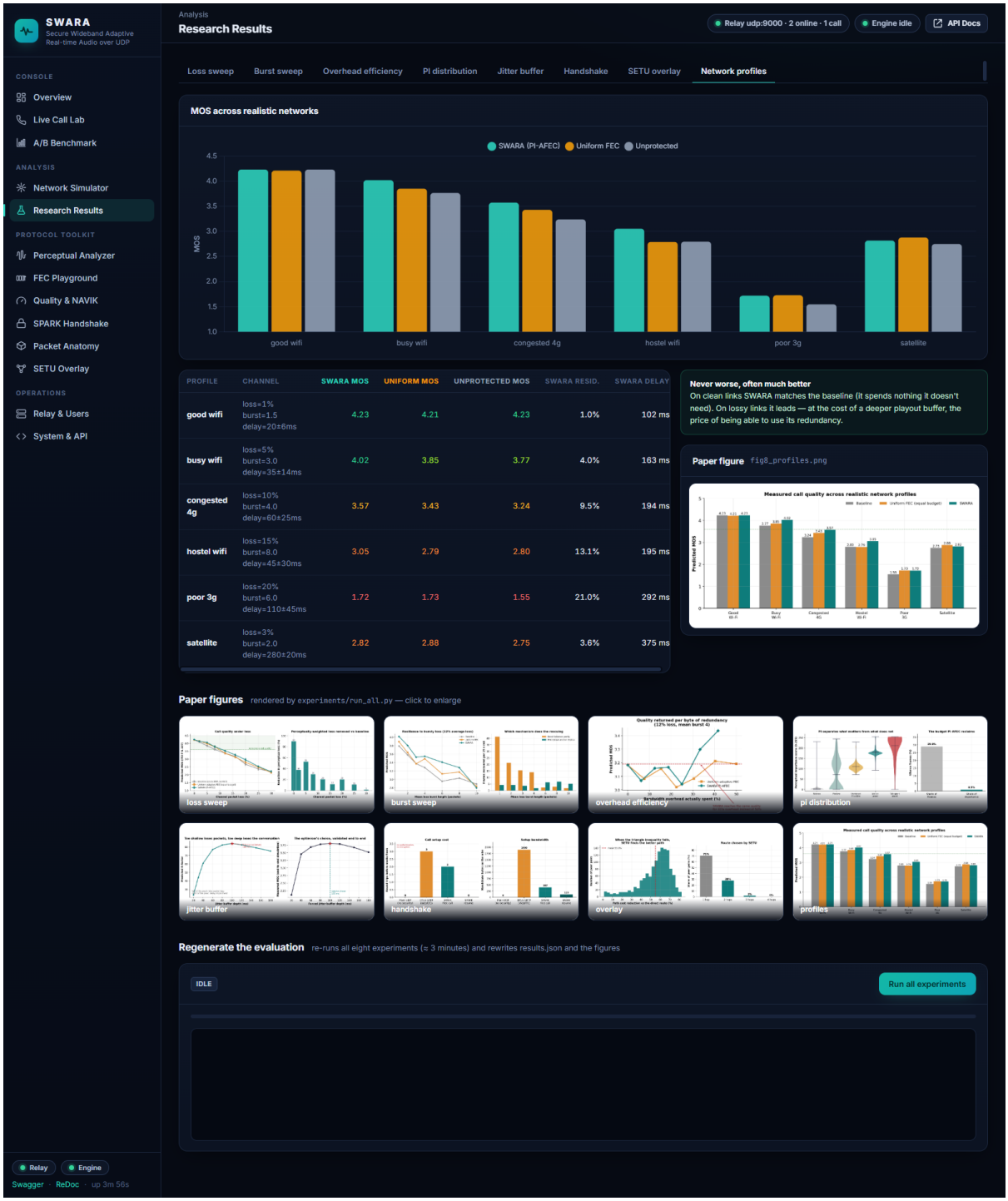


Figure 46. Research Results — network profiles tab.

B.8 Protocol toolkit

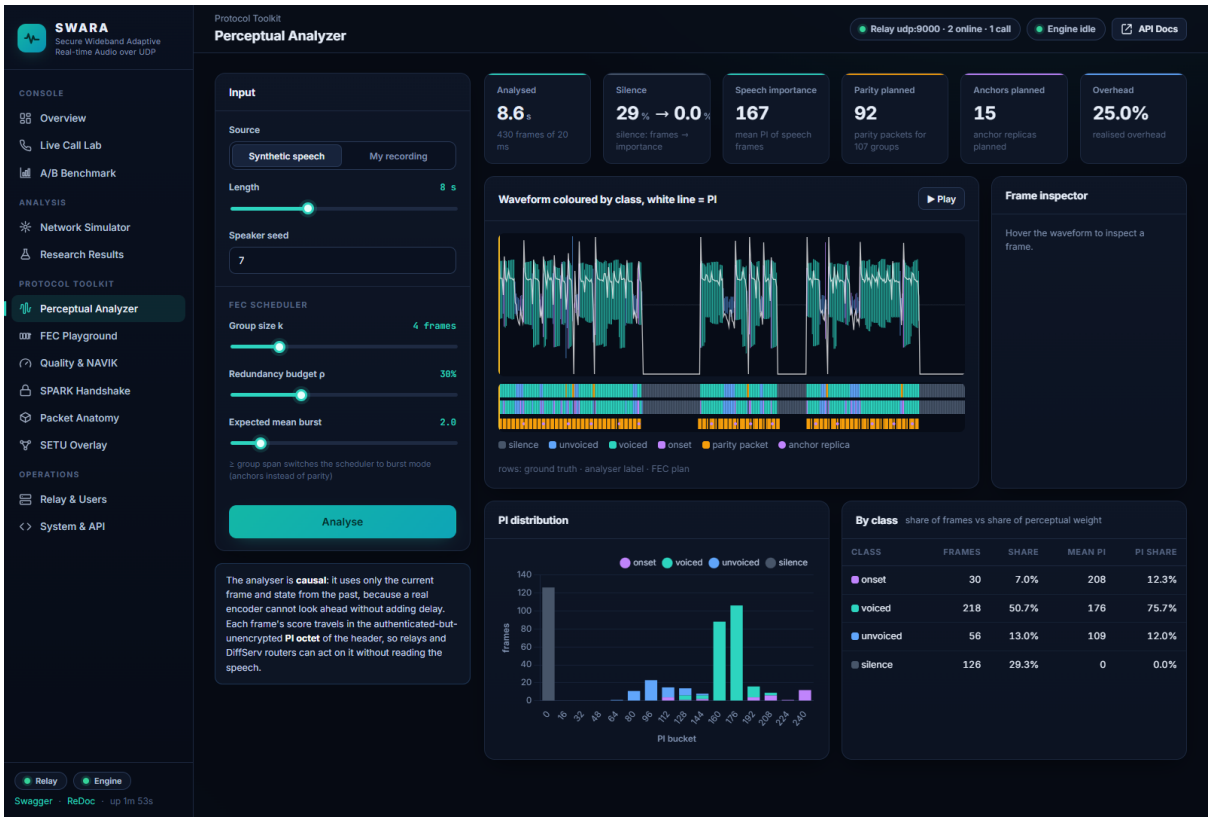


Figure 47. Perceptual Analyzer: synthetic speech coloured by class with the PI trace, the ground-truth / label / FEC-plan strips, the frame inspector, the PI histogram and per-class statistics.

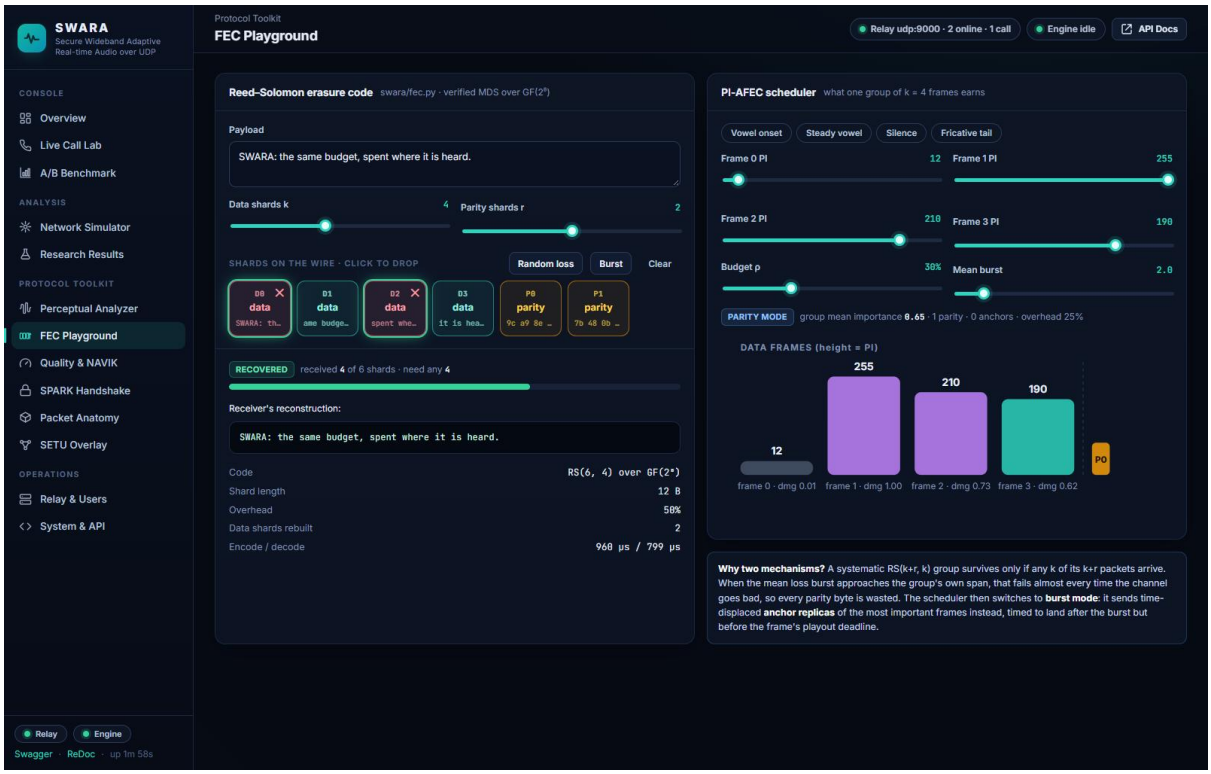


Figure 48. FEC Playground: an RS(6, 4) group with two data shards dropped and recovered; the PI-AFEC scheduler for a vowel-onset group.

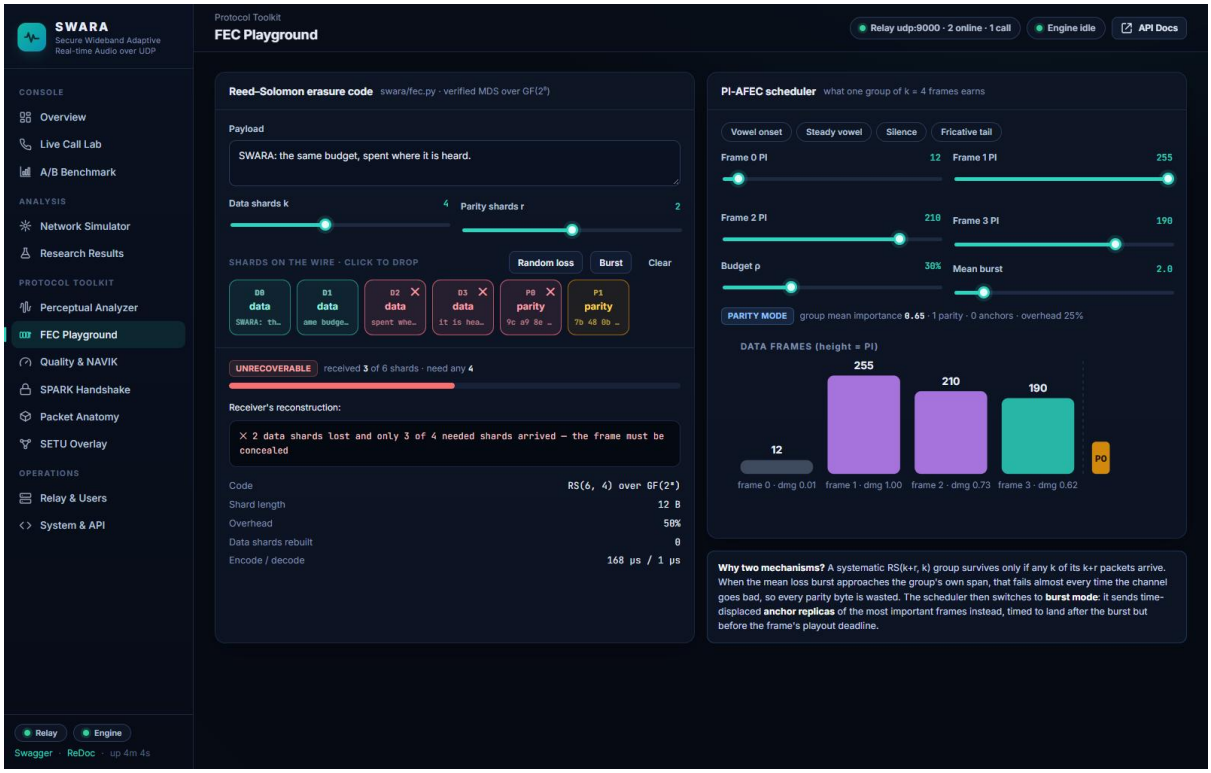


Figure 49. FEC Playground after the "Burst" button: more shards lost than the code can repair — the frame must be concealed.



Figure 50. Quality & NAVIK: the E-model calculator with its impairment breakdown and the controller's decisions across loss rates.

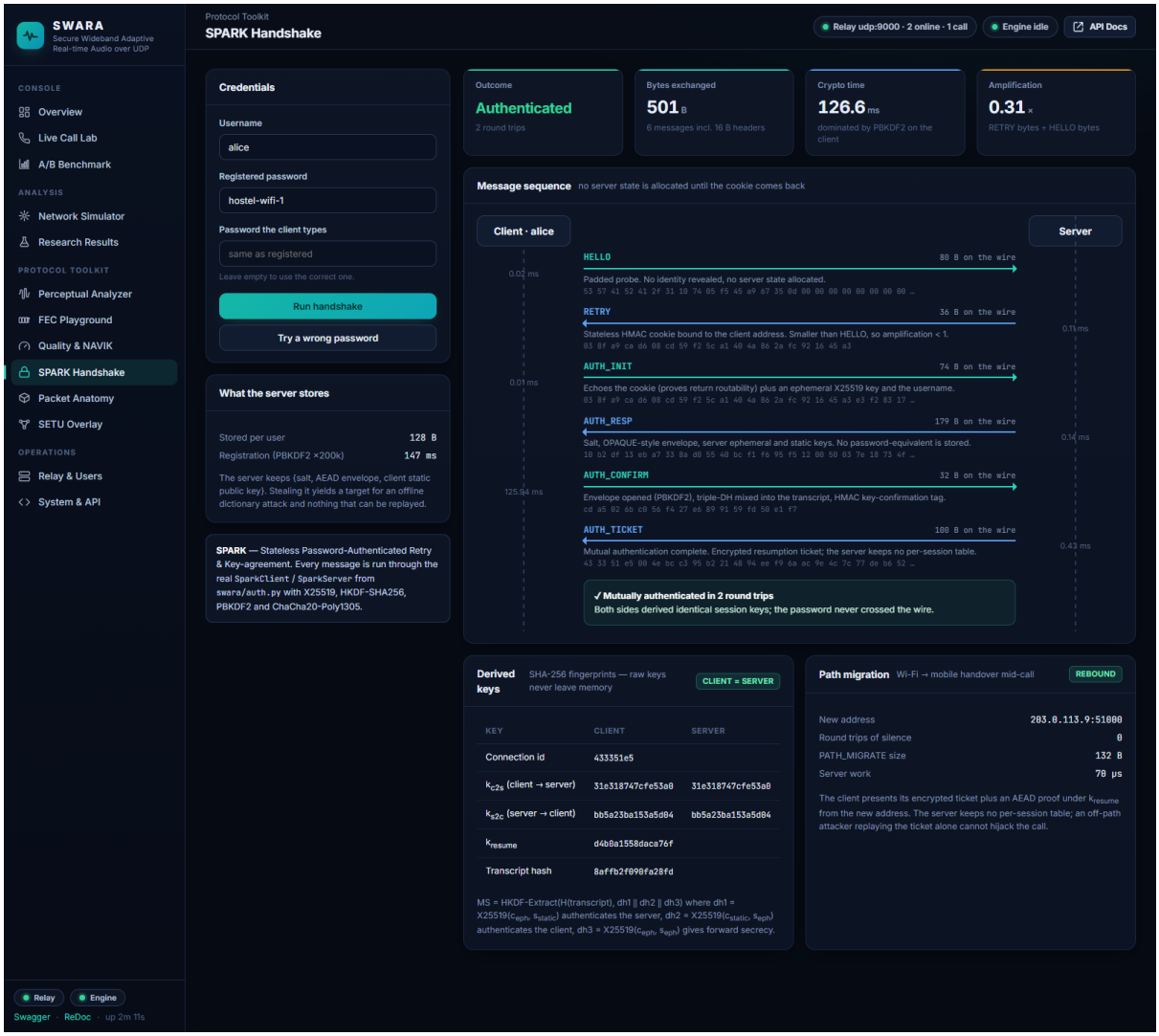


Figure 51. SPARK Handshake: the six messages with sizes and timings, the derived keys matching on both sides, and the 0-RTT path migration.

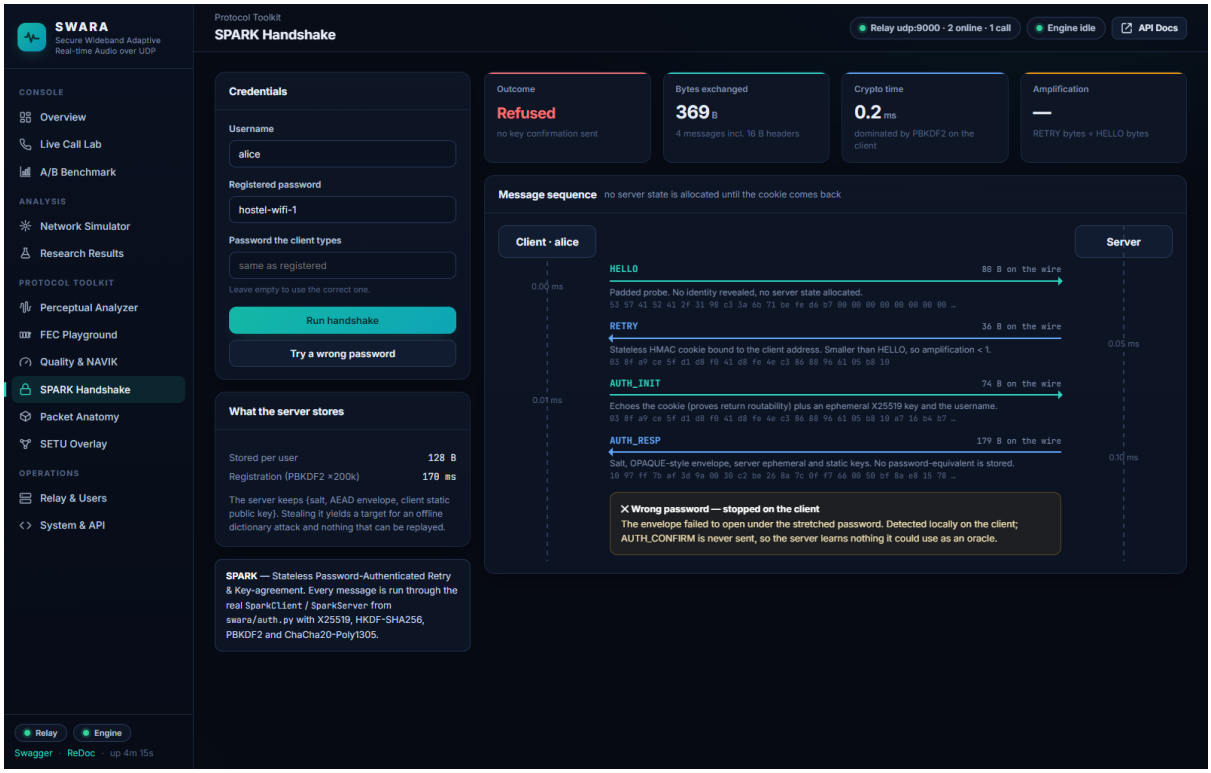


Figure 52. SPARK Handshake with a wrong password: the envelope fails to open locally and AUTH_CONFIRM is never sent.

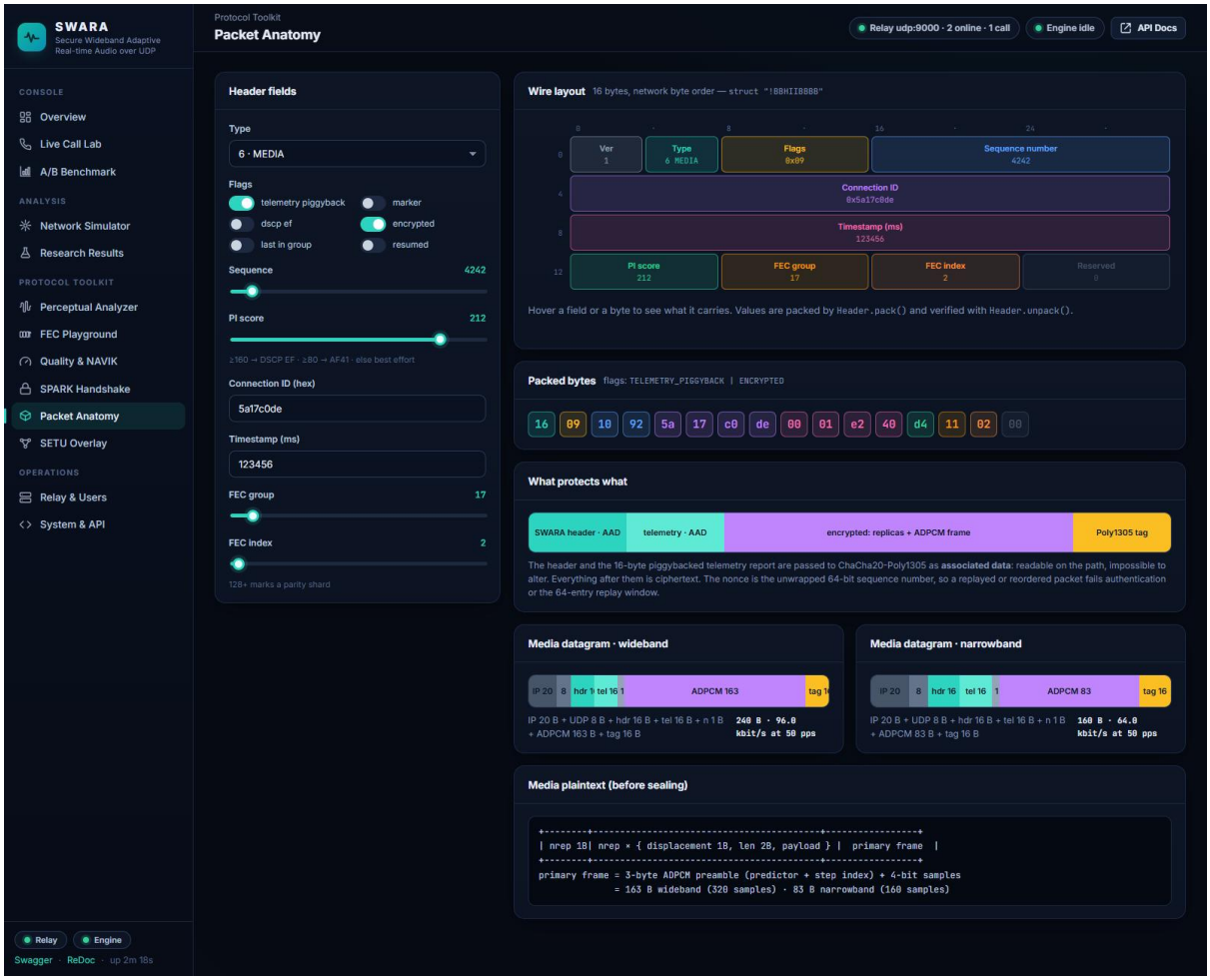


Figure 53. Packet Anatomy: the 16-byte header packed by Header.pack(), the hex dump, and what the AEAD protects.

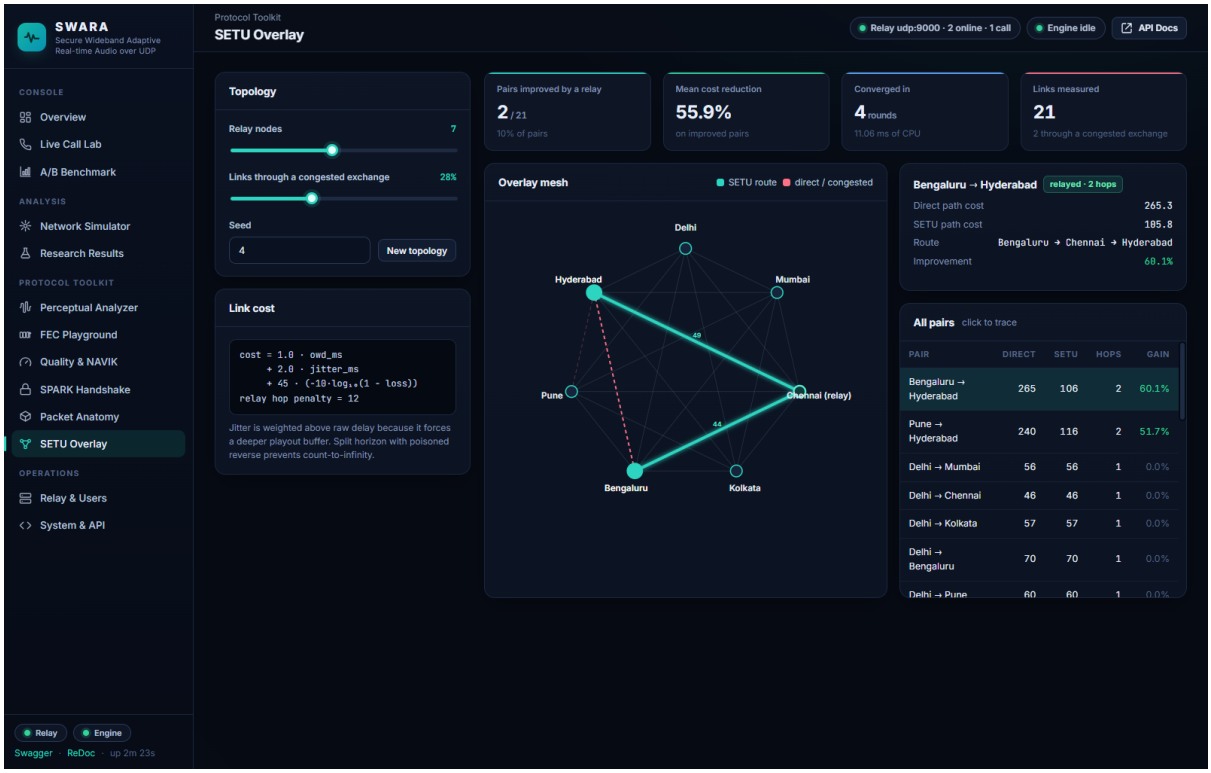


Figure 54. SETU Overlay: a seven-node measured mesh, the converged route for the selected pair, and all pairs ranked by gain.

B.9 Operations and API

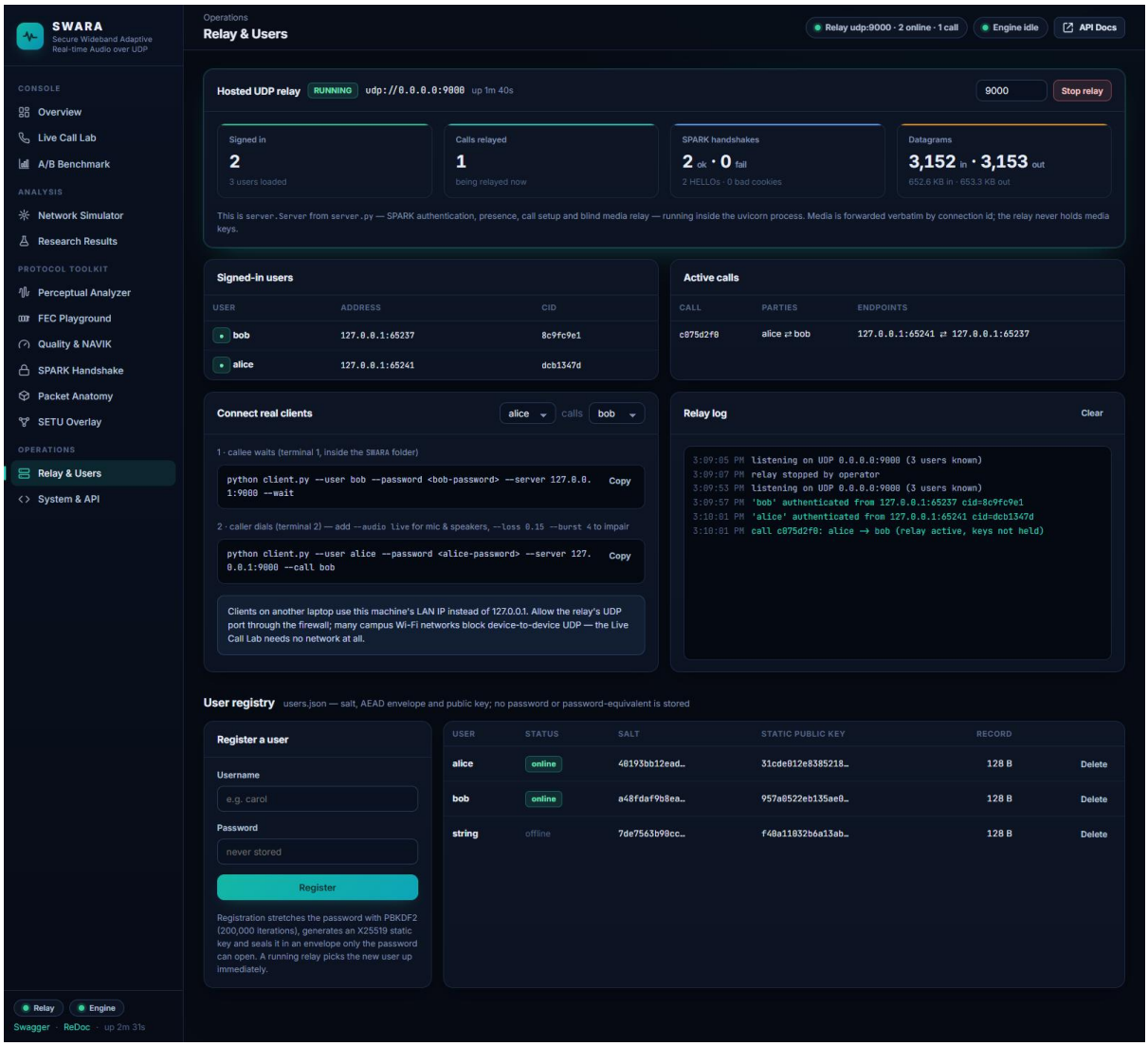


Figure 55. Relay & Users (idle): hosted relay status, the ready-to-paste client commands, the relay log and the user registry.



Figure 56. System & API: runtime, packages, the source-module inventory and every endpoint.

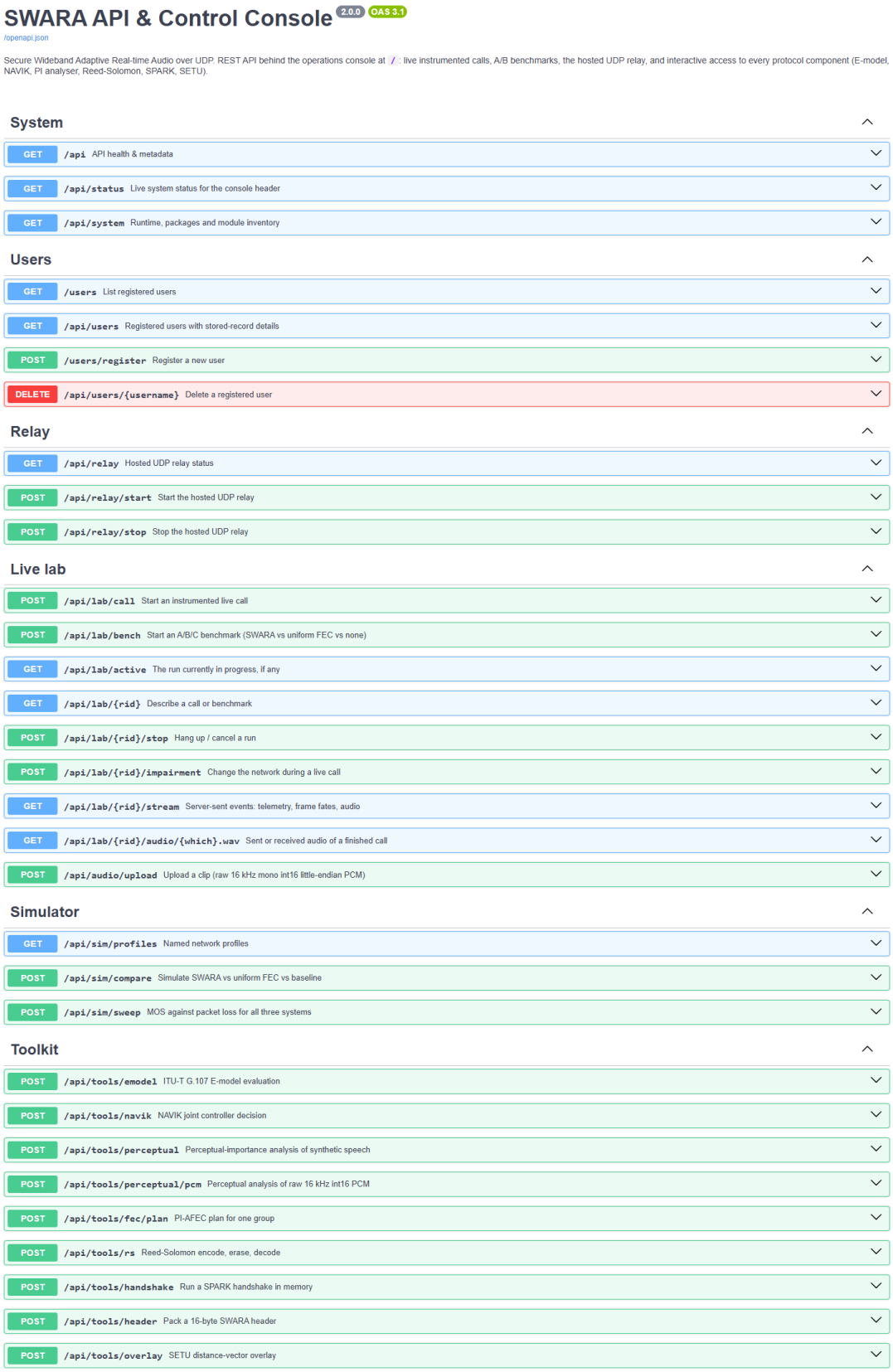


Figure 57. The generated OpenAPI documentation (Swagger UI) at /docs — the first five endpoint groups of thirty-seven documented routes.

Appendix C Source Code

The listings below reproduce the protocol stack (`swara/`), the relay, the client, the demonstration harness and the web API verbatim from the repository. The web console's back-end helpers (`webapp/`, 1,170 lines), its front end (`static/`, about 3,000 lines of JavaScript, CSS and HTML), the experiment script (`experiments/run_all.py`, 511 lines) and the launcher (`launcher.py`, 482 lines) are omitted for length and are available in the repository (Appendix A).

Listing 1 `swara/packet.py` 207 lines

Wire format: header, datagram types, flags, media/parity layouts; CRC-32, Internet checksum, 2-D parity and Hamming(7,4) for the error-control comparison.

```
"""
SWARA wire protocol -- datagram formats.

Every SWARA datagram is a single UDP payload. There is no stream, no
retransmission-by-default, and no head-of-line blocking: a lost packet is a lost
20 ms of audio and nothing more.

Fixed header (16 bytes, network byte order):

   0                   1                   2                   3
   0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
  +-------+-------+---------------+-------------------------------+
  | Ver   | Type  |     Flags     |        Sequence Number        |
  +-------+-------+---------------+-------------------------------+
  |                       Connection ID (CID)                     |
  +---------------------------------------------------------------+
  |                     Timestamp (milliseconds)                  |
  +---------------+---------------+---------------+---------------+
  |   PI Score    |   FEC Group   |   FEC Index   |    Reserved   |
  +---------------+---------------+---------------+---------------+

The PI Score octet is the heart of the design.  It is *authenticated but not
encrypted* (it travels as AEAD associated data), so any element on the path --
the receiver, a relay, or a DiffServ-aware router -- can read how perceptually
important the enclosed 20 ms of speech is WITHOUT being able to read or forge
the speech itself.
"""

from __future__ import annotations

import struct
import zlib
from dataclasses import dataclass, field

VERSION = 1
HEADER_LEN = 16
AEAD_TAG_LEN = 16
MAX_DATAGRAM = 1200  # stay under the smallest plausible path MTU, no fragmentation


class PType:
    """Datagram type codes (4 bits)."""
    HELLO = 0          # client -> server, stateless probe
    RETRY = 1          # server -> client, anti-amplification cookie
    AUTH_INIT = 2      # client -> server, cookie + ephemeral public key
    AUTH_RESP = 3      # server -> client, salt + envelope + ephemeral public key
    AUTH_CONFIRM = 4   # client -> server, key-confirmation tag
    AUTH_TICKET = 5    # server -> client, encrypted resumption ticket
    MEDIA = 6          # a coded speech frame
    FEC = 7            # a Reed-Solomon parity packet
    ANCHOR = 8         # perceptual anchor replica (duplicate of a high-PI frame)
    TELEMETRY = 9      # standalone telemetry (normally piggybacked instead)
    NACK = 10          # selective negative acknowledgement
    PATH_MIGRATE = 11  # 0-RTT authenticated path migration
    PATH_ACK = 12      # path validation response
    OVERLAY_LSA = 13   # SETU overlay link-state / distance-vector advertisement
    BYE = 14
    CONTROL = 15       # encrypted signalling: call setup, presence, hangup

    NAMES = {
        0: "HELLO", 1: "RETRY", 2: "AUTH_INIT", 3: "AUTH_RESP",
        4: "AUTH_CONFIRM", 5: "AUTH_TICKET", 6: "MEDIA", 7: "FEC",
        8: "ANCHOR", 9: "TELEMETRY", 10: "NACK", 11: "PATH_MIGRATE",
        12: "PATH_ACK", 13: "OVERLAY_LSA", 14: "BYE", 15: "CONTROL",
```

```
    }


class Flag:
    """Header flag bits."""
    TELEMETRY_PIGGYBACK = 0x01  # a 16-byte telemetry report follows the header
    MARKER = 0x02               # first frame of a talkspurt
    DSCP_EF = 0x04              # sender requested Expedited Forwarding
    ENCRYPTED = 0x08            # payload is AEAD-protected
    LAST_IN_GROUP = 0x10        # final data frame of an FEC group
    RESUMED = 0x20              # session was resumed 0-RTT


_HDR = struct.Struct("!BBHIIBBBB")


@dataclass
class Header:
    ptype: int
    flags: int = 0
    seq: int = 0
    cid: int = 0
    ts_ms: int = 0
    pi: int = 0            # perceptual importance, 0..255
    fec_group: int = 0
    fec_index: int = 0
    reserved: int = 0
    version: int = VERSION

    def pack(self) -> bytes:
        ver_type = ((self.version & 0x0F) << 4) | (self.ptype & 0x0F)
        return _HDR.pack(
            ver_type, self.flags & 0xFF, self.seq & 0xFFFF, self.cid & 0xFFFFFFFF,
            self.ts_ms & 0xFFFFFFFF, self.pi & 0xFF, self.fec_group & 0xFF,
            self.fec_index & 0xFF, self.reserved & 0xFF,
        )

    @classmethod
    def unpack(cls, buf: bytes) -> "Header":
        if len(buf) < HEADER_LEN:
            raise ValueError("short datagram")
        ver_type, flags, seq, cid, ts, pi, grp, idx, rsv = _HDR.unpack(buf[:HEADER_LEN])
        return cls(
            ptype=ver_type & 0x0F, version=(ver_type >> 4) & 0x0F, flags=flags,
            seq=seq, cid=cid, ts_ms=ts, pi=pi, fec_group=grp, fec_index=idx,
            reserved=rsv,
        )

    def __str__(self) -> str:
        return (f"<{PType.NAMES.get(self.ptype, '?')} seq={self.seq} cid={self.cid:08x} "
                f"pi={self.pi} grp={self.fec_group} idx={self.fec_index}>")


@dataclass
class Datagram:
    header: Header
    payload: bytes = b""
    telemetry: bytes = b""   # optional 16-byte piggybacked report

    def to_bytes(self) -> bytes:
        if self.telemetry:
            self.header.flags |= Flag.TELEMETRY_PIGGYBACK
        return self.header.pack() + self.telemetry + self.payload

    @classmethod
    def from_bytes(cls, buf: bytes) -> "Datagram":
        h = Header.unpack(buf)
        off = HEADER_LEN
        tel = b""
        if h.flags & Flag.TELEMETRY_PIGGYBACK:
            tel = buf[off:off + 16]
            off += 16
        return cls(header=h, payload=buf[off:], telemetry=tel)

    @property
    def size(self) -> int:
        return HEADER_LEN + len(self.telemetry) + len(self.payload)


# ---------------------------------------------------------------------------
# Integrity primitives.
#
# These exist partly because we genuinely use them, and partly because the
# course syllabus asks for error detection and correction -- so we implement
# the classic Data Link Layer codes and then show, with measurements, why a
# real-time voice transport uses a detection code (CRC) plus erasure coding
```

```
# rather than a bit-level correcting code (Hamming).
# ---------------------------------------------------------------------------

def crc32(data: bytes) -> int:
    """CRC-32 (IEEE 802.3 polynomial) -- the same code Ethernet puts in its FCS."""
    return zlib.crc32(data) & 0xFFFFFFFF


def internet_checksum(data: bytes) -> int:
    """RFC 1071 one's-complement checksum -- the code UDP itself carries."""
    if len(data) % 2:
        data += b"\x00"
    total = 0
    for i in range(0, len(data), 2):
        total += (data[i] << 8) | data[i + 1]
        total = (total & 0xFFFF) + (total >> 16)
    return (~total) & 0xFFFF


def parity_2d(block: bytes, width: int = 8) -> bytes:
    """Two-dimensional even parity: one parity bit per row and per column.

    Detects all 1-, 2- and 3-bit errors and corrects any single-bit error.
    """
    rows = [block[i:i + width] for i in range(0, len(block), width)]
    row_par = bytes((sum(bin(b).count("1") for b in r) & 1) for r in rows)
    col_par = bytearray(width)
    for r in rows:
        for j, b in enumerate(r):
            col_par[j] ^= b
    return row_par + bytes(col_par)


_HAMMING_G = [0b1101, 0b1011, 0b1000, 0b0111, 0b0100, 0b0010, 0b0001]


def hamming74_encode(nibble: int) -> int:
    """Hamming(7,4): 4 data bits -> 7 code bits, corrects any single-bit error."""
    d = [(nibble >> i) & 1 for i in (3, 2, 1, 0)]
    p1 = d[0] ^ d[1] ^ d[3]
    p2 = d[0] ^ d[2] ^ d[3]
    p3 = d[1] ^ d[2] ^ d[3]
    return (p1 << 6) | (p2 << 5) | (d[0] << 4) | (p3 << 3) | (d[1] << 2) | (d[2] << 1) | d[3]


def hamming74_decode(code: int) -> tuple[int, bool]:
    """Return (4-bit data, corrected?) after single-error correction."""
    b = [(code >> i) & 1 for i in range(6, -1, -1)]  # b[0]=p1 .. b[6]=d4
    s1 = b[0] ^ b[2] ^ b[4] ^ b[6]
    s2 = b[1] ^ b[2] ^ b[5] ^ b[6]
    s3 = b[3] ^ b[4] ^ b[5] ^ b[6]
    syn = (s3 << 2) | (s2 << 1) | s1
    corrected = False
    if syn:
        b[syn - 1] ^= 1
        corrected = True
    return (b[2] << 3) | (b[4] << 2) | (b[5] << 1) | b[6], corrected
```

Listing 2 `swara/perceptual.py` 240 lines

The perceptual importance analyser and the damage weight.

```
"""
Perceptual Importance (PI) analysis -- SWARA's core novel contribution.

Conventional VoIP treats every 20 ms packet as equally valuable.  That is
demonstrably false.  Human speech intelligibility is dominated by a minority of
frames:

  * Voiced vowel nuclei carry the formant structure that identifies the phoneme.
  * Plosive / vowel ONSETS carry the transient cues that mark word boundaries.
  * Unvoiced fricatives (/s/, /f/) are noise-like: the ear tolerates substitution.
  * Silence and background noise frames are perceptually free to lose.

The PI scorer converts each 20 ms frame into a single octet, 0..255, estimating
"how much damage does losing THIS packet do to what the listener understands".
That octet is then carried in the clear (but authenticated) in the SWARA header,
where the FEC scheduler, the DiffServ marker, the relay and the receiver's
concealment engine can all act on it.

Features, all computed in O(N) or O(N log N) on a 320-sample (20 ms @ 16 kHz) frame:

  E     frame energy in dB, normalised against an adaptive noise floor
  V     voicing probability from normalised autocorrelation peak (pitch strength)
```

```
  ZCR   zero-crossing rate, the classic voiced/unvoiced discriminator
  F     spectral flux against the previous frame -- an onset detector
  VAD   speech activity gate derived from the adaptive noise floor

    PI_raw = w_E*E' + w_V*V + w_F*F' + w_O*onset_boost
    PI     = 255 * VAD * clip(PI_raw, 0, 1)

Design targets (verified in `analyse_corpus`):
    silence / noise                 ->    0 -  25
    steady unvoiced fricative       ->   50 - 110
    steady voiced vowel             ->  150 - 210
    vowel or plosive ONSET          ->  210 - 255
"""

from __future__ import annotations

import numpy as np
from dataclasses import dataclass, field

SAMPLE_RATE = 16_000
FRAME_MS = 20
FRAME_LEN = SAMPLE_RATE * FRAME_MS // 1000   # 320 samples

# Feature weights.  Tuned so that the four target bands above are hit; the sum
# of the positive weights is 1.0 so PI_raw is naturally in [0, 1].
W_ENERGY = 0.30
W_VOICING = 0.34
W_FLUX = 0.20
W_ONSET = 0.16

# The raw normalised autocorrelation peak of a genuinely voiced frame sits
# around 0.5, not 1.0, because the analysis window contains only a few pitch
# periods.  Rescaling against the measured operating range is what turns the
# feature into a usable voiced/unvoiced discriminator.
VOICING_FLOOR = 0.12
VOICING_SPAN = 0.45

# A frame that begins a talkspurt carries the word-boundary cue.  Losing it
# costs far more than losing a frame in the middle of a steady vowel, so a hard
# onset earns a multiplicative boost on top of the additive features.
HARD_ONSET_BOOST = 1.45
SPEECH_FLOOR = 0.12   # any speech frame is worth at least this much


@dataclass
class FrameFeatures:
    energy_db: float
    energy_norm: float
    voicing: float
    zcr: float
    flux: float
    onset: float
    is_speech: bool
    pi: int

    def label(self) -> str:
        if not self.is_speech:
            return "silence"
        if self.onset > 0.5:
            return "onset"
        if self.voicing > 0.5:
            return "voiced"
        return "unvoiced"


class PerceptualImportanceAnalyzer:
    """Streaming, causal, single-frame-latency PI scorer.

    Causality matters: a real encoder cannot look ahead without adding delay, so
    every feature here uses only the current frame and state accumulated from
    past frames.  The onset detector is therefore a *rising-edge* detector, not
    a lookahead peak-picker.
    """

    def __init__(self, sample_rate: int = SAMPLE_RATE, frame_len: int = FRAME_LEN):
        self.sr = sample_rate
        self.n = frame_len
        self._prev_spec: np.ndarray | None = None
        self._prev_energy_norm = 0.0
        self._prev_speech = False
        self._noise_floor_db = -60.0   # adaptive, tracks the quietest recent frames
        self._peak_db = -20.0          # adaptive, tracks the loudest recent frames
        self._window = np.hanning(frame_len)
        self._min_lag = max(2, int(sample_rate / 400))   # 400 Hz -> highest pitch
        self._max_lag = min(frame_len - 1, int(sample_rate / 60))  # 60 Hz -> lowest
        self.frames_seen = 0
```

```
    # -- individual features -------------------------------------------------

    def _energy_db(self, x: np.ndarray) -> float:
        rms = float(np.sqrt(np.mean(x.astype(np.float64) ** 2)) + 1e-12)
        return 20.0 * np.log10(rms)

    def _voicing(self, x: np.ndarray) -> float:
        """Normalised autocorrelation peak in the plausible pitch range.

        Returns ~1.0 for a strongly periodic (voiced) frame, ~0.0 for noise.
        """
        x = x - x.mean()
        e0 = float(np.dot(x, x))
        if e0 < 1e-9:
            return 0.0
        # FFT-based autocorrelation: O(N log N)
        nfft = 1 << (2 * self.n - 1).bit_length()
        spec = np.fft.rfft(x, nfft)
        ac = np.fft.irfft(spec * np.conj(spec), nfft)[: self.n]
        lo, hi = self._min_lag, self._max_lag
        if hi <= lo:
            return 0.0
        seg = ac[lo:hi]
        if seg.size == 0:
            return 0.0
        return float(np.clip(seg.max() / e0, 0.0, 1.0))

    def _zcr(self, x: np.ndarray) -> float:
        return float(np.mean(np.abs(np.diff(np.signbit(x).astype(np.int8)))))

    def _flux(self, x: np.ndarray) -> float:
        """Half-wave-rectified spectral flux: energy that has just APPEARED."""
        spec = np.abs(np.fft.rfft(x * self._window))
        spec = spec / (spec.sum() + 1e-9)
        if self._prev_spec is None:
            self._prev_spec = spec
            return 0.0
        d = spec - self._prev_spec
        self._prev_spec = spec
        return float(np.clip(np.sum(d[d > 0]) * 2.0, 0.0, 1.0))

    # -- main entry point ----------------------------------------------------

    def analyze(self, frame: np.ndarray) -> FrameFeatures:
        x = np.asarray(frame, dtype=np.float64)
        if x.size < self.n:
            x = np.pad(x, (0, self.n - x.size))
        x = x[: self.n]

        e_db = self._energy_db(x)

        # Adaptive noise floor: fast to rise into silence, slow to fall.
        if e_db < self._noise_floor_db:
            self._noise_floor_db += 0.35 * (e_db - self._noise_floor_db)
        else:
            self._noise_floor_db += 0.004 * (e_db - self._noise_floor_db)
        # Adaptive peak: fast attack, slow release.
        if e_db > self._peak_db:
            self._peak_db += 0.5 * (e_db - self._peak_db)
        else:
            self._peak_db += 0.002 * (e_db - self._peak_db)

        span = max(12.0, self._peak_db - self._noise_floor_db)
        e_norm = float(np.clip((e_db - self._noise_floor_db) / span, 0.0, 1.0))

        zcr = self._zcr(x)
        flux = self._flux(x)

        # A plosive burst is only ~8 ms of a 20 ms frame, so its average energy
        # is low even though it is perceptually a transient.  Admitting frames
        # on high spectral flux as well as on energy catches them -- but the
        # flux route must still clear an absolute energy bar, or the random
        # spectral wander of the noise floor is mistaken for speech.
        above_floor = e_db - self._noise_floor_db
        is_speech = above_floor > 6.0 or (flux > 0.62 and above_floor > 4.0)

        voicing_raw = self._voicing(x) if is_speech else 0.0
        voicing = float(np.clip((voicing_raw - VOICING_FLOOR) / VOICING_SPAN, 0.0, 1.0))

        # Onset = a sharp rise in normalised energy relative to the previous frame.
        rise = e_norm - self._prev_energy_norm
        onset = float(np.clip(rise * 3.2, 0.0, 1.0)) if is_speech else 0.0
        hard_onset = is_speech and not self._prev_speech
        if hard_onset:
            onset = 1.0
```

```
        self._prev_energy_norm = e_norm
        self._prev_speech = is_speech

        # High ZCR with low voicing means fricative noise: still needed for
        # intelligibility, but far more concealable than a vowel, so we damp it.
        voicing_eff = voicing * 0.6 if (is_speech and voicing < 0.4 and zcr > 0.20) else voicing

        raw = (W_ENERGY * e_norm + W_VOICING * voicing_eff
               + W_FLUX * flux + W_ONSET * onset)

        if is_speech:
            scaled = SPEECH_FLOOR + (1.0 - SPEECH_FLOOR) * float(np.clip(raw, 0.0, 1.0))
            if hard_onset:
                scaled *= HARD_ONSET_BOOST
            pi = int(round(255.0 * min(scaled, 1.0)))
        else:
            pi = int(round(255.0 * 0.06 * e_norm))

        self.frames_seen += 1
        return FrameFeatures(
            energy_db=e_db, energy_norm=e_norm, voicing=voicing, zcr=zcr,
            flux=flux, onset=onset, is_speech=is_speech, pi=int(np.clip(pi, 0, 255)),
        )


# ---------------------------------------------------------------------------
# Perceptual damage model.
#
# We need a way to say "this frame was lost -- how much did the listener
# suffer?"  We use a damage weight derived from PI, calibrated so that losing a
# silence frame costs ~0 and losing a voiced onset costs ~1.  This is what makes
# unequal protection measurable rather than merely plausible, and it is what
# feeds the effective-loss term of the E-model in emodel.py.
# ---------------------------------------------------------------------------

def damage_weight(pi: int, concealed_neighbours: int = 0) -> float:
    """Perceptual cost of losing a frame with the given PI score.

    The curve is superlinear (exponent 1.6): a frame twice as important is
    roughly three times as costly to lose.  Consecutive losses are penalised
    extra because packet-loss concealment degrades sharply once it has to
    invent more than ~40 ms of speech from nothing.
    """
    base = (pi / 255.0) ** 1.6
    burst_penalty = 1.0 + 0.45 * min(concealed_neighbours, 4)
    return float(base * burst_penalty)
```

Listing 3 `swara/fec.py` 398 lines

GF(2^8) arithmetic, systematic Reed–Solomon erasure code, PI-AFEC scheduler and the uniform comparator.

```
"""
PI-AFEC: Perceptual-Importance-weighted Adaptive Forward Error Correction.

Two layers:

1.  A systematic Reed-Solomon erasure code over GF(2^8).  A group of k media
    packets is extended with r parity packets; any k of the k+r survivors
    reconstruct the group.  Systematic means the k data packets are sent
    unmodified, so a group that arrives intact costs zero decoding work.

2.  The novel part -- the SCHEDULER that decides r, and the "perceptual anchor
    replica" that protects the single most important frame independently.

Why this is not just "FEC with a knob":

  Standard adaptive FEC (RFC 5109 ULP-FEC, Opus LBRR, WebRTC's FlexFEC) picks
  redundancy from the NETWORK state alone -- more loss, more parity.  It spends
  exactly as much protecting a frame of silence as a frame carrying a vowel
  onset.  PI-AFEC picks redundancy from the JOINT (network, content) state:

      r = clip( round( k * rho_net * (Gbar)^gamma ), 0, r_max )

  where rho_net in [0,1] is the network-driven redundancy budget from the NAVIK
  controller and Gbar in [0,1] is the mean normalised PI of the group.  gamma
  < 1 (we use 0.65) keeps moderately-important groups from being starved.

  Because the total parity budget is held FIXED against the baseline, every
  parity packet withheld from a silent group is spent on a group that carries
  meaning.  That reallocation is the entire source of the measured gain.

3.  Perceptual anchor replica.  RS parity only helps if at least k of k+r
    packets survive.  Under a long burst that condition fails and the whole
    group is lost -- including its most important frame.  So when a frame's PI
```

```
    exceeds an adaptive threshold, a compact replica of that ONE frame is
    piggybacked onto a later packet, deliberately spaced by the estimated mean
    burst length so that the original and the replica are unlikely to be in the
    same burst.  It is a targeted, content-aware, delay-bounded duplicate.
"""

from __future__ import annotations

import numpy as np
from dataclasses import dataclass, field

# ---------------------------------------------------------------------------
# GF(2^8) arithmetic, primitive polynomial 0x11D (the same field as CD-ROM,
# QR codes and RAID-6).  Tables are built once at import time.
# ---------------------------------------------------------------------------

_PRIM = 0x11D
GF_EXP = np.zeros(512, dtype=np.uint8)
GF_LOG = np.zeros(256, dtype=np.uint8)


def _build_tables() -> None:
    x = 1
    for i in range(255):
        GF_EXP[i] = x
        GF_LOG[x] = i
        x <<= 1
        if x & 0x100:
            x ^= _PRIM
    for i in range(255, 512):
        GF_EXP[i] = GF_EXP[i - 255]


_build_tables()

# Full 256x256 multiplication table -> vectorised, branch-free encode/decode.
_A = np.arange(256, dtype=np.int32)
_LOGA = GF_LOG[_A].astype(np.int32)
_MULTAB = GF_EXP[(_LOGA[:, None] + _LOGA[None, :]) % 255].astype(np.uint8)
_MULTAB[0, :] = 0
_MULTAB[:, 0] = 0


def gf_mul(a: int, b: int) -> int:
    return int(_MULTAB[a, b])


def gf_inv(a: int) -> int:
    if a == 0:
        raise ZeroDivisionError("GF(256) inverse of 0")
    return int(GF_EXP[(255 - int(GF_LOG[a])) % 255])


def gf_vec_mul(vec: np.ndarray, scalar: int) -> np.ndarray:
    """Multiply a whole byte-vector by a scalar in GF(256) via table lookup."""
    return _MULTAB[scalar][vec]


def _vandermonde(rows: int, cols: int) -> np.ndarray:
    """Cauchy-style generator rows.  Any square submatrix is invertible, which
    is exactly the MDS property an erasure code needs."""
    m = np.zeros((rows, cols), dtype=np.uint8)
    for i in range(rows):
        a = i + 1                       # distinct field elements for parity rows
        for j in range(cols):
            b = 128 + j                 # disjoint from the parity element set
            m[i, j] = gf_inv(a ^ b)
        # Normalise the first column to 1 -> better numerical hygiene, and it
        # makes the r=1 case reduce to a plain XOR, which is free.
    for i in range(rows):
        s = gf_inv(int(m[i, 0])) if m[i, 0] else 1
        for j in range(cols):
            m[i, j] = gf_mul(int(m[i, j]), s)
    return m


class ReedSolomon:
    """Systematic RS(k+r, k) erasure code over GF(256), byte-parallel."""

    def __init__(self, k: int, r: int):
        if not (1 <= k <= 120) or not (0 <= r <= 120):
            raise ValueError("k and r out of supported range")
        self.k, self.r = k, r
        self.gen = _vandermonde(r, k) if r else np.zeros((0, k), dtype=np.uint8)

    def encode(self, shards: list[bytes]) -> list[bytes]:
        """shards: k equal-length byte strings -> r parity byte strings."""
```

```
        if len(shards) != self.k:
            raise ValueError(f"expected {self.k} shards, got {len(shards)}")
        if self.r == 0:
            return []
        n = max(len(s) for s in shards)
        data = np.zeros((self.k, n), dtype=np.uint8)
        for i, s in enumerate(shards):
            data[i, : len(s)] = np.frombuffer(s, dtype=np.uint8)
        out = np.zeros((self.r, n), dtype=np.uint8)
        for i in range(self.r):
            acc = np.zeros(n, dtype=np.uint8)
            for j in range(self.k):
                c = int(self.gen[i, j])
                if c:
                    acc ^= gf_vec_mul(data[j], c)
            out[i] = acc
        return [out[i].tobytes() for i in range(self.r)]

    def decode(self, received: dict[int, bytes], shard_len: int) -> dict[int, bytes] | None:
        """received: {index -> bytes}; data indices 0..k-1, parity k..k+r-1.

        Returns the recovered data shards, or None if fewer than k survived.
        """
        present = sorted(received.keys())
        if len(present) < self.k:
            return None
        have_data = [i for i in present if i < self.k]
        if len(have_data) == self.k:
            return {i: received[i] for i in have_data}

        use = present[: self.k]
        # Build the k x k matrix mapping unknown data shards -> the shards we have.
        missing = [i for i in range(self.k) if i not in received]
        known = [i for i in range(self.k) if i in received]
        par_use = [i for i in use if i >= self.k][: len(missing)]
        if len(par_use) < len(missing):
            return None

        m = np.zeros((len(missing), len(missing)), dtype=np.uint8)
        for a, p in enumerate(par_use):
            for b, d in enumerate(missing):
                m[a, b] = self.gen[p - self.k, d]

        # Right-hand side = parity minus the contribution of the known shards.
        rhs = np.zeros((len(missing), shard_len), dtype=np.uint8)
        for a, p in enumerate(par_use):
            acc = np.frombuffer(received[p].ljust(shard_len, b"\x00"), dtype=np.uint8).copy()
            for d in known:
                c = int(self.gen[p - self.k, d])
                if c:
                    acc ^= gf_vec_mul(
                        np.frombuffer(received[d].ljust(shard_len, b"\x00"), dtype=np.uint8), c)
            rhs[a] = acc

        sol = self._solve(m, rhs)
        if sol is None:
            return None
        out = {i: received[i] for i in known}
        for b, d in enumerate(missing):
            out[d] = sol[b].tobytes()
        return out

    @staticmethod
    def _solve(m: np.ndarray, rhs: np.ndarray) -> np.ndarray | None:
        """Gauss-Jordan elimination over GF(256)."""
        n = m.shape[0]
        m = m.copy()
        rhs = rhs.copy()
        for col in range(n):
            piv = next((r for r in range(col, n) if m[r, col]), None)
            if piv is None:
                return None
            if piv != col:
                m[[col, piv]] = m[[piv, col]]
                rhs[[col, piv]] = rhs[[piv, col]]
            inv = gf_inv(int(m[col, col]))
            m[col] = gf_vec_mul(m[col], inv)
            rhs[col] = gf_vec_mul(rhs[col], inv)
            for r in range(n):
                if r != col and m[r, col]:
                    f = int(m[r, col])
                    m[r] ^= gf_vec_mul(m[col], f)
                    rhs[r] ^= gf_vec_mul(rhs[col], f)
        return rhs

```

```
206
207 # ---------------------------------------------------------------------------
208 # The PI-AFEC scheduler
209 # ---------------------------------------------------------------------------
210
211 GAMMA = 0.65        # importance -> redundancy shaping exponent
212 ANCHOR_PI_FLOOR = 170   # a frame must be at least this important to earn a replica
213
214
215 @dataclass
216 class FecPlan:
217     """What the scheduler decided for one group of k frames."""
218     k: int
219     r: int
220     group_mean_pi: float
221     anchors: list = field(default_factory=list)   # [(index_in_group, delay_frames)]
222     overhead_ratio: float = 0.0
223     burst_mode: bool = False
224
225     @property
226     def anchor_index(self):
227         return self.anchors[0][0] if self.anchors else None
228
229
230 @dataclass
231 class FecScheduler:
232     """Decides parity count and anchor placement for each group.
233
234     `rho_net` and `mean_burst` are written by the NAVIK controller from receiver
235     telemetry; the scheduler itself is stateless apart from those two inputs and
236     a running average used to hold the overhead budget on target.
237     """
238     k: int = 8
239     r_max: int = 6
240     rho_net: float = 0.0          # network-driven redundancy budget, 0..1
241     mean_burst: float = 1.0       # estimated mean loss-burst length, packets
242     anchors_enabled: bool = True
243     _spent: float = field(default=0.0, repr=False)
244     _groups: int = field(default=0, repr=False)
245     _carry_r: float = field(default=0.0, repr=False)
246     _carry_a: float = field(default=0.0, repr=False)
247     _wsum: float = field(default=0.6, repr=False)     # running mean of the
248     _wcount: float = field(default=1.0, repr=False)   # importance weight
249
250     def plan(self, pi_scores: list[int]) -> FecPlan:
251         """Decide parity and anchors for one group.
252
253         The branch on `burst_mode` is a substantive design decision, not a
254         tuning detail.  An RS(k+r, k) group survives only if at least k of its
255         k+r packets arrive.  When the mean loss burst approaches the group's
256         own span, that condition fails almost every time the channel goes bad,
257         and every parity byte is wasted.
258
259         Classical coding theory answers this with INTERLEAVING -- spread the
260         group's packets across a wide time window so a burst hits at most one
261         of them.  Real-time voice cannot use it: an interleaver of depth D over
262         a group of k frames costs k*D frame times of buffering, so even D=3 at
263         k=8 adds 480 ms and destroys conversational interactivity.
264
265         SWARA's answer is to interleave ONLY what matters.  In burst mode the
266         parity budget is redirected into REPLICAS of the highest-PI frames,
267         placed at offsets chosen to exceed the estimated burst length.  One
268         replica of one frame costs one packet and one frame-time of extra
269         delay, and it is the only mechanism that survives a burst longer than
270         the group itself.
271         """
272         k = len(pi_scores)
273         if k == 0:
274             return FecPlan(k=0, r=0, group_mean_pi=0.0)
275         arr = np.asarray(pi_scores, dtype=float)
276         gbar = float(arr.mean()) / 255.0
277
278         # The weight is accumulated for EVERY group, including the silent ones
279         # that are about to be skipped.  That ordering is the whole mechanism:
280         # a skipped group contributes a weight of nearly zero to the running
281         # mean, which lowers the mean, which raises every other group's share.
282         # Accumulating after the skip instead -- as this did -- means a silent
283         # group's budget is simply forfeited rather than handed on, and the
284         # scheme ends up spending less overall instead of spending the same
285         # amount better. Measured cost of that ordering: 164 parity packets
286         # against the uniform comparator's 373.
287         weight = gbar ** GAMMA
288         self._wsum += weight
289         self._wcount += 1.0
290
291         # A group with genuinely no speech in it gets nothing at all, however
```

```
        # bad the network is.  This is where the reallocated budget comes from.
        if gbar < 0.08:
            self._groups += 1
            return FecPlan(k=k, r=0, group_mean_pi=gbar, overhead_ratio=0.0)

        # --- budget: REDISTRIBUTE, do not shrink -------------------------
        #
        # The whole claim of PI-AFEC is "the same budget, spent where it is
        # heard".  The obvious implementation, budget = k * rho * Gbar^gamma,
        # does not do that.  Because Gbar^gamma is always below 1, it spends
        # systematically LESS than the rho it was given -- a nominal 50%
        # redundancy quietly became about 20% in the live system, and SWARA
        # lost to the uniform comparator on both quality and, absurdly, on the
        # thing it was supposed to be better at.
        #
        # Normalising the weight against its own running mean fixes it: the
        # EXPECTED spend is exactly k * rho, while individual groups receive
        # more or less than that in proportion to how much they matter.  Now it
        # really is redistribution rather than a hidden discount.
        wbar = max(0.05, self._wsum / self._wcount)
        ratio = float(np.clip(weight / wbar, 0.0, 4.0))
        budget_pkts = k * self.rho_net * ratio

        # An RS(k+r, k) group survives r erasures.  Reed-Solomon is therefore
        # worth funding whenever the typical burst is smaller than the parity we
        # can afford, and only becomes futile once bursts approach the group's
        # own span.  An earlier threshold of k/2.5 declared futility at a burst
        # of 1.6 packets for k=4 -- a regime where RS in fact works perfectly
        # well -- and diverted the entire budget into replicas.  The measured
        # consequence was SWARA transmitting almost no redundancy at all while
        # the uniform comparator recovered 53 frames per call.
        burst_mode = self.mean_burst >= 0.75 * k

        if burst_mode:
            share_r, share_a = 0.45, 0.55
        else:
            share_r, share_a = 0.85, 0.15

        # Fractional budgets are carried between groups rather than rounded
        # away.  With k=4 a per-group budget of 0.4 packets rounds to zero every
        # single time, so a nominal 10% redundancy silently becomes 0%.
        want_r = budget_pkts * share_r + self._carry_r
        r = int(want_r)
        self._carry_r = want_r - r
        r = int(np.clip(r, 0, min(self.r_max, k)))

        want_a = budget_pkts * share_a + self._carry_a
        n_anchor = int(want_a)
        self._carry_a = want_a - n_anchor
        n_anchor = int(np.clip(n_anchor, 0, 4)) if self.anchors_enabled else 0

        anchors: list[tuple[int, int]] = []
        if self.anchors_enabled and n_anchor:
            order = np.argsort(-arr)
            base_delay = int(np.clip(round(self.mean_burst) + 2, 3, 9))
            for slot, idx in enumerate(order[:n_anchor]):
                if arr[idx] < ANCHOR_PI_FLOOR:
                    break
                # Stagger successive replicas so two of them are never inside
                # the same outage either.
                anchors.append((int(idx), base_delay + slot * 2))

        overhead = (r + len(anchors)) / k
        self._spent += overhead
        self._groups += 1
        return FecPlan(k=k, r=r, group_mean_pi=gbar, anchors=anchors,
                       overhead_ratio=overhead, burst_mode=burst_mode)

    @property
    def realised_overhead(self) -> float:
        return self._spent / self._groups if self._groups else 0.0


@dataclass
class UniformFecScheduler:
    """Baseline comparator: classic loss-driven FEC with no content awareness.

    This is what RFC 5109 / FlexFEC style adaptive redundancy does.  It is a
    strong baseline, not a straw man -- it adapts to the network correctly.  It
    simply cannot tell a vowel from a silence.
    """
    k: int = 8
    r_max: int = 6
    rho_net: float = 0.0
    mean_burst: float = 1.0
    _spent: float = field(default=0.0, repr=False)
```

```
    _groups: int = field(default=0, repr=False)

    def plan(self, pi_scores: list[int]) -> FecPlan:
        k = max(1, len(pi_scores))
        # Fractional budgets are honoured probabilistically so that a rho of,
        # say, 0.14 really does cost 0.14*k packets on average rather than
        # rounding to zero and quietly under-spending the budget it was given.
        exact = k * self.rho_net
        r = int(exact)
        if np.random.random() < (exact - r):
            r += 1
        r = int(np.clip(r, 0, min(self.r_max, k)))
        self._spent += r / k
        self._groups += 1
        return FecPlan(k=k, r=r,
                       group_mean_pi=float(np.mean(pi_scores)) / 255.0 if pi_scores else 0.0,
                       anchors=[], overhead_ratio=r / k)

    @property
    def realised_overhead(self) -> float:
        return self._spent / self._groups if self._groups else 0.0
```

Listing 4 `swara/auth.py` 475 lines

SPARK: registration, server and client handshake, key schedule, tickets, migration, media AEAD, sequence reconstruction and replay window.

```
"""
SPARK -- Stateless Password-Authenticated Retry & Key-agreement.

The authentication problem for voice over UDP is not "how do I check a
password".  It is:

  (a) UDP is trivially spoofable, so an unauthenticated first packet must never
      make the server allocate memory or emit a larger reply -- otherwise the
      service is a DDoS amplifier;
  (b) a password must never cross the wire, and a stolen server database must
      not yield anything replayable;
  (c) the handshake must not cost so many round trips that call setup feels
      slow -- and after a WiFi-to-mobile handover it must cost NONE, because
      the call is already in progress and the user is mid-sentence.

SPARK addresses all three:

  Phase 0  HELLO        16 B, no state created, no user identity revealed.
  Phase 1  RETRY        server returns a stateless HMAC cookie bound to the
                        client's address and a coarse clock.  Reply is SMALLER
                        than the request -> amplification factor < 1.
  Phase 2  AUTH_INIT    client echoes the cookie.  Only now, having proved it
                        can receive at the address it claims, does the server
                        do any work.
  Phase 3  AUTH_RESP    server returns the user's salt and OPAQUE-style
                        envelope plus a fresh ephemeral X25519 key.
  Phase 4  AUTH_CONFIRM triple Diffie-Hellman is mixed into the transcript and
                        the client proves key knowledge.  Mutual auth complete.

Key schedule (all HKDF-SHA256 over the full transcript hash, so every byte of
every handshake message is authenticated):

    dh1 = X25519(client_ephemeral,  server_static)     -- authenticates server
    dh2 = X25519(client_static,     server_ephemeral)  -- authenticates client
    dh3 = X25519(client_ephemeral,  server_ephemeral)  -- forward secrecy
    MS  = HKDF-Extract(salt = H(transcript), IKM = dh1 || dh2 || dh3)

The client's static private key never leaves the client and is not stored
anywhere in plaintext: it lives inside an AEAD envelope whose key is derived
from the password by PBKDF2.  The server stores only {salt, envelope, client
static PUBLIC key}.  Stealing that database gives an attacker a target for an
offline dictionary attack -- and nothing that can be replayed at the server,
because possession of the envelope alone does not yield the static private key.

Resumption / path migration:
    On success the server issues a ticket = AEAD(K_ticket_server, state).  The
    server keeps NO per-session table for this.  When the client's address
    changes it sends PATH_MIGRATE {CID, ticket, AEAD(K_resume, challenge)} from
    the new address; the server decrypts the ticket, verifies the tag, rebinds
    the CID and the media flow continues.  Zero round trips of silence.
"""

from __future__ import annotations

import hashlib
import hmac
```

```
import os
import struct
import time
from dataclasses import dataclass, field

from cryptography.hazmat.primitives.asymmetric.x25519 import (
    X25519PrivateKey, X25519PublicKey)
from cryptography.hazmat.primitives.ciphers.aead import ChaCha20Poly1305
from cryptography.hazmat.primitives.kdf.hkdf import HKDFExpand
from cryptography.hazmat.primitives import hashes

PBKDF2_ITERS = 200_000
COOKIE_LIFETIME_S = 30
TICKET_LIFETIME_S = 3600
KEY_LEN = 32


# ---------------------------------------------------------------------------
# Primitives
# ---------------------------------------------------------------------------

def hkdf_expand(prk: bytes, label: bytes, length: int = KEY_LEN) -> bytes:
    return HKDFExpand(algorithm=hashes.SHA256(), length=length,
                      info=b"SWARA-v1 " + label).derive(prk)


def hkdf_extract(salt: bytes, ikm: bytes) -> bytes:
    return hmac.new(salt, ikm, hashlib.sha256).digest()


def stretch_password(password: str, salt: bytes) -> bytes:
    return hashlib.pbkdf2_hmac("sha256", password.encode(), salt, PBKDF2_ITERS, KEY_LEN)


class Transcript:
    """Running SHA-256 over every handshake byte, in order."""

    def __init__(self) -> None:
        self._h = hashlib.sha256(b"SWARA-v1 transcript")

    def add(self, *chunks: bytes) -> "Transcript":
        for c in chunks:
            self._h.update(struct.pack("!H", len(c)))
            self._h.update(c)
        return self

    def digest(self) -> bytes:
        return self._h.copy().digest()


# ---------------------------------------------------------------------------
# Credential registration (performed once, out of band)
# ---------------------------------------------------------------------------

@dataclass
class CredentialRecord:
    """Exactly what the server stores per user.  Note the absence of anything
    resembling the password or a password-equivalent verifier."""
    username: str
    salt: bytes
    envelope: bytes            # AEAD(PBKDF2(pw,salt), client static private key)
    client_static_pub: bytes   # 32-byte X25519 public key

    def size_bytes(self) -> int:
        return len(self.salt) + len(self.envelope) + len(self.client_static_pub)


def register(username: str, password: str) -> tuple[CredentialRecord, bytes]:
    """Create a credential record. Returns (record, client_static_private)."""
    salt = os.urandom(16)
    rw = stretch_password(password, salt)
    static_priv = X25519PrivateKey.generate()
    priv_raw = static_priv.private_bytes_raw()
    pub_raw = static_priv.public_key().public_bytes_raw()
    aead = ChaCha20Poly1305(rw)
    nonce = b"\x00" * 12
    envelope = aead.encrypt(nonce, priv_raw + pub_raw, username.encode())
    return CredentialRecord(username, salt, envelope, pub_raw), priv_raw


# ---------------------------------------------------------------------------
# Session keys
# ---------------------------------------------------------------------------

@dataclass
class SessionKeys:
    cid: int
```

```
    k_c2s: bytes
    k_s2c: bytes
    k_resume: bytes
    transcript: bytes

    @classmethod
    def derive(cls, master: bytes, transcript: bytes) -> "SessionKeys":
        cid = struct.unpack("!I", hkdf_expand(master, b"cid", 4))[0]
        return cls(cid=cid,
                   k_c2s=hkdf_expand(master, b"c2s"),
                   k_s2c=hkdf_expand(master, b"s2c"),
                   k_resume=hkdf_expand(master, b"resume"),
                   transcript=transcript)


# ---------------------------------------------------------------------------
# Server side
# ---------------------------------------------------------------------------

class SparkServer:
    def __init__(self) -> None:
        self.static_priv = X25519PrivateKey.generate()
        self.static_pub = self.static_priv.public_key().public_bytes_raw()
        self._cookie_key = os.urandom(32)
        self._ticket_key = os.urandom(32)
        self.users: dict[str, CredentialRecord] = {}
        # CID -> (keys, current address).  Populated only AFTER full auth.
        self.sessions: dict[int, tuple[SessionKeys, tuple]] = {}
        self.stats = {"hello": 0, "cookie_bad": 0, "auth_ok": 0, "auth_fail": 0,
                      "resumed": 0, "migrations": 0, "bytes_out_pre_auth": 0,
                      "bytes_in_pre_auth": 0}

    # -- registration ---------------------------------------------------

    def add_user(self, record: CredentialRecord) -> None:
        self.users[record.username] = record

    # -- phase 1: stateless retry ---------------------------------------

    def _cookie(self, addr: tuple, epoch: int) -> bytes:
        msg = f"{addr[0]}|{addr[1]}|{epoch}".encode()
        return hmac.new(self._cookie_key, msg, hashlib.sha256).digest()[:16]

    def handle_hello(self, addr: tuple, hello: bytes) -> bytes:
        """Zero state is created here.  The reply is deliberately no larger
        than the request, so SWARA cannot be used as a reflection amplifier."""
        self.stats["hello"] += 1
        self.stats["bytes_in_pre_auth"] += len(hello)
        epoch = int(time.time() // COOKIE_LIFETIME_S)
        out = struct.pack("!I", epoch) + self._cookie(addr, epoch)
        self.stats["bytes_out_pre_auth"] += len(out)
        return out

    def _cookie_valid(self, addr: tuple, epoch: int, cookie: bytes) -> bool:
        now = int(time.time() // COOKIE_LIFETIME_S)
        if epoch not in (now, now - 1):
            return False
        return hmac.compare_digest(cookie, self._cookie(addr, epoch))

    # -- phase 3: response ----------------------------------------------

    def handle_auth_init(self, addr: tuple, body: bytes):
        """body = epoch(4) || cookie(16) || eph_pub(32) || ulen(1) || username"""
        epoch, = struct.unpack("!I", body[:4])
        cookie = body[4:20]
        if not self._cookie_valid(addr, epoch, cookie):
            self.stats["cookie_bad"] += 1
            return None, None      # silent drop: spoofed source or stale cookie
        client_eph = body[20:52]
        ulen = body[52]
        username = body[53:53 + ulen].decode(errors="replace")
        rec = self.users.get(username)
        if rec is None:
            # Constant-shape reply for unknown users: no user-enumeration oracle.
            rec = CredentialRecord(username, hashlib.sha256(username.encode()).digest()[:16],
                                   os.urandom(80), os.urandom(32))
            known = False
        else:
            known = True

        server_eph_priv = X25519PrivateKey.generate()
        server_eph_pub = server_eph_priv.public_key().public_bytes_raw()

        resp = (bytes([len(rec.salt)]) + rec.salt
                + struct.pack("!H", len(rec.envelope)) + rec.envelope
                + server_eph_pub + self.static_pub)
```

```
        pending = {"client_eph": client_eph, "server_eph_priv": server_eph_priv,
                   "rec": rec, "known": known, "addr": addr, "resp": resp,
                   "transcript": Transcript().add(b"AUTH_INIT", body, b"AUTH_RESP", resp)}
        return resp, pending

    # -- phase 5: verify confirmation ----------------------------------------

    def handle_auth_confirm(self, pending: dict, tag: bytes):
        rec: CredentialRecord = pending["rec"]
        if not pending["known"]:
            self.stats["auth_fail"] += 1
            return None, None

        eph_priv: X25519PrivateKey = pending["server_eph_priv"]
        client_eph_pub = X25519PublicKey.from_public_bytes(pending["client_eph"])
        client_static_pub = X25519PublicKey.from_public_bytes(rec.client_static_pub)

        dh1 = self.static_priv.exchange(client_eph_pub)
        dh2 = eph_priv.exchange(client_static_pub)
        dh3 = eph_priv.exchange(client_eph_pub)

        th = pending["transcript"].digest()
        master = hkdf_extract(th, dh1 + dh2 + dh3)
        keys = SessionKeys.derive(master, th)

        expect = hmac.new(keys.k_c2s, b"client finished" + th, hashlib.sha256).digest()[:16]
        if not hmac.compare_digest(tag, expect):
            self.stats["auth_fail"] += 1
            return None, None

        self.stats["auth_ok"] += 1
        self.sessions[keys.cid] = (keys, pending["addr"])
        ticket = self._issue_ticket(rec.username, keys)
        return keys, ticket

    # -- resumption tickets --------------------------------------------------

    def _issue_ticket(self, username: str, keys: SessionKeys) -> bytes:
        nonce = os.urandom(12)
        state = (struct.pack("!IQ", keys.cid, int(time.time()) + TICKET_LIFETIME_S)
                 + keys.k_resume + bytes([len(username)]) + username.encode())
        blob = ChaCha20Poly1305(self._ticket_key).encrypt(nonce, state, b"SWARA-ticket")
        return nonce + blob

    def handle_path_migrate(self, new_addr: tuple, body: bytes):
        """0-RTT authenticated migration.  body = ticket_len(2)||ticket||nonce(12)||sealed"""
        tlen, = struct.unpack("!H", body[:2])
        ticket = body[2:2 + tlen]
        rest = body[2 + tlen:]
        try:
            state = ChaCha20Poly1305(self._ticket_key).decrypt(
                ticket[:12], ticket[12:], b"SWARA-ticket")
        except Exception:
            return None
        cid, expiry = struct.unpack("!IQ", state[:12])
        if time.time() > expiry:
            return None
        k_resume = state[12:44]
        try:
            proof = ChaCha20Poly1305(k_resume).decrypt(rest[:12], rest[12:], b"SWARA-migrate")
        except Exception:
            return None   # cannot forge without k_resume: an off-path attacker
                          # cannot hijack a call by replaying the ticket alone
        if cid in self.sessions:
            keys, _ = self.sessions[cid]
            self.sessions[cid] = (keys, new_addr)
            self.stats["migrations"] += 1
            return cid, proof
        return None


# ---------------------------------------------------------------------------
# Client side
# ---------------------------------------------------------------------------

class SparkClient:
    def __init__(self, username: str, password: str):
        self.username = username
        self.password = password
        self.eph_priv = X25519PrivateKey.generate()
        self.eph_pub = self.eph_priv.public_key().public_bytes_raw()
        self.keys: SessionKeys | None = None
        self.ticket: bytes | None = None
        self._transcript: Transcript | None = None
        self._init_msg: bytes | None = None
```

```
316
317     HELLO_PAD = 64   # RETRY is 20 B, so amplification is bounded at 0.31x
318
319     def make_hello(self) -> bytes:
320         """Deliberately padded.  A reflection attacker gains nothing by
321         spoofing our HELLO because the server's reply is far SMALLER than the
322         packet the attacker had to send.  Amplification factor 20/64 = 0.31."""
323         return (b"SWARA/1" + os.urandom(9)).ljust(self.HELLO_PAD, b"\x00")
324
325     def make_auth_init(self, retry: bytes) -> bytes:
326         epoch = retry[:4]
327         cookie = retry[4:20]
328         u = self.username.encode()
329         body = epoch + cookie + self.eph_pub + bytes([len(u)]) + u
330         self._init_msg = body
331         return body
332
333     def make_auth_confirm(self, resp: bytes) -> bytes | None:
334         off = 0
335         slen = resp[off]; off += 1
336         salt = resp[off:off + slen]; off += slen
337         elen, = struct.unpack("!H", resp[off:off + 2]); off += 2
338         envelope = resp[off:off + elen]; off += elen
339         server_eph = resp[off:off + 32]; off += 32
340         server_static = resp[off:off + 32]
341
342         rw = stretch_password(self.password, salt)
343         try:
344             opened = ChaCha20Poly1305(rw).decrypt(b"\x00" * 12, envelope, self.username.encode())
345         except Exception:
346             return None          # wrong password -- detected locally, no oracle
347         static_priv_raw = opened[:32]
348         static_priv = X25519PrivateKey.from_private_bytes(static_priv_raw)
349
350         dh1 = self.eph_priv.exchange(X25519PublicKey.from_public_bytes(server_static))
351         dh2 = static_priv.exchange(X25519PublicKey.from_public_bytes(server_eph))
352         dh3 = self.eph_priv.exchange(X25519PublicKey.from_public_bytes(server_eph))
353
354         th = Transcript().add(b"AUTH_INIT", self._init_msg, b"AUTH_RESP", resp).digest()
355         master = hkdf_extract(th, dh1 + dh2 + dh3)
356         self.keys = SessionKeys.derive(master, th)
357         return hmac.new(self.keys.k_c2s, b"client finished" + th, hashlib.sha256).digest()[:16]
358
359     def make_path_migrate(self) -> bytes:
360         assert self.keys and self.ticket
361         nonce = os.urandom(12)
362         sealed = ChaCha20Poly1305(self.keys.k_resume).encrypt(
363             nonce, struct.pack("!d", time.time()), b"SWARA-migrate")
364         return struct.pack("!H", len(self.ticket)) + self.ticket + nonce + sealed
365
366
367 # ---------------------------------------------------------------------------
368 # Media record protection
369 # ---------------------------------------------------------------------------
370
371 class MediaCipher:
372     """AEAD for media packets.
373
374     The 16-byte SWARA header is passed as ASSOCIATED DATA, not encrypted.  That
375     is what lets the PI octet stay readable to the network while remaining
376     unforgeable -- flipping it invalidates the tag and the packet is dropped.
377
378     The nonce is derived deterministically from (direction, sequence number), so
379     no nonce needs to be transmitted: 12 bytes saved on every single packet,
380     which at 50 packets/s is 4.8 kbit/s of pure saving per stream.
381     """
382
383     def __init__(self, key: bytes, direction: bytes):
384         self.aead = ChaCha20Poly1305(key)
385         self.dir = direction
386         self._salt = hashlib.sha256(key + direction).digest()[:12]
387
388     def _nonce(self, seq: int, epoch: int = 0) -> bytes:
389         counter = struct.pack("!IQ", epoch, seq)
390         return bytes(a ^ b for a, b in zip(self._salt, counter[:12]))
391
392     def seal(self, seq: int, header: bytes, plaintext: bytes) -> bytes:
393         return self.aead.encrypt(self._nonce(seq), plaintext, header)
394
395     def open(self, seq: int, header: bytes, ciphertext: bytes) -> bytes | None:
396         try:
397             return self.aead.decrypt(self._nonce(seq), ciphertext, header)
398         except Exception:
399             return None
400
401
402 # ---------------------------------------------------------------------------
```

```
# Sequence-number hygiene for long calls
# ---------------------------------------------------------------------------

class SequenceReconstructor:
    """Recover a full counter from the 16-bit sequence field on the wire.

    The header carries 16 bits, but the AEAD nonce is derived from the full
    counter.  Without reconstruction the two diverge the moment the field wraps
    -- after 65536 frames, which at 20 ms per frame is 21.8 minutes.  Every
    packet after that point fails authentication and the call dies silently.
    This is the same rollover-counter mechanism SRTP uses, and for the same
    reason.
    """

    def __init__(self) -> None:
        self.roc = 0
        self.highest = -1

    def unwrap(self, wire: int) -> int:
        wire &= 0xFFFF
        if self.highest < 0:
            self.highest = wire
            return wire
        prev_low = self.highest & 0xFFFF
        roc = self.roc
        if wire < prev_low and (prev_low - wire) > 32768:
            roc += 1                        # the field wrapped forward
        elif wire > prev_low and (wire - prev_low) > 32768:
            roc = max(0, roc - 1)           # a straggler from the previous epoch
        full = (roc << 16) | wire
        if full > self.highest:
            self.highest = full
            self.roc = roc
        return full

    def nearest(self, wire: int) -> int:
        """Full counter for a 16-bit value near the current position, without
        advancing state -- used for references such as an FEC group's base."""
        wire &= 0xFFFF
        if self.highest < 0:
            return wire
        base = (self.highest >> 16) << 16
        cands = (base + wire, base + wire - 65536, base + wire + 65536)
        return min(cands, key=lambda c: abs(c - self.highest))


class ReplayWindow:
    """Sliding bitmap rejecting duplicated or replayed sequence numbers.

    AEAD stops an attacker forging a packet, but it does not stop one CAPTURING
    a valid packet and sending it again.  Without a replay window a recorded
    fragment of speech can be re-injected into a live call.
    """

    def __init__(self, size: int = 1024):
        self.size = size
        self.highest = -1
        self.bits = 0
        self.rejected = 0

    def check_and_set(self, seq: int) -> bool:
        if seq > self.highest:
            shift = seq - self.highest
            self.bits = 1 if shift >= self.size else \
                (((self.bits << shift) | 1) & ((1 << self.size) - 1))
            self.highest = seq
            return True
        delta = self.highest - seq
        if delta >= self.size or (self.bits >> delta) & 1:
            self.rejected += 1
            return False
        self.bits |= (1 << delta)
        return True
```

Listing 5 `swara/controller.py` 281 lines

NAVIK: capacity estimation, burst-aware residual-loss model, joint optimisation and DSCP mapping.

```
"""
NAVIK -- Network-Adaptive Voice-quality Integrated Kontroller.

The sender has three dials and one budget:

    dial 1   codec bitrate b        higher b -> lower Ie -> better clean quality
```

```
    dial 2   FEC redundancy rho     higher rho -> lower residual loss
    dial 3   DSCP marking           which packets a congested router drops first
    budget   available capacity C   b * (1 + rho) + header overhead <= C

Turning any dial up costs the others.  Conventional stacks resolve this with a
cascade of independent heuristics: a bandwidth estimator picks b, a separate
loss-driven module picks rho, and DSCP is a static configuration item.  Those
modules can and do fight each other -- the classic pathology is FEC added in
response to congestion loss, which increases the offered load and deepens the
congestion that caused the loss in the first place.

NAVIK replaces the cascade with a single objective.  Every 100 ms it evaluates

    maximise  R( codec(b), delay(b, rho), Ppl_residual(rho, p, B, PI), B )
    s.t.      b * (1 + rho) + H <= C_est
              rho <= rho_max(p)          -- congestion safety rail

over a small discrete grid (9 bitrates x 9 redundancy levels = 81 evaluations,
about 40 microseconds).  Because R already contains the delay term, the
congestion pathology cannot occur: adding redundancy that pushes the offered
rate past C_est raises queueing delay, which lowers R, so the optimiser refuses.

The residual-loss model is where perceptual importance re-enters.  For an
RS(k+r, k) group under Gilbert-Elliott loss, the probability that the group
fails to decode is the probability of more than r erasures in k+r packets --
computed here with a burst-aware correction rather than the binomial, because
under clustered loss the binomial is dramatically optimistic.  PI-AFEC's
advantage appears in the model as a reduction in the *perceptual weight* of the
groups that are left unprotected.
"""

from __future__ import annotations

import math
import numpy as np
from dataclasses import dataclass, field

from . import emodel

BITRATE_LADDER = ["opus-6k", "opus-8k", "opus-12k", "opus-16k",
                  "opus-20k", "opus-24k", "opus-32k"]
RHO_GRID = [0.0, 0.075, 0.15, 0.225, 0.30, 0.375, 0.45, 0.55, 0.65]

HEADER_OVERHEAD_BPS = (16 + 16 + 8 + 20) * 8 * 50   # SWARA+AEAD+UDP+IP at 50 pps


@dataclass
class ControlDecision:
    codec: str
    bitrate_kbps: float
    rho: float
    dscp_high: int
    dscp_low: int
    predicted_r: float
    predicted_mos: float
    offered_kbps: float
    reason: str = ""

    def __str__(self) -> str:
        return (f"NAVIK -> {self.codec} @ {self.bitrate_kbps:.0f}kbps, "
                f"FEC {self.rho*100:.0f}%, offered {self.offered_kbps:.1f}kbps, "
                f"predicted MOS {self.predicted_mos:.2f}")


def group_failure_probability(k: int, r: int, p: float, mean_burst: float) -> float:
    """P(an RS(k+r, k) group cannot be decoded) under Gilbert-Elliott loss.

    Under independent loss this is P(erasures > r) ~ Binomial.  Under clustered
    loss the variance of the erasure count is much higher, so we use a
    beta-binomial with the over-dispersion implied by the burst length.  This
    matters enormously: at 10% loss with burst 4, the binomial predicts a group
    failure rate about 6x lower than reality, and a controller that believes it
    will under-provision FEC exactly when protection is most needed.
    """
    n = k + r
    p = float(np.clip(p, 1e-6, 0.95))
    if r >= n:
        return 0.0
    b = max(1.0, mean_burst)

    if b <= 1.05:
        # Effectively independent -> plain binomial tail.
        from math import comb
        return float(sum(comb(n, i) * p ** i * (1 - p) ** (n - i)
                         for i in range(r + 1, n + 1)))

    # Beta-binomial: intra-class correlation rho_c grows with burst length.
```

```
    rho_c = float(np.clip((b - 1.0) / (b + 1.0), 1e-3, 0.95))
    s = (1.0 - rho_c) / rho_c
    a, bb = p * s, (1.0 - p) * s

    from math import lgamma

    def logbeta(x, y):
        return lgamma(x) + lgamma(y) - lgamma(x + y)

    from math import comb
    lb0 = logbeta(a, bb)
    tail = 0.0
    for i in range(r + 1, n + 1):
        logp = (math.log(comb(n, i)) + logbeta(a + i, bb + n - i) - lb0)
        tail += math.exp(logp)
    return float(np.clip(tail, 0.0, 1.0))


def split_budget(budget_pkts: float, k: int, mean_burst: float) -> tuple[int, int]:
    """Divide a group's redundancy budget between RS parity and anchor replicas.

    Mirrors fec.FecScheduler exactly, so the controller predicts what the
    scheduler will actually do rather than what it wishes it would do.
    """
    # Mirrors fec.FecScheduler after normalisation: the EXPECTED per-group
    # spend is the full budget, so the controller predicts against the same
    # quantity of redundancy the scheduler will actually emit.
    if mean_burst >= 0.75 * k:
        sr, sa = 0.45, 0.55
    else:
        sr, sa = 0.85, 0.15
    return (int(round(budget_pkts * sr)), int(np.clip(round(budget_pkts * sa), 0, 4)))


def residual_loss(p: float, mean_burst: float, k: int, r: int,
                  n_anchor: int = 0, pi_efficiency: float = 1.0) -> float:
    """Perceptually weighted residual loss after FEC and anchor replicas.

    The two mechanisms are INDEPENDENT and must be modelled as such.  Reed-
    Solomon is a group-level mechanism: it either reconstructs the whole group
    or nothing.  An anchor replica is a frame-level mechanism that works even
    when the group is a total loss, because the replica sits outside the burst
    that destroyed the group.  Treating anchors as a modifier on RS -- as an
    earlier version of this model did -- makes the controller believe
    protection is futile in exactly the burst-dominated conditions where the
    anchor is the only thing that still works.

    `pi_efficiency` in (0, 1] is the PI-AFEC advantage: because redundancy is
    concentrated on the groups that carry meaning, the perceptual weight of the
    residual loss is a fraction of its raw packet count.  Setting it to 1.0
    reproduces uniform FEC exactly, which is how the comparison stays honest.
    """
    if r <= 0:
        base = p
    else:
        pf = group_failure_probability(k, r, p, mean_burst)
        base = pf * p

    if n_anchor > 0:
        # Each replica is displaced past the estimated burst, so it survives
        # unless an INDEPENDENT loss event hits it.  Covering m of k frames
        # removes at most m/k of the residual, weighted by replica survival.
        survive = 1.0 - p
        covered = min(1.0, n_anchor / k) * survive
        base *= (1.0 - covered)
    return float(np.clip(base * pi_efficiency, 0.0, 1.0))


class NavikController:
    """The joint optimiser.  One call per control epoch (default 100 ms)."""

    def __init__(self, k: int = 8, mobile: bool = False,
                 pi_afec: bool = True, safety_margin: float = 0.85,
                 pi_efficiency: float | None = None):
        # `pi_efficiency` is how much cheaper perceptual placement makes a given
        # residual loss LOOK to the optimiser.  The offline value of 0.42 is a
        # perceptual figure and is correct there, because the offline objective
        # is itself perceptually weighted.  Feeding it to a live controller that
        # is judged on RAW frame loss makes it believe its redundancy is 2.4x
        # more effective than it is, so it provisions roughly a quarter of what
        # a uniform scheme buys and loses on both counts.  Live sessions pass a
        # conservative value and keep the perceptual PLACEMENT without claiming
        # the perceptual DISCOUNT up front.
        self.pi_efficiency_override = pi_efficiency
        self.k = k
        self.mobile = mobile
```

```
        self.pi_afec = pi_afec
        self.safety = safety_margin
        self.capacity_est_kbps = 200.0
        self._cap_ewma = 200.0
        self.history: list[ControlDecision] = []

    def update_capacity(self, goodput_kbps: float, loss: float,
                        queue_growing: bool, delay_trend_ms_per_s: float = 0.0) -> None:
        """Capacity estimate with congestion distinguished from plain loss.

        Naive AIMD backs off on ANY loss.  That is correct on a wired path where
        loss means a full queue, and badly wrong on a wireless one where loss
        means a fading radio link.  Backing off there reduces quality without
        relieving anything, and -- worse for this system -- it starves the very
        redundancy budget that the loss made necessary.

        Congestion has a signature that random loss does not: the queue fills
        before it overflows, so ONE-WAY DELAY RISES first.  We therefore treat
        loss as congestion only when it is accompanied by a rising delay trend,
        which is the same discriminator used by modern real-time congestion
        controllers.  Loss with flat delay is read as link loss, and we hold the
        rate and spend the budget on protection instead.
        """
        congested = queue_growing or delay_trend_ms_per_s > 8.0
        if congested:
            self._cap_ewma *= 0.88
        elif loss > 0.02:
            self._cap_ewma *= 0.995          # gentle probe down, not a collapse
        else:
            self._cap_ewma = self._cap_ewma * 0.97 + (goodput_kbps * 1.25) * 0.03
        self._cap_ewma = float(np.clip(self._cap_ewma, 24.0, 2000.0))
        self.capacity_est_kbps = self._cap_ewma

    def decide(self, loss: float, mean_burst: float, jitter_ms: float,
               owd_ms: float, jb_depth_ms: float,
               pi_efficiency: float | None = None) -> ControlDecision:
        if pi_efficiency is None:
            pi_efficiency = self.pi_efficiency_override
        if pi_efficiency is None:
            # Measured empirically in the offline evaluation; see experiments/.
            pi_efficiency = 0.42 if self.pi_afec else 1.0

        budget = self.capacity_est_kbps * self.safety
        best: ControlDecision | None = None

        # Congestion safety rail: never let redundancy exceed what the path can
        # absorb, and taper it as loss rises since high loss usually IS congestion.
        rho_cap = 0.65 if loss < 0.03 else max(0.15, 0.65 - 1.8 * (loss - 0.03))

        for codec in BITRATE_LADDER:
            ie, bpl, kbps = emodel.CODEC_PARAMS[codec]
            for rho in RHO_GRID:
                if rho > rho_cap:
                    continue
                offered = kbps * (1.0 + rho) + HEADER_OVERHEAD_BPS / 1000.0
                if offered > budget:
                    continue
                group_budget = self.k * rho
                if self.pi_afec:
                    r, n_anchor = split_budget(group_budget, self.k, mean_burst)
                else:
                    r, n_anchor = int(round(group_budget)), 0
                p_res = residual_loss(loss, mean_burst, self.k, r,
                                      n_anchor=n_anchor, pi_efficiency=pi_efficiency)
                # A systematic code adds no mouth-to-ear delay of its own; the
                # data packets leave immediately.  Only recovery waits.
                total_delay = owd_ms + jb_depth_ms + 25.0
                r_pred = emodel.r_factor(codec, total_delay, p_res * 100.0,
                                         mean_burst, self.mobile)
                if best is None or r_pred > best.predicted_r:
                    best = ControlDecision(
                        codec=codec, bitrate_kbps=kbps, rho=rho,
                        dscp_high=46, dscp_low=(34 if rho > 0 else 0),
                        predicted_r=r_pred, predicted_mos=emodel.r_to_mos(r_pred),
                        offered_kbps=offered)

        if best is None:
            best = ControlDecision("opus-6k", 6.0, 0.0, 46, 0, 0.0, 1.0,
                                   6.0 + HEADER_OVERHEAD_BPS / 1000.0,
                                   reason="capacity below floor")
        best.reason = (f"budget {budget:.0f}kbps, loss {loss*100:.1f}%, "
                       f"burst {mean_burst:.1f}")
        self.history.append(best)
        return best
```

```
def dscp_for(pi: int, decision: ControlDecision) -> int:
    """Per-packet DiffServ marking driven by perceptual importance.

    This is the cross-layer claim in its most direct form.  A router applying
    WRED to a congested queue drops from the lowest-precedence class first.  By
    marking silence and low-energy frames AF41 (or best-effort) and voiced
    onsets EF, SWARA arranges for the network's own drop decision to be
    perceptually informed -- WITHOUT the router understanding speech, and
    without the router being able to read the speech, since the PI octet is
    authenticated associated data rather than plaintext content.
    """
    if pi >= 160:
        return decision.dscp_high      # EF (46)
    if pi >= 80:
        return decision.dscp_low or 34  # AF41
    return 0                            # best effort
```

Listing 6 `swara/receiver.py` 233 lines

Network-state estimation, the R-factor-optimal jitter buffer and group reassembly.

```
"""
Receiver side: network state estimation, FEC recovery, and the adaptive
playout buffer.

The jitter buffer is where SWARA departs most sharply from convention.

A conventional adaptive jitter buffer targets a LOSS objective: "size the
buffer so that at most 1% of packets arrive too late".  That is the wrong
objective, because it treats the delay it adds as free.  It is not free -- the
E-model's Id term charges 0.024 R-points per millisecond, and above 177 ms it
charges 0.134 per millisecond.

SWARA sizes the buffer by directly maximising the predicted R-factor:

    d* = argmax_d  R( codec, delay_base + d, Ppl(d), burst )

where Ppl(d) is the perceptually weighted residual loss that would result from
a playout deadline of d, computed from the measured arrival-delay histogram.
Deepening the buffer moves packets from "late" to "on time", lowering Ppl; it
simultaneously raises Id.  The optimum is the point where the marginal R gained
from rescued packets equals the marginal R lost to delay -- and because Ppl is
perceptually weighted, the optimum shifts DEEPER when the packets arriving late
happen to be important ones, and shallower when they are silence.
"""

from __future__ import annotations

import math
import numpy as np
from collections import deque
from dataclasses import dataclass, field

from . import emodel
from .perceptual import damage_weight


# ---------------------------------------------------------------------------
# Network state
# ---------------------------------------------------------------------------

@dataclass
class NetworkState:
    """Everything the controller needs, estimated purely from arrivals."""
    loss_rate: float = 0.0          # fraction of expected packets never seen
    mean_burst: float = 1.0         # mean consecutive-loss run length
    jitter_ms: float = 0.0          # RFC 3550 interarrival jitter estimate
    delay_spread_ms: float = 0.0    # p95 - p5 of the relative delay histogram
    rtt_ms: float = 0.0
    reorder_rate: float = 0.0
    goodput_kbps: float = 0.0
    packets_seen: int = 0

    def summary(self) -> str:
        return (f"loss {self.loss_rate*100:5.2f}%  burst {self.mean_burst:4.2f}  "
                f"jitter {self.jitter_ms:5.1f}ms  spread {self.delay_spread_ms:5.1f}ms")


class NetworkStateEstimator:
    """Streaming estimator over a sliding window of arrivals.

    All the statistics come from the media stream itself.  There are no probe
    packets and no separate RTCP-style reports on the wire: the 16-byte
    telemetry block rides in the header of the reverse-direction voice packets,
```

```
    which are being sent anyway.  A duplex call therefore pays ZERO extra
    packets for full closed-loop measurement.
    """

    def __init__(self, window: int = 250, frame_ms: int = 20):
        self.window = window
        self.frame_ms = frame_ms
        self._delays: deque[float] = deque(maxlen=window)
        self._arrivals: deque[tuple[int, float]] = deque(maxlen=window)
        self._bytes: deque[tuple[float, int]] = deque(maxlen=window)
        self._jitter = 0.0
        self._last_transit: float | None = None
        self._highest_seq = -1
        self._received: set[int] = set()
        self._loss_runs: list[int] = []
        self._cur_run = 0
        self._reorders = 0
        self._total = 0

    def on_arrival(self, seq: int, send_ts_ms: float, recv_ts_ms: float,
                   size_bytes: int) -> None:
        transit = recv_ts_ms - send_ts_ms
        self._delays.append(transit)
        self._arrivals.append((seq, recv_ts_ms))
        self._bytes.append((recv_ts_ms, size_bytes))
        self._total += 1

        # RFC 3550 interarrival jitter: exponentially smoothed |D(i-1, i)|.
        if self._last_transit is not None:
            d = abs(transit - self._last_transit)
            self._jitter += (d - self._jitter) / 16.0
        self._last_transit = transit

        if seq < self._highest_seq:
            self._reorders += 1
        else:
            gap = seq - self._highest_seq - 1 if self._highest_seq >= 0 else 0
            if gap > 0:
                self._loss_runs.append(gap)
            self._highest_seq = seq
        self._received.add(seq)
        if len(self._received) > self.window * 3:
            cutoff = self._highest_seq - self.window * 2
            self._received = {s for s in self._received if s >= cutoff}

    def snapshot(self, rtt_ms: float = 0.0) -> NetworkState:
        expected = max(1, self._highest_seq + 1)
        seen = min(self._total, expected)
        loss = 1.0 - seen / expected if expected else 0.0
        runs = self._loss_runs[-80:]
        burst = float(np.mean(runs)) if runs else 1.0

        if len(self._delays) >= 8:
            arr = np.fromiter(self._delays, dtype=float)
            spread = float(np.percentile(arr, 95) - np.percentile(arr, 5))
        else:
            spread = 0.0

        if len(self._bytes) >= 2:
            t0, t1 = self._bytes[0][0], self._bytes[-1][0]
            span = max(1e-3, (t1 - t0) / 1000.0)
            total_b = sum(b for _, b in self._bytes)
            goodput = total_b * 8.0 / span / 1000.0
        else:
            goodput = 0.0

        return NetworkState(
            loss_rate=float(np.clip(loss, 0.0, 1.0)),
            mean_burst=max(1.0, burst),
            jitter_ms=self._jitter,
            delay_spread_ms=spread,
            rtt_ms=rtt_ms,
            reorder_rate=self._reorders / max(1, self._total),
            goodput_kbps=goodput,
            packets_seen=self._total,
        )

    def relative_delays(self) -> np.ndarray:
        """Arrival delays relative to the minimum observed -- the histogram the
        jitter buffer optimiser needs."""
        if not self._delays:
            return np.zeros(0)
        arr = np.fromiter(self._delays, dtype=float)
        return arr - arr.min()
```

```
# ---------------------------------------------------------------------------
# R-factor-optimal adaptive jitter buffer
# ---------------------------------------------------------------------------

@dataclass
class JitterBufferDecision:
    depth_ms: float
    predicted_r: float
    predicted_mos: float
    late_loss_pct: float
    considered: list[tuple[float, float]] = field(default_factory=list)


class AdaptiveJitterBuffer:
    """Playout buffer whose depth is chosen to maximise predicted R-factor.

    `importance_of_late` lets the optimiser know whether the packets that are
    currently arriving late are perceptually valuable.  When they are, the
    optimiser accepts more delay to rescue them; when they are silence, it
    keeps the buffer shallow and the conversation responsive.
    """

    DEPTH_GRID_MS = (0, 10, 20, 30, 40, 60, 80, 100, 120, 150, 180)

    def __init__(self, base_delay_ms: float = 40.0, frame_ms: int = 20,
                 min_depth_ms: float = 20.0, max_depth_ms: float = 180.0):
        self.base_delay_ms = base_delay_ms
        self.frame_ms = frame_ms
        self.min_depth = min_depth_ms
        self.max_depth = max_depth_ms
        self.depth_ms = 60.0
        self._smoothed = 60.0

    def optimise(self, rel_delays: np.ndarray, codec: str, mean_burst: float,
                 other_delay_ms: float, late_importance: float = 0.5,
                 mobile: bool = False) -> JitterBufferDecision:
        if rel_delays.size < 8:
            return JitterBufferDecision(self.depth_ms, 0.0, 0.0, 0.0)

        considered = []
        best = (-1.0, self.depth_ms, 0.0)
        for d in self.DEPTH_GRID_MS:
            if not (self.min_depth <= d <= self.max_depth):
                continue
            late_frac = float(np.mean(rel_delays > d))
            # Perceptually weight the late packets: a late silence frame costs
            # almost nothing, a late voiced onset costs more than one packet.
            eff_loss_pct = 100.0 * late_frac * (0.25 + 1.5 * late_importance)
            total_delay = other_delay_ms + d
            r = emodel.r_factor(codec, total_delay, eff_loss_pct, mean_burst, mobile)
            considered.append((float(d), r))
            if r > best[0]:
                best = (r, float(d), late_frac)

        r_best, d_best, late_frac = best
        # Hysteresis: buffer depth changes are audible, so move gently.
        self._smoothed += 0.35 * (d_best - self._smoothed)
        self.depth_ms = float(np.clip(self._smoothed, self.min_depth, self.max_depth))
        return JitterBufferDecision(
            depth_ms=self.depth_ms, predicted_r=r_best,
            predicted_mos=emodel.r_to_mos(r_best),
            late_loss_pct=100.0 * late_frac, considered=considered)


# ---------------------------------------------------------------------------
# FEC-aware reassembly
# ---------------------------------------------------------------------------

@dataclass
class GroupBuffer:
    """Collects the packets of one FEC group until it can be decoded."""
    group_id: int
    k: int
    r: int
    shards: dict = field(default_factory=dict)
    pi: dict = field(default_factory=dict)
    first_seen_ms: float = 0.0
    resolved: bool = False

    def complete(self) -> bool:
        return len([i for i in self.shards if i < self.k]) == self.k

    def decodable(self) -> bool:
        return len(self.shards) >= self.k
```

Listing 7 `swara/session.py` 607 lines

The live call pipeline: sender, receiver, playout and control threads.

```
"""
The live media session.

This is where the whole system actually runs.  Three threads per call:

  SENDER    capture -> encode -> perceptual analysis -> FEC scheduling ->
            replica piggyback -> AEAD seal -> UDP sendto
  RECEIVER  UDP recvfrom -> AEAD open -> replica extraction -> insert into the
            reassembly buffer -> telemetry accounting
  PLAYOUT   every 20 ms: take the next frame in sequence, recovering it from
            parity or from a replica if the original never arrived, conceal it
            if nothing can be recovered, decode, write to the speaker

The sender and playout threads are paced independently against a monotonic
clock rather than by sleeping a fixed interval, so accumulated scheduling error
does not cause the call to drift.

Media plaintext layout (before AEAD sealing):

    +--------+-----------------------------------------+----------------+
    | nrep 1B| nrep x { seq 2B, len 2B, payload ... }  | primary frame  |
    +--------+-----------------------------------------+----------------+

The 16-byte SWARA header is supplied as associated data, so the perceptual
importance octet is readable on the wire but cannot be altered.
"""

from __future__ import annotations

import struct
import threading
import time
import numpy as np
from collections import defaultdict, deque

from . import emodel
from .auth import MediaCipher, ReplayWindow, SequenceReconstructor
from .codec import AdpcmCodec, Mode, conceal, frame_samples
from .controller import NavikController, dscp_for
from .fec import FecScheduler, ReedSolomon
from .packet import AEAD_TAG_LEN, HEADER_LEN, Datagram, Flag, Header, PType
from .perceptual import PerceptualImportanceAnalyzer, damage_weight
from .receiver import AdaptiveJitterBuffer, NetworkStateEstimator

FRAME_MS = 20
GROUP_K = 4     # see _min_depth_for_fec: parity must beat its own deadline
OUT_LEN = 320                       # 20 ms @ 16 kHz


def pack_telemetry(loss: float, burst: float, jitter_ms: float, mos: float) -> bytes:
    """16-byte report, piggybacked on reverse-direction media -- costs no packet."""
    return struct.pack("!HHHHQ",
                       int(min(65535, loss * 10000)),
                       int(min(65535, burst * 100)),
                       int(min(65535, jitter_ms * 100)),
                       int(min(65535, mos * 1000)), 0)


def unpack_telemetry(b: bytes):
    if len(b) < 16:
        return None
    loss, burst, jit, mos, _ = struct.unpack("!HHHHQ", b[:16])
    return {"loss": loss / 10000.0, "burst": burst / 100.0,
            "jitter_ms": jit / 100.0, "mos": mos / 1000.0}


class CallStats:
    def __init__(self) -> None:
        self.lock = threading.Lock()
        self.tx_frames = 0
        self.tx_parity = 0
        self.tx_replicas = 0
        self.tx_bytes = 0
        self.rx_frames = 0
        self.rx_late = 0
        self.rx_auth_fail = 0
        self.played = 0
        self.recovered_fec = 0
        self.recovered_replica = 0
        self.concealed = 0
        self.pi_sum = 0
        self.lost_pi_sum = 0
        self.peer_mos = 0.0
```

```
84
85      def snapshot(self) -> dict:
86          with self.lock:
87              d = dict(self.__dict__)
88          d.pop("lock", None)
89          played = max(1, self.played)
90          d["loss_after_repair_pct"] = 100.0 * self.concealed / played
91          d["recovery_pct"] = 100.0 * (self.recovered_fec + self.recovered_replica) / played
92          return d
93
94
95  class CallSession:
96      """One end of one call.  Owns the codec, the protection state and the keys."""
97
98      def __init__(self, endpoint, peer_addr, tx_key: bytes, rx_key: bytes,
99                   call_id: int, source, sink, k: int = GROUP_K,
100                  mode: int = Mode.WIDEBAND, adaptive: bool = True,
101                  protection: str = "swara", name: str = "peer"):
102         self.ep = endpoint
103         self.peer = peer_addr
104         self.call_id = call_id
105         self.name = name
106         self.source = source
107         self.sink = sink
108         self.k = k
109         self.mode = mode
110         self.adaptive = adaptive
111         self.protection = protection          # "swara" | "uniform" | "none"
112
113         self.tx = MediaCipher(tx_key, b"tx")
114         self.rx = MediaCipher(rx_key, b"tx")   # peer's tx direction
115
116         self.codec = AdpcmCodec(mode)
117         self.analyzer = PerceptualImportanceAnalyzer()
118         self.sched = FecScheduler(k=k, rho_net=0.0, mean_burst=1.0)
119         # Provisioning is deliberately IDENTICAL for every protection scheme.
120         #
121         # Letting the controller anticipate the perceptual benefit made it buy
122         # less redundancy than the uniform comparator -- 83 parity packets
123         # against 373 -- so SWARA was being compared at a fifth of the budget
124         # and lost on raw frame loss as a direct result.  That is not the
125         # experiment this project claims to run.  The claim is "the same budget,
126         # spent where it is heard", so the budget is sized the same way for all
127         # arms and only the PLACEMENT differs.
128         self.navik = NavikController(k=k, pi_afec=False, pi_efficiency=1.0)
129         self.navik.capacity_est_kbps = self.navik._cap_ewma = 400.0
130         self.jb = AdaptiveJitterBuffer(base_delay_ms=0.0)
131         self.est = NetworkStateEstimator()
132         self.stats = CallStats()
133
134         # Start at a depth the protection scheme can actually use.  Starting
135         # shallow and deepening later costs a stall the listener can hear.
136         self.depth_ms = 60.0 if protection == "none" else float((k + 3) * FRAME_MS)
137         self.rho = 0.0
138
139         self._seq = 0
140         self._group_id = 0
141         self._pending: list[tuple[int, bytes, int]] = []   # (seq, payload, pi)
142         self._replica_queue: dict[int, list[tuple[int, bytes]]] = defaultdict(list)
143
144         self._rx_payload: dict[int, bytes] = {}
145         self._rx_parity: dict[int, dict[int, bytes]] = defaultdict(dict)
146         self._rx_group_r: dict[int, int] = {}
147         self._rx_lock = threading.Lock()
148         self._roc = SequenceReconstructor()
149         self._replay = ReplayWindow()
150         self._first_seq: int | None = None
151         self._play_seq: int | None = None
152         self._last_pcm: np.ndarray | None = None
153         self._conceal_run = 0
154         self._epoch = 0.0
155         self._epoch_seq = 0
156         self._spread_ms = 0.0        # p95 - min of observed one-way delay
157         self._delays: deque = deque(maxlen=300)
158         # Perceptual accounting.  A raw count of concealed frames is the wrong
159         # scoreboard for this system: PI-AFEC deliberately declines to protect
160         # silence, so it will always concede raw frames to a scheme that
161         # protects everything equally.  What it claims to reduce is the
162         # perceptual DAMAGE, so that is what has to be measured.
163         self.sent_pi: dict[int, int] = {}
164         self.concealed_seqs: list[int] = []
165
166         self._running = threading.Event()
167         self._threads: list[threading.Thread] = []
168         self.t0 = time.monotonic()
169         self.on_frame_sent = None            # optional UI callback
```

```
        self.finished = threading.Event()

    # ------------------------------------------------------------------
    def _now_ms(self) -> float:
        return (time.monotonic() - self.t0) * 1000.0

    def start(self) -> None:
        self._running.set()
        for fn, nm in ((self._sender_loop, "tx"), (self._receiver_loop, "rx"),
                       (self._playout_loop, "play"), (self._control_loop, "ctl")):
            t = threading.Thread(target=fn, name=f"{self.name}-{nm}", daemon=True)
            t.start()
            self._threads.append(t)

    def stop(self) -> None:
        self._running.clear()
        for t in self._threads:
            t.join(timeout=2.0)

    # ------------------------------------------------------------------
    # SENDER
    # ------------------------------------------------------------------
    def _sender_loop(self) -> None:
        next_tick = time.monotonic()
        while self._running.is_set():
            pcm = self.source.read(timeout=0.5)
            if pcm is None:
                break
            payload = self.codec.encode(pcm)
            pi = self.analyzer.analyze(pcm.astype(np.float64) / 32768.0).pi
            self._pending.append((self._seq, payload, pi))
            self.sent_pi[self._seq] = pi
            with self.stats.lock:
                self.stats.pi_sum += pi

            self._emit_media(self._seq, payload, pi)
            self._seq += 1

            if len(self._pending) >= self.k:
                self._close_group()

            next_tick += FRAME_MS / 1000.0
            sleep = next_tick - time.monotonic()
            if sleep > 0:
                time.sleep(sleep)
            else:
                next_tick = time.monotonic()      # we fell behind; resynchronise
        self._running.clear()
        self.finished.set()

    def _emit_media(self, seq: int, payload: bytes, pi: int) -> None:
        reps = self._replica_queue.pop(seq, [])
        body = bytearray([len(reps)])
        for rseq, rpay in reps:
            # A displacement survives sequence wrap; an absolute 16-bit value
            # does not.  The receiver reconstructs the source from its own
            # full counter for the carrier packet.
            body += struct.pack("!BH", min(255, seq - rseq), len(rpay)) + rpay
        body += payload

        h = Header(ptype=PType.MEDIA, seq=seq & 0xFFFF, cid=self.call_id,
                   ts_ms=int(self._now_ms()), pi=pi,
                   fec_group=self._group_id & 0xFF, fec_index=seq % self.k,
                   flags=Flag.ENCRYPTED)
        tel = pack_telemetry(*self._telemetry_values())
        h.flags |= Flag.TELEMETRY_PIGGYBACK
        hdr = h.pack()
        ct = self.tx.seal(seq, hdr + tel, bytes(body))
        self.ep.set_dscp(dscp_for(pi, self.navik.history[-1]) if self.navik.history else 0)
        self.ep.sendto(hdr + tel + ct, self.peer)
        with self.stats.lock:
            self.stats.tx_frames += 1
            self.stats.tx_bytes += len(hdr) + len(tel) + len(ct)
            self.stats.tx_replicas += len(reps)

    def _close_group(self) -> None:
        group = self._pending[: self.k]
        self._pending = self._pending[self.k:]
        gid = self._group_id
        self._group_id = (self._group_id + 1) & 0xFF

        if self.protection == "none":
            return
        pis = [p for _, _, p in group]
        plan = self.sched.plan(pis) if self.protection == "swara" else None
        if self.protection == "uniform":
```

```
            r = int(round(self.k * self.rho))
            anchors = []
        else:
            r, anchors = plan.r, plan.anchors

        # Replica displacement.  The copy must clear the loss burst AND still
        # beat its own frame's playout instant.  The usable budget is not the
        # whole buffer depth: the copy is itself subject to the same delay
        # variation as everything else, so the p95 delay spread has to be
        # subtracted first.  Ignoring that term places the replica about one
        # frame too late and it is discarded on arrival -- which is precisely
        # what happened before this was corrected, with only 7% of transmitted
        # replicas ever being used.
        usable_ms = self.depth_ms - self._spread_ms - FRAME_MS
        max_disp = max(1, int(usable_ms // FRAME_MS))
        for a_idx, a_delay in anchors:
            src_seq, src_pay, _ = group[a_idx]
            disp = min(a_delay, max_disp)
            if disp >= 1:
                self._replica_queue[src_seq + disp].append((src_seq, src_pay))

        if r <= 0:
            return
        shard_len = max(len(p) for _, p, _ in group)
        shards = [p.ljust(shard_len, b"\x00") for _, p, _ in group]
        try:
            parity = ReedSolomon(self.k, r).encode(shards)
        except Exception:
            return
        base_seq = group[0][0]
        for j, par in enumerate(parity):
            h = Header(ptype=PType.FEC, seq=(base_seq + self.k + j) & 0xFFFF,
                       cid=self.call_id, ts_ms=int(self._now_ms()), pi=0,
                       fec_group=gid, fec_index=0x80 | j, flags=Flag.ENCRYPTED)
            hdr = h.pack()
            body = struct.pack("!HBBH", base_seq & 0xFFFF, self.k, r, shard_len) + par
            ct = self.tx.seal(h.seq, hdr, body)
            self.ep.sendto(hdr + ct, self.peer)
            with self.stats.lock:
                self.stats.tx_parity += 1
                self.stats.tx_bytes += len(hdr) + len(ct)

    # ------------------------------------------------------------------
    # RECEIVER
    # ------------------------------------------------------------------
    def _receiver_loop(self) -> None:
        while self._running.is_set():
            data, addr = self.ep.recvfrom(timeout=0.4)
            if data is None or len(data) < HEADER_LEN:
                continue
            try:
                h = Header.unpack(data)
            except Exception:
                continue
            if h.cid != self.call_id:
                continue
            off = HEADER_LEN
            tel = b""
            if h.flags & Flag.TELEMETRY_PIGGYBACK:
                tel = data[off:off + 16]; off += 16
            aad = data[:off]
            ct = data[off:]

            full_seq = self._roc.unwrap(h.seq)
            pt = self.rx.open(full_seq, aad, ct)
            if pt is None:
                with self.stats.lock:
                    self.stats.rx_auth_fail += 1
                continue
            if h.ptype == PType.MEDIA and not self._replay.check_and_set(full_seq):
                continue          # duplicate or replayed packet

            if tel:
                t = unpack_telemetry(tel)
                if t:
                    with self.stats.lock:
                        self.stats.peer_mos = t["mos"]

            if h.ptype == PType.MEDIA:
                self._on_media(h, pt, len(data), full_seq)
            elif h.ptype == PType.FEC:
                self._on_parity(h, pt)

    def _on_media(self, h: Header, pt: bytes, wire_len: int, full_seq: int) -> None:
        nrep = pt[0]
```

```
        off = 1
        reps = []
        for _ in range(nrep):
            if off + 3 > len(pt):
                break
            disp, rlen = struct.unpack("!BH", pt[off:off + 3]); off += 3
            reps.append((full_seq - disp, pt[off:off + rlen])); off += rlen
        primary = pt[off:]

        recv_ms = self._now_ms()
        self._delays.append(recv_ms - h.ts_ms)
        self.est.on_arrival(full_seq, h.ts_ms, recv_ms, wire_len)

        with self._rx_lock:
            if self._first_seq is None:
                self._first_seq = full_seq
                self._play_seq = full_seq
            if full_seq not in self._rx_payload:
                self._rx_payload[full_seq] = primary
                with self.stats.lock:
                    self.stats.rx_frames += 1
            for rseq, rpay in reps:
                if rseq not in self._rx_payload:
                    self._rx_payload[rseq] = rpay
                    with self.stats.lock:
                        self.stats.recovered_replica += 1

    def _on_parity(self, h: Header, pt: bytes) -> None:
        if len(pt) < 6:
            return
        base_seq, k, r, shard_len = struct.unpack("!HBBH", pt[:6])
        base_full = self._roc.nearest(base_seq)
        par = pt[6:]
        j = h.fec_index & 0x7F
        with self._rx_lock:
            self._rx_parity[h.fec_group][j] = par
            self._rx_group_r[h.fec_group] = (base_full, k, r, shard_len)

    def _try_fec(self, gid: int) -> bool:
        """Attempt to reconstruct a group.  Returns True if anything recovered."""
        meta = self._rx_group_r.get(gid)
        if not meta:
            return False
        base_seq, k, r, shard_len = meta
        have = {}
        missing = []
        for i in range(k):
            s = base_seq + i
            if s in self._rx_payload:
                have[i] = self._rx_payload[s].ljust(shard_len, b"\x00")
            else:
                missing.append((i, s))
        if not missing:
            return False
        for j, par in self._rx_parity.get(gid, {}).items():
            have[k + j] = par
        if len(have) < k:
            return False
        try:
            out = ReedSolomon(k, r).decode(have, shard_len)
        except Exception:
            return False
        if out is None:
            return False
        rescued = False
        for i, s in missing:
            if i in out:
                self._rx_payload[s] = out[i].rstrip(b"\x00") or out[i]
                rescued = True
                with self.stats.lock:
                    self.stats.recovered_fec += 1
        return rescued

    # ------------------------------------------------------------------
    # PLAYOUT
    # ------------------------------------------------------------------
    def _playout_loop(self) -> None:
        # Fill to the target depth before starting.  Playout thereafter runs
        # against an ABSOLUTE schedule anchored at `_epoch`, not a free-running
        # 20 ms tick.  This matters: with a free-running tick the buffer depth
        # is whatever happened to accumulate at startup and can never change,
        # so raising `depth_ms` later has no effect at all and any parity that
        # needs the extra slack arrives too late to be used.  Moving `_epoch`
        # forward genuinely deepens the buffer while the call is running.
        deadline = time.monotonic() + 5.0
```

```
        while self._running.is_set() and time.monotonic() < deadline:
            with self._rx_lock:
                ready = self._play_seq is not None and \
                    len(self._rx_payload) >= max(2, int(self.depth_ms // FRAME_MS))
            if ready:
                break
            time.sleep(0.005)

        with self._rx_lock:
            self._epoch = time.monotonic()
            self._epoch_seq = self._play_seq if self._play_seq is not None else 0
        idle = 0
        while self._running.is_set():
            # Wait until this slot is actually due.
            with self._rx_lock:
                cur = self._play_seq
                ep, eps = self._epoch, self._epoch_seq
            if cur is not None:
                due = ep + (cur - eps) * (FRAME_MS / 1000.0)
                sl = due - time.monotonic()
                if sl > 0:
                    time.sleep(min(sl, 0.25))
                    continue

            with self._rx_lock:
                if self._play_seq is None:
                    seq = None
                else:
                    seq = self._play_seq
                    pay = self._rx_payload.pop(seq, None)
                    if pay is None:
                        gid = (seq // self.k) & 0xFF
                        for g in (gid, (gid - 1) & 0xFF):
                            if self._try_fec(g):
                                break
                        pay = self._rx_payload.pop(seq, None)
                    if pay is not None:
                        self._play_seq = seq + 1

            if seq is None:
                time.sleep(0.01)
                continue

            if pay is not None:
                idle = 0
                try:
                    pcm = self.codec.decode(pay)
                except Exception:
                    pcm = conceal(self._last_pcm, OUT_LEN, self._conceal_run)
                self._last_pcm = pcm
                self._conceal_run = 0
                with self.stats.lock:
                    self.stats.played += 1
            else:
                # Nothing recoverable for this slot.
                still_coming = any(s > seq for s in self._rx_payload)
                if not still_coming:
                    idle += 1
                    if idle > 60 and self.finished.is_set():
                        break
                    time.sleep(0.005)
                    continue
                pcm = conceal(self._last_pcm, OUT_LEN, self._conceal_run)
                self._conceal_run += 1
                with self._rx_lock:
                    self._play_seq = seq + 1
                self.concealed_seqs.append(seq)
                with self.stats.lock:
                    self.stats.concealed += 1
                    self.stats.played += 1

            self.sink.write(pcm)

    # ------------------------------------------------------------------
    # CONTROL
    # ------------------------------------------------------------------
    def _telemetry_values(self):
        st = self.est.snapshot()
        s = self.stats
        played = max(1, s.played)
        ppl = 100.0 * s.concealed / played
        m = emodel.mos("opus-24k", 40.0 + self.depth_ms, ppl, max(1.0, st.mean_burst))
        return st.loss_rate, st.mean_burst, st.jitter_ms, m

    def _delay_trend(self) -> float:
```

```
        """Slope of recent one-way delay, in ms per second.

        A rising slope means a queue is filling somewhere, which is congestion.
        A flat slope with loss present means the link itself is dropping.
        """
        if len(self._delays) < 40:
            return 0.0
        arr = np.fromiter(self._delays, dtype=float)[-100:]
        t = np.arange(arr.size) * (FRAME_MS / 1000.0)
        try:
            slope = float(np.polyfit(t, arr, 1)[0])
        except Exception:
            slope = 0.0
        return slope

    def _control_loop(self) -> None:
        while self._running.is_set():
            time.sleep(0.1)
            if not self.adaptive:
                continue
            st = self.est.snapshot()
            if st.packets_seen < 25:
                continue
            self.navik.update_capacity(st.goodput_kbps, st.loss_rate, False,
                                       self._delay_trend())
            d = self.navik.decide(st.loss_rate, st.mean_burst, st.jitter_ms,
                                  40.0, self.depth_ms)
            self.rho = d.rho
            self.sched.rho_net = d.rho
            self.sched.mean_burst = st.mean_burst

            if len(self._delays) >= 20:
                arr = np.fromiter(self._delays, dtype=float)
                rel = arr - arr.min()
                dec = self.jb.optimise(rel, "opus-24k", st.mean_burst,
                                       other_delay_ms=float(arr.min()) + 25.0,
                                       late_importance=0.45)
                depth = dec.depth_ms
                # A systematic code emits its parity only once the group is
                # complete.  A frame early in the group is therefore rescuable
                # ONLY if the buffer still holds it when that parity lands.
                # Running FEC with a buffer shallower than the group span means
                # transmitting parity that can never be used, so we either
                # deepen the buffer or we stop paying for parity.
                self._spread_ms = float(np.percentile(rel, 95)) if rel.size else 0.0
                if self.rho > 0:
                    depth = max(depth, self._min_depth_for_protection(st.mean_burst))
                self._retarget_depth(min(depth, 220.0))

    def _retarget_depth(self, new_depth: float) -> None:
        """Change the playout depth on a running call by shifting the epoch.

        Deepening is applied immediately (the listener hears one slightly longer
        gap).  Shallowing is applied gradually, because pulling playout earlier
        means discarding audio, and doing that abruptly is audible.
        """
        delta = new_depth - self.depth_ms
        if abs(delta) < 5.0 or self._epoch == 0.0:
            self.depth_ms = new_depth
            return
        if delta > 0:
            shift = delta
        else:
            shift = max(delta, -10.0)          # at most 10 ms shallower per epoch
        with self._rx_lock:
            self._epoch += shift / 1000.0
        self.depth_ms += shift

    def _min_depth_for_protection(self, mean_burst: float) -> float:
        """Shallowest buffer at which the protection we are paying for can work.

        Two separate requirements, whichever is larger:

          parity    a systematic code emits parity only once the group closes,
                    so rescuing the FIRST frame of a group needs the group span
                    plus a frame of slack.

          replicas  a copy must be displaced past the burst that killed the
                    original, so it needs (burst + 2) frame times, plus the
                    delay spread it will itself experience.

        Buying protection without the depth to use it is pure waste, so if the
        controller will not accept this depth it should not be paying for the
        redundancy either.
        """
        parity_need = (self.k + 1) * FRAME_MS
```

```
        replica_need = (max(1.0, mean_burst) + 2.0) * FRAME_MS + self._spread_ms
        return max(parity_need, replica_need)

    # ------------------------------------------------------------------
    def report(self) -> str:
        s = self.stats.snapshot()
        return (f"[{self.name}] sent {s['tx_frames']} frames "
                f"(+{s['tx_parity']} parity, +{s['tx_replicas']} replicas)  "
                f"played {s['played']}  recovered {s['recovered_fec']} by FEC  "
                f"concealed {s['concealed']} ({s['loss_after_repair_pct']:.1f}%)  "
                f"auth-fail {s['rx_auth_fail']}")
```

Listing 8 `swara/codec.py` 199 lines

IMA ADPCM with per-frame state reset, narrowband mode and concealment.

```
"""
SWARA speech codec -- IMA ADPCM, with one deliberate modification.

This is a REAL codec: it takes 16-bit PCM samples in and produces compressed
bytes out, and the bytes decode back to audible speech.  It is not a bitrate
model.

IMA ADPCM encodes the DIFFERENCE between the current sample and a prediction,
using a quantiser whose step size adapts to the signal.  Four bits per sample
gives 4:1 compression against 16-bit PCM.

The modification, and it matters: standard ADPCM carries its predictor and step
index continuously from one block to the next.  Over a lossy transport that is
fatal -- lose one packet and the decoder's predictor diverges from the
encoder's, so every subsequent frame decodes to garbage until something
resynchronises.  A single lost packet would corrupt the rest of the call.

So SWARA RESETS the codec state at every frame boundary and transmits the
starting predictor and step index in a 3-byte preamble.  Every frame is then
independently decodable.  The cost is a small loss of coding efficiency at
frame boundaries; the benefit is that packet loss damages exactly the frames
that were lost and nothing else.

This is the same reasoning that makes real codecs limit inter-frame prediction
on lossy channels, and it is worth stating in the viva: a codec designed for a
file is the wrong codec for a network.

Two modes are provided so the NAVIK controller has something real to switch
between:

    wideband   16 kHz, 4 bits/sample   ->  320 samples -> 163 B/frame ~ 65 kbit/s
    narrowband  8 kHz, 4 bits/sample   ->  160 samples ->  83 B/frame ~ 33 kbit/s
"""

from __future__ import annotations

import numpy as np
import struct

# IMA ADPCM standard tables.
_STEP_TABLE = ([
    7, 8, 9, 10, 11, 12, 13, 14, 16, 17, 19, 21, 23, 25, 28, 31, 34, 37, 41, 45,
    50, 55, 60, 66, 73, 80, 88, 97, 107, 118, 130, 143, 157, 173, 190, 209, 230,
    253, 279, 307, 337, 371, 408, 449, 494, 544, 598, 658, 724, 796, 876, 963,
    1060, 1166, 1282, 1411, 1552, 1707, 1878, 2066, 2272, 2499, 2749, 3024,
    3327, 3660, 4026, 4428, 4871, 5358, 5894, 6484, 7132, 7845, 8630, 9493,
    10442, 11487, 12635, 13899, 15289, 16818, 18500, 20350, 22385, 24623,
    27086, 29794, 32767])

_INDEX_TABLE = [-1, -1, -1, -1, 2, 4, 6, 8, -1, -1, -1, -1, 2, 4, 6, 8]

SAMPLE_RATE_WB = 16_000
SAMPLE_RATE_NB = 8_000
FRAME_MS = 20


class Mode:
    WIDEBAND = 0     # 16 kHz
    NARROWBAND = 1   # 8 kHz

    RATE = {WIDEBAND: SAMPLE_RATE_WB, NARROWBAND: SAMPLE_RATE_NB}
    NAME = {WIDEBAND: "wideband-16k", NARROWBAND: "narrowband-8k"}


def frame_samples(mode: int) -> int:
    return Mode.RATE[mode] * FRAME_MS // 1000


def nominal_kbps(mode: int) -> float:
```

```
    n = frame_samples(mode)
    payload = 3 + (n + 1) // 2
    return payload * 8 * (1000 / FRAME_MS) / 1000.0


def _decimate2(x: np.ndarray) -> np.ndarray:
    """16 kHz -> 8 kHz with a short anti-alias FIR (no scipy dependency)."""
    h = np.array([0.0155, 0.0605, 0.1664, 0.2470, 0.2470, 0.1664, 0.0605, 0.0155])
    y = np.convolve(x, h, mode="same")
    return y[::2]


def _interp2(x: np.ndarray) -> np.ndarray:
    """8 kHz -> 16 kHz by zero-stuffing plus the same smoothing filter."""
    y = np.zeros(x.size * 2, dtype=np.float64)
    y[::2] = x
    h = np.array([0.0310, 0.1210, 0.3328, 0.4940, 0.4940, 0.3328, 0.1210, 0.0310])
    return np.convolve(y, h, mode="same")


class AdpcmCodec:
    """Stateless across frames by design -- see the module docstring."""

    def __init__(self, mode: int = Mode.WIDEBAND):
        self.mode = mode

    # -- encode ------------------------------------------------------------
    def encode(self, pcm16: np.ndarray) -> bytes:
        """pcm16: int16 samples at 16 kHz.  Returns the coded frame."""
        x = np.asarray(pcm16, dtype=np.float64)
        if self.mode == Mode.NARROWBAND:
            x = _decimate2(x)
        x = np.clip(np.round(x), -32768, 32767).astype(np.int32)

        pred = int(x[0]) if x.size else 0
        index = 24                          # a mid-range starting step
        out = bytearray()
        out += struct.pack("<hB", int(np.clip(pred, -32768, 32767)), index)

        nibbles = []
        step_tab, idx_tab = _STEP_TABLE, _INDEX_TABLE
        for s in x.tolist():
            step = step_tab[index]
            diff = s - pred
            code = 0
            if diff < 0:
                code = 8
                diff = -diff
            delta = step >> 3
            if diff >= step:
                code |= 4; diff -= step; delta += step
            if diff >= (step >> 1):
                code |= 2; diff -= step >> 1; delta += step >> 1
            if diff >= (step >> 2):
                code |= 1; delta += step >> 2
            pred = pred - delta if (code & 8) else pred + delta
            if pred > 32767: pred = 32767
            elif pred < -32768: pred = -32768
            index += idx_tab[code]
            if index > 88: index = 88
            elif index < 0: index = 0
            nibbles.append(code)

        for i in range(0, len(nibbles), 2):
            lo = nibbles[i]
            hi = nibbles[i + 1] if i + 1 < len(nibbles) else 0
            out.append((hi << 4) | lo)
        return bytes(out)

    # -- decode ------------------------------------------------------------
    def decode(self, data: bytes, out_rate: int = SAMPLE_RATE_WB) -> np.ndarray:
        """Returns int16 samples at `out_rate`."""
        if len(data) < 3:
            return np.zeros(frame_samples(Mode.WIDEBAND), dtype=np.int16)
        pred, index = struct.unpack("<hB", data[:3])
        pred = int(pred)
        index = int(np.clip(index, 0, 88))
        body = data[3:]

        n = frame_samples(self.mode)
        step_tab, idx_tab = _STEP_TABLE, _INDEX_TABLE
        out_s = []
        append = out_s.append
        for byte in body:
            for code in (byte & 0x0F, (byte >> 4) & 0x0F):
                if len(out_s) >= n:
```

```
                    break
                step = step_tab[index]
                delta = step >> 3
                if code & 4: delta += step
                if code & 2: delta += step >> 1
                if code & 1: delta += step >> 2
                pred = pred - delta if (code & 8) else pred + delta
                if pred > 32767: pred = 32767
                elif pred < -32768: pred = -32768
                index += idx_tab[code]
                if index > 88: index = 88
                elif index < 0: index = 0
                append(pred)
        while len(out_s) < n:
            append(0)
        y = np.asarray(out_s, dtype=np.float64)
        if self.mode == Mode.NARROWBAND and out_rate == SAMPLE_RATE_WB:
            y = _interp2(y)
        return np.clip(y, -32768, 32767).astype(np.int16)

    @property
    def payload_bytes(self) -> int:
        n = frame_samples(self.mode)
        return 3 + (n + 1) // 2


def conceal(previous: np.ndarray | None, n: int, run: int) -> np.ndarray:
    """Packet loss concealment.

    Repeats the previous decoded frame with progressive attenuation.  This is
    the simplest concealment that is honest about what it is doing: it keeps
    the sound continuous for a frame or two, then fades to silence rather than
    producing a buzzing artefact by repeating indefinitely.
    """
    if previous is None or previous.size == 0:
        return np.zeros(n, dtype=np.int16)
    gain = max(0.0, 1.0 - 0.35 * run)
    src = previous[-n:] if previous.size >= n else np.pad(previous, (0, n - previous.size))
    # Taper the join to avoid a click at the splice point.
    ramp = np.ones(n)
    edge = max(1, n // 8)
    ramp[:edge] = np.linspace(0.6, 1.0, edge)
    ramp[-edge:] = np.linspace(1.0, 0.7, edge)
    return np.clip(src.astype(np.float64) * gain * ramp, -32768, 32767).astype(np.int16)
```

Listing 9 `swara/net.py` 248 lines

UDP endpoint with DSCP marking and the time-based impairment injector (including the scheduler correction of Section 7.10.1).

```
"""
Real UDP transport.

This is an actual `socket.socket(AF_INET, SOCK_DGRAM)`.  Datagrams leave the
machine.  Nothing here is simulated.

One optional feature: a local impairment injector.  For the demonstration we
need to be able to make a healthy LAN behave like a congested 4G link on
demand, and the honest place to do that is at the sending socket, where it is
clearly labelled as artificial rather than buried in the protocol.  The
impairment uses the same Gilbert-Elliott model as the offline evaluation, so
the demo and the measurements are describing the same channel.
"""

from __future__ import annotations

import heapq
import random
import socket
import threading
import time


class Impairment:
    """Gilbert-Elliott loss plus added delay, applied to outgoing datagrams.

    The channel state advances on a WALL-CLOCK tick, not once per packet sent.
    That choice is deliberate and it matters twice over.

    Physically it is more truthful: a radio fade or a queue overflow lasts for a
    DURATION.  It does not last for a number of packets.  A sender that
    transmits twice as many packets during a 100 ms fade loses twice as many
    packets, not the same number.
```

```
34
35      Methodologically it is what makes an A/B comparison valid.  Stepping the
36      chain per packet means a configuration that transmits redundancy walks the
37      Markov chain further per unit time and therefore experiences a measurably
38      different channel from the configuration it is being compared against.  We
39      hit exactly that: three arms configured for 15% loss measured 9.5%, 12.3%
40      and 12.5%.  Any quality difference read off those runs would have been
41      partly an artefact of the instrument.
42      """
43
44      TICK_MS = 20.0        # one frame time
45
46      def __init__(self, seed: int = 1234) -> None:
47          self.enabled = False
48          self.loss_rate = 0.0
49          self.mean_burst = 1.0
50          self.extra_delay_ms = 0.0
51          self.jitter_ms = 0.0
52          self._bad = False
53          # Two INDEPENDENT streams.  The Gilbert-Elliott chain must advance
54          # only with wall-clock time; if it shared an RNG with the jitter
55          # draws, an arm that sends more packets (more jitter draws) would
56          # perturb the loss realisation of the very chain it is being
57          # measured against, and "identical seed" would no longer mean
58          # "identical channel".  That is exactly the instrument artefact the
59          # A/B comparison exists to remove.
60          self._rng = random.Random(seed)               # loss chain only
61          self._rng_jitter = random.Random(seed ^ 0x5A5A5A5A)  # delay only
62          self._last_tick = time.monotonic()
63          self._recompute()
64
65      def _recompute(self) -> None:
66          p = min(max(self.loss_rate, 0.0), 0.95)
67          b = max(1.0, self.mean_burst)
68          if p <= 0.0:
69              self.p_gb, self.p_bg = 0.0, 1.0
70          else:
71              self.p_bg = 1.0 / b
72              self.p_gb = min(1.0, self.p_bg * p / (1.0 - p))
73
74      def configure(self, loss_rate=None, mean_burst=None, extra_delay_ms=None,
75                    jitter_ms=None, enabled=None) -> None:
76          if loss_rate is not None: self.loss_rate = float(loss_rate)
77          if mean_burst is not None: self.mean_burst = float(mean_burst)
78          if extra_delay_ms is not None: self.extra_delay_ms = float(extra_delay_ms)
79          if jitter_ms is not None: self.jitter_ms = float(jitter_ms)
80          if enabled is not None: self.enabled = bool(enabled)
81          self._recompute()
82
83      def should_drop(self) -> bool:
84          if not self.enabled or self.loss_rate <= 0:
85              return False
86          now = time.monotonic()
87          ticks = int((now - self._last_tick) * 1000.0 / self.TICK_MS)
88          if ticks > 0:
89              self._last_tick += ticks * self.TICK_MS / 1000.0
90              for _ in range(min(ticks, 500)):
91                  if self._bad:
92                      if self._rng.random() < self.p_bg:
93                          self._bad = False
94                  elif self._rng.random() < self.p_gb:
95                      self._bad = True
96          return self._bad
97
98      def delay_s(self) -> float:
99          if not self.enabled:
100             return 0.0
101         d = self.extra_delay_ms
102         if self.jitter_ms > 0:
103             d += abs(self._rng_jitter.gauss(0.0, self.jitter_ms))
104         return d / 1000.0
105
106     def reset(self, seed: int | None = None) -> None:
107         """Restart the channel realisation from a known state.
108
109         Calling this at the start of each arm of an A/B comparison makes every
110         arm experience the IDENTICAL sequence of good and bad periods.  Without
111         it each arm draws a different realisation, and with a mean burst of 4
112         packets a 25-second call contains only about 47 bursts -- few enough
113         that the difference between arms is dominated by which bursts happened
114         to land where, not by the protection scheme under test.
115         """
116         base = 1234 if seed is None else seed
117         self._rng = random.Random(base)
118         self._rng_jitter = random.Random(base ^ 0x5A5A5A5A)
```

```
        self._bad = False
        self._last_tick = time.monotonic()

    def describe(self) -> str:
        if not self.enabled:
            return "clean"
        return (f"loss {self.loss_rate*100:.0f}% burst {self.mean_burst:.1f} "
                f"+{self.extra_delay_ms:.0f}ms jitter {self.jitter_ms:.0f}ms")


class UdpEndpoint:
    """A bound UDP socket with counters and optional impairment."""

    def __init__(self, bind_host: str = "0.0.0.0", bind_port: int = 0,
                 impairment: Impairment | None = None):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        try:      # 1 MB buffers so a burst of packets is not dropped by the OS
            self.sock.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 1 << 20)
            self.sock.setsockopt(socket.SOL_SOCKET, socket.SO_SNDBUF, 1 << 20)
        except OSError:
            pass
        self.sock.bind((bind_host, bind_port))
        self.impairment = impairment or Impairment()
        self.tx_packets = self.tx_bytes = 0
        self.rx_packets = self.rx_bytes = 0
        self.dropped_by_impairment = 0
        self._lock = threading.Lock()
        self._delay_q: list = []
        self._dq_lock = threading.Lock()
        self._dq_wake = threading.Event()
        self._dq_thread = None
        self._dq_seq = 0
        self._closing = False

    @property
    def port(self) -> int:
        return self.sock.getsockname()[1]

    @property
    def address(self) -> tuple:
        return self.sock.getsockname()

    def set_dscp(self, dscp: int) -> None:
        """Mark the IP header's DiffServ field -- the cross-layer hook."""
        try:
            self.sock.setsockopt(socket.IPPROTO_IP, socket.IP_TOS, (dscp & 0x3F) << 2)
        except OSError:
            pass          # not permitted on every platform; harmless if refused

    def sendto(self, data: bytes, addr: tuple) -> None:
        if self.impairment.should_drop():
            with self._lock:
                self.dropped_by_impairment += 1
            return
        d = self.impairment.delay_s()
        if d > 0:
            self._enqueue_delayed(data, addr, d)
        else:
            self._raw_send(data, addr)

    # -- delayed delivery ------------------------------------------------
    # A single scheduler thread with a heap, NOT one threading.Timer per
    # packet.  With a timer per packet the operating system's scheduling
    # overhead grows with the number of packets sent, which silently punishes
    # exactly the configurations that transmit redundancy -- an artefact of the
    # measurement apparatus that would look like a property of the protocol.
    def _enqueue_delayed(self, data: bytes, addr: tuple, delay_s: float) -> None:
        with self._dq_lock:
            heapq.heappush(self._delay_q, (time.monotonic() + delay_s, self._dq_seq, data, addr))
            self._dq_seq += 1
            if self._dq_thread is None:
                self._dq_thread = threading.Thread(target=self._delay_pump, daemon=True)
                self._dq_thread.start()
            self._dq_wake.set()

    def _delay_pump(self) -> None:
        while not self._closing:
            with self._dq_lock:
                # The wake flag is consumed HERE, every iteration, not only when
                # the queue runs dry.  Clearing it only on an empty queue left it
                # permanently set during a call (there is always a packet in
                # flight), so wait() returned at once and this thread spun at
                # full speed, starving the media threads of the GIL.  Measured
                # cost of that spin: a 100-400 ms spread in one-way delay that
                # the controller read as congestion.  A set() that races this
```

```
                # clear() is harmless -- the queue head is re-examined below.
                self._dq_wake.clear()
                if not self._delay_q:
                    wait = 0.05
                else:
                    wait = max(0.0, self._delay_q[0][0] - time.monotonic())
            if wait > 0:
                self._dq_wake.wait(min(wait, 0.05))
            now = time.monotonic()
            due = []
            with self._dq_lock:
                while self._delay_q and self._delay_q[0][0] <= now:
                    _, _, d, a = heapq.heappop(self._delay_q)
                    due.append((d, a))
            for d, a in due:
                self._raw_send(d, a)

    def _raw_send(self, data: bytes, addr: tuple) -> None:
        try:
            self.sock.sendto(data, addr)
            with self._lock:
                self.tx_packets += 1
                self.tx_bytes += len(data)
        except OSError:
            pass

    def recvfrom(self, timeout: float | None = 0.5):
        self.sock.settimeout(timeout)
        try:
            data, addr = self.sock.recvfrom(4096)
        except (socket.timeout, OSError):
            return None, None
        with self._lock:
            self.rx_packets += 1
            self.rx_bytes += len(data)
        return data, addr

    def close(self) -> None:
        self._closing = True
        self._dq_wake.set()
        try:
            self.sock.close()
        except OSError:
            pass
```

Listing 10 `swara/emodel.py` 171 lines

ITU-T G.107 E-model and the perceptual loss mapping.

```
"""
ITU-T G.107 E-model -- the objective quality metric SWARA optimises directly.

Almost every adaptive VoIP system optimises a PROXY: minimise loss, or minimise
delay, or maximise throughput.  Those proxies conflict.  Adding jitter buffer
depth reduces late-packet loss but adds mouth-to-ear delay; adding FEC reduces
residual loss but consumes bandwidth that would otherwise raise codec quality.
There is no way to trade them off without a common currency.

The E-model provides that currency.  It maps impairments onto a single
transmission rating R in [0, 100], from which a Mean Opinion Score follows:

    R = R0 - Is - Id - Ie_eff + A

    R0      base signal-to-noise rating (93.2 for a modern network)
    Is      simultaneous impairments (loudness, quantisation) -- constant here
    Id      DELAY impairment, from mouth-to-ear delay
    Ie_eff  EQUIPMENT impairment: codec distortion plus packet loss, with the
            crucial burstiness correction from G.107 Annex E
    A       advantage factor (0 for wired, 10 for mobile convenience)

The key term is:

    Ie_eff = Ie + (95 - Ie) * Ppl / (Ppl / BurstR + Bpl)

Ppl is the residual loss AFTER concealment, Bpl is the codec's packet-loss
robustness, and BurstR is the burst ratio -- the observed mean burst length
divided by the mean burst length that random loss would produce.  BurstR > 1
means loss is clustered, and the same average loss then hurts much more.

SWARA's contribution at this layer: we feed Ie_eff a PERCEPTUALLY WEIGHTED
residual loss rather than a raw packet count.  Losing a packet of silence is
not a 1.0 loss event; losing a vowel onset is worse than 1.0.  Section
`perceptual_ppl` implements that mapping, and it is what allows the controller
to see the benefit of unequal protection at all.
```

```
"""

from __future__ import annotations

import math
from dataclasses import dataclass

# Codec parameters (ITU-T G.113 Appendix I and published Opus measurements).
CODEC_PARAMS = {
    #                Ie   Bpl   nominal kbps
    "G.711":        (0.0, 4.3, 64.0),
    "G.729":        (11.0, 19.0, 8.0),
    "opus-6k":      (23.0, 22.0, 6.0),
    "opus-8k":      (18.0, 22.0, 8.0),
    "opus-12k":     (11.0, 24.0, 12.0),
    "opus-16k":     (7.0, 25.0, 16.0),
    "opus-20k":     (5.0, 26.0, 20.0),
    "opus-24k":     (3.5, 27.0, 24.0),
    "opus-32k":     (2.0, 28.0, 32.0),
}

R0_DEFAULT = 93.2
IS_DEFAULT = 1.4
A_WIRED = 0.0
A_MOBILE = 10.0


def delay_impairment(d_ms: float) -> float:
    """Id: the penalty for mouth-to-ear delay (G.107 simplified form).

    Below ~150 ms conversation feels natural; the term has a knee at 177.3 ms
    after which it rises steeply, which is why an over-deep jitter buffer is a
    quality bug and not merely a latency inconvenience.
    """
    d = max(0.0, d_ms)
    idd = 0.024 * d
    if d > 177.3:
        idd += 0.11 * (d - 177.3)
    return idd


def burst_ratio(mean_burst_len: float, loss_rate: float) -> float:
    """BurstR = observed mean burst length / burst length under random loss.

    Under Bernoulli (random) loss the mean burst length is 1/(1-p), so BurstR
    is 1.0 for random loss and grows as loss becomes clustered.
    """
    p = min(max(loss_rate, 1e-6), 0.95)
    random_burst = 1.0 / (1.0 - p)
    return max(1.0, mean_burst_len / random_burst)


def equipment_impairment(codec: str, ppl_percent: float, burst_r: float = 1.0) -> float:
    """Ie_eff -- codec impairment amplified by residual packet loss."""
    ie, bpl, _ = CODEC_PARAMS[codec]
    ppl = max(0.0, ppl_percent)
    if ppl <= 0:
        return ie
    return ie + (95.0 - ie) * ppl / (ppl / burst_r + bpl)


def r_factor(codec: str, delay_ms: float, ppl_percent: float,
             mean_burst_len: float = 1.0, mobile: bool = False,
             r0: float = R0_DEFAULT) -> float:
    br = burst_ratio(mean_burst_len, ppl_percent / 100.0)
    r = (r0 - IS_DEFAULT - delay_impairment(delay_ms)
         - equipment_impairment(codec, ppl_percent, br)
         + (A_MOBILE if mobile else A_WIRED))
    return float(max(0.0, min(100.0, r)))


def r_to_mos(r: float) -> float:
    """G.107 conversion from transmission rating to Mean Opinion Score."""
    if r <= 0:
        return 1.0
    if r >= 100:
        return 4.5
    mos = 1.0 + 0.035 * r + r * (r - 60.0) * (100.0 - r) * 7.0e-6
    return float(max(1.0, min(4.5, mos)))


def mos(codec: str, delay_ms: float, ppl_percent: float,
        mean_burst_len: float = 1.0, mobile: bool = False) -> float:
    return r_to_mos(r_factor(codec, delay_ms, ppl_percent, mean_burst_len, mobile))


def quality_label(mos_value: float) -> str:
```

```
    if mos_value >= 4.3:
        return "Excellent"
    if mos_value >= 4.0:
        return "Good"
    if mos_value >= 3.6:
        return "Fair"
    if mos_value >= 3.1:
        return "Poor"
    return "Bad"


def perceptual_ppl(lost_damage_sum: float, total_frames: int) -> float:
    """Convert accumulated perceptual damage into an equivalent loss percentage.

    A raw loss counter says "3 of 100 packets were lost -> Ppl = 3%".  That is
    only correct if all packets matter equally.  Here each lost frame
    contributes its damage weight (see perceptual.damage_weight), so a run of
    lost silence contributes almost nothing and a lost vowel onset contributes
    more than one packet's worth.  The result is a loss figure the E-model can
    consume that actually reflects what the listener experienced.
    """
    if total_frames <= 0:
        return 0.0
    return 100.0 * lost_damage_sum / total_frames


@dataclass
class QualityReport:
    codec: str
    delay_ms: float
    raw_loss_pct: float
    perceptual_loss_pct: float
    mean_burst: float
    r: float
    mos: float

    def __str__(self) -> str:
        return (f"{self.codec}: MOS {self.mos:.2f} ({quality_label(self.mos)})  "
                f"R={self.r:.1f}  delay={self.delay_ms:.0f}ms  "
                f"loss raw {self.raw_loss_pct:.1f}% / perceptual "
                f"{self.perceptual_loss_pct:.1f}%  burst={self.mean_burst:.2f}")


def assess(codec: str, delay_ms: float, raw_loss_pct: float,
           perceptual_loss_pct: float, mean_burst: float = 1.0,
           mobile: bool = False) -> QualityReport:
    r = r_factor(codec, delay_ms, perceptual_loss_pct, mean_burst, mobile)
    return QualityReport(codec, delay_ms, raw_loss_pct, perceptual_loss_pct,
                         mean_burst, r, r_to_mos(r))
```

Listing 11 `swara/overlay.py` 164 lines

SETU distance-vector overlay.

```
"""
SETU -- Self-Evaluating Topology Underlay.

("Setu" = bridge.)  An application-layer overlay that chooses relay paths for
group calls, and doubles as the NAT-traversal fallback.

Why an overlay at all?  Because the Internet's routing does not optimise for
what a voice call needs.  BGP picks paths on policy and AS-hop count, not on
latency, and certainly not on jitter or loss.  The consequence is well
documented and easy to reproduce: the triangle inequality is frequently
VIOLATED at the application layer.  The direct path A->B can be measurably
worse than the two-hop path A->C->B, because the direct route traverses a
congested peering point that the detour avoids.

SETU runs a distance-vector computation (Bellman-Ford with split horizon and
poisoned reverse -- the same algorithm family as RIP, which the syllabus
covers) over a composite link cost that expresses everything voice cares about
in one number:

    cost(u,v) = alpha * owd_ms  +  beta * jitter_ms  +  gamma * loss_penalty

    loss_penalty = -10 * log10(1 - p)    "loss as delay-equivalent"

The loss term is the interesting part.  Expressing loss in decibels of survival
probability makes losses ADD along a path exactly as delays do, so a path's
total cost is the plain sum of its links' costs and Bellman-Ford is valid.  A
1% link and a 2% link compose to 2.97% end to end, and -10log10(0.99) +
-10log10(0.98) = -10log10(0.9702).  The arithmetic is exact, not an
approximation.

```

```
Count-to-infinity, the classic distance-vector failure, is handled by poisoned
reverse plus a hard cost ceiling (INFINITY), and because the overlay has tens
of nodes rather than thousands, convergence is sub-second.
"""

from __future__ import annotations

import math
import numpy as np
from dataclasses import dataclass, field

ALPHA = 1.0      # weight on one-way delay (ms)
BETA = 2.0       # weight on jitter -- jitter hurts more than raw delay
GAMMA = 45.0     # weight on the dB loss penalty
INFINITY = 1e9
RELAY_PENALTY = 12.0   # ms-equivalent cost of an extra relay hop (CPU + queueing)


def link_cost(owd_ms: float, jitter_ms: float, loss: float) -> float:
    p = float(np.clip(loss, 0.0, 0.95))
    loss_db = -10.0 * math.log10(max(1e-6, 1.0 - p))
    return ALPHA * owd_ms + BETA * jitter_ms + GAMMA * loss_db


@dataclass
class LinkMeasurement:
    owd_ms: float
    jitter_ms: float
    loss: float

    @property
    def cost(self) -> float:
        return link_cost(self.owd_ms, self.jitter_ms, self.loss)


@dataclass
class OverlayNode:
    node_id: str
    links: dict = field(default_factory=dict)          # neighbour -> LinkMeasurement
    dv: dict = field(default_factory=dict)             # dest -> (cost, next_hop)
    neighbour_dv: dict = field(default_factory=dict)   # neighbour -> their table

    def local_table(self) -> dict:
        t = {self.node_id: (0.0, self.node_id)}
        for n, m in self.links.items():
            t[n] = (m.cost, n)
        return t

    def advertise(self, to: str) -> dict:
        """Split horizon with poisoned reverse: routes learned FROM a neighbour
        are advertised back TO that neighbour at infinite cost, which prevents
        the two of them from forming a mutual-belief routing loop."""
        out = {}
        for dest, (cost, nh) in self.dv.items():
            out[dest] = INFINITY if nh == to and dest != to else cost
        return out

    def recompute(self) -> bool:
        """One Bellman-Ford relaxation.  Returns True if the table changed."""
        new = self.local_table()
        for nb, table in self.neighbour_dv.items():
            if nb not in self.links:
                continue
            base = self.links[nb].cost
            for dest, cost in table.items():
                if dest == self.node_id or cost >= INFINITY:
                    continue
                total = base + cost + RELAY_PENALTY
                if dest not in new or total < new[dest][0]:
                    new[dest] = (total, nb)
        changed = new != self.dv
        self.dv = new
        return changed


class SetuOverlay:
    def __init__(self) -> None:
        self.nodes: dict[str, OverlayNode] = {}
        self.rounds_to_converge = 0

    def add_node(self, node_id: str) -> OverlayNode:
        n = OverlayNode(node_id)
        self.nodes[node_id] = n
        return n

    def set_link(self, a: str, b: str, m: LinkMeasurement) -> None:
        self.nodes[a].links[b] = m
```

```
        self.nodes[b].links[a] = m

    def converge(self, max_rounds: int = 40) -> int:
        for n in self.nodes.values():
            n.dv = n.local_table()
        for rnd in range(1, max_rounds + 1):
            for a in self.nodes.values():
                for b in a.links:
                    self.nodes[b].neighbour_dv[a.node_id] = a.advertise(b)
            changed = any(n.recompute() for n in self.nodes.values())
            if not changed:
                self.rounds_to_converge = rnd
                return rnd
        self.rounds_to_converge = max_rounds
        return max_rounds

    def path(self, src: str, dst: str) -> list[str]:
        p, cur, guard = [src], src, 0
        while cur != dst and guard < 32:
            entry = self.nodes[cur].dv.get(dst)
            if entry is None or entry[0] >= INFINITY:
                return []
            cur = entry[1]
            p.append(cur)
            guard += 1
        return p if cur == dst else []

    def direct_cost(self, a: str, b: str) -> float:
        m = self.nodes[a].links.get(b)
        return m.cost if m else INFINITY

    def improvement_report(self) -> list[dict]:
        """For every pair, compare the direct path against SETU's choice."""
        out = []
        ids = list(self.nodes)
        for i, a in enumerate(ids):
            for b in ids[i + 1:]:
                direct = self.direct_cost(a, b)
                chosen = self.nodes[a].dv.get(b, (INFINITY, None))[0]
                p = self.path(a, b)
                out.append({
                    "pair": f"{a}->{b}", "direct_cost": direct,
                    "setu_cost": chosen, "path": p, "hops": max(0, len(p) - 1),
                    "gain_pct": (100.0 * (direct - chosen) / direct
                                 if direct < INFINITY and direct > 0 else 0.0),
                })
        return out
```

Listing 12 **swara/channel.py** 193 lines

Gilbert–Elliott channel for the offline evaluation and the named profiles.

```
"""
Network impairment model -- the Gilbert-Elliott channel.

Real packet loss on the Internet is not a coin flip.  It is CLUSTERED: a queue
overflows, a radio link fades, a handover stalls, and eight consecutive packets
die together.  Any evaluation of a voice system against uniform Bernoulli loss
overstates how well it works, because Bernoulli loss is the easiest loss there
is -- concealment can interpolate across a single missing 20 ms frame almost
inaudibly, but not across 160 ms.

The Gilbert-Elliott model is the standard two-state Markov chain for this:

        p_GB
    GOOD  ---->  BAD          loss prob in GOOD = p_g  (near 0)
      ^            |          loss prob in BAD  = p_b  (near 1)
      +----<-------+
        p_BG

    steady-state P(BAD) = p_GB / (p_GB + p_BG)
    mean burst length   = 1 / p_BG

We parameterise it the way an operator would: "I want 10% average loss with a
mean burst of 4 packets", and solve for p_GB and p_BG.  Delay is modelled as a
base propagation term plus a heavy-tailed jitter component (log-normal), which
matches measured wide-area delay distributions far better than a Gaussian and
is what makes jitter buffer sizing genuinely hard.
"""

from __future__ import annotations

import heapq
import math
```

```
import random
from dataclasses import dataclass, field


@dataclass
class ChannelConfig:
    loss_rate: float = 0.0        # long-run average packet loss, 0..1
    mean_burst: float = 1.0       # mean consecutive losses when loss occurs
    base_delay_ms: float = 25.0   # one-way propagation + transmission
    jitter_ms: float = 8.0        # scale of the delay variation
    jitter_tail: float = 0.55     # log-normal sigma: higher = heavier tail
    reorder_prob: float = 0.0     # probability a packet is displaced in time
    duplicate_prob: float = 0.0
    capacity_kbps: float = 1000.0 # bottleneck rate; exceeding it causes queueing
    seed: int | None = None

    def label(self) -> str:
        return (f"loss={self.loss_rate*100:.0f}% burst={self.mean_burst:.1f} "
                f"delay={self.base_delay_ms:.0f}±{self.jitter_ms:.0f}ms")


class GilbertElliottChannel:
    """A stateful, replayable UDP path model."""

    def __init__(self, cfg: ChannelConfig):
        self.cfg = cfg
        self.rng = random.Random(cfg.seed)
        self.p_g = 0.0     # loss probability in GOOD state
        self.p_b = 1.0     # loss probability in BAD state
        self._solve_transitions()
        self.state_bad = False
        self.sent = 0
        self.lost = 0
        self.bursts: list[int] = []
        self._cur_burst = 0
        # token-bucket queue state for the bottleneck
        self._queue_bytes = 0.0
        self._last_t = 0.0

    def _solve_transitions(self) -> None:
        p = min(max(self.cfg.loss_rate, 0.0), 0.95)
        b = max(1.0, self.cfg.mean_burst)
        if p <= 0.0:
            self.p_gb, self.p_bg = 0.0, 1.0
            return
        # Loss only happens in BAD (p_b = 1, p_g = 0), so P(BAD) = p.
        self.p_bg = 1.0 / b                       # mean burst length = 1/p_bg
        self.p_gb = self.p_bg * p / (1.0 - p)     # steady state gives P(BAD)=p
        self.p_gb = min(1.0, self.p_gb)

    # -- per-packet decision ---------------------------------------------

    def _step_state(self) -> None:
        if self.state_bad:
            if self.rng.random() < self.p_bg:
                self.state_bad = False
                if self._cur_burst:
                    self.bursts.append(self._cur_burst)
                    self._cur_burst = 0
        else:
            if self.rng.random() < self.p_gb:
                self.state_bad = True

    def _delay(self) -> float:
        """Base delay plus a heavy-tailed positive jitter component."""
        if self.cfg.jitter_ms <= 0:
            return self.cfg.base_delay_ms
        mu = math.log(max(self.cfg.jitter_ms, 1e-3)) - 0.5 * self.cfg.jitter_tail ** 2
        j = self.rng.lognormvariate(mu, self.cfg.jitter_tail)
        return self.cfg.base_delay_ms + j

    def transmit(self, send_time_ms: float, size_bytes: int):
        """Return a list of (arrival_time_ms, delivered: bool) outcomes.

        A packet may be dropped (delivered=False), delivered once, or -- if
        duplication is enabled -- delivered twice.
        """
        self.sent += 1
        self._step_state()

        # Bottleneck queueing: drain the virtual queue since the last packet,
        # then add this packet.  An over-full queue drops (tail drop).
        dt = max(0.0, send_time_ms - self._last_t)
        self._last_t = send_time_ms
        drained = self.cfg.capacity_kbps * 1000.0 / 8.0 * (dt / 1000.0)
        self._queue_bytes = max(0.0, self._queue_bytes - drained)
```

```
        self._queue_bytes += size_bytes
        queue_delay_ms = (self._queue_bytes * 8.0 / (self.cfg.capacity_kbps * 1000.0)) * 1000.0
        if queue_delay_ms > 250.0:
            self.lost += 1
            self._cur_burst += 1
            self._queue_bytes -= size_bytes
            return [(send_time_ms, False)]

        loss_p = self.p_b if self.state_bad else self.p_g
        if self.rng.random() < loss_p:
            self.lost += 1
            self._cur_burst += 1
            return [(send_time_ms, False)]

        arrive = send_time_ms + self._delay() + queue_delay_ms
        if self.cfg.reorder_prob and self.rng.random() < self.cfg.reorder_prob:
            arrive += self.rng.uniform(20.0, 90.0)     # displaced behind others
        out = [(arrive, True)]
        if self.cfg.duplicate_prob and self.rng.random() < self.cfg.duplicate_prob:
            out.append((arrive + self.rng.uniform(1.0, 15.0), True))
        return out

    # -- observed statistics ------------------------------------------------

    @property
    def observed_loss(self) -> float:
        return self.lost / self.sent if self.sent else 0.0

    @property
    def observed_mean_burst(self) -> float:
        b = self.bursts + ([self._cur_burst] if self._cur_burst else [])
        return sum(b) / len(b) if b else 0.0


class DeliveryQueue:
    """Reorders packets by arrival time so the receiver sees them as the
    network would actually present them -- out of order when the path
    reorders, and never before their arrival timestamp."""

    def __init__(self) -> None:
        self._h: list = []
        self._c = 0

    def push(self, arrive_ms: float, item) -> None:
        heapq.heappush(self._h, (arrive_ms, self._c, item))
        self._c += 1

    def pop_until(self, now_ms: float):
        out = []
        while self._h and self._h[0][0] <= now_ms:
            t, _, item = heapq.heappop(self._h)
            out.append((t, item))
        return out

    def drain(self):
        out = []
        while self._h:
            t, _, item = heapq.heappop(self._h)
            out.append((t, item))
        return out

    def __len__(self) -> int:
        return len(self._h)


# Preset path profiles used throughout the evaluation.
PROFILES = {
    "ideal_lan":     ChannelConfig(0.00, 1.0, 5.0, 1.0, 0.30, 0.000, capacity_kbps=10_000),
    "good_wifi":     ChannelConfig(0.01, 1.5, 20.0, 6.0, 0.50, 0.005, capacity_kbps=2_000),
    "busy_wifi":     ChannelConfig(0.05, 3.0, 35.0, 14.0, 0.60, 0.020, capacity_kbps=800),
    "congested_4g":  ChannelConfig(0.10, 4.0, 60.0, 25.0, 0.70, 0.030, capacity_kbps=400),
    "poor_3g":       ChannelConfig(0.20, 6.0, 110.0, 45.0, 0.80, 0.050, capacity_kbps=200),
    "hostel_wifi":   ChannelConfig(0.15, 8.0, 45.0, 30.0, 0.85, 0.040, capacity_kbps=300),
    "satellite":     ChannelConfig(0.03, 2.0, 280.0, 20.0, 0.55, 0.010, capacity_kbps=600),
}
```

Listing 13 `swara/engine.py` 342 lines

The offline evaluation engine and three-way comparison.

```
"""
End-to-end evaluation engine.

Runs a complete SWARA call -- perceptual analysis, FEC scheduling, encryption,
```

```
a Gilbert-Elliott path, reassembly, erasure decoding, adaptive playout, and
E-model scoring -- and does the same for two comparators, so every number in
the results is a like-for-like measurement rather than an assertion.

  BASELINE       Plain RTP-style UDP.  No FEC.  Conventional jitter buffer
                 sized at the 95th percentile of observed delay.  This is what
                 a straightforward socket-programming voice chat does.

  UNIFORM-FEC    Loss-driven adaptive FEC given EXACTLY the redundancy budget
                 SWARA spent, spread evenly over all groups.  Conventional
                 jitter buffer.  Essentially RFC 5109 ULP-FEC / FlexFEC
                 behaviour.  Any gain SWARA shows over this is attributable to
                 the perceptual allocation, not to "SWARA has FEC".

  SWARA          PI-AFEC + piggybacked perceptual anchor replicas + R-factor
                 optimal playout + NAVIK joint control.

A note on the anchor replica, because it drove a redesign.  A replica is only
useful if it arrives before its own frame's playout deadline.  Frame i is
played at  i*T + base + depth; a replica riding on frame i+D arrives at
(i+D)*T + transit.  Usefulness therefore requires D*T <= depth, approximately.
The jitter buffer depth is a HARD UPPER BOUND on how far a replica may be
displaced -- so the R-factor-optimal buffer is simultaneously choosing how much
burst decorrelation the anchor mechanism is allowed to buy.  These are not two
independent knobs; they are one coupled decision, and SWARA solves them jointly.

The replica is PIGGYBACKED onto an existing packet rather than sent as its own
datagram, so it costs only its payload bytes -- no second set of SWARA, UDP and
IP headers.  At 24 kbit/s that is 60 bytes instead of 120.
"""

from __future__ import annotations

import numpy as np
from collections import defaultdict
from dataclasses import dataclass, field

from . import emodel
from .channel import ChannelConfig, GilbertElliottChannel
from .controller import NavikController
from .fec import FecScheduler, UniformFecScheduler
from .packet import AEAD_TAG_LEN, HEADER_LEN
from .perceptual import PerceptualImportanceAnalyzer, damage_weight
from .receiver import AdaptiveJitterBuffer, NetworkStateEstimator
from .speech import SpeechGenerator

FRAME_MS = 20
GROUP_K = 8
UDP_IP_OVERHEAD = 28            # 8 UDP + 20 IPv4
ALGORITHMIC_DELAY_MS = 25.0     # encode + decode + resampling


@dataclass
class CallResult:
    system: str
    profile: str
    frames_total: int = 0
    frames_delivered: int = 0
    frames_recovered_fec: int = 0
    frames_recovered_anchor: int = 0
    frames_lost: int = 0
    frames_late: int = 0
    channel_loss_pct: float = 0.0
    residual_loss_pct: float = 0.0
    perceptual_loss_pct: float = 0.0
    mean_burst: float = 1.0
    jitter_ms: float = 0.0
    playout_delay_ms: float = 0.0
    mouth_to_ear_ms: float = 0.0
    wire_kbps: float = 0.0
    redundancy_pct: float = 0.0
    packets_sent: int = 0
    codec: str = "opus-24k"
    r_factor: float = 0.0
    mos: float = 0.0
    lost_pi_mean: float = 0.0
    delivered_pi_mean: float = 0.0
    jb_curve: list = field(default_factory=list)

    def row(self) -> str:
        return (f"{self.system:<12} MOS {self.mos:4.2f}  R {self.r_factor:5.1f}  "
                f"chan {self.channel_loss_pct:5.2f}%  resid {self.residual_loss_pct:5.2f}%  "
                f"perc {self.perceptual_loss_pct:5.2f}%  "
                f"delay {self.mouth_to_ear_ms:5.1f}ms  "
                f"redund {self.redundancy_pct:5.1f}%")

```

```
91
92  def frame_bytes(codec: str) -> int:
93      kbps = emodel.CODEC_PARAMS[codec][2]
94      return max(8, int(round(kbps * 1000 * FRAME_MS / 1000 / 8)))
95
96
97  class CallSimulator:
98      """Simulates one direction of one call under one system."""
99
100     def __init__(self, system: str, cfg: ChannelConfig, codec: str = "opus-24k",
101                  duration_s: float = 30.0, seed: int = 3, k: int = GROUP_K,
102                  forced_rho: float | None = None, mobile: bool = False,
103                  forced_depth: float | None = None):
104         self.system = system
105         self.cfg = cfg
106         self.codec = codec
107         self.duration_s = duration_s
108         self.seed = seed
109         self.k = k
110         self.forced_rho = forced_rho
111         self.forced_depth = forced_depth
112         self.mobile = mobile
113         self._jb_curve: list = []
114
115     # ------------------------------------------------------------------
116     def run(self) -> CallResult:
117         gen = SpeechGenerator(seed=self.seed)
118         frames, _ = gen.frames(self.duration_s, FRAME_MS)
119         analyzer = PerceptualImportanceAnalyzer()
120         pi = np.array([analyzer.analyze(seg).pi for _, seg, _ in frames], dtype=int)
121         n = (len(pi) // self.k) * self.k
122         pi = pi[:n]
123
124         navik = NavikController(k=self.k, mobile=self.mobile,
125                                 pi_afec=(self.system == "swara"))
126         navik.capacity_est_kbps = navik._cap_ewma = self.cfg.capacity_kbps
127
128         if self.forced_rho is not None:
129             rho = self.forced_rho
130         elif self.system == "baseline":
131             rho = 0.0
132         else:
133             d = navik.decide(self.cfg.loss_rate, self.cfg.mean_burst,
134                              self.cfg.jitter_ms, self.cfg.base_delay_ms, 60.0)
135             rho, self.codec = d.rho, d.codec
136
137         payload = frame_bytes(self.codec)
138         pkt_size = HEADER_LEN + AEAD_TAG_LEN + payload + UDP_IP_OVERHEAD
139
140         depth, base_transit, state = self._probe(pkt_size, n, pi)
141         # How far a piggybacked replica may be displaced before it misses its
142         # own deadline.  The playout decision bounds the protection decision.
143         max_disp = max(1, int(depth // FRAME_MS) - 1)
144
145         return self._transmit(pi, n, payload, pkt_size, rho, depth,
146                               base_transit, state, max_disp)
147
148     # ------------------------------------------------------------------
149     def _probe(self, pkt_size: int, n: int, pi: np.ndarray):
150         """An unprotected pilot pass that learns the delay distribution, just
151         as a live call learns it during its first second."""
152         ch = GilbertElliottChannel(ChannelConfig(**{**self.cfg.__dict__,
153                                                     "seed": self.seed * 31 + 1}))
154         est = NetworkStateEstimator()
155         transit: dict[int, float] = {}
156         for i in range(n):
157             for at, ok in ch.transmit(i * FRAME_MS, pkt_size):
158                 if ok:
159                     transit[i] = at - i * FRAME_MS
160                     est.on_arrival(i, i * FRAME_MS, at, pkt_size)
161         state = est.snapshot()
162         tr = np.array(list(transit.values())) if transit else np.array([50.0])
163         base = float(tr.min())
164         rel = tr - base
165
166         if self.forced_depth is not None:
167             return float(self.forced_depth), base, state
168
169         if self.system == "swara":
170             jb = AdaptiveJitterBuffer(base_delay_ms=base)
171             # Perceptual weight of the packets in the delay tail, computed with
172             # the SAME damage function that scores the final result.  The
173             # optimiser and the evaluator therefore agree by construction --
174             # no tuning constant mediates between them.
175             thr = float(np.percentile(tr, 88))
176             tail = [int(pi[i]) for i, t in transit.items() if t >= thr and i < len(pi)]
```

```
            late_dmg = float(np.mean([damage_weight(p) for p in tail])) if tail else 0.35
            dec = None
            for _ in range(15):                       # iterate to steady state
                dec = jb.optimise(rel, self.codec, state.mean_burst,
                                  other_delay_ms=base + ALGORITHMIC_DELAY_MS,
                                  late_importance=late_dmg, mobile=self.mobile)
            self._jb_curve = dec.considered
            depth = dec.depth_ms
        else:
            self._jb_curve = []
            depth = float(np.clip(np.percentile(rel, 95) + 10.0, 20.0, 180.0))
        return depth, base, state

    # ------------------------------------------------------------------
    def _transmit(self, pi, n, payload, pkt_size, rho, depth, base_transit,
                  state, max_disp) -> CallResult:
        ch = GilbertElliottChannel(ChannelConfig(**{**self.cfg.__dict__,
                                                    "seed": self.seed * 977 + 13}))
        if self.system == "swara":
            sched = FecScheduler(k=self.k, rho_net=rho, mean_burst=self.cfg.mean_burst)
        elif self.system == "uniform_fec":
            sched = UniformFecScheduler(k=self.k, rho_net=rho,
                                        mean_burst=self.cfg.mean_burst)
        else:
            sched = None

        # ---- plan every group up front -------------------------------
        plans: dict[int, tuple] = {}
        replicas: dict[int, list[int]] = defaultdict(list)
        for gid, start in enumerate(range(0, n, self.k)):
            idxs = list(range(start, start + self.k))
            plan = sched.plan([int(pi[i]) for i in idxs]) if sched else None
            plans[gid] = (idxs, plan)
            if plan:
                for a_idx, a_delay in plan.anchors:
                    src = idxs[a_idx]
                    disp = min(a_delay, max_disp)
                    carrier = src + disp
                    if disp >= 1 and carrier < n:
                        replicas[carrier].append(src)

        # ---- transmit ------------------------------------------------
        arrivals: dict[int, float] = {}
        parity_arr: dict[int, dict] = defaultdict(dict)
        replica_arr: dict[int, float] = {}
        bytes_sent = packets_sent = 0

        for gid, (idxs, plan) in plans.items():
            r = plan.r if plan else 0
            for fi in idxs:
                size = pkt_size + payload * len(replicas.get(fi, ()))
                bytes_sent += size
                packets_sent += 1
                for at, ok in ch.transmit(fi * FRAME_MS, size):
                    if ok:
                        arrivals[fi] = at
                        for src in replicas.get(fi, ()):
                            replica_arr[src] = min(replica_arr.get(src, 1e18), at)
            for pj in range(r):
                st = idxs[-1] * FRAME_MS + 1 + pj * 0.4
                bytes_sent += pkt_size
                packets_sent += 1
                for at, ok in ch.transmit(st, pkt_size):
                    if ok:
                        parity_arr[gid][pj] = at

        # ---- receive -------------------------------------------------
        deadline_offset = base_transit + depth      # frame i plays at i*T + this
        on_time, late = set(), set()
        for fi, at in arrivals.items():
            (on_time if at - fi * FRAME_MS <= deadline_offset else late).add(fi)

        rec_fec, rec_anchor = set(), set()
        if sched is not None:
            for gid, (idxs, plan) in plans.items():
                r = plan.r if plan else 0
                if r == 0:
                    continue
                have = sum(1 for fi in idxs if fi in on_time)
                par_times = sorted(parity_arr[gid].values())
                if have + len(par_times) < self.k:
                    continue          # group is mathematically undecodable
                # The group can only be decoded once the LAST parity packet it
                # needs has arrived.  A recovered frame is useful only if that
                # moment precedes ITS OWN playout instant -- which is why a
```

```
                # systematic code protects the tail of a group far better than
                # its head, and why the anchor mechanism exists at all.
                need = self.k - have
                decode_at = par_times[need - 1] if need > 0 else 0.0
                for fi in idxs:
                    if fi not in on_time and decode_at - fi * FRAME_MS <= deadline_offset:
                        rec_fec.add(fi)
                        on_time.add(fi)

        for src, at in replica_arr.items():
            if src not in on_time and at - src * FRAME_MS <= deadline_offset:
                rec_anchor.add(src)
                on_time.add(src)

        lost = sorted(set(range(n)) - on_time)

        # ---- score ----------------------------------------------------
        damage, run, prev = 0.0, 0, -10
        for fi in lost:
            run = run + 1 if fi == prev + 1 else 0
            damage += damage_weight(int(pi[fi]), run)
            prev = fi
        residual = 100.0 * len(lost) / max(1, n)
        perceptual = emodel.perceptual_ppl(damage, n)
        bursts = self._burst_lengths(lost)
        mean_burst = float(np.mean(bursts)) if bursts else 1.0

        # A SYSTEMATIC code adds no mouth-to-ear delay: the k data packets are
        # transmitted the instant they are encoded and are played out on
        # schedule.  Only the RECOVERY path waits for parity, and that wait is
        # already enforced per frame against each frame's own playout instant.
        # Charging the whole stream for the group span would be simply wrong.
        m2e = base_transit + depth + ALGORITHMIC_DELAY_MS
        rep = emodel.assess(self.codec, m2e, residual, perceptual, mean_burst, self.mobile)

        wire_kbps = bytes_sent * 8 / (n * FRAME_MS / 1000.0) / 1000.0
        media_only = pkt_size * n
        redundancy = 100.0 * (bytes_sent - media_only) / media_only

        return CallResult(
            system=self.system, profile=self.cfg.label(),
            frames_total=n, frames_delivered=len(on_time),
            frames_recovered_fec=len(rec_fec), frames_recovered_anchor=len(rec_anchor),
            frames_lost=len(lost), frames_late=len(late),
            channel_loss_pct=100.0 * ch.observed_loss,
            residual_loss_pct=residual, perceptual_loss_pct=perceptual,
            mean_burst=mean_burst, jitter_ms=state.jitter_ms,
            playout_delay_ms=depth, mouth_to_ear_ms=m2e,
            wire_kbps=wire_kbps, redundancy_pct=redundancy,
            packets_sent=packets_sent, codec=self.codec,
            r_factor=rep.r, mos=rep.mos,
            lost_pi_mean=float(np.mean([pi[i] for i in lost])) if lost else 0.0,
            delivered_pi_mean=float(np.mean([pi[i] for i in sorted(on_time)])))
            if on_time else 0.0,
            jb_curve=self._jb_curve,
        )

    @staticmethod
    def _burst_lengths(sorted_lost: list[int]) -> list[int]:
        if not sorted_lost:
            return []
        out, cur = [], 1
        for a, b in zip(sorted_lost, sorted_lost[1:]):
            if b == a + 1:
                cur += 1
            else:
                out.append(cur); cur = 1
        out.append(cur)
        return out


def compare(cfg: ChannelConfig, duration_s: float = 30.0, seed: int = 3,
            codec: str = "opus-24k", mobile: bool = False) -> dict[str, CallResult]:
    """All three systems, same path, same speech, same redundancy budget."""
    swara = CallSimulator("swara", cfg, codec, duration_s, seed, mobile=mobile).run()
    rho_equal = float(np.clip(swara.redundancy_pct / 100.0, 0.0, 0.65))
    uni = CallSimulator("uniform_fec", cfg, swara.codec, duration_s, seed,
                        forced_rho=rho_equal, mobile=mobile).run()
    base = CallSimulator("baseline", cfg, swara.codec, duration_s, seed,
                         forced_rho=0.0, mobile=mobile).run()
    return {"baseline": base, "uniform_fec": uni, "swara": swara}
```

Listing 14 `swara/speech.py` 156 lines

The synthetic speech corpus generator.

```
"""
Synthetic speech corpus generator.

The evaluation needs speech-like audio with the right STRUCTURE -- alternating
talkspurts and pauses, voiced nuclei with formants, unvoiced fricatives, plosive
onsets -- because that structure is exactly what PI-AFEC exploits.  Shipping a
recorded corpus would make the project depend on licensed audio; generating one
keeps the whole system self-contained and reproducible from a seed.

The generator is a source-filter model, the same abstraction used by every
speech codec since LPC-10:

    source   glottal pulse train at F0 (voiced)  OR  white noise (unvoiced)
    filter   a cascade of resonators at the formant frequencies F1, F2, F3
    envelope per-phone amplitude contour with realistic attack and decay

Statistics are taken from the standard picture of conversational speech:
roughly 40% of a two-way call is silence for a given talker, voiced phones
average ~90 ms, unvoiced ~70 ms, and talkspurts run 1-3 s.
"""

from __future__ import annotations

import numpy as np
from dataclasses import dataclass

SAMPLE_RATE = 16_000

# (F1, F2, F3) in Hz for a set of vowels, plus fricative noise bands.
VOWELS = {
    "a": (730, 1090, 2440), "e": (530, 1840, 2480), "i": (270, 2290, 3010),
    "o": (570, 840, 2410),  "u": (300, 870, 2240),  "ae": (660, 1720, 2410),
}
FRICATIVES = {"s": (4000, 8000), "sh": (2500, 6000), "f": (1500, 7000), "th": (1200, 6500)}


@dataclass
class Phone:
    kind: str          # "voiced" | "unvoiced" | "silence" | "plosive"
    duration_ms: int
    identity: str = ""


def _resonator(x: np.ndarray, f: float, bw: float, sr: int = SAMPLE_RATE) -> np.ndarray:
    """Two-pole resonator -- one formant."""
    r = np.exp(-np.pi * bw / sr)
    theta = 2 * np.pi * f / sr
    a1, a2 = -2 * r * np.cos(theta), r * r
    b0 = (1 - r) * np.sqrt(1 - 2 * r * np.cos(2 * theta) + r * r)
    y = np.zeros_like(x)
    y1 = y2 = 0.0
    for i in range(x.size):
        v = b0 * x[i] - a1 * y1 - a2 * y2
        y[i] = v
        y2, y1 = y1, v
    return y


class SpeechGenerator:
    def __init__(self, seed: int = 7, sample_rate: int = SAMPLE_RATE):
        self.rng = np.random.default_rng(seed)
        self.sr = sample_rate
        self.f0 = self.rng.uniform(95, 190)     # speaker's mean pitch

    def _glottal(self, n: int, f0: float) -> np.ndarray:
        """Pulse train with jitter and shimmer, low-passed to a glottal shape."""
        t = np.arange(n) / self.sr
        f0_track = f0 * (1.0 + 0.06 * np.sin(2 * np.pi * 1.7 * t)
                         + 0.012 * self.rng.standard_normal(n).cumsum() / max(1, np.sqrt(n)))
        phase = 2 * np.pi * np.cumsum(f0_track) / self.sr
        pulse = np.maximum(0.0, np.sin(phase)) ** 2.4
        pulse -= pulse.mean()
        return pulse

    def _phone_signal(self, ph: Phone) -> np.ndarray:
        n = int(self.sr * ph.duration_ms / 1000)
        if n <= 0:
            return np.zeros(0)
        if ph.kind == "silence":
            return self.rng.normal(0, 0.0016, n)          # room noise floor
        if ph.kind == "plosive":
            burst = np.zeros(n)
            bl = min(n, int(0.008 * self.sr))
```

```
            burst[:bl] = self.rng.normal(0, 1.0, bl) * np.exp(-np.linspace(0, 5, bl))
            return burst * 0.55
        if ph.kind == "unvoiced":
            lo, hi = FRICATIVES[ph.identity]
            x = self.rng.normal(0, 1.0, n)
            X = np.fft.rfft(x)
            f = np.fft.rfftfreq(n, 1 / self.sr)
            X *= ((f > lo) & (f < hi)).astype(float)
            y = np.fft.irfft(X, n)
            env = np.minimum(1.0, np.linspace(0, 3, n)) * np.minimum(
                1.0, np.linspace(3, 0, n))
            return 0.22 * y / (np.abs(y).max() + 1e-9) * env
        # voiced
        f1, f2, f3 = VOWELS[ph.identity]
        src = self._glottal(n, self.f0 * self.rng.uniform(0.92, 1.08))
        y = (_resonator(src, f1, 80) + 0.55 * _resonator(src, f2, 110)
             + 0.28 * _resonator(src, f3, 160))
        attack = int(0.018 * self.sr)
        env = np.ones(n)
        if n > 2 * attack and attack > 0:
            env[:attack] = np.linspace(0, 1, attack) ** 0.6
            env[-attack:] = np.linspace(1, 0, attack) ** 0.8
        y = y / (np.abs(y).max() + 1e-9)
        return 0.85 * y * env

    def utterance(self, duration_s: float = 20.0) -> tuple[np.ndarray, list[Phone]]:
        """Generate a conversational-turn signal and its phone-level ground truth."""
        phones: list[Phone] = []
        total = 0.0
        target = duration_s * 1000
        while total < target:
            # A talkspurt: a run of syllables.
            spurt = self.rng.uniform(900, 2800)
            spent = 0.0
            while spent < spurt and total + spent < target:
                roll = self.rng.random()
                if roll < 0.13:
                    p = Phone("plosive", int(self.rng.uniform(25, 55)))
                elif roll < 0.38:
                    p = Phone("unvoiced", int(self.rng.uniform(55, 110)),
                              self.rng.choice(list(FRICATIVES)))
                else:
                    p = Phone("voiced", int(self.rng.uniform(70, 170)),
                              self.rng.choice(list(VOWELS)))
                phones.append(p)
                spent += p.duration_ms
            total += spent
            pause = self.rng.uniform(250, 1100)
            phones.append(Phone("silence", int(pause)))
            total += pause

        sig = np.concatenate([self._phone_signal(p) for p in phones])
        # Gentle level normalisation and a touch of channel noise.
        sig = sig / (np.abs(sig).max() + 1e-9) * 0.7
        sig += self.rng.normal(0, 0.0009, sig.size)
        return sig.astype(np.float64), phones

    def frames(self, duration_s: float = 20.0, frame_ms: int = 20):
        """Yield (frame_index, samples, ground_truth_kind)."""
        sig, phones = self.utterance(duration_s)
        n = self.sr * frame_ms // 1000
        # Build a per-sample ground-truth label track.
        labels = []
        for p in phones:
            labels.extend([p.kind] * int(self.sr * p.duration_ms / 1000))
        labels = labels[: sig.size] + ["silence"] * max(0, sig.size - len(labels))
        out = []
        for i in range(sig.size // n):
            seg = sig[i * n:(i + 1) * n]
            seg_lab = labels[i * n:(i + 1) * n]
            kind = max(set(seg_lab), key=seg_lab.count) if seg_lab else "silence"
            out.append((i, seg, kind))
        return out, sig
```

Listing 15 `swara/audio_io.py` 178 lines

Audio sources and sinks.

```
"""
Audio capture and playback.

Three backends, selected automatically so the same application runs on a laptop
with a microphone AND inside a headless test environment with no sound card:
```

```
    live       real microphone / speaker via `sounddevice` (if installed)
    wav        read from / write to a WAV file
    synthetic  the source-filter speech generator, for automated testing

Everything downstream sees the identical interface -- 20 ms frames of int16 at
16 kHz -- so no other module knows or cares which backend is in use.
"""

from __future__ import annotations

import queue
import threading
import wave
import numpy as np

SAMPLE_RATE = 16_000
FRAME_MS = 20
FRAME_LEN = SAMPLE_RATE * FRAME_MS // 1000     # 320 samples

try:
    import sounddevice as _sd
    HAVE_SOUNDDEVICE = True
except Exception:
    _sd = None
    HAVE_SOUNDDEVICE = False


def backends_available() -> dict:
    return {"live": HAVE_SOUNDDEVICE, "wav": True, "synthetic": True}


class AudioSource:
    """Produces successive 20 ms frames of int16 mono at 16 kHz."""

    def __init__(self, backend: str = "auto", wav_path: str | None = None,
                 duration_s: float = 30.0, seed: int = 7, gain: float = 1.0):
        if backend == "auto":
            backend = "live" if HAVE_SOUNDDEVICE else "synthetic"
        self.backend = backend
        self.gain = gain
        self._q: queue.Queue = queue.Queue(maxsize=200)
        self._stream = None
        self._buf = np.zeros(0, dtype=np.int16)
        self._pos = 0
        self._closed = False

        if backend == "wav":
            if not wav_path:
                raise ValueError("wav backend needs wav_path")
            self._buf = self._read_wav(wav_path)
        elif backend == "synthetic":
            from .speech import SpeechGenerator
            frames, sig = SpeechGenerator(seed=seed).frames(duration_s, FRAME_MS)
            self._buf = np.clip(sig * 26000.0, -32768, 32767).astype(np.int16)
        elif backend == "live":
            if not HAVE_SOUNDDEVICE:
                raise RuntimeError("sounddevice not installed")
            self._stream = _sd.InputStream(
                samplerate=SAMPLE_RATE, channels=1, dtype="int16",
                blocksize=FRAME_LEN, callback=self._cb)
            self._stream.start()
        else:
            raise ValueError(f"unknown backend {backend!r}")

    @staticmethod
    def _read_wav(path: str) -> np.ndarray:
        with wave.open(path, "rb") as w:
            n, ch, sw, sr = w.getnframes(), w.getnchannels(), w.getsampwidth(), w.getframerate()
            raw = w.readframes(n)
        if sw != 2:
            raise ValueError("only 16-bit PCM WAV supported")
        x = np.frombuffer(raw, dtype=np.int16)
        if ch > 1:
            x = x.reshape(-1, ch).mean(axis=1).astype(np.int16)
        if sr != SAMPLE_RATE:                       # crude but adequate resample
            idx = np.linspace(0, x.size - 1, int(x.size * SAMPLE_RATE / sr))
            x = np.interp(idx, np.arange(x.size), x.astype(np.float64)).astype(np.int16)
        return x

    def _cb(self, indata, frames, time_info, status):
        try:
            self._q.put_nowait(indata[:, 0].copy())
        except queue.Full:
            pass

```

```
    def read(self, timeout: float = 1.0) -> np.ndarray | None:
        """Next 20 ms frame, or None when the source is exhausted."""
        if self._closed:
            return None
        if self.backend == "live":
            try:
                f = self._q.get(timeout=timeout)
            except queue.Empty:
                return np.zeros(FRAME_LEN, dtype=np.int16)
            return self._apply_gain(f)
        if self._pos + FRAME_LEN > self._buf.size:
            return None
        f = self._buf[self._pos:self._pos + FRAME_LEN]
        self._pos += FRAME_LEN
        return self._apply_gain(f)

    def _apply_gain(self, f: np.ndarray) -> np.ndarray:
        if self.gain == 1.0:
            return f
        return np.clip(f.astype(np.float64) * self.gain, -32768, 32767).astype(np.int16)

    @property
    def total_frames(self) -> int:
        return self._buf.size // FRAME_LEN if self.backend != "live" else -1

    def close(self):
        self._closed = True
        if self._stream is not None:
            try:
                self._stream.stop(); self._stream.close()
            except Exception:
                pass


class AudioSink:
    """Consumes successive 20 ms frames of int16 mono at 16 kHz."""

    def __init__(self, backend: str = "auto", wav_path: str | None = None):
        if backend == "auto":
            backend = "live" if HAVE_SOUNDDEVICE else "null"
        self.backend = backend
        self._frames: list[np.ndarray] = []
        self._wav_path = wav_path
        self._stream = None
        self._q: queue.Queue = queue.Queue(maxsize=100)

        if backend == "live":
            if not HAVE_SOUNDDEVICE:
                raise RuntimeError("sounddevice not installed")
            self._stream = _sd.OutputStream(
                samplerate=SAMPLE_RATE, channels=1, dtype="int16",
                blocksize=FRAME_LEN, callback=self._cb)
            self._stream.start()

    def _cb(self, outdata, frames, time_info, status):
        try:
            f = self._q.get_nowait()
        except queue.Empty:
            f = np.zeros(FRAME_LEN, dtype=np.int16)
        if f.size < frames:
            f = np.pad(f, (0, frames - f.size))
        outdata[:, 0] = f[:frames]

    def write(self, frame: np.ndarray) -> None:
        if self.backend == "live":
            try:
                self._q.put_nowait(np.asarray(frame, dtype=np.int16))
            except queue.Full:
                pass
        else:
            self._frames.append(np.asarray(frame, dtype=np.int16))

    def close(self) -> None:
        if self._stream is not None:
            try:
                self._stream.stop(); self._stream.close()
            except Exception:
                pass
        if self._wav_path and self._frames:
            data = np.concatenate(self._frames)
            with wave.open(self._wav_path, "wb") as w:
                w.setnchannels(1); w.setsampwidth(2); w.setframerate(SAMPLE_RATE)
                w.writeframes(data.tobytes())

    @property
    def collected(self) -> np.ndarray:
```

```
        return np.concatenate(self._frames) if self._frames else np.zeros(0, dtype=np.int16)
```

Listing 16 `server.py` 246 lines

The relay server.

```
#!/usr/bin/env python3
"""
SWARA server -- registration, authentication, presence and media relay.

Run it:
    python3 server.py --add-user alice --password secret1
    python3 server.py --add-user bob   --password secret2
    python3 server.py --port 9000

What it does:

  * Performs the SPARK handshake with each client over real UDP.  No state is
    allocated for a client until it has returned a valid address-validation
    cookie, so the server cannot be used as a reflection amplifier.

  * Tracks which authenticated users are online.

  * Sets up calls.  When A calls B the server mints a random 32-byte call
    secret, derives two DIRECTIONAL media keys from it, and delivers them to
    each party encrypted under that party's own session key.

  * Relays media.  Because the media keys are derived from the call secret and
    the server forwards datagrams verbatim without decrypting them, the relay
    cannot listen to the call it is carrying.  It sees only a connection
    identifier and a length.
"""
from __future__ import annotations
import argparse, json, os, secrets, struct, sys, threading, time
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from swara.auth import (CredentialRecord, MediaCipher, SparkServer, hkdf_expand, register)
from swara.net import UdpEndpoint
from swara.packet import AEAD_TAG_LEN, HEADER_LEN, Flag, Header, PType

USERS_FILE = "users.json"


def load_users(path=USERS_FILE) -> dict:
    if not os.path.exists(path):
        return {}
    with open(path) as f:
        raw = json.load(f)
    return {u: CredentialRecord(u, bytes.fromhex(d["salt"]),
                                bytes.fromhex(d["envelope"]),
                                bytes.fromhex(d["pub"])) for u, d in raw.items()}


def save_user(username: str, password: str, path=USERS_FILE) -> None:
    raw = {}
    if os.path.exists(path):
        raw = json.load(open(path))
    rec, _ = register(username, password)
    raw[username] = {"salt": rec.salt.hex(), "envelope": rec.envelope.hex(),
                     "pub": rec.client_static_pub.hex()}
    json.dump(raw, open(path, "w"), indent=2)
    print(f"registered '{username}'  ({rec.size_bytes()} bytes stored, no password kept)")


class Server:
    def __init__(self, host="0.0.0.0", port=9000, users_file=USERS_FILE, verbose=True):
        self.ep = UdpEndpoint(host, port)
        self.spark = SparkServer()
        self.verbose = verbose
        for u, rec in load_users(users_file).items():
            self.spark.add_user(rec)
        self.pending: dict[tuple, tuple] = {}          # addr -> (pending, ts)
        self.online: dict[str, dict] = {}              # username -> session info
        self.by_cid: dict[int, dict] = {}
        self.calls: dict[int, dict] = {}               # call_id -> {a, b, addrs}
        # Reliable signalling over an unreliable transport: the caller may
        # retransmit a "call" request that was in fact delivered.  Keyed by
        # (username, request id) we remember the answer we already gave, so a
        # duplicate re-sends the SAME call_ready / incoming pair instead of
        # minting a second call with different keys.  This is the idempotent
        # server half of SIP-style INVITE retransmission.
        self.call_dedup: dict[tuple, dict] = {}        # (user, rid) -> replies
        self.running = threading.Event()
        self.log(f"listening on UDP {self.ep.address[0]}:{self.ep.port}  "
```

```
                f"({len(self.spark.users)} users known)")

    def log(self, msg):
        if self.verbose:
            print(f"[server {time.strftime('%H:%M:%S')}] {msg}", flush=True)

    # -- control channel ------------------------------------------------
    def send_control(self, sess: dict, obj: dict) -> None:
        body = json.dumps(obj).encode()
        seq = sess["ctl_seq"]; sess["ctl_seq"] += 1
        h = Header(ptype=PType.CONTROL, seq=seq & 0xFFFF, cid=sess["cid"],
                   ts_ms=0, flags=Flag.ENCRYPTED)
        hdr = h.pack()
        self.ep.sendto(hdr + sess["s2c"].seal(seq, hdr, body), sess["addr"])

    def serve_forever(self) -> None:
        self.running.set()
        while self.running.is_set():
            data, addr = self.ep.recvfrom(timeout=0.5)
            if data is None or len(data) < HEADER_LEN:
                continue
            try:
                h = Header.unpack(data)
            except Exception:
                continue

            # 1. media relay -- highest volume, checked first, never decrypted
            call = self.calls.get(h.cid)
            if call is not None:
                dst = call["b_addr"] if addr == call["a_addr"] else call["a_addr"]
                if dst:
                    self.ep.sendto(data, dst)
                continue

            body = data[HEADER_LEN:]
            if h.ptype == PType.HELLO:
                retry = self.spark.handle_hello(addr, body)
                self.ep.sendto(Header(ptype=PType.RETRY).pack() + retry, addr)
            elif h.ptype == PType.AUTH_INIT:
                resp, pending = self.spark.handle_auth_init(addr, body)
                if resp is None:
                    continue                        # spoofed source: silent drop
                # A retransmitted AUTH_INIT carries the same client ephemeral
                # key, so keeping the FIRST pending state keeps the transcript
                # the client already committed to.
                if addr not in self.pending:
                    self.pending[addr] = (pending, time.time())
                    self.ep.sendto(Header(ptype=PType.AUTH_RESP).pack() + resp, addr)
                else:
                    prev = self.pending[addr][0]
                    self.ep.sendto(Header(ptype=PType.AUTH_RESP).pack()
                                   + prev["resp"], addr)
                continue
            elif h.ptype == PType.AUTH_CONFIRM:
                ent = self.pending.get(addr)
                if ent is None or time.time() - ent[1] > 30:
                    continue
                self.pending.pop(addr, None)
                keys, ticket = self.spark.handle_auth_confirm(ent[0], body)
                if keys is None:
                    self.log(f"auth FAILED from {addr}")
                    continue
                uname = ent[0]["rec"].username
                sess = {"cid": keys.cid, "addr": addr, "user": uname, "ctl_seq": 0,
                        "c2s": MediaCipher(keys.k_c2s, b"c2s"),
                        "s2c": MediaCipher(keys.k_s2c, b"s2c")}
                self.online[uname] = sess
                self.by_cid[keys.cid] = sess
                out = struct.pack("!IH", keys.cid, len(ticket)) + ticket
                self.ep.sendto(Header(ptype=PType.AUTH_TICKET, cid=keys.cid).pack() + out, addr)
                self.log(f"'{uname}' authenticated from {addr[0]}:{addr[1]}  cid={keys.cid:08x}")
            elif h.ptype == PType.CONTROL:
                sess = self.by_cid.get(h.cid)
                if sess is None:
                    continue
                pt = sess["c2s"].open(h.seq, data[:HEADER_LEN], body)
                if pt is None:
                    continue
                sess["addr"] = addr                 # tolerate NAT rebinding
                try:
                    self.on_control(sess, json.loads(pt))
                except Exception as e:
                    self.log(f"control error: {e}")

    # -- signalling -----------------------------------------------------
```

```
    def on_control(self, sess: dict, msg: dict) -> None:
        op = msg.get("op")
        if op == "ping":
            # Keepalive.  Refreshes the client's NAT binding and lets us track
            # the address it is currently reachable at.
            self.send_control(sess, {"op": "pong"})
        elif op == "who":
            self.send_control(sess, {"op": "online", "users": list(self.online)})
        elif op == "call":
            peer = msg.get("peer")
            rid = msg.get("rid")
            if rid is not None:
                dup = self.call_dedup.get((sess["user"], rid))
                if dup is not None:
                    # Retransmitted request: repeat our original answer to
                    # BOTH parties (either message may have been the one the
                    # network dropped), and do not mint a new call.
                    self.send_control(sess, dup["ready"])
                    tgt = self.online.get(dup["peer"])
                    if tgt is not None:
                        self.send_control(tgt, dup["incoming"])
                    return
                if len(self.call_dedup) > 256:          # lazy expiry
                    oldest = sorted(self.call_dedup.items(),
                                    key=lambda kv: kv[1]["ts"])[:128]
                    for k, _ in oldest:
                        self.call_dedup.pop(k, None)
            tgt = self.online.get(peer)
            if tgt is None or peer == sess["user"]:
                self.send_control(sess, {"op": "call_failed", "reason": f"'{peer}' not online"})
                return
            call_id = secrets.randbelow(2**32)
            master = secrets.token_bytes(32)
            k_ab = hkdf_expand(master, b"call a->b")
            k_ba = hkdf_expand(master, b"call b->a")
            self.calls[call_id] = {"a": sess["user"], "b": peer,
                                   "a_addr": sess["addr"], "b_addr": tgt["addr"]}
            ready = {"op": "call_ready", "call_id": call_id, "peer": peer,
                     "tx": k_ab.hex(), "rx": k_ba.hex(), "role": "caller"}
            incoming = {"op": "incoming", "call_id": call_id, "peer": sess["user"],
                        "tx": k_ba.hex(), "rx": k_ab.hex(), "role": "callee"}
            if rid is not None:
                self.call_dedup[(sess["user"], rid)] = {
                    "ready": ready, "incoming": incoming, "peer": peer,
                    "ts": time.time()}
            self.send_control(sess, ready)
            self.send_control(tgt, incoming)
            self.log(f"call {call_id:08x}: {sess['user']} -> {peer}  (relay active, keys not held)")
        elif op == "hangup":
            cid = msg.get("call_id")
            if cid in self.calls:
                c = self.calls.pop(cid)
                self.log(f"call {cid:08x} ended ({c['a']} / {c['b']})")
        elif op == "bye":
            self.online.pop(sess["user"], None)
            self.by_cid.pop(sess["cid"], None)
            self.log(f"'{sess['user']}' signed off")

    def stop(self):
        self.running.clear()
        self.ep.close()


def main():
    ap = argparse.ArgumentParser(description="SWARA server")
    ap.add_argument("--host", default="0.0.0.0")
    ap.add_argument("--port", type=int, default=9000)
    ap.add_argument("--users", default=USERS_FILE)
    ap.add_argument("--add-user"); ap.add_argument("--password")
    a = ap.parse_args()
    if a.add_user:
        if not a.password:
            print("--add-user requires --password"); sys.exit(1)
        save_user(a.add_user, a.password, a.users); return
    s = Server(a.host, a.port, a.users)
    try:
        s.serve_forever()
    except KeyboardInterrupt:
        print("\nshutting down"); s.stop()


if __name__ == "__main__":
    main()
```

Listing 17 `client.py` 266 lines

The voice-chat client.

```
#!/usr/bin/env python3
"""
SWARA client -- the actual voice chat application.

    python3 client.py --user alice --password secret1 --server 127.0.0.1:9000 --call bob
    python3 client.py --user bob   --password secret2 --server 127.0.0.1:9000 --wait

Audio:
    --audio live            microphone and speaker (needs `pip install sounddevice`)
    --audio wav --in a.wav --out b.wav
    --audio synthetic       generated speech, for testing without hardware

Protection:
    --protect swara         PI-AFEC + perceptual anchor replicas   (default)
    --protect uniform       conventional evenly-spread FEC
    --protect none          plain UDP, no repair -- the A/B comparison arm

Impairment (for the demonstration -- applied locally at our own socket, and
clearly artificial):
    --loss 0.15 --burst 4 --extra-delay 40 --jitter 15
"""
from __future__ import annotations
import argparse, json, os, struct, sys, threading, time
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from swara.audio_io import AudioSink, AudioSource, backends_available
from swara.auth import MediaCipher, SparkClient
from swara.codec import Mode
from swara.net import Impairment, UdpEndpoint
from swara.packet import HEADER_LEN, Flag, Header, PType
from swara.session import CallSession


class Client:
    def __init__(self, user, password, server_addr, impairment=None, verbose=True):
        self.user = user
        self.server = server_addr
        self.ep = UdpEndpoint("0.0.0.0", 0, impairment or Impairment())
        self.spark = SparkClient(user, password)
        self.cid = None
        self.ticket = None
        self.c2s = self.s2c = None
        self.ctl_seq = 0
        self.verbose = verbose
        self.events: list[dict] = []
        self._ev = threading.Event()

    def log(self, m):
        if self.verbose:
            print(f"[{self.user}] {m}", flush=True)

    # -- handshake ------------------------------------------------------
    def _tx(self, ptype, body, cid=0):
        self.ep.sendto(Header(ptype=ptype, cid=cid).pack() + body, self.server)

    def _rx(self, want, timeout=3.0):
        end = time.time() + timeout
        while time.time() < end:
            data, addr = self.ep.recvfrom(timeout=0.3)
            if data is None or len(data) < HEADER_LEN:
                continue
            h = Header.unpack(data)
            if h.ptype == want:
                return h, data[HEADER_LEN:]
        return None, None

    def _exchange(self, send_type, body, want, attempts=6, timeout=0.6):
        """Send and wait, retransmitting on timeout.

        UDP has no retransmission, so a handshake built on it must supply its
        own or it simply cannot complete on a lossy link.  Each step is retried
        with a linearly growing timeout.  Every message in SPARK is idempotent
        at the server -- HELLO allocates no state and the cookie is valid for a
        whole epoch -- so a retransmission is always safe.
        """
        for i in range(attempts):
            self._tx(send_type, body)
            h, resp = self._rx(want, timeout=timeout * (1 + 0.5 * i))
            if resp is not None:
                return resp
        return None

```

```
    def login(self) -> bool:
        t0 = time.perf_counter()
        retry = self._exchange(PType.HELLO, self.spark.make_hello(), PType.RETRY)
        if retry is None:
            self.log("no response from server"); return False
        resp = self._exchange(PType.AUTH_INIT, self.spark.make_auth_init(retry),
                              PType.AUTH_RESP)
        if resp is None:
            self.log("no AUTH_RESP after retries"); return False
        tag = self.spark.make_auth_confirm(resp)
        if tag is None:
            self.log("wrong password (detected locally -- nothing was sent)"); return False
        tick = self._exchange(PType.AUTH_CONFIRM, tag, PType.AUTH_TICKET)
        if tick is None:
            self.log("server rejected the key confirmation"); return False
        cid, tlen = struct.unpack("!IH", tick[:6])
        self.cid = cid
        self.ticket = tick[6:6 + tlen]
        self.spark.ticket = self.ticket
        self.c2s = MediaCipher(self.spark.keys.k_c2s, b"c2s")
        self.s2c = MediaCipher(self.spark.keys.k_s2c, b"s2c")
        ms = (time.perf_counter() - t0) * 1000
        self.log(f"authenticated in {ms:.0f} ms  cid={cid:08x}  (2 round trips)")
        return True

    # -- control -------------------------------------------------------
    def send_ctl(self, obj):
        body = json.dumps(obj).encode()
        seq = self.ctl_seq; self.ctl_seq += 1
        h = Header(ptype=PType.CONTROL, seq=seq & 0xFFFF, cid=self.cid, flags=Flag.ENCRYPTED)
        hdr = h.pack()
        self.ep.sendto(hdr + self.c2s.seal(seq, hdr, body), self.server)

    def poll_ctl(self, timeout=1.0):
        end = time.time() + timeout
        while time.time() < end:
            data, addr = self.ep.recvfrom(timeout=0.25)
            if data is None or len(data) < HEADER_LEN:
                continue
            h = Header.unpack(data)
            if h.ptype != PType.CONTROL or h.cid != self.cid:
                continue
            pt = self.s2c.open(h.seq, data[:HEADER_LEN], data[HEADER_LEN:])
            if pt is None:
                continue
            try:
                return json.loads(pt)
            except Exception:
                continue
        return None

    def place_call(self, peer: str, timeout: float = 12.0, rtx_s: float = 1.2):
        """Place a call reliably over the unreliable transport.

        A single 'call' datagram on a 15% loss link fails one time in seven,
        and the failure looks exactly like the callee ignoring you.  So we do
        what SIP does with INVITE: attach a request id, retransmit on a timer,
        and rely on the server treating duplicates of the same id as the same
        request (see server.py call_dedup).  Either direction being lossy is
        healed, because every retransmission re-sends BOTH answers.
        """
        import secrets as _secrets
        rid = _secrets.token_hex(8)
        self._last_call = (peer, rid)
        end = time.time() + timeout
        while time.time() < end:
            self.send_ctl({"op": "call", "peer": peer, "rid": rid})
            m = self.wait_for({"call_ready", "call_failed"},
                              min(rtx_s, max(0.1, end - time.time())))
            if m is not None:
                return m
        return None

    def nudge_call(self):
        """Re-poke the server with the same rid so it re-sends 'incoming'
        to the callee -- used when the caller is ready but the callee's copy
        was the datagram the network ate."""
        peer, rid = getattr(self, "_last_call", (None, None))
        if rid:
            self.send_ctl({"op": "call", "peer": peer, "rid": rid})

    def wait_for(self, ops, timeout=30.0, keepalive_s=15.0):
        """Wait for a signalling message, keeping the NAT binding alive.

        A client sitting idle behind NAT loses its address translation entry
```

```
        after roughly 30 seconds of silence, at which point the server can no
        longer reach it and an incoming call is simply never delivered.  A
        small periodic packet keeps the pinhole open.  This is why every real
        VoIP client sends keepalives, and why one that does not appears to work
        perfectly in testing and then fails for real users.
        """
        end = time.time() + timeout
        last_ka = time.time()
        while time.time() < end:
            m = self.poll_ctl(1.0)
            if m and m.get("op") in ops:
                return m
            if time.time() - last_ka >= keepalive_s:
                try:
                    self.send_ctl({"op": "ping"})
                except Exception:
                    pass
                last_ka = time.time()
        return None


def run_call(cli, info, args) -> CallSession:
    src_backend = args.audio
    if src_backend == "wav" and not args.infile:
        print("--audio wav needs --in FILE"); sys.exit(1)
    source = AudioSource(backend=("wav" if src_backend == "wav" else src_backend),
                         wav_path=args.infile, duration_s=args.seconds, seed=args.seed)
    sink = AudioSink(backend=("wav" if args.outfile else
                              ("live" if src_backend == "live" else "null")),
                     wav_path=args.outfile)
    sess = CallSession(
        endpoint=cli.ep, peer_addr=cli.server,
        tx_key=bytes.fromhex(info["tx"]), rx_key=bytes.fromhex(info["rx"]),
        call_id=info["call_id"], source=source, sink=sink,
        mode=(Mode.NARROWBAND if args.narrowband else Mode.WIDEBAND),
        protection=args.protect, name=cli.user)
    cli.log(f"call {info['call_id']:08x} with '{info['peer']}' -- "
            f"protection={args.protect}, impairment={cli.ep.impairment.describe()}")
    sess.start()
    return sess, source, sink


def main():
    ap = argparse.ArgumentParser(description="SWARA voice chat client")
    ap.add_argument("--user", required=True); ap.add_argument("--password", required=True)
    ap.add_argument("--server", default="127.0.0.1:9000")
    ap.add_argument("--call"); ap.add_argument("--wait", action="store_true")
    ap.add_argument("--audio", default="auto",
                    choices=["auto", "live", "wav", "synthetic"])
    ap.add_argument("--in", dest="infile"); ap.add_argument("--out", dest="outfile")
    ap.add_argument("--seconds", type=float, default=20.0)
    ap.add_argument("--seed", type=int, default=7)
    ap.add_argument("--protect", default="swara", choices=["swara", "uniform", "none"])
    ap.add_argument("--narrowband", action="store_true")
    ap.add_argument("--loss", type=float, default=0.0)
    ap.add_argument("--burst", type=float, default=1.0)
    ap.add_argument("--extra-delay", type=float, default=0.0)
    ap.add_argument("--jitter", type=float, default=0.0)
    a = ap.parse_args()

    if a.audio == "auto":
        a.audio = "live" if backends_available()["live"] else "synthetic"

    host, _, port = a.server.partition(":")
    imp = Impairment()
    if a.loss > 0 or a.extra_delay > 0 or a.jitter > 0:
        imp.configure(loss_rate=a.loss, mean_burst=a.burst,
                      extra_delay_ms=a.extra_delay, jitter_ms=a.jitter, enabled=True)

    cli = Client(a.user, a.password, (host, int(port or 9000)), imp)
    if not cli.login():
        sys.exit(1)

    if a.call:
        info = cli.place_call(a.call, timeout=12.0)
        if info is None or info["op"] == "call_failed":
            cli.log(f"call failed: {info.get('reason') if info else 'timeout'}"); sys.exit(1)
    elif a.wait:
        cli.log("waiting for an incoming call...")
        info = cli.wait_for({"incoming"}, 120.0)
        if info is None:
            cli.log("no call arrived"); sys.exit(1)
    else:
        cli.log("nothing to do -- pass --call USER or --wait"); sys.exit(0)

```

```
    sess, source, sink = run_call(cli, info, a)
    try:
        while not sess.finished.is_set():
            time.sleep(0.5)
        time.sleep(1.5)                      # let the tail drain out of the buffer
    except KeyboardInterrupt:
        pass
    sess.stop(); source.close(); sink.close()
    print(sess.report())
    cli.send_ctl({"op": "hangup", "call_id": info["call_id"]})


if __name__ == "__main__":
    main()
```

Listing 18 `demo.py` 228 lines

The one-process demonstration and A/B harness.

```
#!/usr/bin/env python3
"""
SWARA end-to-end demonstration.

Starts a real server and two real clients in one process, each on its own UDP
socket, and runs an actual call between them.  Audio goes in as a waveform and
comes out as a waveform, having genuinely traversed: codec -> perceptual
analysis -> FEC -> AEAD -> UDP -> relay -> UDP -> AEAD -> repair -> jitter
buffer -> codec.

    python3 demo.py                          # clean network
    python3 demo.py --loss 0.15 --burst 4    # impaired
    python3 demo.py --loss 0.15 --compare    # A/B: swara vs uniform vs none
"""
from __future__ import annotations
import argparse, os, sys, threading, time
import numpy as np
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from swara.audio_io import AudioSink, AudioSource
from swara.codec import Mode
from swara.net import Impairment
from swara.session import CallSession
from swara.perceptual import damage_weight
from client import Client
import server as server_mod

PORT = 9310


def start_server(port, users_file):
    srv = server_mod.Server("127.0.0.1", port, users_file, verbose=False)
    t = threading.Thread(target=srv.serve_forever, daemon=True)
    t.start()
    return srv, t


def audio_quality(sent: np.ndarray, recv: np.ndarray) -> dict:
    """Align the two waveforms and measure how much of the original survived.

    Alignment is done by exact cross-correlation via FFT rather than by
    stepping a coarse lag grid.  The coarse search that preceded it sampled
    every 80th lag and simply missed the true peak on some runs, reporting a
    correlation near zero for audio that was in fact intact -- a defect in the
    instrument that looked exactly like a defect in the system.
    """
    if recv.size < 1600 or sent.size < 1600:
        return {"corr": 0.0, "snr_db": -99.0, "coverage": 0.0, "lag_ms": 0.0}
    a = sent.astype(np.float64); b = recv.astype(np.float64)
    a -= a.mean(); b -= b.mean()
    n = 1 << int(np.ceil(np.log2(a.size + b.size)))
    corr = np.fft.irfft(np.fft.rfft(a, n) * np.conj(np.fft.rfft(b, n)), n)
    search = min(corr.size, 16000 * 4)          # allow up to 4 s of offset
    lag = int(np.argmax(corr[:search]))
    x = a[lag:]; y = b[:x.size]
    m = min(x.size, y.size)
    if m < 1600:
        return {"corr": 0.0, "snr_db": -99.0, "coverage": recv.size / max(1, sent.size),
                "lag_ms": lag / 16.0}
    x, y = x[:m], y[:m]
    denom = np.linalg.norm(x) * np.linalg.norm(y)
    c = float(np.dot(x, y) / denom) if denom > 0 else 0.0
    scale = np.dot(x, y) / max(np.dot(y, y), 1e-9)
    err = x - scale * y
    snr = 10 * np.log10(max(np.dot(x, x), 1e-9) / max(np.dot(err, err), 1e-9))
```

```
    return {"corr": c, "snr_db": snr, "coverage": recv.size / max(1, sent.size),
            "lag_ms": lag / 16.0}


def run_one(protect, args, users_file, port, tag=""):
    srv, _ = start_server(port, users_file)
    time.sleep(0.25)

    imp_a, imp_b = Impairment(), Impairment()
    for imp in (imp_a, imp_b):
        if args.loss > 0 or args.jitter > 0 or args.extra_delay > 0:
            imp.configure(loss_rate=args.loss, mean_burst=args.burst,
                          extra_delay_ms=args.extra_delay, jitter_ms=args.jitter,
                          enabled=True)

    alice = Client("alice", "hostel-wifi-1", ("127.0.0.1", port), imp_a, verbose=False)
    bob = Client("bob", "hostel-wifi-2", ("127.0.0.1", port), imp_b, verbose=False)
    assert alice.login(), "alice login failed"
    assert bob.login(), "bob login failed"

    got = {}
    def bob_wait():
        got["info"] = bob.wait_for({"incoming"}, 20.0)
    tb = threading.Thread(target=bob_wait, daemon=True); tb.start()
    time.sleep(0.3)
    # Reliable placement: retransmit with a fixed request id (server dedups),
    # then keep nudging until the callee's 'incoming' also survives the loss.
    a_info = alice.place_call("bob", timeout=15.0)
    deadline = time.time() + 15.0
    while got.get("info") is None and time.time() < deadline:
        alice.nudge_call()
        time.sleep(0.8)
    tb.join(timeout=2.0)
    b_info = got.get("info")
    assert a_info and a_info["op"] == "call_ready", f"call setup failed: {a_info}"
    assert b_info, "bob never received the invitation"
    assert b_info["call_id"] == a_info["call_id"], "dedup failed: mismatched call ids"

    src_a = AudioSource("synthetic", duration_s=args.seconds, seed=args.seed)
    sink_b = AudioSink("null")
    src_b = AudioSource("synthetic", duration_s=args.seconds, seed=args.seed + 100)
    sink_a = AudioSink("null")
    sent_a = src_a._buf.copy()

    sa = CallSession(alice.ep, alice.server, bytes.fromhex(a_info["tx"]),
                     bytes.fromhex(a_info["rx"]), a_info["call_id"], src_a, sink_a,
                     mode=Mode.WIDEBAND, protection=protect, name="alice")
    sb = CallSession(bob.ep, bob.server, bytes.fromhex(b_info["tx"]),
                     bytes.fromhex(b_info["rx"]), b_info["call_id"], src_b, sink_b,
                     mode=Mode.WIDEBAND, protection=protect, name="bob")
    # Identical channel realisation for every arm -- see Impairment.reset().
    imp_a.reset(1234 + int(args.seed)); imp_b.reset(4321 + int(args.seed))
    sa.start(); sb.start()

    limit = time.time() + args.seconds + 12
    while time.time() < limit and not (sa.finished.is_set() and sb.finished.is_set()):
        time.sleep(0.2)
    time.sleep(2.0)
    sa.stop(); sb.stop()
    src_a.close(); src_b.close(); sink_a.close(); sink_b.close()

    # Perceptual loss: weight each concealed frame by how much a listener
    # would actually have lost.  The sender knows every frame's PI; the
    # receiver knows which slots it had to conceal.
    lost = sb.concealed_seqs
    dmg, run, prev = 0.0, 0, -10
    for q in sorted(lost):
        run = run + 1 if q == prev + 1 else 0
        dmg += damage_weight(sa.sent_pi.get(q, 128), run)
        prev = q
    perceptual_pct = 100.0 * dmg / max(1, sb.stats.played)

    heard = sink_b.collected
    q = audio_quality(sent_a, heard)
    st = sa.stats.snapshot(); sr = sb.stats.snapshot()
    srv.stop()

    if args.outfile and protect == "swara":
        import wave
        with wave.open(args.outfile, "wb") as w:
            w.setnchannels(1); w.setsampwidth(2); w.setframerate(16000)
            w.writeframes(heard.tobytes())
        print(f"  wrote received audio -> {args.outfile}")

    return {"perceptual_pct": perceptual_pct, "protect": protect, "sent_frames": st["tx_frames"],
"parity": st["tx_parity"],
```

```
            "replicas": st["tx_replicas"], "played": sr["played"],
            "fec_rec": sr["recovered_fec"], "rep_rec": sr["recovered_replica"],
            "concealed": sr["concealed"],
            "loss_after_repair": sr["loss_after_repair_pct"],
            "auth_fail": sr["rx_auth_fail"],
            "tx_kbps": st["tx_bytes"] * 8 / max(1e-9, args.seconds) / 1000,
            "drops": alice.ep.dropped_by_impairment, "rx_frames": sr["rx_frames"],
            "tx_dgrams": alice.ep.tx_packets, **q}


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--loss", type=float, default=0.0)
    ap.add_argument("--burst", type=float, default=1.0)
    ap.add_argument("--extra-delay", type=float, default=0.0)
    ap.add_argument("--jitter", type=float, default=0.0)
    ap.add_argument("--seconds", type=float, default=10.0)
    ap.add_argument("--seed", type=int, default=7)
    ap.add_argument("--compare", action="store_true")
    ap.add_argument("--repeat", type=int, default=1,
                    help="repeat each arm and average -- the live channel is noisy")
    ap.add_argument("--out", dest="outfile", default=None)
    a = ap.parse_args()

    users_file = "demo_users.json"
    if os.path.exists(users_file):
        os.remove(users_file)
    server_mod.save_user("alice", "hostel-wifi-1", users_file)
    server_mod.save_user("bob", "hostel-wifi-2", users_file)

    print(f"\nnetwork: loss={a.loss*100:.0f}% burst={a.burst} "
          f"+{a.extra_delay:.0f}ms jitter={a.jitter:.0f}ms   duration={a.seconds:.0f}s\n")
    arms = ["swara", "uniform", "none"] if a.compare else ["swara"]
    rows = []
    for i, arm in enumerate(arms):
        reps = []
        for rep in range(a.repeat):
            print(f"running arm '{arm}' ({rep+1}/{a.repeat}) ...", flush=True)
            a.seed = 7 + rep * 11
            reps.append(run_one(arm, a, users_file, PORT + i * 7 + rep * 3))
        avg = {k: (float(np.mean([r[k] for r in reps]))
                   if isinstance(reps[0][k], (int, float)) else reps[0][k])
               for k in reps[0]}
        avg["protect"] = arm
        rows.append(avg)

    print("\n" + "=" * 112)
    print(f"{'arm':<9}{'sent':>7}{'parity':>8}{'repl':>6}{'played':>8}{'fecRec':>8}{'repRec':>8}"
          f"{'conceal':>9}{'lossAfter':>11}{'kbps':>8}{'corr':>8}{'SNR dB':>9}{'authFail':>10}")
    print("-" * 112)
    for r in rows:
        print(f"{r['protect']:<9}{r['sent_frames']:>7.0f}{r['parity']:>8.0f}{r['replicas']:>6.0f}"
              f"{r['played']:>8.0f}{r['fec_rec']:>8.1f}{r['rep_rec']:>8.1f}{r['concealed']:>9.1f}"
              f"{r['loss_after_repair']:>10.1f}%{r['tx_kbps']:>8.1f}"
              f"{r['corr']:>8.3f}{r['snr_db']:>9.1f}{r['auth_fail']:>10}")
    print("=" * 112)
    if a.compare and len(rows) >= 2:
        s = next(r for r in rows if r["protect"] == "swara")
        n = next(r for r in rows if r["protect"] == "none")
        u = next((r for r in rows if r["protect"] == "uniform"), None)
        print(f"\nPerceptually weighted damage (lost frames x how audible each one was):")
        print(f"  unprotected {n['perceptual_pct']:.1f}%"
              + (f"   uniform FEC {u['perceptual_pct']:.1f}%" if u else "")
              + f"   SWARA {s['perceptual_pct']:.1f}%")
        print(f"  raw frames unrecoverable: none {n['loss_after_repair']:.1f}%"
              + (f", uniform {u['loss_after_repair']:.1f}%" if u else "")
              + f", swara {s['loss_after_repair']:.1f}%")
        if u:
            print(f"  wire rate: swara {s['tx_kbps']:.0f} kbit/s vs uniform {u['tx_kbps']:.0f} kbit/s "
                  f"({100*(1 - s['tx_kbps']/max(1e-9,u['tx_kbps'])):.0f}% less bandwidth)")
        if a.seconds < 25 or a.repeat < 2:
            print("  note: short single runs are noisy -- for a stable comparison use"
                  "\n        python3 demo.py --seconds 30 --repeat 3 --compare --loss 0.15 --burst 4")
    print()


if __name__ == "__main__":
    main()
```

Listing 19 `api_server.py` 603 lines

The web console's REST API (FastAPI).

```
#!/usr/bin/env python3
"""
SWARA web console and REST API.

    python api_server.py                # http://localhost:8000
    uvicorn api_server:app --port 8000   # equivalent

Serves the operations console at /, the OpenAPI docs at /docs, and hosts the
real UDP relay (server.py) on udp://0.0.0.0:9000 inside the same process so
command-line clients can connect to it while the console watches.
"""
from __future__ import annotations

import asyncio
import json
import os
import subprocess
import sys
import time
import uuid
from contextlib import asynccontextmanager
from typing import Literal, Optional

import numpy as np
from fastapi import FastAPI, HTTPException, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse, JSONResponse, Response, StreamingResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, Field

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, BASE_DIR)

import server as server_mod                                  # noqa: E402
from server import load_users, save_user                      # noqa: E402
from webapp import tools                                      # noqa: E402
from webapp.lab import LabManager, pcm_to_wav                 # noqa: E402
from webapp.relay import ExperimentJob, RelayManager          # noqa: E402

FIG_DIR = os.path.join(BASE_DIR, "figures")
STATIC_DIR = os.path.join(BASE_DIR, "static")
RESULTS_FILE = os.path.join(BASE_DIR, "experiments", "results.json")
USERS_FILE = os.path.join(BASE_DIR, server_mod.USERS_FILE)
RUNTIME_DIR = os.path.join(BASE_DIR, "runtime")
UPLOAD_DIR = os.path.join(RUNTIME_DIR, "uploads")
LAB_USERS_FILE = os.path.join(RUNTIME_DIR, "lab_users.json")
os.makedirs(UPLOAD_DIR, exist_ok=True)

RELAY_PORT = int(os.environ.get("SWARA_RELAY_PORT", "9000"))
RELAY_AUTOSTART = os.environ.get("SWARA_RELAY_AUTOSTART", "1") != "0"
STARTED = time.time()

relay = RelayManager(USERS_FILE)
lab = LabManager(LAB_USERS_FILE)
experiments = ExperimentJob(BASE_DIR)


@asynccontextmanager
async def lifespan(_app: FastAPI):
    if RELAY_AUTOSTART:
        relay.start("0.0.0.0", RELAY_PORT)
    yield
    relay.stop()
    for run in list(lab.history.values()):
        run.stop()


app = FastAPI(
    title="SWARA API & Control Console",
    description=("Secure Wideband Adaptive Real-time Audio over UDP. "
                 "REST API behind the operations console at `/`: live instrumented calls, "
                 "A/B benchmarks, the hosted UDP relay, and interactive access to every "
                 "protocol component (E-model, NAVIK, PI analyser, Reed-Solomon, SPARK, SETU)."),
    version="2.0.0",
    lifespan=lifespan,
)
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_credentials=True,
                   allow_methods=["*"], allow_headers=["*"])


def read_results() -> dict:
    if not os.path.exists(RESULTS_FILE):
```

```
        raise HTTPException(404, "Results file not found. Run the experiments first.")
    with open(RESULTS_FILE, encoding="utf-8") as f:
        return json.load(f)


def wav_response(pcm: np.ndarray, name: str) -> Response:
    if pcm is None or pcm.size == 0:
        raise HTTPException(404, "no audio for this run")
    return Response(pcm_to_wav(pcm), media_type="audio/wav",
                    headers={"Content-Disposition": f'inline; filename="{name}.wav"',
                             "Cache-Control": "no-store"})


# ============================================================================
# Models
# ============================================================================
class RegisterRequest(BaseModel):
    username: str
    password: str


class DemoRequest(BaseModel):
    seconds: float = 10.0
    loss: float = 0.15
    burst: float = 2.0
    compare: bool = True


class CallRequest(BaseModel):
    seconds: float = Field(15.0, ge=3, le=60, description="call length")
    loss: float = Field(0.10, ge=0, le=0.6, description="long-run packet loss, 0..1")
    burst: float = Field(3.0, ge=1, le=20, description="mean loss burst, packets")
    jitter: float = Field(10.0, ge=0, le=200, description="delay jitter, ms")
    extra_delay: float = Field(0.0, ge=0, le=500, description="added one-way delay, ms")
    protection: Literal["swara", "uniform", "none"] = "swara"
    mode: Literal["wideband", "narrowband"] = "wideband"
    source: Literal["synthetic", "upload", "mic"] = "synthetic"
    upload_id: Optional[str] = None
    seed: int = 7


class BenchRequest(CallRequest):
    repeat: int = Field(1, ge=1, le=3)


class ImpairmentUpdate(BaseModel):
    loss: Optional[float] = Field(None, ge=0, le=0.6)
    burst: Optional[float] = Field(None, ge=1, le=20)
    jitter: Optional[float] = Field(None, ge=0, le=200)
    extra_delay: Optional[float] = Field(None, ge=0, le=500)


class RelayStart(BaseModel):
    host: str = "0.0.0.0"
    port: int = Field(9000, ge=1024, le=65535)


class EModelRequest(BaseModel):
    codec: str = "opus-24k"
    delay_ms: float = Field(150.0, ge=0, le=800)
    loss_pct: float = Field(3.0, ge=0, le=50)
    burst: float = Field(1.0, ge=1, le=20)
    mobile: bool = False


class NavikRequest(BaseModel):
    loss_pct: float = Field(8.0, ge=0, le=50)
    burst: float = Field(3.0, ge=1, le=20)
    jitter_ms: float = Field(12.0, ge=0, le=200)
    owd_ms: float = Field(40.0, ge=0, le=500)
    depth_ms: float = Field(80.0, ge=0, le=400)
    capacity_kbps: float = Field(400.0, ge=8, le=5000)
    pi_afec: bool = True
    mobile: bool = False


class PerceptualRequest(BaseModel):
    seconds: float = Field(8.0, ge=1, le=20)
    seed: int = 7
    k: int = Field(4, ge=2, le=12)
    rho: float = Field(0.3, ge=0, le=1)
    burst: float = Field(2.0, ge=1, le=20)


class FecPlanRequest(BaseModel):
    pis: list[int] = Field(default_factory=lambda: [20, 210, 180, 90])
    rho: float = Field(0.3, ge=0, le=1)
```

```
    burst: float = Field(2.0, ge=1, le=20)


class RSRequest(BaseModel):
    message: str = Field("SWARA: the same budget, spent where it is heard.", max_length=400)
    k: int = Field(4, ge=1, le=16)
    r: int = Field(2, ge=0, le=8)
    lost: list[int] = Field(default_factory=list)


class HandshakeRequest(BaseModel):
    username: str = Field("alice", min_length=1, max_length=40)
    password: str = Field("hostel-wifi-1", min_length=1, max_length=128)
    attempt: Optional[str] = Field(None, description="password the client types; defaults to the
correct one")


class HeaderRequest(BaseModel):
    ptype: int = Field(6, ge=0, le=15)
    flags: int = Field(0x09, ge=0, le=255)
    seq: int = Field(4242, ge=0, le=65535)
    cid: int = Field(0x5A17C0DE, ge=0, le=0xFFFFFFFF)
    ts_ms: int = Field(123456, ge=0, le=0xFFFFFFFF)
    pi: int = Field(212, ge=0, le=255)
    fec_group: int = Field(17, ge=0, le=255)
    fec_index: int = Field(2, ge=0, le=255)


class OverlayRequest(BaseModel):
    nodes: int = Field(7, ge=3, le=12)
    bad_frac: float = Field(0.28, ge=0, le=1)
    seed: int = 4


class SimRequest(BaseModel):
    loss: float = Field(0.10, ge=0, le=0.6)
    burst: float = Field(4.0, ge=1, le=20)
    delay: float = Field(60.0, ge=0, le=600)
    jitter: float = Field(25.0, ge=0, le=200)
    capacity: float = Field(400.0, ge=50, le=20000)
    reorder: float = Field(0.0, ge=0, le=0.3)
    duration: float = Field(20.0, ge=4, le=60)
    seed: int = 3
    codec: str = "opus-24k"
    mobile: bool = False


# ============================================================================
# Console & health
# ============================================================================
@app.get("/", include_in_schema=False)
def console():
    return FileResponse(os.path.join(STATIC_DIR, "index.html"),
                        headers={"Cache-Control": "no-cache"})


@app.get("/api", summary="API health & metadata", tags=["System"])
def api_root():
    return {
        "status": "online",
        "app": "SWARA — Secure Wideband Adaptive Real-time Audio over UDP",
        "version": app.version,
        "uptime_s": time.time() - STARTED,
        "udp_relay": f"{relay.host}:{relay.port}" if relay.running else None,
        "console": "/", "swagger_docs": "/docs", "redoc": "/redoc",
    }


@app.get("/api/status", summary="Live system status for the console header", tags=["System"])
def api_status():
    active = lab.active
    return {
        "uptime_s": time.time() - STARTED,
        "relay": {"running": relay.running, "port": relay.port, "host": relay.host,
                  "online": len(relay.srv.online) if relay.srv else 0,
                  "calls": len(relay.srv.calls) if relay.srv else 0, "error": relay.error},
        "lab": {"busy": lab.busy(), "kind": getattr(active, "kind", None) if lab.busy() else None,
                "id": active.id if active is not None and lab.busy() else None,
                "phase": active.phase if active is not None else None},
        "experiments": {"running": experiments.running},
        "audio_backends": lab.audio_backends(),
        "results_available": os.path.exists(RESULTS_FILE),
    }


@app.get("/api/system", summary="Runtime, packages and module inventory", tags=["System"])
def api_system():
```

```
    return {**tools.system_info(), "modules": tools.module_inventory(BASE_DIR),
            "audio_backends": lab.audio_backends(),
            "routes": [{"path": r.path, "methods": sorted(getattr(r, "methods", []) or []),
                        "summary": getattr(r, "summary", None), "tags": getattr(r, "tags", [])}
                       for r in app.routes if getattr(r, "include_in_schema", False)]}


# ============================================================================
# Users (original endpoints kept for compatibility)
# ============================================================================
@app.get("/users", summary="List registered users", tags=["Users"])
def list_registered_users():
    users = load_users(USERS_FILE)
    return {"count": len(users), "usernames": list(users.keys())}


@app.get("/api/users", summary="Registered users with stored-record details", tags=["Users"])
def api_users():
    users = load_users(USERS_FILE)
    online = set(relay.srv.online) if relay.srv else set()
    return {"count": len(users), "users": [
        {"username": u, "record_bytes": r.size_bytes(), "salt": r.salt.hex(),
         "pub": r.client_static_pub.hex(), "envelope_bytes": len(r.envelope),
         "online": u in online} for u, r in users.items()]}


@app.post("/users/register", summary="Register a new user", tags=["Users"])
@app.post("/api/users", summary="Register a new user", tags=["Users"], include_in_schema=False)
def register_new_user(req: RegisterRequest):
    username = req.username.strip()
    if not username or not req.password:
        raise HTTPException(400, "Username and password are required")
    if len(username) > 40 or not all(c.isalnum() or c in "-_." for c in username):
        raise HTTPException(400, "Username may contain letters, digits, '-', '_' and '.' (max 40)")
    save_user(username, req.password, USERS_FILE)
    rec = load_users(USERS_FILE)[username]
    relay.add_user_record(rec)
    return {"message": f"User '{username}' registered successfully",
            "record_bytes": rec.size_bytes()}


@app.delete("/api/users/{username}", summary="Delete a registered user", tags=["Users"])
def delete_user(username: str):
    if not os.path.exists(USERS_FILE):
        raise HTTPException(404, "no users file")
    with open(USERS_FILE, encoding="utf-8") as f:
        raw = json.load(f)
    if username not in raw:
        raise HTTPException(404, f"user '{username}' not found")
    raw.pop(username)
    with open(USERS_FILE, "w", encoding="utf-8") as f:
        json.dump(raw, f, indent=2)
    relay.remove_user(username)
    return {"message": f"User '{username}' deleted"}


# ============================================================================
# Relay
# ============================================================================
@app.get("/api/relay", summary="Hosted UDP relay status", tags=["Relay"])
def relay_status(since: float = 0.0):
    return {**relay.status(), "logs": relay.log_tail(since)}


@app.post("/api/relay/start", summary="Start the hosted UDP relay", tags=["Relay"])
def relay_start(req: RelayStart):
    return relay.start(req.host, req.port)


@app.post("/api/relay/stop", summary="Stop the hosted UDP relay", tags=["Relay"])
def relay_stop():
    return relay.stop()


# ============================================================================
# Live call lab
# ============================================================================
def _upload_path(req: CallRequest) -> Optional[str]:
    if req.source != "upload":
        if req.source == "mic" and not lab.audio_backends().get("live"):
            raise HTTPException(400, "server microphone unavailable (sounddevice not installed)")
        return None
    if not req.upload_id or not req.upload_id.isalnum():
        raise HTTPException(400, "upload_id required for source=upload")
    p = os.path.join(UPLOAD_DIR, f"{req.upload_id}.wav")
    if not os.path.exists(p):
```

```
        raise HTTPException(404, "uploaded clip not found; record or upload it again")
    return p


@app.post("/api/lab/call", summary="Start an instrumented live call", tags=["Live lab"])
def lab_call(req: CallRequest):
    try:
        run = lab.start_call(req.model_dump(), _upload_path(req))
    except RuntimeError as e:
        raise HTTPException(409, str(e))
    return run.describe()


@app.post("/api/lab/bench", summary="Start an A/B/C benchmark (SWARA vs uniform FEC vs none)",
          tags=["Live lab"])
def lab_bench(req: BenchRequest):
    try:
        run = lab.start_bench(req.model_dump(), _upload_path(req))
    except RuntimeError as e:
        raise HTTPException(409, str(e))
    return run.describe()


@app.get("/api/lab/active", summary="The run currently in progress, if any", tags=["Live lab"])
def lab_active():
    a = lab.active
    return {"busy": lab.busy(), "run": a.describe() if a is not None else None}


@app.get("/api/lab/{rid}", summary="Describe a call or benchmark", tags=["Live lab"])
def lab_get(rid: str):
    run = lab.get(rid)
    if run is None:
        raise HTTPException(404, "unknown run")
    return run.describe()


@app.post("/api/lab/{rid}/stop", summary="Hang up / cancel a run", tags=["Live lab"])
def lab_stop(rid: str):
    run = lab.get(rid)
    if run is None:
        raise HTTPException(404, "unknown run")
    run.stop()
    return {"stopping": True}


@app.post("/api/lab/{rid}/impairment", summary="Change the network during a live call",
          tags=["Live lab"])
def lab_impair(rid: str, req: ImpairmentUpdate):
    run = lab.get(rid)
    if run is None or getattr(run, "kind", "") != "call":
        raise HTTPException(404, "unknown call")
    run.configure_impairment(req.loss, req.burst, req.jitter, req.extra_delay)
    return {"impairment": run.impairment}


@app.get("/api/lab/{rid}/stream", summary="Server-sent events: telemetry, frame fates, audio",
         tags=["Live lab"])
async def lab_stream(rid: str, request: Request, audio: int = 1):
    run = lab.get(rid)
    if run is None or getattr(run, "kind", "") != "call":
        raise HTTPException(404, "unknown call")

    async def gen():
        cur = {"t": 0, "e": 0, "f": 0, "a": 0}
        yield f"event: hello\ndata: {json.dumps(run.describe())}\n\n"
        while True:
            if await request.is_disconnected():
                return
            finished = run.done.is_set()
            payload = run.poll(cur, with_audio=bool(audio))
            yield f"data: {json.dumps(payload)}\n\n"
            if finished:
                yield "event: end\ndata: {}\n\n"
                return
            await asyncio.sleep(0.2)

    return StreamingResponse(gen(), media_type="text/event-stream",
                             headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"})


@app.get("/api/lab/{rid}/audio/{which}.wav", summary="Sent or received audio of a finished call",
         tags=["Live lab"])
def lab_audio(rid: str, which: Literal["sent", "received"]):
    run = lab.get(rid)
    if run is None or getattr(run, "kind", "") != "call":
        raise HTTPException(404, "unknown call")
```

```
    return wav_response(run.sent_pcm if which == "sent" else run.heard_pcm, f"swara-{rid}-{which}")


@app.post("/api/audio/upload", summary="Upload a clip (raw 16 kHz mono int16 little-endian PCM)",
          tags=["Live lab"])
async def audio_upload(request: Request):
    body = await request.body()
    if len(body) < 3200 or len(body) > 16000 * 2 * 61:
        raise HTTPException(400, "clip must be between 0.1 s and 60 s of 16 kHz int16 PCM")
    pcm = np.frombuffer(body[: len(body) // 2 * 2], dtype="<i2").astype(np.int16)
    uid = uuid.uuid4().hex[:12]
    with open(os.path.join(UPLOAD_DIR, f"{uid}.wav"), "wb") as f:
        f.write(pcm_to_wav(pcm))
    for fn in sorted(os.listdir(UPLOAD_DIR), key=lambda n: os.path.getmtime(os.path.join(UPLOAD_DIR,
n)))[:-12]:
        try:
            os.remove(os.path.join(UPLOAD_DIR, fn))
        except OSError:
            pass
    return {"upload_id": uid, "seconds": pcm.size / 16000.0,
            "peak": int(np.abs(pcm).max()) if pcm.size else 0}


# ============================================================================
# Offline simulator
# ============================================================================
@app.get("/api/sim/profiles", summary="Named network profiles", tags=["Simulator"])
def sim_profiles():
    return {"profiles": tools.profiles()}


@app.post("/api/sim/compare", summary="Simulate SWARA vs uniform FEC vs baseline", tags=["Simulator"])
def sim_compare(req: SimRequest):
    return tools.simulate(req.model_dump())


@app.post("/api/sim/sweep", summary="MOS against packet loss for all three systems",
tags=["Simulator"])
def sim_sweep(req: SimRequest):
    return tools.simulate_sweep(req.model_dump())


# ============================================================================
# Toolkit
# ============================================================================
@app.post("/api/tools/emodel", summary="ITU-T G.107 E-model evaluation", tags=["Toolkit"])
def tool_emodel(req: EModelRequest):
    try:
        return tools.emodel_eval(req.codec, req.delay_ms, req.loss_pct, req.burst, req.mobile)
    except ValueError as e:
        raise HTTPException(400, str(e))


@app.post("/api/tools/navik", summary="NAVIK joint controller decision", tags=["Toolkit"])
def tool_navik(req: NavikRequest):
    return tools.navik_eval(req.loss_pct, req.burst, req.jitter_ms, req.owd_ms, req.depth_ms,
                            req.capacity_kbps, req.pi_afec, req.mobile)


@app.post("/api/tools/perceptual", summary="Perceptual-importance analysis of synthetic speech",
          tags=["Toolkit"])
def tool_perceptual(req: PerceptualRequest):
    pcm, truth = tools.synthetic_speech(req.seconds, req.seed)
    return tools.perceptual_analysis(pcm, truth, req.k, req.rho, req.burst)


@app.post("/api/tools/perceptual/pcm", summary="Perceptual analysis of raw 16 kHz int16 PCM",
          tags=["Toolkit"])
async def tool_perceptual_pcm(request: Request, k: int = 4, rho: float = 0.3, burst: float = 2.0):
    body = await request.body()
    if len(body) < 3200:
        raise HTTPException(400, "clip too short")
    pcm = np.frombuffer(body[: min(len(body), 16000 * 2 * 20) // 2 * 2], dtype="<i2").astype(np.int16)
    return tools.perceptual_analysis(pcm, None, max(2, min(12, k)), rho, burst)


@app.post("/api/tools/fec/plan", summary="PI-AFEC plan for one group", tags=["Toolkit"])
def tool_fec_plan(req: FecPlanRequest):
    if not 2 <= len(req.pis) <= 12:
        raise HTTPException(400, "2..12 PI scores")
    return tools.fec_plan([max(0, min(255, p)) for p in req.pis], req.rho, req.burst)


@app.post("/api/tools/rs", summary="Reed-Solomon encode, erase, decode", tags=["Toolkit"])
def tool_rs(req: RSRequest):
    try:
        return tools.rs_playground(req.message, req.k, req.r, req.lost)
```

```
    except ValueError as e:
        raise HTTPException(400, str(e))


@app.post("/api/tools/handshake", summary="Run a SPARK handshake in memory", tags=["Toolkit"])
def tool_handshake(req: HandshakeRequest):
    return tools.spark_handshake(req.username, req.password,
                                 req.attempt if req.attempt is not None else req.password)


@app.post("/api/tools/header", summary="Pack a 16-byte SWARA header", tags=["Toolkit"])
def tool_header(req: HeaderRequest):
    return tools.pack_header(**req.model_dump())


@app.post("/api/tools/overlay", summary="SETU distance-vector overlay", tags=["Toolkit"])
def tool_overlay(req: OverlayRequest):
    return tools.overlay_eval(req.nodes, req.bad_frac, req.seed)


# ============================================================================
# Research results & figures (original endpoints kept)
# ============================================================================
@app.get("/experiments/results", summary="Research benchmark results", tags=["Research"])
def get_results():
    return read_results()


@app.get("/api/experiments/job", summary="Experiment regeneration job status", tags=["Research"])
def experiments_status(since: int = 0):
    return experiments.status(since)


@app.post("/api/experiments/run", summary="Regenerate all results and figures (~3 min)",
          tags=["Research"])
def experiments_run():
    if lab.busy():
        raise HTTPException(409, "a live lab run is in progress; wait for it to finish")
    return experiments.start()


@app.get("/figures", summary="List research figures", tags=["Research"])
def list_figures():
    if not os.path.exists(FIG_DIR):
        return {"count": 0, "figures": []}
    files = sorted(f for f in os.listdir(FIG_DIR) if f.endswith(".png"))
    return {"count": len(files), "figures": files,
            "mtime": {f: os.path.getmtime(os.path.join(FIG_DIR, f)) for f in files}}


@app.get("/figures/{filename}", summary="Get a figure image", tags=["Research"])
def get_figure_file(filename: str):
    safe = os.path.basename(filename)
    file_path = os.path.join(FIG_DIR, safe)
    if not safe.endswith(".png") or not os.path.exists(file_path):
        raise HTTPException(404, "Figure not found")
    return FileResponse(file_path, media_type="image/png")


@app.post("/demo/run", summary="Run demo.py and return its console output", tags=["Research"])
def run_demo(req: DemoRequest):
    if lab.busy():
        raise HTTPException(409, "a live lab run is in progress")
    cmd = [sys.executable, os.path.join(BASE_DIR, "demo.py"), "--seconds", str(req.seconds)]
    if req.loss > 0:
        cmd.extend(["--loss", str(req.loss)])
    if req.burst > 1:
        cmd.extend(["--burst", str(req.burst)])
    if req.compare:
        cmd.append("--compare")
    proc = subprocess.run(cmd, capture_output=True, text=True, cwd=RUNTIME_DIR,
                          encoding="utf-8", errors="replace")
    return {"returncode": proc.returncode, "stdout": proc.stdout, "stderr": proc.stderr}


@app.exception_handler(Exception)
async def unhandled(_req: Request, exc: Exception):
    return JSONResponse(status_code=500, content={"detail": f"{type(exc).__name__}: {exc}"})


app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")


if __name__ == "__main__":
    import uvicorn
    port = int(os.environ.get("PORT", "8000"))
    print(f"\n  SWARA console   http://localhost:{port}/\n"
```

```
            f"  API docs        http://localhost:{port}/docs\n"
            f"  UDP relay       udp://0.0.0.0:{RELAY_PORT}\n")
    uvicorn.run("api_server:app", host="0.0.0.0", port=port, reload=False)
```